Java Source Code Compilation

Total Files: 321

---

1. `1. Two Sum.java`

```
/*

Given an array of integers, return indices of the two numbers such that they add up to a specific target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

Example:

Given nums = [2, 7, 11, 15], target = 9,

Because nums[0] + nums[1] = 2 + 7 = 9,
return [0, 1].

*/

import java.util.*;

class Solution {

    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (map.containsKey(complement)) {
                return new int[]{map.get(complement), i};
            }
            map.put(nums[i], i);
        }
        throw new IllegalArgumentException("No two sum solution");
    }
}
```

2. `2. Add Two Numbers.java`

```
/*
 * You are given two non-empty linked lists representing two non-negative integers. The
digits are stored in reverse order and each of their nodes contain a single digit. Add
the two numbers and return it as a linked list.
 *
 * You may assume the two numbers do not contain any leading zero, except the number 0
itself.
 *
 * Example:
 *
 * Input: (2 -> 4 -> 3) + (5 -> 6 -> 4)
 * Output: 7 -> 0 -> 8
 * Explanation: 342 + 465 = 807.
 */

class Solution {

    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        int carry = 0;
        ListNode p, dummy = new ListNode(0);
        p = dummy;
        while (l1 != null || l2 != null || carry != 0) {
            if (l1 != null) {
                carry += l1.val;
                l1 = l1.next;
            }
            if (l2 != null) {
                carry += l2.val;
                l2 = l2.next;
            }
            p.next = new ListNode(carry % 10);
            carry /= 10;
            p = p.next;
        }
        return dummy.next;
    }
}
```

3. `3. Longest Substring Without Repeating Characters.java`

```
/*
 * Given a string, find the length of the longest substring without repeating characters.
 *
 * Example 1:
 *
 * Input: "abcabcbb"
 * Output: 3
 * Explanation: The answer is "abc", with the length of 3.
 * Example 2:
 *
 * Input: "bbbbb"
 * Output: 1
 * Explanation: The answer is "b", with the length of 1.
 * Example 3:
 *
 * Input: "pwwkew"
 * Output: 3
 * Explanation: The answer is "wke", with the length of 3.
 * Note that the answer must be a substring, "pwke" is a subsequence and not a substring.
 *
 */

public class Solution {

    public int lengthOfLongestSubstring(String s) {

        Set<Character> set = new HashSet<>();

        int i = 0, j = 0, n = s.length();
        int ans = 0;
        while (i < n && j < n) {
            if (!set.contains(s.charAt(j))) {
                set.add(s.charAt(j));
                j++;
                ans = Math.max(ans, j - i);
            } else {
                set.remove(s.charAt(i));
                i++;
            }
        }

        return ans;
    }
}
```

4. `5. Longest Palindromic Substring.java`

```
/*
 * Given a string s, find the longest palindromic substring in s. You may assume that the
maximum length of s is 1000.
 *
 * Example 1:
 *
 * Input: "babad"
 * Output: "bab"
 * Note: "aba" is also a valid answer.
 * Example 2:
 *
 * Input: "cbbd"
 * Output: "bb"
 *
 */

public class Solution {

    public String longestPalindrome(String s) {
        if (s == null || s.length() == 0) {
            return "";
        }
        int len = s.length();
        boolean[][] dp = new boolean[len][len];
        int start = 0;
        int end = 0;
        int max = 0;
        for (int i = 0; i < s.length(); i++) {
            for (int j = 0; j <= i; j++) {
                if (s.charAt(i) == s.charAt(j) && (i - j <= 2 || dp[j + 1][i - 1])) {
                    dp[j][i] = true;
                }
                if (dp[j][i] && max < i - j + 1) {
                    max = i - j + 1;
                    start = j;
                    end = i;
                }
            }
        }
        return s.substring(start, end + 1);
    }
}
```

5. `6. ZigZag Conversion.java`

```
/*
 * The string "PAYPALISHIRING" is written in a zigzag pattern on a given number of rows
like this: (you may want to display this pattern in a fixed font for better legibility)
 *
 * P   A   H   N
 * A P L S I I G
 * Y   I   R
 * And then read line by line: "PAHNAPLSIIGYIR"
 *
 * Write the code that will take a string and make this conversion given a number of
rows:
 *
 * string convert(string s, int numRows);
 * Example 1:
 *
 * Input: s = "PAYPALISHIRING", numRows = 3
 * Output: "PAHNAPLSIIGYIR"
 * Example 2:
 *
 * Input: s = "PAYPALISHIRING", numRows = 4
 * Output: "PINALSIGYAHRPI"
 * Explanation:
 *
 * P     I    N
 * A   L S  I G
 * Y A   H R
 * P     I
 *
 */

public class Solution {

    public String convert(String s, int rows) {

        if (rows == 1) {
            return s;
        }

        char[] c = s.toCharArray();

        StringBuffer[] sb = new StringBuffer[rows];

        for (int i = 0; i < sb.length; i++) {
            sb[i] = new StringBuffer();
        }

        int ar = 0;
        boolean direction = true;//True means going down
        for (int i = 0; i < s.length(); i++) {
            if (rows == 1) {
                return s;
```

```
            }

            char temp = c[i];
            // System.out.println(ar);

            sb[ar].append(temp);

            if (ar == 0) {
                direction = true;
            } else {
                if (ar == rows - 1) {
                    direction = false;
                }
            }

            if (direction == true) {
                ar++;
            } else {
                ar--;
            }

        }

        for (int i = 1; i < rows; i++) {
            sb[0].append(sb[i]);
        }
        System.out.println(sb[0].toString());
        return sb[0].toString();

    }
}
```

6. `7. Reverse Integer.java`

```
/*
 * Given a 32-bit signed integer, reverse digits of an integer.
 *
 * Example 1:
 *
 * Input: 123
 * Output: 321
 * Example 2:
 *
 * Input: -123
 * Output: -321
 * Example 3:
 *
 * Input: 120
 * Output: 21
 * Note:
 * Assume we are dealing with an environment which could only store integers within the
32-bit signed integer range: [−231,  231 − 1]. For the purpose of this problem, assume
that your function returns 0 when the reversed integer overflows.
 *
 */

class Solution {

    public int reverse(int x) {
        long rev = 0;
        while (x != 0) {
            rev = rev * 10 + x % 10;
            x = x / 10;
            if (rev > Integer.MAX_VALUE || rev < Integer.MIN_VALUE) {
                return 0;
            }
        }
        return (int) rev;

    }

}
```

7. `8. String to Integer (atoi).java`

```
/*
 * Implement atoi which converts a string to an integer.
 *
 * The function first discards as many whitespace characters as necessary until the first
non-whitespace character is found. Then, starting from this character, takes an optional
initial plus or minus sign followed by as many numerical digits as possible, and
interprets them as a numerical value.
 *
 * The string can contain additional characters after those that form the integral
number, which are ignored and have no effect on the behavior of this function.
 *
 * If the first sequence of non-whitespace characters in str is not a valid integral
number, or if no such sequence exists because either str is empty or it contains only
whitespace characters, no conversion is performed.
 *
 * If no valid conversion could be performed, a zero value is returned.
 *
 * Note:
 *
 * Only the space character ' ' is considered as whitespace character.
 * Assume we are dealing with an environment which could only store integers within the
32-bit signed integer range: [−231,  231 − 1]. If the numerical value is out of the range
of representable values, INT_MAX (231 − 1) or INT_MIN (−231) is returned.
 * Example 1:
 *
 * Input: "42"
 * Output: 42
 * Example 2:
 *
 * Input: "   -42"
 * Output: -42
 * Explanation: The first non-whitespace character is '-', which is the minus sign.
 * Then take as many numerical digits as possible, which gets 42.
 * Example 3:
 *
 * Input: "4193 with words"
 * Output: 4193
 * Explanation: Conversion stops at digit '3' as the next character is not a numerical
digit.
 * Example 4:
 *
 * Input: "words and 987"
 * Output: 0
 * Explanation: The first non-whitespace character is 'w', which is not a numerical
 * digit or a +/- sign. Therefore no valid conversion could be performed.
 * Example 5:
 *
 * Input: "-91283472332"
 * Output: -2147483648
 * Explanation: The number "-91283472332" is out of the range of a 32-bit signed integer.
 * Thefore INT_MIN (−231) is returned.
```

```
 *
 */

class Solution {

    public int myAtoi(String str) {
        if (str == null || str.length() < 1) {
            return 0;
        }

        // trim white spaces
        str = str.trim();

        char flag = '+';

        // check negative or positive
        int i = 0;
        if (str.charAt(0) == '-') {
            flag = '-';
            i++;
        } else if (str.charAt(0) == '+') {
            i++;
        }
        // use double to store result
        double result = 0;

        // calculate value
        while (str.length() > i && str.charAt(i) >= '0' && str.charAt(i) <= '9') {
            result = result * 10 + (str.charAt(i) - '0');
            i++;
        }

        if (flag == '-') {
            result = -result;
        }

        // handle max and min
        if (result > Integer.MAX_VALUE) {
            return Integer.MAX_VALUE;
        }

        if (result < Integer.MIN_VALUE) {
            return Integer.MIN_VALUE;
        }

        return (int) result;
    }
}
```

8. `9. Palindrome Number.java`

```
/*
 *
 * Determine whether an integer is a palindrome. An integer is a palindrome when it reads
the same backward as forward.
 *
 * Example 1:
 *
 * Input: 121
 * Output: true
 * Example 2:
 *
 * Input: -121
 * Output: false
 * Explanation: From left to right, it reads -121. From right to left, it becomes 121-.
Therefore it is not a palindrome.
 * Example 3:
 *
 * Input: 10
 * Output: false
 * Explanation: Reads 01 from right to left. Therefore it is not a palindrome.
 */

class Solution {

    public boolean isPalindrome(int x) {
        /*
         * First we check for the false cases. i) if the number is negative then
         * it is not possible for palindrome ii) if the number is zero, then it
         * not a palindrome(the terminating condition also) iii) if the last
         * digit is zero, then it can't be palindrome
         *
         * Then we just are doing reverse of half of the numbers. So that's why
         * we have taken the reverse, and the comparision x > rev tells us that
         * we need to do stop at the half way mark. This is done to prevent the
         * overflow, which may happen for long integers.
         */
        if (x < 0 || (x != 0 && x % 10 == 0)) {
            return false;
        }
        int rev = 0;

        /*
         * Here we are just doing half of the actual work, since we are just
         * calculating the half part of the given integer.
         *
         */
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x = x / 10;
        }
        return (x == rev || x == rev / 10);
```

```
    }
}
```

9. `10. Regular Expression Matching.java`

```
/*
 *
 * Given an input string (s) and a pattern (p), implement regular expression matching
with support for '.' and '*'.
 *
 * '.' Matches any single character.
 * '*' Matches zero or more of the preceding element.
 * The matching should cover the entire input string (not partial).
 *
 * Note:
 *
 * s could be empty and contains only lowercase letters a-z.
 * p could be empty and contains only lowercase letters a-z, and characters like . or *.
 * Example 1:
 *
 * Input:
 * s = "aa"
 * p = "a"
 * Output: false
 * Explanation: "a" does not match the entire string "aa".
 * Example 2:
 *
 * Input:
 * s = "aa"
 * p = "a*"
 * Output: true
 * Explanation: '*' means zero or more of the precedeng element, 'a'. Therefore, by
repeating 'a' once, it becomes "aa".
 * Example 3:
 *
 * Input:
 * s = "ab"
 * p = ".*"
 * Output: true
 * Explanation: ".*" means "zero or more (*) of any character (.)".
 * Example 4:
 *
 * Input:
 * s = "aab"
 * p = "c*a*b"
 * Output: true
 * Explanation: c can be repeated 0 times, a can be repeated 1 time. Therefore it matches
"aab".
 * Example 5:
 *
 * Input:
 * s = "mississippi"
 * p = "mis*is*p*."
 * Output: false
 */
```

```
class Solution {

    public boolean isMatch(String s, String p) {
        // base case
        if (p.length() == 0) {
            return s.length() == 0;
        }
        // special case
        if (p.length() == 1) {
            // if the length of s is 0, return false
            if (s.length() < 1) {
                return false;
            } //if the first does not match, return false
            else if ((p.charAt(0) != s.charAt(0)) && (p.charAt(0) != '.')) {
                return false;
            } // otherwise, compare the rest of the string of s and p.
            else {
                return isMatch(s.substring(1), p.substring(1));
            }
        }
        // case 1: when the second char of p is not '*'
        if (p.charAt(1) != '*') {
            if (s.length() < 1) {
                return false;
            }
            if ((p.charAt(0) != s.charAt(0)) && (p.charAt(0) != '.')) {
                return false;
            } else {
                return isMatch(s.substring(1), p.substring(1));
            }
        } // case 2: when the second char of p is '*', complex case.
        else {
            //case 2.1: a char & '*' can stand for 0 element
            if (isMatch(s, p.substring(2))) {
                return true;
            }
            //case 2.2: a char & '*' can stand for 1 or more preceding element,so try
every sub string
            int i = 0;
            while (i < s.length() && (s.charAt(i) == p.charAt(0) || p.charAt(0) == '.'))
{
                if (isMatch(s.substring(i + 1), p.substring(2))) {
                    return true;
                }
                i++;
            }
            return false;
        }
    }
}
```

10. `11. Container With Most Water.java`

```
/*
 * Given n non-negative integers a1, a2, ..., an , where each represents a point at coordinate (i, ai). n vertical lines are drawn such that the two endpoints of line i is at (i, ai) and (i, 0). Find two lines, which together with x-axis forms a container, such that the container contains the most water.
 *
 * Note: You may not slant the container and n is at least 2.
 *
 * The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.
 *
 * Example:
 *
 * Input: [1,8,6,2,5,4,8,3,7]
 * Output: 49
 *
 */

public class Solution {

    public int maxArea(int[] height) {

        if (height == null || height.length < 2) {
            return 0;
        }

        int max = 0;
        int left = 0;
        int right = height.length - 1;

        while (left < right) {
            max = Math.max(max, (right - left) * Math.min(height[left], height[right]));
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }

        return max;

    }
}
```

11. `12. Integer to Roman.java`

```
/*
 *
 * Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.
 *
 * Symbol       Value
 * I             1
 * V             5
 * X             10
 * L             50
 * C             100
 * D             500
 * M             1000
 * For example, two is written as II in Roman numeral, just two one's added together. Twelve is written as, XII, which is simply X + II. The number twenty seven is written as XXVII, which is XX + V + II.
 *
 * Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:
 *
 * I can be placed before V (5) and X (10) to make 4 and 9.
 * X can be placed before L (50) and C (100) to make 40 and 90.
 * C can be placed before D (500) and M (1000) to make 400 and 900.
 * Given an integer, convert it to a roman numeral. Input is guaranteed to be within the range from 1 to 3999.
 *
 * Example 1:
 *
 * Input: 3
 * Output: "III"
 * Example 2:
 *
 * Input: 4
 * Output: "IV"
 * Example 3:
 *
 * Input: 9
 * Output: "IX"
 * Example 4:
 *
 * Input: 58
 * Output: "LVIII"
 * Explanation: L = 50, V = 5, III = 3.
 * Example 5:
 *
 * Input: 1994
 * Output: "MCMXCIV"
 * Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
 *
 */
```

```
class Solution {

    public static String intToRoman(int num) {
        String M[] = {"", "M", "MM", "MMM"};
        String C[] = {"", "C", "CC", "CCC", "CD", "D", "DC", "DCC", "DCCC", "CM"};
        String X[] = {"", "X", "XX", "XXX", "XL", "L", "LX", "LXX", "LXXX", "XC"};
        String I[] = {"", "I", "II", "III", "IV", "V", "VI", "VII", "VIII", "IX"};

        return M[num / 1000] + C[(num % 1000) / 100] + X[(num % 100) / 10] + I[num % 10];
    }
}
```

12. `13. Roman to Integer.java`

```
/*
 * Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.
 *
 * Symbol       Value
 * I             1
 * V             5
 * X             10
 * L             50
 * C             100
 * D             500
 * M             1000
 * For example, two is written as II in Roman numeral, just two one's added together. Twelve is written as, XII, which is simply X + II. The number twenty seven is written as XXVII, which is XX + V + II.
 *
 * Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:
 *
 * I can be placed before V (5) and X (10) to make 4 and 9.
 * X can be placed before L (50) and C (100) to make 40 and 90.
 * C can be placed before D (500) and M (1000) to make 400 and 900.
 * Given a roman numeral, convert it to an integer. Input is guaranteed to be within the range from 1 to 3999.
 *
 * Example 1:
 *
 * Input: "III"
 * Output: 3
 * Example 2:
 *
 * Input: "IV"
 * Output: 4
 * Example 3:
 *
 * Input: "IX"
 * Output: 9
 * Example 4:
 *
 * Input: "LVIII"
 * Output: 58
 * Explanation: L = 50, V= 5, III = 3.
 * Example 5:
 *
 * Input: "MCMXCIV"
 * Output: 1994
 * Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
 */

class Solution {
```

```
int value(char r) {
    if (r == 'I') {
        return 1;
    }
    if (r == 'V') {
        return 5;
    }
    if (r == 'X') {
        return 10;
    }
    if (r == 'L') {
        return 50;
    }
    if (r == 'C') {
        return 100;
    }
    if (r == 'D') {
        return 500;
    }
    if (r == 'M') {
        return 1000;
    }
    return -1;
}

// Finds decimal value of a given romal numeral
int romanToInt(String str) {
    // Initialize result
    int res = 0;

    for (int i = 0; i < str.length(); i++) {
        // Getting value of symbol s[i]
        int s1 = value(str.charAt(i));

        // Getting value of symbol s[i+1]
        if (i + 1 < str.length()) {
            int s2 = value(str.charAt(i + 1));

            // Comparing both values
            if (s1 >= s2) {
                // Value of current symbol is greater
                // or equalto the next symbol
                res = res + s1;
            } else {
                res = res + s2 - s1;
                i++; // Value of current symbol is
                // less than the next symbol
            }
        } else {
            res = res + s1;
            i++;
        }
    }
```

```
        return res;
    }
}
```

13. `14. Longest Common Prefix.java`

```
/*
 * Write a function to find the longest common prefix string amongst an array of strings.
 *
 * If there is no common prefix, return an empty string "".
 *
 * Example 1:
 *
 * Input: ["flower","flow","flight"]
 * Output: "fl"
 * Example 2:
 *
 * Input: ["dog","racecar","car"]
 * Output: ""
 * Explanation: There is no common prefix among the input strings.
 * Note:
 *
 * All given inputs are in lowercase letters a-z.
 */

class Solution {

    public String longestCommonPrefix(String[] strs) {
        if (strs.length == 0) {
            return "";
        }
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (strs[i].indexOf(prefix) != 0) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) {
                    return "";
                }
            }
        }
        return prefix;
    }
}
```

14. `15. 3Sum.java`

```
/*
 *
 * Given an array nums of n integers, are there elements a, b, c in nums such that a + b
+ c = 0? Find all unique triplets in the array which gives the sum of zero.
 *
 * Note:
 *
 * The solution set must not contain duplicate triplets.
 *
 * Example:
 *
 * Given array nums = [-1, 0, 1, 2, -1, -4],
 *
 * A solution set is:
 * [
 * [-1, 0, 1],
 * [-1, -1, 2]
 * ]
 *
 */

class Solution {

    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> result = new ArrayList<List<Integer>>();
        if (nums == null || nums.length < 3) {
            return result;
        }
        Arrays.sort(nums);
        for (int i = 0; i < nums.length - 2; i++) {
            if (i == 0 || nums[i] > nums[i - 1]) {
                int j = i + 1;
                int k = nums.length - 1;
                while (j < k) {
                    if (nums[i] + nums[j] + nums[k] == 0) {
                        List<Integer> l = new ArrayList<Integer>();
                        l.add(nums[i]);
                        l.add(nums[j]);
                        l.add(nums[k]);
                        result.add(l);
                        j++;
                        k--;

                        //handle duplicate here
                        while (j < k && nums[j] == nums[j - 1]) {
                            j++;
                        }
                        while (j < k && nums[k] == nums[k + 1]) {
                            k--;
                        }
                    } else if (nums[i] + nums[j] + nums[k] < 0) {
```

```
                    j++;
                } else {
                    k--;
                }
            }
        }
        return result;
    }
}
```

15. `16. 3Sum Closest.java`

```
/*
 *
 * Given an array nums of n integers and an integer target, find three integers in nums such that the sum is closest to target. Return the sum of the three integers. You may assume that each input would have exactly one solution.
 *
 * Example:
 *
 * Given array nums = [-1, 2, 1, -4], and target = 1.
 *
 * The sum that is closest to the target is 2. (-1 + 2 + 1 = 2)
 *
 */

class Solution {

    public int threeSumClosest(int[] nums, int target) {
        int min = Integer.MAX_VALUE;
        int result = 0;

        Arrays.sort(nums);

        for (int i = 0; i < nums.length; i++) {
            int j = i + 1;
            int k = nums.length - 1;
            while (j < k) {
                int sum = nums[i] + nums[j] + nums[k];
                int diff = Math.abs(sum - target);

                if (diff == 0) {
                    return sum;
                }

                if (diff < min) {
                    min = diff;
                    result = sum;
                }
                if (sum <= target) {
                    j++;
                } else {
                    k--;
                }
            }
        }

        return result;
    }
}
```

16. `17. Letter Combinations of a Phone Number.java`

```
/*
 *
 * Given a string containing digits from 2-9 inclusive, return all possible letter
combinations that the number could represent.
 *
 * A mapping of digit to letters (just like on the telephone buttons) is given below.
Note that 1 does not map to any letters.
 *
 * Example:
 *
 * Input: "23"
 * Output: ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"].
 * Note:
 *
 * Although the above answer is in lexicographical order, your answer could be in any
order you want.
 *
 */

class Solution {

    public List<String> letterCombinations(String digits) {
        HashMap<Integer, String> map = new HashMap<Integer, String>();
        map.put(2, "abc");
        map.put(3, "def");
        map.put(4, "ghi");
        map.put(5, "jkl");
        map.put(6, "mno");
        map.put(7, "pqrs");
        map.put(8, "tuv");
        map.put(9, "wxyz");
        map.put(0, "");

        ArrayList<String> result = new ArrayList<String>();

        if (digits == null || digits.length() == 0) {
            return result;
        }

        ArrayList<Character> temp = new ArrayList<Character>();
        getString(digits, temp, result, map);

        return result;
    }

    public void getString(String digits, ArrayList<Character> temp, ArrayList<String>
result, HashMap<Integer, String> map) {
        if (digits.length() == 0) {
            char[] arr = new char[temp.size()];
            for (int i = 0; i < temp.size(); i++) {
                arr[i] = temp.get(i);
```

```
            }
            result.add(String.valueOf(arr));
            return;
        }

        Integer curr = Integer.valueOf(digits.substring(0, 1));
        String letters = map.get(curr);
        for (int i = 0; i < letters.length(); i++) {

            temp.add(letters.charAt(i));
            getString(digits.substring(1), temp, result, map);
            temp.remove(temp.size() - 1);
        }
    }
}
```

17. `18. 4Sum.java`

```
/*
 *
 * Given an array nums of n integers and an integer target, are there elements a, b, c,
and d in nums such that a + b + c + d = target? Find all unique quadruplets in the array
which gives the sum of target.
 *
 * Note:
 *
 * The solution set must not contain duplicate quadruplets.
 *
 * Example:
 *
 * Given array nums = [1, 0, -1, 0, -2, 2], and target = 0.
 *
 * A solution set is:
 * [
 * [-1,  0, 0, 1],
 * [-2, -1, 1, 2],
 * [-2,  0, 0, 2]
 * ]
 *
 */

class Solution {

    public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> result = new ArrayList<List<Integer>>();

        if (nums == null || nums.length < 4) {
            return result;
        }

        Arrays.sort(nums);

        for (int i = 0; i < nums.length - 3; i++) {
            if (i != 0 && nums[i] == nums[i - 1]) {
                continue;
            }
            for (int j = i + 1; j < nums.length - 2; j++) {
                if (j != i + 1 && nums[j] == nums[j - 1]) {
                    continue;
                }
                int k = j + 1;
                int l = nums.length - 1;
                while (k < l) {
                    if (nums[i] + nums[j] + nums[k] + nums[l] < target) {
                        k++;
                    } else if (nums[i] + nums[j] + nums[k] + nums[l] > target) {
                        l--;
                    } else {
                        List<Integer> t = new ArrayList<Integer>();
```

```
                        t.add(nums[i]);
                        t.add(nums[j]);
                        t.add(nums[k]);
                        t.add(nums[l]);
                        result.add(t);

                        k++;
                        l--;

                        while (k < l && nums[l] == nums[l + 1]) {
                            l--;
                        }

                        while (k < l && nums[k] == nums[k - 1]) {
                            k++;
                        }
                    }

                }
            }
        }

        return result;
    }
}
```

18. `19. Remove Nth Node From End of List.java`

```
/*
 * Given a linked list, remove the n-th node from the end of list and return its head.
 *
 * Example:
 *
 * Given linked list: 1->2->3->4->5, and n = 2.
 *
 * After removing the second node from the end, the linked list becomes 1->2->3->5.
 * Note:
 *
 * Given n will always be valid.
 *
 * Follow up:
 *
 * Could you do this in one pass?
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode removeNthFromEnd(ListNode head, int n) {
        if (head == null) {
            return null;
        }

        //get length of list
        ListNode p = head;
        int len = 0;
        while (p != null) {
            len++;
            p = p.next;
        }

        //if remove first node
        int fromStart = len - n + 1;
        if (fromStart == 1) {
            return head.next;
        }

        //remove non-first node
        p = head;
        int i = 0;
        while (p != null) {
```

```
            i++;
            if (i == fromStart - 1) {
                p.next = p.next.next;
            }
            p = p.next;
        }

        return head;
    }
}
```

19. `20. Valid Parentheses.java`

```
/*
 * Given a string containing just the characters '(', ')', '{', '}', '[' and ']',
determine if the input string is valid.
 *
 * An input string is valid if:
 *
 * Open brackets must be closed by the same type of brackets.
 * Open brackets must be closed in the correct order.
 * Note that an empty string is also considered valid.
 *
 * Example 1:
 *
 * Input: "()"
 * Output: true
 * Example 2:
 *
 * Input: "()[]{}"
 * Output: true
 * Example 3:
 *
 * Input: "(]"
 * Output: false
 * Example 4:
 *
 * Input: "([)]"
 * Output: false
 * Example 5:
 *
 * Input: "{[]}"
 * Output: true
 *
 */

class Solution {

    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<Character>();
        // Iterate through string until empty
        for (int i = 0; i < s.length(); i++) {
            // Push any open parentheses onto stack
            if (s.charAt(i) == '(' || s.charAt(i) == '[' || s.charAt(i) == '{') {
                stack.push(s.charAt(i));
            }

            //Check stack for corresponding closing parentheses, false if not valid
            //The case for empty stack has to be checked since the input could be just be
), where there will be nothing pushed on the //stack, so we need to tackle that.
            //Also in order to confirm that we are taking the right combination of () or
any other, we need to check for the peek of ( and // the charAt i = )
            else if (s.charAt(i) == ')' && !stack.empty() && stack.peek() == '(') {
                stack.pop();
```

```
            } else if (s.charAt(i) == ']' && !stack.empty() && stack.peek() == '[') {
                stack.pop();
            } else if (s.charAt(i) == '}' && !stack.empty() && stack.peek() == '{') {
                stack.pop();
            } else {
                return false;
            }
        }
        // return true if no open parentheses left in stack
        return stack.empty();
    }
}
```

20. `21. Merge Two Sorted Lists.java`

```
/*
 * Merge two sorted linked lists and return it as a new list. The new list should be made
by splicing together the nodes of the first two lists.
 *
 * Example:
 *
 * Input: 1->2->4, 1->3->4
 * Output: 1->1->2->3->4->4
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode head = new ListNode(0);
        ListNode p = head;

        ListNode p1 = l1;
        ListNode p2 = l2;
        while (p1 != null && p2 != null) {
            if (p1.val < p2.val) {
                p.next = p1;
                p1 = p1.next;
            } else {
                p.next = p2;
                p2 = p2.next;
            }
            p = p.next;
        }

        if (p1 != null) {
            p.next = p1;
        }

        if (p2 != null) {
            p.next = p2;
        }

        return head.next;
    }
}
```

21. `22. Generate Parentheses.java`

```
/*
 * Given n pairs of parentheses, write a function to generate all combinations of well-
formed parentheses.
 *
 * For example, given n = 3, a solution set is:
 *
 * [
 * "((()))",
 * "(()())",
 * "(())()",
 * "()(())",
 * "()()()"
 * ]
 *
 */

 /*
 * left and right represents the remaining number of ( and ) that need to be added.
 * When left > right, there are more ")" placed than "(". Such cases are wrong and the
method stops.
 */
class Solution {

    public List<String> generateParenthesis(int n) {
        ArrayList<String> result = new ArrayList<String>();
        dfs(result, "", n, n);
        return result;
    }

    public void dfs(ArrayList<String> result, String s, int left, int right) {

        if (left > right) {
            return;
        }

        if (left == 0 && right == 0) {
            result.add(s);
            return;
        }

        if (left > 0) {
            dfs(result, s + "(", left - 1, right);
        }

        if (right > 0) {
            dfs(result, s + ")", left, right - 1);
        }
    }
}
```

22. `23. Merge k Sorted Lists.java`

```
/*
 * Merge k sorted linked lists and return it as one sorted list. Analyze and describe its
complexity.
 *
 * Example:
 *
 * Input:
 * [
 * 1->4->5,
 * 1->3->4,
 * 2->6
 * ]
 * Output: 1->1->2->3->4->4->5->6
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode mergeKLists(ListNode[] lists) {
        if (lists == null || lists.length == 0) {
            return null;
        }

        PriorityQueue<ListNode> queue = new PriorityQueue<ListNode>((a, b) -> (a.val -
b.val));

        ListNode head = new ListNode(0);
        ListNode p = head;

        for (ListNode list : lists) {
            if (list != null) {
                queue.offer(list);
            }
        }

        while (!queue.isEmpty()) {
            ListNode n = queue.poll();
            p.next = n;
            p = p.next;

            if (n.next != null) {
                queue.offer(n.next);
            }
```

```
        }

        return head.next;

    }
}
```

23. `24. Swap Nodes in Pairs.java`

```
/*
 * Given a linked list, swap every two adjacent nodes and return its head.
 *
 * You may not modify the values in the list's nodes, only nodes itself may be changed.
 *
 * Example:
 *
 * Given 1->2->3->4, you should return the list as 2->1->4->3.
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode swapPairs(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode h = new ListNode(0);
        h.next = head;
        ListNode p = h;

        while (p.next != null && p.next.next != null) {
            //use t1 to track first node
            ListNode t1 = p;
            p = p.next;
            t1.next = p.next;

            //use t2 to track next node of the pair
            ListNode t2 = p.next.next;
            p.next.next = p;
            p.next = t2;
        }

        return h.next;
    }
}
```

24. `25. Reverse Nodes in k-Group.java`

```
/*
Given a linked list, reverse the nodes of a linked list k at a time and return its
modified list.

k is a positive integer and is less than or equal to the length of the linked list. If
the number of nodes is not a multiple of k then left-out nodes in the end should remain
as it is.

Example:

Given this linked list: 1->2->3->4->5

For k = 2, you should return: 2->1->4->3->5

For k = 3, you should return: 3->2->1->4->5

Note:

Only constant extra memory is allowed.
You may not alter the values in the list's nodes, only nodes itself may be changed.

*/

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode reverseKGroup(ListNode head, int k) {
        if (head == null || k == 1) {
            return head;
        }

        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode pre = dummy;
        int i = 0;

        ListNode p = head;
        while (p != null) {
            i++;
            if (i % k == 0) {
                pre = reverse(pre, p.next);
                p = pre.next;
            } else {
                p = p.next;
```

```
            }
        }

        return dummy.next;
    }

/*
 * 0->1->2->3->4->5->6
 * |           |
 * pre        next
 *
 * after calling pre = reverse(pre, next)
 *
 * 0->3->2->1->4->5->6
 *          |  |
 *          pre next
 */
    public ListNode reverse(ListNode start, ListNode end) {
        ListNode prev = start.next;
        ListNode curr = prev.next;

        while (curr != end) {
            prev.next = curr.next;
            curr.next = start.next;
            start.next = curr;
            curr = prev.next;
        }

        return prev;
    }

    /*
    In the reverse function we are going to have the following iterations.
    First we get this as input:
          0 -> 1-> 2-> 3-> 4-> 5-> 6
          |    |   |       |
        pre  last curr    next
    After Iteration 1:
          0 -> 2-> 1-> 3->  4-> 5-> 6
          |        |   |    |
         pre     last curr next

    After Iteration 2:
          0 -> 3-> 2-> 1-> 4-> 5-> 6
          |            |   |
         pre         last next


    */
}
```

25. `26. Remove Duplicates from Sorted Array.java`

```
/*
 * Given a sorted array nums, remove the duplicates in-place such that each element
appear only once and return the new length.
 *
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * Example 1:
 *
 * Given nums = [1,1,2],
 *
 * Your function should return length = 2, with the first two elements of nums being 1
and 2 respectively.
 *
 * It doesn't matter what you leave beyond the returned length.
 * Example 2:
 *
 * Given nums = [0,0,1,1,1,2,2,3,3,4],
 *
 * Your function should return length = 5, with the first five elements of nums being
modified to 0, 1, 2, 3, and 4 respectively.
 *
 * It doesn't matter what values are set beyond the returned length.
 * Clarification:
 *
 * Confused why the returned value is an integer but your answer is an array?
 *
 * Note that the input array is passed in by reference, which means modification to the
input array will be known to the caller as well.
 *
 * Internally you can think of this:
 *
 * // nums is passed in by reference. (i.e., without making a copy)
 * int len = removeDuplicates(nums);
 *
 * // any modification to nums in your function would be known by the caller.
 * // using the length returned by your function, it prints the first len elements.
 * for (int i = 0; i < len; i++) {
 * print(nums[i]);
 * }
 *
 *
 */

class Solution {

    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) {
            return 0;
        }
        int i = 0;
```

```
        for (int j = 1; j < nums.length; j++) {
            if (nums[j] != nums[i]) {
                i++;
                nums[i] = nums[j];
            }
        }
        return i + 1;
    }
}
```

26. `27. Remove Element.java`

```
/*
 * Given an array nums and a value val, remove all instances of that value in-place and
return the new length.
 *
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * The order of elements can be changed. It doesn't matter what you leave beyond the new
length.
 *
 * Example 1:
 *
 * Given nums = [3,2,2,3], val = 3,
 *
 * Your function should return length = 2, with the first two elements of nums being 2.
 *
 * It doesn't matter what you leave beyond the returned length.
 * Example 2:
 *
 * Given nums = [0,1,2,2,3,0,4,2], val = 2,
 *
 * Your function should return length = 5, with the first five elements of nums
containing 0, 1, 3, 0, and 4.
 *
 * Note that the order of those five elements can be arbitrary.
 *
 * It doesn't matter what values are set beyond the returned length.
 * Clarification:
 *
 * Confused why the returned value is an integer but your answer is an array?
 *
 * Note that the input array is passed in by reference, which means modification to the
input array will be known to the caller as well.
 *
 * Internally you can think of this:
 *
 * // nums is passed in by reference. (i.e., without making a copy)
 * int len = removeElement(nums, val);
 *
 * // any modification to nums in your function would be known by the caller.
 * // using the length returned by your function, it prints the first len elements.
 * for (int i = 0; i < len; i++) {
 * print(nums[i]);
 * }
 *
 */

class Solution {

    public int removeElement(int[] nums, int val) {
        int i = 0;
```

```
        for (int j = 0; j < nums.length; j++) {
            if (nums[j] != val) {
                nums[i] = nums[j];
                i++;
            }
        }
        return i;
    }
}
```

27. `28. Implement strStr().java`

```
/*
 * Implement strStr().
 *
 * Return the index of the first occurrence of needle in haystack, or -1 if needle is not
part of haystack.
 *
 * Example 1:
 *
 * Input: haystack = "hello", needle = "ll"
 * Output: 2
 * Example 2:
 *
 * Input: haystack = "aaaaa", needle = "bba"
 * Output: -1
 * Clarification:
 *
 * What should we return when needle is an empty string? This is a great question to ask
during an interview.
 *
 * For the purpose of this problem, we will return 0 when needle is an empty string. This
is consistent to C's strstr() and Java's indexOf().
 *
 */

class Solution {

    public int strStr(String haystack, String needle) {
        return haystack.indexOf(needle);
    }
}
```

28. `29. Divide Two Integers.java`

```
/*
 * Given two integers dividend and divisor, divide two integers without using
multiplication, division and mod operator.
 *
 * Return the quotient after dividing dividend by divisor.
 *
 * The integer division should truncate toward zero.
 *
 * Example 1:
 *
 * Input: dividend = 10, divisor = 3
 * Output: 3
 * Example 2:
 *
 * Input: dividend = 7, divisor = -3
 * Output: -2
 * Note:
 *
 * Both dividend and divisor will be 32-bit signed integers.
 * The divisor will never be 0.
 * Assume we are dealing with an environment which could only store integers within the
32-bit signed integer range: [−231,  231 − 1]. For the purpose of this problem, assume
that your function returns 231 − 1 when the division result overflows.
 *
 */

class Solution {

    public int divide(int dividend, int divisor) {
        long divid = 0, divis = 0;
        long count = 0;

        int sign = ((dividend < 0) ^ (divisor < 0)) ? -1 : 1;
        divid = Math.abs((long) dividend);
        divis = Math.abs((long) divisor);

        while (divid >= divis) {
            divid -= divis;
            count++;
        }

        int ans;
        if (count > Integer.MAX_VALUE) { //Handle overflow.
            ans = (sign == 1) ? Integer.MAX_VALUE : Integer.MIN_VALUE;
        } else {
            ans = (int) (sign * count);
        }
        return ans;
    }
}
```

29. `31. Next Permutation.java`

```
/*
 * Implement next permutation, which rearranges numbers into the lexicographically next
greater permutation of numbers.
 *
 * If such arrangement is not possible, it must rearrange it as the lowest possible order
(ie, sorted in ascending order).
 *
 * The replacement must be in-place and use only constant extra memory.
 *
 * Here are some examples. Inputs are in the left-hand column and its corresponding
outputs are in the right-hand column.
 *
 * 1,2,3 → 1,3,2
 * 3,2,1 → 1,2,3
 * 1,1,5 → 1,5,1
 *
 */

class Solution {

    public void nextPermutation(int[] nums) {
        if (nums == null || nums.length < 2) {
            return;
        }

        int p = 0;
        for (int i = nums.length - 2; i >= 0; i--) {
            if (nums[i] < nums[i + 1]) {
                p = i;
                break;
            }
        }

        int q = 0;
        for (int i = nums.length - 1; i > p; i--) {
            if (nums[i] > nums[p]) {
                q = i;
                break;
            }
        }

        if (p == 0 && q == 0) {
            reverse(nums, 0, nums.length - 1);
            return;
        }

        int temp = nums[p];
        nums[p] = nums[q];
        nums[q] = temp;

        if (p < nums.length - 1) {
```

```
            reverse(nums, p + 1, nums.length - 1);
        }
    }

    public void reverse(int[] nums, int left, int right) {
        while (left < right) {
            int temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;
            left++;
            right--;
        }
    }
}
```

30. `33. Search in Rotated Sorted Array.java`

```
/*
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 *
 * (i.e., [0,1,2,4,5,6,7] might become [4,5,6,7,0,1,2]).
 *
 * You are given a target value to search. If found in the array return its index,
otherwise return -1.
 *
 * You may assume no duplicate exists in the array.
 *
 * Your algorithm's runtime complexity must be in the order of O(log n).
 *
 * Example 1:
 *
 * Input: nums = [4,5,6,7,0,1,2], target = 0
 * Output: 4
 * Example 2:
 *
 * Input: nums = [4,5,6,7,0,1,2], target = 3
 * Output: -1
 *
 */

class Solution {

    public int search(int[] nums, int target) {
        return binarySearch(nums, 0, nums.length - 1, target);
    }

    public int binarySearch(int[] nums, int left, int right, int target) {
        if (left > right) {
            return -1;
        }

        int mid = left + (right - left) / 2;

        if (target == nums[mid]) {
            return mid;
        }

        if (nums[left] <= nums[mid]) {
            if (nums[left] <= target && target < nums[mid]) {
                return binarySearch(nums, left, mid - 1, target);
            } else {
                return binarySearch(nums, mid + 1, right, target);
            }
        } else if (nums[mid] < target && target <= nums[right]) {
            return binarySearch(nums, mid + 1, right, target);
        } else {
            return binarySearch(nums, left, mid - 1, target);
```

```
        }
    }
}
```

31. `34. Find First and Last Position of Element in Sorted Array.java`

```
/*
Given an array of integers nums sorted in ascending order, find the starting and ending
position of a given target value.

Your algorithm's runtime complexity must be in the order of O(log n).

If the target is not found in the array, return [-1, -1].

Example 1:

Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
Example 2:

Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]

*/

public class Solution {
    public int[] searchRange(int[] A, int target) {
        int[] range = {A.length, -1};
        searchRange(A, target, 0, A.length - 1, range);
        if (range[0] > range[1]) range[0] = -1;
        return range;
    }

    public void searchRange(int[] A, int target, int left, int right, int[] range) {
        if (left > right) return;
        int mid = left + (right - left) / 2;
        if (A[mid] == target) {
            if (mid < range[0]) {
                range[0] = mid;
                searchRange(A, target, left, mid - 1, range);
            }
            if (mid > range[1]) {
                range[1] = mid;
                searchRange(A, target, mid + 1, right, range);
            }
        } else if (A[mid] < target) {
            searchRange(A, target, mid + 1, right, range);
        } else {
            searchRange(A, target, left, mid - 1, range);
        }
    }
}
```

32. `35. Search Insert Position.java`

```
/*
 * Given a sorted array and a target value, return the index if the target is found. If
not, return the index where it would be if it were inserted in order.
 *
 * You may assume no duplicates in the array.
 *
 * Example 1:
 *
 * Input: [1,3,5,6], 5
 * Output: 2
 * Example 2:
 *
 * Input: [1,3,5,6], 2
 * Output: 1
 * Example 3:
 *
 * Input: [1,3,5,6], 7
 * Output: 4
 * Example 4:
 *
 * Input: [1,3,5,6], 0
 * Output: 0
 *
 */

class Solution {

    public static int searchInsert(int[] nums, int target) {

        int position = -1;

        for (int i = 0; i < nums.length; i++) {
            if (target == nums[i]) {
                position = i;
                break;
            } else if (target > nums[i]) {
                continue;
            } else {
                position = i;
                break;
            }
        }

        if (position == -1) {
            position = nums.length;
        }

        return position;
    }
}
```

33. `36. Valid Sudoku.java`

```
/*
 * Determine if a 9x9 Sudoku board is valid. Only the filled cells need to be validated
according to the following rules:
 *
 * Each row must contain the digits 1-9 without repetition.
 * Each column must contain the digits 1-9 without repetition.
 * Each of the 9 3x3 sub-boxes of the grid must contain the digits 1-9 without
repetition.
 *
 * A partially filled sudoku which is valid.
 *
 * The Sudoku board could be partially filled, where empty cells are filled with the
character '.'
 *
 * Example 1:
 *
 * Input:
 * [
 * ["5","3",".",".","7",".",".",".","."],
 * ["6",".",".","1","9","5",".",".","."],
 * [".","9","8",".",".",".",".","6","."],
 * ["8",".",".",".","6",".",".",".","3"],
 * ["4",".",".","8",".","3",".",".","1"],
 * ["7",".",".",".","2",".",".",".","6"],
 * [".","6",".",".",".",".","2","8","."],
 * [".",".",".","4","1","9",".",".","5"],
 * [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: true
 * Example 2:
 *
 * Input:
 * [
 * ["8","3",".",".","7",".",".",".","."],
 * ["6",".",".","1","9","5",".",".","."],
 * [".","9","8",".",".",".",".","6","."],
 * ["8",".",".",".","6",".",".",".","3"],
 * ["4",".",".","8",".","3",".",".","1"],
 * ["7",".",".",".","2",".",".",".","6"],
 * [".","6",".",".",".",".","2","8","."],
 * [".",".",".","4","1","9",".",".","5"],
 * [".",".",".",".","8",".",".","7","9"]
 * ]
 * Output: false
 * Explanation: Same as Example 1, except with the 5 in the top left corner being
 * modified to 8. Since there are two 8's in the top left 3x3 sub-box, it is invalid.
 * Note:
 *
 * A Sudoku board (partially filled) could be valid but is not necessarily solvable.
 * Only the filled cells need to be validated according to the mentioned rules.
 * The given board contain only digits 1-9 and the character '.'.
```

```
 * The given board size is always 9x9.
 *
 */

class Solution {

    public boolean isValidSudoku(char[][] board) {
        if (board == null || board.length != 9 || board[0].length != 9) {
            return false;
        }
// check each column
        for (int i = 0; i < 9; i++) {
            boolean[] m = new boolean[9];
            for (int j = 0; j < 9; j++) {
                if (board[i][j] != '.') {
                    if (m[(int) (board[i][j] - '1')]) {
                        return false;
                    }
                    m[(int) (board[i][j] - '1')] = true;
                }
            }
        }
//check each row
        for (int j = 0; j < 9; j++) {
            boolean[] m = new boolean[9];
            for (int i = 0; i < 9; i++) {
                if (board[i][j] != '.') {
                    if (m[(int) (board[i][j] - '1')]) {
                        return false;
                    }
                    m[(int) (board[i][j] - '1')] = true;
                }
            }
        }
//check each 3*3 matrix
        for (int block = 0; block < 9; block++) {
            boolean[] m = new boolean[9];
            for (int i = block / 3 * 3; i < block / 3 * 3 + 3; i++) {
                for (int j = block % 3 * 3; j < block % 3 * 3 + 3; j++) {
                    if (board[i][j] != '.') {
                        if (m[(int) (board[i][j] - '1')]) {
                            return false;
                        }
                        m[(int) (board[i][j] - '1')] = true;
                    }
                }
            }
        }
        return true;
    }
}
```

34. `37. Sudoku Solver.java`

```
/*
 * Write a program to solve a Sudoku puzzle by filling the empty cells.
 *
 * A sudoku solution must satisfy all of the following rules:
 *
 * Each of the digits 1-9 must occur exactly once in each row.
 * Each of the digits 1-9 must occur exactly once in each column.
 * Each of the the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of
the grid.
 * Empty cells are indicated by the character '.'.
 *
 *
 * A sudoku puzzle...
 *
 *
 * ...and its solution numbers marked in red.
 *
 * Note:
 *
 * The given board contain only digits 1-9 and the character '.'.
 * You may assume that the given Sudoku puzzle will have a single unique solution.
 * The given board size is always 9x9.
 *
 */

class Solution {

    public void solveSudoku(char[][] board) {
        solve(board);
    }

    public boolean solve(char[][] board) {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                if (board[i][j] != '.') {
                    continue;
                }
                for (int k = 1; k <= 9; k++) {
                    board[i][j] = (char) (k + '0');
                    if (isValid(board, i, j) && solve(board)) {
                        return true;
                    }
                    board[i][j] = '.';
                }
                return false;
            }
        }
        return true;
    }

    public boolean isValid(char[][] board, int i, int j) {
```

```
        HashSet<Character> set = new HashSet<Character>();
        for (int k = 0; k < 9; k++) {
            if (set.contains(board[i][k])) {
                return false;
            }
            if (board[i][k] != '.') {
                set.add(board[i][k]);
            }
        }
        set.clear();
        for (int k = 0; k < 9; k++) {
            if (set.contains(board[k][j])) {
                return false;
            }
            if (board[k][j] != '.') {
                set.add(board[k][j]);
            }
        }
        set.clear();
        for (int m = 0; m < 3; m++) {
            for (int n = 0; n < 3; n++) {
                int x = i / 3 * 3 + m;
                int y = j / 3 * 3 + n;
                if (set.contains(board[x][y])) {
                    return false;
                }
                if (board[x][y] != '.') {
                    set.add(board[x][y]);
                }
            }
        }
        return true;
    }
}
```

35. `38. Count and Say.java`

```
/*
 * The count-and-say sequence is the sequence of integers with the first five terms as
following:
 *
 * 1.     1
 * 2.     11
 * 3.     21
 * 4.     1211
 * 5.     111221
 * 1 is read off as "one 1" or 11.
 * 11 is read off as "two 1s" or 21.
 * 21 is read off as "one 2, then one 1" or 1211.
 *
 * Given an integer n where 1 ≤ n ≤ 30, generate the nth term of the count-and-say
sequence.
 *
 * Note: Each term of the sequence of integers will be represented as a string.
 *
 *
 *
 * Example 1:
 *
 * Input: 1
 * Output: "1"
 * Example 2:
 *
 * Input: 4
 * Output: "1211"
 *
 */

class Solution {

    public String countAndSay(int n) {
        if (n <= 0) {
            return null;
        }

        String result = "1";
        int i = 1;

        while (i < n) {
            StringBuilder sb = new StringBuilder();
            int count = 1;
            for (int j = 1; j < result.length(); j++) {
                if (result.charAt(j) == result.charAt(j - 1)) {
                    count++;
                } else {
                    sb.append(count);
                    sb.append(result.charAt(j - 1));
                    count = 1;
```

```
                }
            }

            sb.append(count);
            sb.append(result.charAt(result.length() - 1));
            result = sb.toString();
            i++;
        }

        return result;
    }
}
```

36. `39. Combination Sum.java`

```
/*
 * Given a set of candidate numbers (candidates) (without duplicates) and a target number
(target), find all unique combinations in candidates where the candidate numbers sums to
target.
 *
 * The same repeated number may be chosen from candidates unlimited number of times.
 *
 * Note:
 *
 * All numbers (including target) will be positive integers.
 * The solution set must not contain duplicate combinations.
 * Example 1:
 *
 * Input: candidates = [2,3,6,7], target = 7,
 * A solution set is:
 * [
 * [7],
 * [2,2,3]
 * ]
 * Example 2:
 *
 * Input: candidates = [2,3,5], target = 8,
 * A solution set is:
 * [
 * [2,2,2,2],
 * [2,3,3],
 * [3,5]
 * ]
 *
 */

class Solution {

    public List<List<Integer>> combinationSum(int[] nums, int target) {
        List<List<Integer>> list = new ArrayList<>();
        Arrays.sort(nums);
        backtrack(list, new ArrayList<>(), nums, target, 0);
        return list;
    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums,
int remain, int start) {
        if (remain < 0) {
            return;
        } else {
            if (remain == 0) {
                list.add(new ArrayList<>(tempList));
            } else {
                for (int i = start; i < nums.length; i++) {
                    tempList.add(nums[i]);
                    backtrack(list, tempList, nums, remain - nums[i], i); // not i + 1
```

```
because we can reuse same elements
                    tempList.remove(tempList.size() - 1);
                }
            }
        }
    }
}
```

37. `40. Combination Sum II.java`

```
/*
 * Given a collection of candidate numbers (candidates) and a target number (target),
find all unique combinations in candidates where the candidate numbers sums to target.
 *
 * Each number in candidates may only be used once in the combination.
 *
 * Note:
 *
 * All numbers (including target) will be positive integers.
 * The solution set must not contain duplicate combinations.
 * Example 1:
 *
 * Input: candidates = [10,1,2,7,6,1,5], target = 8,
 * A solution set is:
 * [
 * [1, 7],
 * [1, 2, 5],
 * [2, 6],
 * [1, 1, 6]
 * ]
 * Example 2:
 *
 * Input: candidates = [2,5,2,1,2], target = 5,
 * A solution set is:
 * [
 * [1,2,2],
 * [5]
 * ]
 *
 */

class Solution {

    public List<List<Integer>> combinationSum2(int[] nums, int target) {
        List<List<Integer>> list = new ArrayList<>();
        Arrays.sort(nums);
        backtrack(list, new ArrayList<>(), nums, target, 0);
        return list;

    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums,
int remain, int start) {
        if (remain < 0) {
            return;
        } else {
            if (remain == 0) {
                list.add(new ArrayList<>(tempList));
            } else {
                for (int i = start; i < nums.length; i++) {
                    if (i > start && nums[i] == nums[i - 1]) {
```

```
                    continue; // skip duplicates
                }
                tempList.add(nums[i]);
                backtrack(list, tempList, nums, remain - nums[i], i + 1);
                tempList.remove(tempList.size() - 1);
            }
        }
    }
}
```

38. `42. Trapping Rain Water.java`

```
/*
 * Given n non-negative integers representing an elevation map where the width of each
bar is 1, compute how much water it is able to trap after raining.
 *
 * The above elevation map is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this
case, 6 units of rain water (blue section) are being trapped. Thanks Marcos for
contributing this image!
 *
 * Example:
 *
 * Input: [0,1,0,2,1,0,1,3,2,1,2,1]
 * Output: 6
 *
 */

class Solution {

    public int trap(int[] height) {
        if (height.length <= 2) {
            return 0;
        }
        int[] left = new int[height.length];
        int[] right = new int[height.length];
        left[0] = height[0];
        right[height.length - 1] = height[height.length - 1];
        int result = 0;

        for (int i = 1; i < height.length; i++) {
            left[i] = Math.max(height[i], left[i - 1]);
        }
        for (int i = height.length - 2; i >= 0; i--) {
            right[i] = Math.max(height[i], right[i + 1]);
        }

        for (int i = 0; i < height.length; i++) {
            result += Math.min(left[i], right[i]) - height[i];
            // int min = Math.min(left[i], right[i]);
            // if(height[i] < Math.min(left[i], right[i])) {
            //  result += min - height[i];
            // }
        }
        return result;
    }
}
```

39. `43. Multiply Strings.java`

```
/*
 *
 * Given two non-negative integers num1 and num2 represented as strings, return the
product of num1 and num2, also represented as a string.
 *
 * Example 1:
 *
 * Input: num1 = "2", num2 = "3"
 * Output: "6"
 * Example 2:
 *
 * Input: num1 = "123", num2 = "456"
 * Output: "56088"
 * Note:
 *
 * The length of both num1 and num2 is < 110.
 * Both num1 and num2 contain only digits 0-9.
 * Both num1 and num2 do not contain any leading zero, except the number 0 itself.
 * You must not use any built-in BigInteger library or convert the inputs to integer
directly
 *
 */

class Solution {

    public String multiply(String num1, String num2) {
        String n1 = new StringBuilder(num1).reverse().toString();
        String n2 = new StringBuilder(num2).reverse().toString();

        int[] d = new int[num1.length() + num2.length()];

        //multiply each digit and sum at the corresponding positions
        for (int i = 0; i < n1.length(); i++) {
            for (int j = 0; j < n2.length(); j++) {
                d[i + j] += (n1.charAt(i) - '0') * (n2.charAt(j) - '0');
            }
        }

        StringBuilder sb = new StringBuilder();

        //calculate each digit
        for (int i = 0; i < d.length; i++) {
            int mod = d[i] % 10;
            int carry = d[i] / 10;
            if (i + 1 < d.length) {
                d[i + 1] += carry;
            }
            sb.insert(0, mod);
        }

        //remove front 0's
```

```
        while (sb.charAt(0) == '0' && sb.length() > 1) {
            sb.deleteCharAt(0);
        }

        return sb.toString();
    }
}
```

40. `45. Jump Game II.java`

```
/*
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 *
 * Each element in the array represents your maximum jump length at that position.
 *
 * Your goal is to reach the last index in the minimum number of jumps.
 *
 * Example:
 *
 * Input: [2,3,1,1,4]
 * Output: 2
 * Explanation: The minimum number of jumps to reach the last index is 2.
 * Jump 1 step from index 0 to 1, then 3 steps to the last index.
 * Note:
 *
 * You can assume that you can always reach the last index.
 *
 */

class Solution {

    public int jump(int[] A) {
        int step_count = 0;
        int last_jump_max = 0;
        int current_jump_max = 0;
        for (int i = 0; i < A.length - 1; i++) {
            current_jump_max = Math.max(current_jump_max, i + A[i]);
            if (i == last_jump_max) {
                step_count++;
                last_jump_max = current_jump_max;
            }
        }
        return step_count;
    }
}
```

41. `46. Permutations.java`

```
/*
 * Given a collection of distinct integers, return all possible permutations.
 *
 * Example:
 *
 * Input: [1,2,3]
 * Output:
 * [
 * [1,2,3],
 * [1,3,2],
 * [2,1,3],
 * [2,3,1],
 * [3,1,2],
 * [3,2,1]
 * ]
 *
 */

class Solution {

    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> list = new ArrayList<>();
        // Arrays.sort(nums); // not necessary
        backtrack(list, new ArrayList<>(), nums);
        return list;
    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums)
{
        if (tempList.size() == nums.length) {
            list.add(new ArrayList<>(tempList));
        } else {
            for (int i = 0; i < nums.length; i++) {
                if (tempList.contains(nums[i])) {
                    continue; // element already exists, skip
                }
                tempList.add(nums[i]);
                backtrack(list, tempList, nums);
                tempList.remove(tempList.size() - 1);
            }
        }
    }
}
```

42. `47. Permutations II.java`

```
/*
 * Given a collection of numbers that might contain duplicates, return all possible
unique permutations.
 *
 * Example:
 *
 * Input: [1,1,2]
 * Output:
 * [
 * [1,1,2],
 * [1,2,1],
 * [2,1,1]
 * ]
 *
 */

public class Solution {

    public List<List<Integer>> permuteUnique(int[] nums) {
        List<List<Integer>> list = new ArrayList<>();
        Arrays.sort(nums);
        backtrack(list, new ArrayList<>(), nums, new boolean[nums.length]);
        return list;
    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums,
boolean[] used) {
        if (tempList.size() == nums.length) {
            list.add(new ArrayList<>(tempList));
        } else {
            for (int i = 0; i < nums.length; i++) {
                if (used[i] || i > 0 && nums[i] == nums[i - 1] && !used[i - 1]) {
                    continue;
                }
                used[i] = true;
                tempList.add(nums[i]);
                backtrack(list, tempList, nums, used);
                used[i] = false;
                tempList.remove(tempList.size() - 1);
            }
        }
    }
}
```

43. `48. Rotate Image.java`

```
/*
 * You are given an n x n 2D matrix representing an image.
 *
 * Rotate the image by 90 degrees (clockwise).
 *
 * Note:
 *
 * You have to rotate the image in-place, which means you have to modify the input 2D
matrix directly. DO NOT allocate another 2D matrix and do the rotation.
 *
 * Example 1:
 *
 * Given input matrix =
 * [
 * [1,2,3],
 * [4,5,6],
 * [7,8,9]
 * ],
 *
 * rotate the input matrix in-place such that it becomes:
 * [
 * [7,4,1],
 * [8,5,2],
 * [9,6,3]
 * ]
 * Example 2:
 *
 * Given input matrix =
 * [
 * [ 5, 1, 9,11],
 * [ 2, 4, 8,10],
 * [13, 3, 6, 7],
 * [15,14,12,16]
 * ],  *
 * rotate the input matrix in-place such that it becomes:
 * [
 * [15,13, 2, 5],
 * [14, 3, 4, 1],
 * [12, 6, 8, 9],
 * [16, 7,10,11]
 * ]
 *
 */

public class Solution {

    public void rotate(int[][] matrix) {
        for (int i = 0; i < matrix.length; i++) {
            for (int j = i; j < matrix[0].length; j++) {
                int temp = 0;
                temp = matrix[i][j];
```

```
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix.length / 2; j++) {
                int temp = 0;
                temp = matrix[i][j];
                matrix[i][j] = matrix[i][matrix.length - 1 - j];
                matrix[i][matrix.length - 1 - j] = temp;
            }
        }
    }
}
```

44. `49. Group Anagrams.java`

```
/*
 * Given an array of strings, group anagrams together.
 *
 * Example:
 *
 * Input: ["eat", "tea", "tan", "ate", "nat", "bat"],
 * Output:
 * [
 * ["ate","eat","tea"],
 * ["nat","tan"],
 * ["bat"]
 * ]
 * Note:
 *
 * All inputs will be in lowercase.
 * The order of your output does not matter.
 *
 */

class Solution {

    public List<List<String>> groupAnagrams(String[] strs) {
        if (strs.length == 0) {
            return new ArrayList();
        }
        Map<String, List> ans = new HashMap<String, List>();
        for (String s : strs) {
            char[] ca = s.toCharArray();
            Arrays.sort(ca);
            String key = String.valueOf(ca);
            if (!ans.containsKey(key)) {
                ans.put(key, new ArrayList());
            }
            ans.get(key).add(s);
        }
        return new ArrayList(ans.values());
    }
}
```

45. `50. Pow(x, n).java`

```
/*
 * Implement pow(x, n), which calculates x raised to the power n (xn).
 *
 * Example 1:
 *
 * Input: 2.00000, 10
 * Output: 1024.00000
 * Example 2:
 *
 * Input: 2.10000, 3
 * Output: 9.26100
 * Example 3:
 *
 * Input: 2.00000, -2
 * Output: 0.25000
 * Explanation: 2-2 = 1/22 = 1/4 = 0.25
 * Note:
 *
 * -100.0 < x < 100.0
 * n is a 32-bit signed integer, within the range [−231, 231 − 1]
 *
 */

class Solution {

    private double fastPow(double x, long n) {
        if (n == 0) {
            return 1.0;
        }

        double half = fastPow(x, n / 2);

        if (n % 2 == 0) {
            return half * half;
        } else {
            return half * half * x;
        }
    }

    public double myPow(double x, int n) {
        long N = n;
        if (N < 0) {
            x = 1 / x;
            N = -N;
        }

        return fastPow(x, N);
    }
}
```

46. `53. Maximum Subarray.java`

```
/*
 *
 * Given an integer array nums, find the contiguous subarray (containing at least one
number) which has the largest sum and return its sum.
 *
 * Example:
 *
 * Input: [-2,1,-3,4,-1,2,1,-5,4],
 * Output: 6
 * Explanation: [4,-1,2,1] has the largest sum = 6.
 *
 */

class Solution {

    public int maxSubArray(int[] nums) {
        int max_so_far = nums[0];
        int curr_max = nums[0];

        for (int i = 1; i < nums.length; i++) {
            curr_max = Math.max(nums[i], curr_max + nums[i]);
            max_so_far = Math.max(max_so_far, curr_max);
        }
        return max_so_far;
    }
}
```

47. `54. Spiral Matrix.java`

```
/*
 *
 * Given a matrix of m x n elements (m rows, n columns), return all elements of the
matrix in spiral order.
 *
 * Example 1:
 *
 * Input:
 * [
 * [ 1, 2, 3 ],
 * [ 4, 5, 6 ],
 * [ 7, 8, 9 ]
 * ]
 * Output: [1,2,3,6,9,8,7,4,5]
 * Example 2:
 *
 * Input:
 * [
 * [1, 2, 3, 4],
 * [5, 6, 7, 8],
 * [9,10,11,12]
 * ]
 * Output: [1,2,3,4,8,12,11,10,9,5,6,7]
 *
 */

public class Solution {

    public List<Integer> spiralOrder(int[][] matrix) {
        List<Integer> list = new ArrayList<Integer>();
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return list;
        }
        int row = matrix.length;
        int col = matrix[0].length;
        int left = 0, right = col - 1;
        int top = 0, bottom = row - 1;
        while (true) {
            for (int i = left; i <= right; i++) {
                list.add(matrix[top][i]);
            }
            top++;
            if (top > bottom) {
                break;
            }
            for (int i = top; i <= bottom; i++) {
                list.add(matrix[i][right]);
            }
            right--;
            if (right < left) {
                break;
```

```
            }
            for (int i = right; i >= left; i--) {
                list.add(matrix[bottom][i]);
            }
            bottom--;
            if (bottom < top) {
                break;
            }
            for (int i = bottom; i >= top; i--) {
                list.add(matrix[i][left]);
            }
            left++;
            if (left > right) {
                break;
            }
        }
        return list;
    }
}
```

48. `55. Jump Game.java`

```
/*
 *
 * Given an array of non-negative integers, you are initially positioned at the first
index of the array.
 *
 * Each element in the array represents your maximum jump length at that position.
 *
 * Determine if you are able to reach the last index.
 *
 * Example 1:
 *
 * Input: [2,3,1,1,4]
 * Output: true
 * Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
 * Example 2:
 *
 * Input: [3,2,1,0,4]
 * Output: false
 * Explanation: You will always arrive at index 3 no matter what. Its maximum
 * jump length is 0, which makes it impossible to reach the last index.
 *
 */

class Solution {

    /*
     * The basic idea is this: at each step, we keep track of the furthest
     * reachable index. The nature of the problem (eg. maximal jumps where you
     * can hit a range of targets instead of singular jumps where you can only
     * hit one target) is that for an index to be reachable, each of the
     * previous indices have to be reachable.
     *
     * Hence, it suffices that we iterate over each index, and If we ever
     * encounter an index that is not reachable, we abort and return false. By
     * the end, we will have iterated to the last index. If the loop finishes,
     * then the last index is reachable.
     */

    public boolean canJump(int[] nums) {
        int reachable = 0;
        for (int i = 0; i < nums.length; ++i) {
            if (i > reachable) {
                return false;
            }
            reachable = Math.max(reachable, i + nums[i]);
        }
        return true;
    }
}
```

49. `56. Merge Intervals.java`

```
/*
 *
 * Given a collection of intervals, merge all overlapping intervals.
 *
 * Example 1:
 *
 * Input: [[1,3],[2,6],[8,10],[15,18]]
 * Output: [[1,6],[8,10],[15,18]]
 * Explanation: Since intervals [1,3] and [2,6] overlaps, merge them into [1,6].
 * Example 2:
 *
 * Input: [[1,4],[4,5]]
 * Output: [[1,5]]
 * Explanation: Intervals [1,4] and [4,5] are considered overlapping.
 * NOTE: input types have been changed on April 15, 2019. Please reset to default code
definition to get new method signature.
 *
 */

/**
 * Definition for an interval.
 * public class Interval {
 *     int start;
 *     int end;
 *     Interval() { start = 0; end = 0; }
 *     Interval(int s, int e) { start = s; end = e; }
 * }
 */

/*
 * Method 1: By sorting based on the start value and then comparing two consecutive
intervals to check if they could be merged or not
 */
class Solution {

    public List<Interval> merge(List<Interval> list) {
        if (list.size() == 0) {
            return new ArrayList<Interval>();
        }

        Collections.sort(list, new Comparator<Interval>() {
            public int compare(Interval i1, Interval i2) {
                return i1.start - i2.start;
            }
        });

        List<Interval> res = new ArrayList();
        Interval prev = list.get(0);

        for (int i = 1; i < list.size(); i++) {
            Interval current = list.get(i);
```

```
            if (current.start <= prev.end) {
                Interval merged = new Interval(prev.start, Math.max(current.end,
prev.end));
                prev = merged;
            } else {
                res.add(prev);
                prev = current;
            }
        }
        res.add(prev);
        return res;
    }
}


/*
 * Method 2: First sort the intervals and then Using Stack compare 2 consecutive
 * intervals can be merged or not
 */
class Solution {

    public List<Interval> merge(List<Interval> list)
    if (list.size() <= 0)
            return new ArrayList<Interval>();

        List<Interval> res = new ArrayList<>();

    // Create an empty stack of intervals
    Stack<Interval> stack = new Stack<>();

    // sort the intervals in increasing order of start time
    Collections.sort (list,  new Comparator<Interval>(){


    public int compare(Interval i1, Interval i2) {
        return i1.start - i2.start;
    }
}
);

        // push the first interval to stack
        stack.push(list.get(0));

        for (int i = 1 ; i < list.size(); i++)
        {
            // get interval from stack top
            Interval current = stack.peek();

            // if current interval is not overlapping with stack top,
            // push it to the stack
            if (current.end < list.get(i).start)
                stack.push(list.get(i));

            // Otherwise update the ending time of top if ending of current
```

```
            // interval is more
            else if (current.end < list.get(i).end)
            {
                current.end = list.get(i).end;
                stack.pop();
                stack.push(current);
            }
        }

        while (!stack.isEmpty())
        {
            Interval t = stack.pop();
            res.add(t);
        }
    return res;
    }
}
```

50. `58. Length of Last Word.java`

```
/*
 * Given a string s consists of upper/lower-case alphabets and empty space characters '
', return the length of last word in the string.
 *
 * If the last word does not exist, return 0.
 *
 * Note: A word is defined as a character sequence consists of non-space characters only.
 *
 * Example:
 *
 * Input: "Hello World"
 * Output: 5
 *
 */

class Solution {

    public int lengthOfLastWord(String s) {
        if (s.length() == 0 || s.matches("^\\s*$")) {
            return 0;
        }

        String[] s1 = s.split("\\s");

        return s1[s1.length - 1].length();

    }
}
```

51. `59. Spiral Matrix II.java`

```
/*
 *
 * Given a positive integer n, generate a square matrix filled with elements from 1 to n2
in spiral order.
 *
 * Example:
 *
 * Input: 3
 * Output:
 * [
 * [ 1, 2, 3 ],
 * [ 8, 9, 4 ],
 * [ 7, 6, 5 ]
 * ]
 *
 */

class Solution {

    public int[][] generateMatrix(int n) {
        int[][] matrix = new int[n][n];

        int row = n;
        int col = n;

        int left = 0, right = col - 1;
        int top = 0, bottom = row - 1;
        int count = 0;

        while (true) {
            for (int i = left; i <= right; i++) {
                matrix[top][i] = ++count;
            }
            top++;
            if (top > bottom) {
                break;
            }

            for (int i = top; i <= bottom; i++) {
                matrix[i][right] = ++count;
            }
            right--;
            if (right < left) {
                break;
            }

            for (int i = right; i >= left; i--) {
                matrix[bottom][i] = ++count;
            }
            bottom--;
            if (bottom < top) {
```

```
                break;
            }

            for (int i = bottom; i >= top; i--) {
                matrix[i][left] = ++count;
            }
            left++;
            if (left > right) {
                break;
            }
        }
        return matrix;

    }
}
```

52. `60. Permutation Sequence.java`

```
/*
 *
 * The set [1,2,3,...,n] contains a total of n! unique permutations.
 *
 * By listing and labeling all of the permutations in order, we get the following
sequence for n = 3:
 *
 * "123"
 * "132"
 * "213"
 * "231"
 * "312"
 * "321"
 * Given n and k, return the kth permutation sequence.
 *
 * Note:
 *
 * Given n will be between 1 and 9 inclusive.
 * Given k will be between 1 and n! inclusive.
 * Example 1:
 *
 * Input: n = 3, k = 3
 * Output: "213"
 * Example 2:
 *
 * Input: n = 4, k = 9
 * Output: "2314"
 *
 */

class Solution {

    public String getPermutation(int n, int k) {
        int[] numbers = new int[n];
        for (int i = 1; i <= n; i++) {
            numbers[i - 1] = i;
        }
        return perm(numbers, k);
    }

    private String perm(int[] input, int k) {
        if (input.length == 0) {
            return "";
        }
        int len = input.length - 1;
        int cnt = factorial(len);
        int pos = 0;
        while (k - cnt > 0) {
            k -= cnt;
            pos++;
        }
```

```
        String result = Integer.toString(input[pos]);
        int[] rem = new int[len];
        int ix = 0;
        for (int i = 0; i <= len; i++) {
            if (i == pos) {
                continue;
            }
            rem[ix++] = input[i];
        }
        result += perm(rem, k);
        return result;
    }

    private int factorial(int n) {
        int result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }
}

/*
Explanation:

Let's say n = 5 and k = 40;
Currently we have the following individual numbers:

1 2 3 4 5
Among n! = 120 permutations (n-1)! = 24 of them will begin with "1", the next 24 of them
will begin with "2", the next 24 will begin with "3" and so on.
In which case 40th element will begin with "2".

Now the problem becomes among the following numbers, what is the 40 - 24 = 16th
permutation:

1 3 4 5
The length of this set is 4, so (4-1)! = 6 of them will begin with "1", the next 6 will
begin with "3" and the next 6 wil begin with "4".
In which case 16th element will begin with a "4".

Now the problem is what is the 16 - 12 = 4th permutation of the following set:

1 3 5
2 of them will begin with a "1" and 3rd and 4th permutation will begin with a "3" and we
need the 2nd permutation from the remaining set which is:

1 5
1st permutation will begin with a "1" and 2nd will begin with "5".

Overall result is:
```

```
24351
*/
```

53. `61. Rotate List.java`

```
/*
 *
 * Given a linked list, rotate the list to the right by k places, where k is non-
negative.
 *
 * Example 1:
 *
 * Input: 1->2->3->4->5->NULL, k = 2
 * Output: 4->5->1->2->3->NULL
 * Explanation:
 * rotate 1 steps to the right: 5->1->2->3->4->NULL
 * rotate 2 steps to the right: 4->5->1->2->3->NULL
 * Example 2:
 *
 * Input: 0->1->2->NULL, k = 4
 * Output: 2->0->1->NULL
 * Explanation:
 * rotate 1 steps to the right: 2->0->1->NULL
 * rotate 2 steps to the right: 1->2->0->NULL
 * rotate 3 steps to the right: 0->1->2->NULL
 * rotate 4 steps to the right: 2->0->1->NULL
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null || k == 0) {
            return head;
        }

        //make it a cricle, break from k postion far from the head
        ListNode index = head;
        int len = 1;// int len to record the length of list

        while (index.next != null) {
            index = index.next;
            len++;
        }

        index.next = head; //Connecting the last element with the head of the list, so
that we have a circular list!
```

```
        for (int i = 0; i < len - k % len; i++) {
            index = index.next;
        }

        ListNode result = index.next;//Cutoff the list after the k nodes are traversed
        index.next = null;
        return result;
    }
}
```

54. `62. Unique Paths.java`

```
/*
 *
 * A robot is located at the top-left corner of a m x n grid (marked 'Start' in the
diagram below).
 *
 * The robot can only move either down or right at any point in time. The robot is trying
to reach the bottom-right corner of the grid (marked 'Finish' in the diagram below).
 *
 * How many possible unique paths are there?
 *
 *
 * Above is a 7 x 3 grid. How many possible unique paths are there?
 *
 * Note: m and n will be at most 100.
 *
 * Example 1:
 *
 * Input: m = 3, n = 2
 * Output: 3
 * Explanation:
 * From the top-left corner, there are a total of 3 ways to reach the bottom-right
corner:
 * 1. Right -> Right -> Down
 * 2. Right -> Down -> Right
 * 3. Down -> Right -> Right
 * Example 2:
 *
 * Input: m = 7, n = 3
 * Output: 28
 *
 */

class Solution {

    public int uniquePaths(int m, int n) {
        if (m == 0 || n == 0) {
            return 0;
        }
        if (m == 1 || n == 1) {
            return 1;
        }

        int[][] dp = new int[m][n];

        //left column
        for (int i = 0; i < m; i++) {
            dp[i][0] = 1;
        }

        //top row
        for (int j = 0; j < n; j++) {
```

```
            dp[0][j] = 1;
        }

        //fill up the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
            }
        }

        return dp[m - 1][n - 1];
    }
}
```

55. `63. Unique Paths II.java`

```
/*
 *
 * A robot is located at the top-left corner of a m x n grid (marked 'Start' in the
diagram below).
 *
 * The robot can only move either down or right at any point in time. The robot is trying
to reach the bottom-right corner of the grid (marked 'Finish' in the diagram below).
 *
 * Now consider if some obstacles are added to the grids. How many unique paths would
there be?
 *
 *
 * An obstacle and empty space is marked as 1 and 0 respectively in the grid.
 *
 * Note: m and n will be at most 100.
 *
 * Example 1:
 *
 * Input:
 * [
 * [0,0,0],
 * [0,1,0],
 * [0,0,0]
 * ]
 * Output: 2
 * Explanation:
 * There is one obstacle in the middle of the 3x3 grid above.
 * There are two ways to reach the bottom-right corner:
 * 1. Right -> Right -> Down -> Down
 * 2. Down -> Down -> Right -> Right
 *
 */

class Solution {

    public int uniquePathsWithObstacles(int[][] obstacleGrid) {
        if (obstacleGrid == null || obstacleGrid.length == 0) {
            return 0;
        }
        int m = obstacleGrid.length;
        int n = obstacleGrid[0].length;
        if (obstacleGrid[0][0] == 1 || obstacleGrid[m - 1][n - 1] == 1) {
            return 0;
        }
        int[][] dp = new int[m][n];
        dp[0][0] = 1;
//left column
        for (int i = 1; i < m; i++) {
            if (obstacleGrid[i][0] == 1) {
                dp[i][0] = 0;
            } else {
```

```
                dp[i][0] = dp[i - 1][0];
            }
        }
//top row
        for (int i = 1; i < n; i++) {
            if (obstacleGrid[0][i] == 1) {
                dp[0][i] = 0;
            } else {
                dp[0][i] = dp[0][i - 1];
            }
        }
//fill up cells inside
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (obstacleGrid[i][j] == 1) {
                    dp[i][j] = 0;
                } else {
                    dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
                }
            }
        }
        return dp[m - 1][n - 1];
    }
}
```

56. `64. Minimum Path Sum.java`

```
/*
 *
 * Given a m x n grid filled with non-negative numbers, find a path from top left to
bottom right which minimizes the sum of all numbers along its path.
 *
 * Note: You can only move either down or right at any point in time.
 *
 * Example:
 *
 * Input:
 * [
 * [1,3,1],
 * [1,5,1],
 * [4,2,1]
 * ]
 * Output: 7
 * Explanation: Because the path 1→3→1→1→1 minimizes the sum.
 *
 */

class Solution {

    public int minPathSum(int[][] grid) {
        if (grid == null || grid.length == 0) {
            return 0;
        }
        int m = grid.length;
        int n = grid[0].length;
        int[][] dp = new int[m][n];
        dp[0][0] = grid[0][0];
// initialize top row
        for (int i = 1; i < n; i++) {
            dp[0][i] = dp[0][i - 1] + grid[0][i];
        }
// initialize left column
        for (int j = 1; j < m; j++) {
            dp[j][0] = dp[j - 1][0] + grid[j][0];
        }
// fill up the dp table
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
//if(dp[i-1][j] > dp[i][j-1]){
                dp[i][j] = Math.min((dp[i][j - 1] + grid[i][j]), (dp[i - 1][j] + grid[i]
[j]));
//}else{
//dp[i][j] = dp[i-1][j] + grid[i][j];
//}
            }
        }
        return dp[m - 1][n - 1];
```

```
    }
}
```

57. `66. Plus One.java`

```
/*
 *
 * Given a non-empty array of digits representing a non-negative integer, plus one to the
integer.
 *
 * The digits are stored such that the most significant digit is at the head of the list,
and each element in the array contain a single digit.
 *
 * You may assume the integer does not contain any leading zero, except the number 0
itself.
 *
 * Example 1:
 *
 * Input: [1,2,3]
 * Output: [1,2,4]
 * Explanation: The array represents the integer 123.
 * Example 2:
 *
 * Input: [4,3,2,1]
 * Output: [4,3,2,2]
 * Explanation: The array represents the integer 4321.
 *
 */

public class Solution {

    public int[] plusOne(int[] digits) {
        for (int i = digits.length - 1; i >= 0; i--) ///Here we are traversing from
unit's place to tens or hundreds place.
        {
            if (digits[i] != 9) {
                digits[i]++;
                break;
            } else {
                digits[i] = 0;
            }
        }
        if (digits[0] == 0) //This means that earlier there was the digit 9 present
there. And because of +1, it has become 0 now. So we need to ////add the digit 1 on the
left hand side of the current zero.
        {
            int[] res = new int[digits.length + 1];
            res[0] = 1;
            return res;
        }
        return digits;
    }
}
/*

General description
```

```
The meaning of the question: For an n sized array, traverse through each of the array
elements, and plus one in it.
So if the digit is 9, then we need to make 0 there. Else just add one to the number.

*/
```

58. `67. Add Binary.java`

```
/*
 *
 * Given two binary strings, return their sum (also a binary string).
 *
 * The input strings are both non-empty and contains only characters 1 or 0.
 *
 * Example 1:
 *
 * Input: a = "11", b = "1"
 * Output: "100"
 * Example 2:
 *
 * Input: a = "1010", b = "1011"
 * Output: "10101"
 *
 */

import java.math.BigInteger;

class Solution {

    public String addBinary(String a, String b) {
        StringBuilder sb = new StringBuilder();
        int i = a.length() - 1;
        int j = b.length() - 1;
        int carry = 0;
        while (i >= 0 || j >= 0) {
            int sum = 0;
            if (i >= 0 && a.charAt(i) == '1') {
                sum++;
            }
            if (j >= 0 && b.charAt(j) == '1') {
                sum++;
            }
            sum += carry;
            if (sum >= 2) {
                carry = 1;
            } else {
                carry = 0;
            }
            sb.insert(0, (char) ((sum % 2) + '0'));
            i--;
            j--;
        }
        if (carry == 1) {
            sb.insert(0, '1');
        }
        return sb.toString();
    }
}
```

59. `69. Sqrt(x).java`

```
/*
 * Example 1:
 * Input: 4
 * Output: 2
 *
 *
 * Example 2:
 *
 * Input: 8
 * Output: 2
 * Explanation: The square root of 8 is 2.82842..., and since
 * the decimal part is truncated, 2 is returned.
 *
 */

class Solution {

    public int mySqrt(int x) {
        Double y = Math.sqrt(x);
        return (int) Math.floor(y);
    }
}
```

60. `70. Climbing Stairs.java`

```
/*
 *
 * You are climbing a stair case. It takes n steps to reach to the top.
 *
 * Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb
to the top?
 *
 * Note: Given n will be a positive integer.
 *
 * Example 1:
 *
 * Input: 2
 * Output: 2
 * Explanation: There are two ways to climb to the top.
 * 1. 1 step + 1 step
 * 2. 2 steps
 * Example 2:
 *
 * Input: 3
 * Output: 3
 * Explanation: There are three ways to climb to the top.
 * 1. 1 step + 1 step + 1 step
 * 2. 1 step + 2 steps
 * 3. 2 steps + 1 step
 *
 */

class Solution {

    public int climbStairs(int n) {
        if (n == 1) {
            return 1;
        }

        int[] dp = new int[n + 1];
        dp[1] = 1;
        dp[2] = 2;
        for (int i = 3; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
}
```

61. `71. Simplify Path.java`

```
/*
 *
 * Given an absolute path for a file (Unix-style), simplify it. Or in other words,
convert it to the canonical path.
 *
 * In a UNIX-style file system, a period . refers to the current directory. Furthermore,
a double period .. moves the directory up a level. For more information, see: Absolute
path vs relative path in Linux/Unix
 *
 * Note that the returned canonical path must always begin with a slash /, and there must
be only a single slash / between two directory names. The last directory name (if it
exists) must not end with a trailing /. Also, the canonical path must be the shortest
string representing the absolute path.
 *
 *
 *
 * Example 1:
 *
 * Input: "/home/"
 * Output: "/home"
 * Explanation: Note that there is no trailing slash after the last directory name.
 * Example 2:
 *
 * Input: "/../"
 * Output: "/"
 * Explanation: Going one level up from the root directory is a no-op, as the root level
is the highest level you can go.
 * Example 3:
 *
 * Input: "/home//foo/"
 * Output: "/home/foo"
 * Explanation: In the canonical path, multiple consecutive slashes are replaced by a
single one.
 * Example 4:
 *
 * Input: "/a/./b/../../c/"
 * Output: "/c"
 * Example 5:
 *
 * Input: "/a/../../b/../c//.//"
 * Output: "/c"
 * Example 6:
 *
 * Input: "/a//b////c/d//././/.."
 * Output: "/a/b/c"
 *
 */

class Solution {

    public String simplifyPath(String path) {
```

```
        String[] dir = path.split("/");
        String[] stack = new String[dir.length];
        int ptr = 0;
        for (int i = 0; i < dir.length; i++) {
            if (dir[i].equals(".") || dir[i].equals("")) {
                continue;
            } else if (dir[i].equals("..")) {
                if (ptr > 0) {
                    ptr--;
                }
            } else {
                stack[ptr] = dir[i];
                ptr++;
            }
        }
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < ptr; i++) {
            result.append("/");
            result.append(stack[i]);
        }
        return result.length() == 0 ? "/" : result.toString();
    }
}
```

62. `72. Edit Distance.java`

```
/*
 *
 * Given two words word1 and word2, find the minimum number of operations required to
convert word1 to word2.
 *
 * You have the following 3 operations permitted on a word:
 *
 * Insert a character
 * Delete a character
 * Replace a character
 * Example 1:
 *
 * Input: word1 = "horse", word2 = "ros"
 * Output: 3
 * Explanation:
 * horse -> rorse (replace 'h' with 'r')
 * rorse -> rose (remove 'r')
 * rose -> ros (remove 'e')
 * Example 2:
 *
 * Input: word1 = "intention", word2 = "execution"
 * Output: 5
 * Explanation:
 * intention -> inention (remove 't')
 * inention -> enention (replace 'i' with 'e')
 * enention -> exention (replace 'n' with 'x')
 * exention -> exection (replace 'n' with 'c')
 * exection -> execution (insert 'u')
 *
 */

class Solution {

    public int minDistance(String word1, String word2) {
        int[][] dp = new int[word2.length() + 1][word1.length() + 1];

        for (int i = 0; i < dp.length; i++) {
            dp[i][0] = i;
        }

        for (int j = 0; j < dp[0].length; j++) {
            dp[0][j] = j;
        }

        char[] w1 = word1.toCharArray();
        char[] w2 = word2.toCharArray();

        for (int i = 1; i < dp.length; i++) {
            for (int j = 1; j < dp[0].length; j++) {
                if (w2[i - 1] == w1[j - 1]) {
                    dp[i][j] = dp[i - 1][j - 1];
```

```
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i][j - 1], dp[i
- 1][j]));
                }
            }
        }

        for (int i = 0; i < dp.length; i++) {
            for (int j = 0; j < dp[0].length; j++) {
                System.out.print(dp[i][j]);
            }
            System.out.println();
        }
        return dp[word2.length()][word1.length()];
    }
}
```

63. `73. Set Matrix Zeroes.java`

```
/*
 *
 * Given a m x n matrix, if an element is 0, set its entire row and column to 0. Do it
in-place.
 *
 * Example 1:
 *
 * Input:
 * [
 * [1,1,1],
 * [1,0,1],
 * [1,1,1]
 * ]
 * Output:
 * [
 * [1,0,1],
 * [0,0,0],
 * [1,0,1]
 * ]
 * Example 2:
 *
 * Input:
 * [
 * [0,1,2,0],
 * [3,4,5,2],
 * [1,3,1,5]
 * ]
 * Output:
 * [
 * [0,0,0,0],
 * [0,4,5,0],
 * [0,3,1,0]
 * ]
 * Follow up:
 *
 * A straight forward solution using O(mn) space is probably a bad idea.
 * A simple improvement uses O(m + n) space, but still not the best solution.
 * Could you devise a constant space solution?
 *
 */

public class Solution {

    public void setZeroes(int[][] matrix) {
        boolean firstRowZero = false;
        boolean firstColumnZero = false;
//set first row and column zero or not
        for (int i = 0; i < matrix.length; i++) {
            if (matrix[i][0] == 0) {
                firstColumnZero = true;
                break;
```

```
            }
        }
        for (int i = 0; i < matrix[0].length; i++) {
            if (matrix[0][i] == 0) {
                firstRowZero = true;
                break;
            }
        }
//mark zeros on first row and column
        for (int i = 1; i < matrix.length; i++) {
            for (int j = 1; j < matrix[0].length; j++) {
                if (matrix[i][j] == 0) {
                    matrix[i][0] = 0;
                    matrix[0][j] = 0;
                }
            }
        }
//use mark to set elements
        for (int i = 1; i < matrix.length; i++) {
            for (int j = 1; j < matrix[0].length; j++) {
                if (matrix[i][0] == 0 || matrix[0][j] == 0) {
                    matrix[i][j] = 0;
                }
            }
        }
//set first column and row
        if (firstColumnZero) {
            for (int i = 0; i < matrix.length; i++) {
                matrix[i][0] = 0;
            }
        }
        if (firstRowZero) {
            for (int i = 0; i < matrix[0].length; i++) {
                matrix[0][i] = 0;
            }
        }
    }
}
```

64. `74. Search a 2D Matrix.java`

```
/*
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix. This matrix
has the following properties:
 *
 * Integers in each row are sorted from left to right.
 * The first integer of each row is greater than the last integer of the previous row.
 * Example 1:
 *
 * Input:
 * matrix = [
 * [1,   3,  5,  7],
 * [10, 11, 16, 20],
 * [23, 30, 34, 50]
 * ]
 * target = 3
 * Output: true
 * Example 2:
 *
 * Input:
 * matrix = [
 * [1,   3,  5,  7],
 * [10, 11, 16, 20],
 * [23, 30, 34, 50]
 * ]
 * target = 13
 * Output: false
 *
 */

class Solution {

    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix.length == 0) {
            return false;
        }

        int i = 0, j = matrix[0].length - 1;

        //Start with the matrix[0][mat[0].len -1] i.e. the right most corner element
        //Check if the value is less or more than the target
        while (i < matrix.length && j >= 0) {
            if (matrix[i][j] == target) {
                return true;
            } else if (matrix[i][j] > target) //Means that the target is less than the
element, so move to the left column
            {
                j--;
            } else { //Value more than the rightmost corner element, so that means move
to the next row for search
                i++;
```

```
            }
        }
        return false;
    }
}
```

65. `75. Sort Colors.java`

```
/*
 *
 * Given an array with n objects colored red, white or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white and blue.
 *
 * Here, we will use the integers 0, 1, and 2 to represent the color red, white, and blue respectively.
 *
 * Note: You are not suppose to use the library's sort function for this problem.
 *
 * Example:
 *
 * Input: [2,0,2,1,1,0]
 * Output: [0,0,1,1,2,2]
 * Follow up:
 *
 * A rather straight forward solution is a two-pass algorithm using counting sort.
 * First, iterate the array counting number of 0's, 1's, and 2's, then overwrite array with total number of 0's, then 1's and followed by 2's.
 * Could you come up with a one-pass algorithm using only constant space?
 *
 */

class Solution {

    public void sortColors(int[] nums) {
        int count0 = 0, count1 = 0, count2 = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 0) {
                count0++;
            } else {
                if (nums[i] == 1) {
                    count1++;
                } else {
                    count2++;
                }
            }
        }

        for (int i = 0; i < nums.length; i++) {
            if (i < count0) {
                nums[i] = 0;
            } else {
                if (i < count0 + count1) {
                    nums[i] = 1;
                } else {
                    nums[i] = 2;
                }
            }
```

```
        }

    }
}
```

66. `77. Combinations.java`

```
/*
 *
 * Given two integers n and k, return all possible combinations of k numbers out of 1 ...
n.
 *
 * Example:
 *
 * Input: n = 4, k = 2
 * Output:
 * [
 * [2,4],
 * [3,4],
 * [2,3],
 * [1,2],
 * [1,3],
 * [1,4],
 * ]
 *
 */

class Solution {

    public List<List<Integer>> combine(int n, int k) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(result, new ArrayList<Integer>(), 1, n, k);
        return result;
    }

    public void backtrack(List<List<Integer>> result, ArrayList<Integer> comb, int start,
int n, int k) {
        if (k == 0) {
            result.add(new ArrayList<>(comb));
        }

        for (int i = start; i <= n; i++) {
            comb.add(i);
            backtrack(result, comb, i + 1, n, k - 1);
            comb.remove(comb.size() - 1);
        }
    }
}

/*
 * Similar approach to the Backtracking question of Subsets I & II, Combination
 * Sum questions https://leetcode.com/problems/subsets-ii/description/
 */
```

67. `78. Subsets.java`

```
/*
 *
 * Given a set of distinct integers, nums, return all possible subsets (the power set).
 *
 * Note: The solution set must not contain duplicate subsets.
 *
 * Example:
 *
 * Input: nums = [1,2,3]
 * Output:
 * [
 * [3],
 * [1],
 * [2],
 * [1,2,3],
 * [1,3],
 * [2,3],
 * [1,2],
 * []
 * ]
 *
 */

 /*
 * This is a typical Backtracking problem. In case you wish to look at generic
Backtracking problems and solutions, then:
 *
 * Reference solution here: https://leetcode.com/problems/subsets/discuss/27281/A-
general-approach-to-backtracking-questions-in-Java-(Subsets-Permutations-Combination-Sum-
Palindrome-Partitioning)
 */
class Solution {

    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> list = new ArrayList<>();
        //Arrays.sort(nums);
        backtrack(list, new ArrayList<>(), nums, 0);
        return list;
    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums,
int start) {
        list.add(new ArrayList<>(tempList));

        //System.out.println("Resultant list: " + list);
        for (int i = start; i < nums.length; i++) {
            tempList.add(nums[i]);
            backtrack(list, tempList, nums, i + 1);
            tempList.remove(tempList.size() - 1);
        }
    }
```

```
}

/*
The way we are performing operations is:
1. Sort the given array.
2. Then call the function of backtracking with an empty set, since empty set is also also a subset of the given array
3. The back track function is basically to make sure that we traverse every possible combination available from our input array
    i) First we start with an empty list, which is also an subset of any given array
    ii) Then we take the first element i.e. 1, and proceed to get all combinations possible with by recursively calling the function
    iii) Once all combinations of first element are done, then we take the next element i.e. 2, and then get all the combinations starting with the number 2.
    iv) We are automatically eliminating subsets with same values i.e [1,2] and [2,1] is same. The duplicates are automatically handled by the for loop since we are starting from element start is incrementing as we go on traversing


    The output looks like this:
    1. Resultant list: [[]]   <----- The empty subset
    ----------------All combinations starting from 1 are explored ----------------------
    2. Resultant list: [[], [1]]
    3. Resultant list: [[], [1], [1, 2]]
    4. Resultant list: [[], [1], [1, 2], [1, 2, 3]]
    5. Resultant list: [[], [1], [1, 2], [1, 2, 3], [1, 3]]
    ----------------All combinations with 1 are done and now starting from 2 are explored ----------------------
    6. Resultant list: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2]]
    7. Resultant list: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3]]
    ----------------All combinations with 2 are done and now starting from 3 are explored ----------------------
    8. Resultant list: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
*/
```

68. `79. Word Search.java`

```
/*
 *
 * Given a 2D board and a word, find if the word exists in the grid.
 *
 * The word can be constructed from letters of sequentially adjacent cell, where
"adjacent" cells are those horizontally or vertically neighboring. The same letter cell
may not be used more than once.
 *
 * Example:
 *
 * board =
 * [
 * ['A','B','C','E'],
 * ['S','F','C','S'],
 * ['A','D','E','E']
 * ]
 *
 * Given word = "ABCCED", return true.
 * Given word = "SEE", return true.
 * Given word = "ABCB", return false.
 *
 */

//Explanation: Given word = SEE. WE are allowed to search for the alphabets from the word
in teh horizontal or vertical direction only. SO we need to search 4 directions that's
why do DFS
//in the given array -> S is available at index= arr[1][3], then E is available down at
index = arr[2][3] and last E is available at index = arr[2][2]
class Solution {

    public boolean exist(char[][] board, String word) {
        int m = board.length;
        int n = board[0].length;
        boolean result = false;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (dfs(board, word, i, j, 0)) {
                    result = true;
                }
            }
        }
        return result;
    }

//i and j are the indexes of the array of characters
//k is the index of the character in the given word. EG: k will be index of S or E or E
in our pursuit to find if the characters are available in the grid
    public boolean dfs(char[][] board, String word, int i, int j, int k) {
        int m = board.length;
        int n = board[0].length;
        if (i < 0 || j < 0 || i >= m || j >= n) {
```

```
            return false;
        }
        if (board[i][j] == word.charAt(k)) {
            char temp = board[i][j];
            board[i][j] = '#';
            if (k == word.length() - 1) {
                return true;
            } else if (dfs(board, word, i - 1, j, k + 1)
                    || dfs(board, word, i + 1, j, k + 1)
                    || dfs(board, word, i, j - 1, k + 1)
                    || dfs(board, word, i, j + 1, k + 1)) {
                return true;
            }
            board[i][j] = temp;
        }
        return false;
    }
}
```

69. `80. Remove Duplicates from Sorted Array II.java`

```
/*
 *
 * Given a sorted array nums, remove the duplicates in-place such that duplicates
appeared at most twice and return the new length.
 *
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * Example 1:
 *
 * Given nums = [1,1,1,2,2,3],
 *
 * Your function should return length = 5, with the first five elements of nums being 1,
1, 2, 2 and 3 respectively.
 *
 * It doesn't matter what you leave beyond the returned length.
 * Example 2:
 *
 * Given nums = [0,0,1,1,1,1,2,3,3],
 *
 * Your function should return length = 7, with the first seven elements of nums being
modified to 0, 0, 1, 1, 2, 3 and 3 respectively.
 *
 * It doesn't matter what values are set beyond the returned length.
 * Clarification:
 *
 * Confused why the returned value is an integer but your answer is an array?
 *
 * Note that the input array is passed in by reference, which means modification to the
input array will be known to the caller as well.
 *
 * Internally you can think of this:
 *
 * // nums is passed in by reference. (i.e., without making a copy)
 * int len = removeDuplicates(nums);
 *
 * // any modification to nums in your function would be known by the caller.
 * // using the length returned by your function, it prints the first len elements.
 * for (int i = 0; i < len; i++) {
 * print(nums[i]);
 * }
 *
 *
 */

class Solution {

    public int removeDuplicates(int[] A) {
        if (A.length <= 2) {
            return A.length;
        }
```

```
        int prev = 1; // point to previous
        int curr = 2; // point to current

        while (curr < A.length) {
            if (A[curr] == A[prev] && A[curr] == A[prev - 1]) {
                curr++;
            } else {
                prev++;
                A[prev] = A[curr];
                curr++;
            }
        }

        return prev + 1;
    }
}
```

70. `81. Search in Rotated Sorted Array II.java`

```
/*
 *
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 *
 * (i.e., [0,0,1,2,2,5,6] might become [2,5,6,0,0,1,2]).
 *
 * You are given a target value to search. If found in the array return true, otherwise
return false.
 *
 * Example 1:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 0
 * Output: true
 * Example 2:
 *
 * Input: nums = [2,5,6,0,0,1,2], target = 3
 * Output: false
 * Follow up:
 *
 * This is a follow up problem to Search in Rotated Sorted Array, where nums may contain
duplicates.
 * Would this affect the run-time complexity? How and why?
 *
 */

class Solution {

    public boolean search(int[] nums, int target) {
        int start = 0, end = nums.length - 1, mid = -1;
        while (start <= end) {
            mid = start + (end - start) / 2;
            if (nums[mid] == target) {
                return true;
            }
            //If we know for sure right side is sorted or left side is unsorted
            if (nums[mid] < nums[end] || nums[mid] < nums[start]) {
                if (target > nums[mid] && target <= nums[end]) {
                    start = mid + 1;
                } else {
                    end = mid - 1;
                }
            } //If we know for sure left side is sorted or right side is unsorted
            else if (nums[mid] > nums[start] || nums[mid] > nums[end]) {
                if (target < nums[mid] && target >= nums[start]) {
                    end = mid - 1;
                } else {
                    start = mid + 1;
                }

                //If we get here, that means nums[start] == nums[mid] == nums[end], then
```

```
shifting out
                //any of the two sides won't change the result but can help remove
duplicate from
                //consideration, here we just use end-- but left++ works too
            } else {
                end--;
            }
        }

        return false;
    }
}
```

71. `82. Remove Duplicates from Sorted List II.java`

```
/*
 *
 * Given a sorted linked list, delete all nodes that have duplicate numbers, leaving only
distinct numbers from the original list.
 *
 * Example 1:
 *
 * Input: 1->2->3->3->4->4->5
 * Output: 1->2->5
 * Example 2:
 *
 * Input: 1->1->1->2->3
 * Output: 2->3
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode deleteDuplicates(ListNode head) {
        ListNode dummy = new ListNode(0);
        ListNode prev = dummy;
        ListNode current = head;

        while (current != null) {
            //We have a duplicate
            if (current.next != null && current.val == current.next.val) {
                //Repeat till the time you remove all the duplicates
                while (current.next != null && current.val == current.next.val) {
                    current = current.next;
                }
            } else {
                //If the 2 consecutive nodes are having different values, then just
append the current element to the resultant list
                prev.next = current;
                prev = prev.next;
            }

            current = current.next;
        }

        prev.next = null;
        return dummy.next;
```

```
    }
}
```

72. `83. Remove Duplicates from Sorted List.java`

```
/*
 *
 * Given a sorted linked list, delete all duplicates such that each element appear only
once.
 *
 * Example 1:
 *
 * Input: 1->1->2
 * Output: 1->2
 * Example 2:
 *
 * Input: 1->1->2->3->3
 * Output: 1->2->3
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
public class Solution {

    public ListNode deleteDuplicates(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode p = head;

        while (p != null && p.next != null) {
            if (p.val == p.next.val) {
                p.next = p.next.next;
            } else {
                p = p.next;
            }
        }

        return head;
    }
}
```

73. `84. Largest Rectangle in Histogram.java`

```
/*
 * Given n non-negative integers representing the histogram's bar height where the width
of each bar is 1, find the area of largest rectangle in the histogram.
 *
 *
 * Above is a histogram where width of each bar is 1, given height = [2,1,5,6,2,3].
 *
 *
 * The largest rectangle is shown in the shaded area, which has area = 10 unit.
 *
 *
 * Example:
 *
 * Input: [2,1,5,6,2,3]
 * Output: 10
 *
 */

class Solution {

    public int largestRectangleArea(int[] height) {
        if (height == null || height.length == 0) {
            return 0;
        }

        Stack<Integer> stack = new Stack<Integer>();
        int max = 0;
        int i = 0;
        while (i < height.length) {
            //push index to stack when the current height is larger than the previous one
            if (stack.isEmpty() || height[i] >= height[stack.peek()]) {
                stack.push(i);
                i++;
            } else {
//calculate max value when the current height is less than the previous one
                int p = stack.pop();
                int h = height[p];
                int w = stack.isEmpty() ? i : i - stack.peek() - 1;
                max = Math.max(h * w, max);
            }
        }

        while (!stack.isEmpty()) {
            int p = stack.pop();
            int h = height[p];
            int w = stack.isEmpty() ? i : i - stack.peek() - 1;
            max = Math.max(h * w, max);
        }
        return max;
    }
}
```

74. `86. Partition List.java`

```
/*
 *
 * Given a linked list and a value x, partition it such that all nodes less than x come
before nodes greater than or equal to x.
 *
 * You should preserve the original relative order of the nodes in each of the two
partitions.
 *
 * Example:
 *
 * Input: head = 1->4->3->2->5->2, x = 3
 * Output: 1->2->2->4->3->5
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode partition(ListNode head, int x) {
        ListNode dummyLeft = new ListNode(42);
        ListNode dummyRight = new ListNode(42);
        ListNode nodeLeft = dummyLeft;
        ListNode nodeRight = dummyRight;
        while (head != null) {
            if (head.val < x) {
                nodeLeft.next = head;
                nodeLeft = nodeLeft.next;
            } else {
                nodeRight.next = head;
                nodeRight = nodeRight.next;
            }
            head = head.next;
        }
        nodeLeft.next = dummyRight.next;
        nodeRight.next = null;
        return dummyLeft.next;

    }
}
```

75. `88. Merge Sorted Array.java`

```
/*
 *
 * Given two sorted integer arrays nums1 and nums2, merge nums2 into nums1 as one sorted
array.
 *
 * Note:
 *
 * The number of elements initialized in nums1 and nums2 are m and n respectively.
 * You may assume that nums1 has enough space (size that is greater or equal to m + n) to
hold additional elements from nums2.
 * Example:
 *
 * Input:
 * nums1 = [1,2,3,0,0,0], m = 3
 * nums2 = [2,5,6],       n = 3
 *
 * Output: [1,2,2,3,5,6]
 *
 */

public class Solution {

    public void merge(int A[], int m, int B[], int n) {

        while (m > 0 && n > 0) {
            if (A[m - 1] > B[n - 1]) {
                A[m + n - 1] = A[m - 1];
                m--;
            } else {
                A[m + n - 1] = B[n - 1];
                n--;
            }
        }

        while (n > 0) {
            A[m + n - 1] = B[n - 1];
            n--;
        }
    }
}
```

76. `89. Gray Code.java`

```
/*
 *
 * The gray code is a binary numeral system where two successive values differ in only
one bit.
 *
 * Given a non-negative integer n representing the total number of bits in the code,
print the sequence of gray code. A gray code sequence must begin with 0.
 *
 * Example 1:
 *
 * Input: 2
 * Output: [0,1,3,2]
 * Explanation:
 * 00 - 0
 * 01 - 1
 * 11 - 3
 * 10 - 2
 *
 * For a given n, a gray code sequence may not be uniquely defined.
 * For example, [0,2,3,1] is also a valid gray code sequence.
 *
 * 00 - 0
 * 10 - 2
 * 11 - 3
 * 01 - 1
 * Example 2:
 *
 * Input: 0
 * Output: [0]
 * Explanation: We define the gray code sequence to begin with 0.
 * A gray code sequence of n has size = 2n, which for n = 0 the size is 20 = 1.
 * Therefore, for n = 0 the gray code sequence is [0].
 *
 */

class Solution {

    public List<Integer> grayCode(int n) {
        List<Integer> list = new LinkedList<Integer>();
        Stack<Integer> stack = new Stack<Integer>();
        list.add(0);
        int i = 0;

        while (i < n) {
            int len = list.size();
            int inc = (int) (Math.pow(2, i));
            System.out.println(len + "  " + inc);
            for (int j = 0; j < len; j++) {
                stack.push(list.get(j));
            }
```

```
            System.out.println(stack);
            while (!stack.isEmpty()) {
                list.add(stack.pop() + inc);
            }
            i++;
        }
        return list;

    }
}
```

77. `90. Subsets II.java`

```
/*
 *
 * Given a collection of integers that might contain duplicates, nums, return all
possible subsets (the power set).
 *
 * Note: The solution set must not contain duplicate subsets.
 *
 * Example:
 *
 * Input: [1,2,2]
 * Output:
 * [
 * [2],
 * [1],
 * [1,2,2],
 * [2,2],
 * [1,2],
 * []
 * ]
 *
 */

class Solution {

    public List<List<Integer>> subsetsWithDup(int[] nums) {
        List<List<Integer>> list = new ArrayList<>();
        Arrays.sort(nums);
        backtrack(list, new ArrayList<>(), nums, 0);
        return list;
    }

    private void backtrack(List<List<Integer>> list, List<Integer> tempList, int[] nums,
int start) {
        list.add(new ArrayList<>(tempList));
        for (int i = start; i < nums.length; i++) {
            if (i > start && nums[i] == nums[i - 1]) {
                continue; // skip duplicates
            }
            tempList.add(nums[i]);
            backtrack(list, tempList, nums, i + 1);
            tempList.remove(tempList.size() - 1);
        }
    }
}
```

78. `91. Decode Ways.java`

```
/*
 *
 * A message containing letters from A-Z is being encoded to numbers using the following
mapping:
 *
 * 'A' -> 1
 * 'B' -> 2
 * ...
 * 'Z' -> 26
 * Given a non-empty string containing only digits, determine the total number of ways to
decode it.
 *
 * Example 1:
 *
 * Input: "12"
 * Output: 2
 * Explanation: It could be decoded as "AB" (1 2) or "L" (12).
 * Example 2:
 *
 * Input: "226"
 * Output: 3
 * Explanation: It could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).
 *
 */

class Solution {

    public int numDecodings(String s) {
        if (s == null || s.length() == 0 || s.charAt(0) == '0') {
            return 0;
        }
        if (s.length() == 1) {
            return 1;
        }

        int[] dp = new int[s.length()];
        dp[0] = 1;
        if (Integer.parseInt(s.substring(0, 2)) > 26) {
            if (s.charAt(1) != '0') {
                dp[1] = 1;
            } else {
                dp[1] = 0;
            }
        } else if (s.charAt(1) != '0') {
            dp[1] = 2;
        } else {
            dp[1] = 1;
        }

        for (int i = 2; i < s.length(); i++) {
            if (s.charAt(i) != '0') {
```

```
                dp[i] += dp[i - 1];
            }

            int val = Integer.parseInt(s.substring(i - 1, i + 1));
            if (val <= 26 && val >= 10) {
                dp[i] += dp[i - 2];
            }
        }

        return dp[s.length() - 1];
    }
}
```

79. `92. Reverse Linked List II.java`

```
/*
 *
 * Reverse a linked list from position m to n. Do it in one-pass.
 *
 * Note: 1 ≤ m ≤ n ≤ length of list.
 *
 * Example:
 *
 * Input: 1->2->3->4->5->NULL, m = 2, n = 4
 * Output: 1->4->3->2->5->NULL
 *
 */


 /*
 * Method 1: First traveling m nodes ahead and then reversing the sublist of length (n -
m) nodes.
 */
class Solution {

    public ListNode reverseBetween(ListNode head, int m, int n) {

        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;

        for (int i = 1; i < m; i++) {
            prev = prev.next;               // Node pointing to a subList
        }
        ListNode current = prev.next;       // First node of a sub-list

        //Basically swapping the nodes so that finally the result is reversed list
        for (int i = 0; i < n - m; i++) {
            // Node that will be reversed, temp node will become sub-list head in each
iteration
            ListNode temp = current.next;
            current.next = temp.next;
            temp.next = prev.next;
            prev.next = temp;
        }

        return dummy.next;
    }
}

/*
 * Method 2: First traverse and mark mth and nth node. Then reverse the sublist
 * based on the mth and nth node marking
 */
class Solution {
```

```
public ListNode reverseBetween(ListNode head, int m, int n) {
    if (m == n) {
        return head;
    }

    ListNode prev = null;//track (m-1)th node
    ListNode first = new ListNode(0);//first's next points to mth
    ListNode second = new ListNode(0);//second's next points to (n+1)th

    int i = 0;
    ListNode p = head;

    while (p != null) {
        i++;
        if (i == m - 1) {
            prev = p;
        }

        if (i == m) {
            first.next = p;
        }

        if (i == n) {
            second.next = p.next;
            p.next = null;
        }

        p = p.next;
    }

    if (first.next == null) {
        return head;
    }

    // reverse list [m, n]
    ListNode p1 = first.next;
    ListNode p2 = p1.next;
    p1.next = second.next;

    while (p1 != null && p2 != null) {
        ListNode t = p2.next;
        p2.next = p1;
        p1 = p2;
        p2 = t;
    }

    //connect to previous part
    if (prev != null) {
        prev.next = p1;
    } else {
        return p1;
    }

    return head;
```

```
    }
}
```

80. `93. Restore IP Addresses.java`

```
/*
 *
 * Given a string containing only digits, restore it by returning all possible valid IP
address combinations.
 *
 * Example:
 *
 * Input: "25525511135"
 * Output: ["255.255.11.135", "255.255.111.35"]
 *
 */

public class Solution {

    public List<String> restoreIpAddresses(String s) {

        List<String> res = new ArrayList<String>();
        int len = s.length();

        for (int i = 1; i < 4 && i < len - 2; i++) {
            for (int j = i + 1; j < i + 4 && j < len - 1; j++) {
                for (int k = j + 1; k < j + 4 && k < len; k++) {
                    String s1 = s.substring(0, i),
                            s2 = s.substring(i, j),
                            s3 = s.substring(j, k),
                            s4 = s.substring(k, len);

                    if (isValid(s1) && isValid(s2) && isValid(s3) && isValid(s4)) {
                        res.add(s1 + "." + s2 + "." + s3 + "." + s4);
                    }
                }
            }
        }
        return res;
    }

    public boolean isValid(String s) {
        if (s.length() > 3 || s.length() == 0 || (s.charAt(0) == '0' && s.length() > 1)
|| Integer.parseInt(s) > 255) {
            return false;
        }
        return true;
    }
}


/*
For an input = "25525511135"

The Strings evaluated are:
Evaluated: 2  5  5  25511135
```

```
Evaluated: 2  5  52  5511135
Evaluated: 2  5  525  511135
Evaluated: 2  55  2  5511135
Evaluated: 2  55  25  511135
Evaluated: 2  55  255  11135
Evaluated: 2  552  5  511135
Evaluated: 2  552  55  11135
Evaluated: 2  552  551  1135
Evaluated: 25  5  2  5511135
Evaluated: 25  5  25  511135
Evaluated: 25  5  255  11135
Evaluated: 25  52  5  511135
Evaluated: 25  52  55  11135
Evaluated: 25  52  551  1135
Evaluated: 25  525  5  11135
Evaluated: 25  525  51  1135
Evaluated: 25  525  511  135
Evaluated: 255  2  5  511135
Evaluated: 255  2  55  11135
Evaluated: 255  2  551  1135
Evaluated: 255  25  5  11135
Evaluated: 255  25  51  1135
Evaluated: 255  25  511  135
Evaluated: 255  255  1  1135
Evaluated: 255  255  11  135  <--- Correct IP address
Evaluated: 255  255  111  35  <--- Correct IP address

Result = ["255.255.11.135","255.255.111.35"]

Remaining are wrong IP addresses for given input conditions
*/
```

81. `94. Binary Tree Inorder Traversal.java`

```
/*
 *
 * Given a binary tree, return the inorder traversal of its nodes' values.
 *
 * Example:
 *
 * Input: [1,null,2,3]
 *     1
 *      \
 *       2
 *      /
 *     3
 *
 * Output: [1,3,2]
 *
 */

class Solution {

    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<Integer>();
        if (root != null) {
            result.addAll(inorderTraversal(root.left));
            result.add(root.val);
            result.addAll(inorderTraversal(root.right));
        }
        return result;
    }
}
```

82. `96. Unique Binary Search Trees.java`

```
/*
 * Given n, how many structurally unique BST's (binary search trees) that store values 1
... n?
 *
 * Example:
 *
 * Input: 3
 * Output: 5
 * Explanation:
 * Given n = 3, there are a total of 5 unique BST's:
 *
 * 1         3     3      2      1
 *  \       /     /      / \      \
 *   3     2     1      1   3      2
 *  /     /       \                 \
 * 2     1         2                 3
 *
 */

class Solution {

    public int numTrees(int n) {
        int[] dp = new int[n + 1];
        dp[0] = 1;
        dp[1] = 1;

        for (int i = 2; i <= n; ++i) {
            for (int j = 1; j <= i; ++j) {
                dp[i] += dp[j - 1] * dp[i - j];
            }
        }

        return dp[n];
    }
}

/*
Solution taken from:
https://github.com/bephrem1/backtobackswe/blob/master/Dynamic%20Programming%2C%20Recursio
n%2C%20%26%20Backtracking/countUniqueBinarySearchTrees.java

Reference video: https://www.youtube.com/watch?v=GgP75HAvrlY

For n = 3
Initial -> dp[0] = 1, dp[1] = 1

1. Iteration 1: i = 2, means we have a total of 2 nodes in our BST.
    i)  dp[2] = dp[2] + dp[0]*dp[1] = 0 + 1*1 = 1 <-- We have 0 node on left and 1 node
on right subtree out of the total 2 nodes, bcoz one node is going to be the root node so
that's why 0 nodes on left and 1 node on right.
    ii) dp[2] = dp[2] + dp[1]*dp[0] = 1 + 1*1 = 2 <-- We are having 1 node on left and 0
```

```
node on the right subtree of 2 nodes.
    The cases of the above one are:
    i)     1            ii)      2
            \                    /
             2                  1

2. Iteration 2: i = 3, means we have a total of 3 nodes in our BST.
    i)  dp[3] = dp[3] + dp[0]*dp[2] = 0 + 1*2 = 2 <-- We have 0 nodes on the left and 2
nodes on the right. Final answer is 2 which means that we have 2 combinations available
for 0 nodes on left and 2 nodes on right.
    ii) dp[3] = dp[3] + dp[1]*dp[1] = 2 + 1*1 = 3 <-- We have 1 nodes on the left and 1
nodes on the right. Final answer is 3 which means that we have 1 combinations available
for 1 nodes on left and 1 nodes on right, with addition of 2 previous combinations
    iii)  dp[3] = dp[3] + dp[2]*dp[0] = 3 + 2*1 = 5 <-- We have 2 nodes on the left and 0
nodes on the right. Final answer is 5 which means that we have total 2 combinations
available for 2 nodes on left and 0 nodes on right, with addition of 3 previous
combinations

     The cases of the above one are:
    i) 0 nodes on the left and 2 nodes on the right = 2 combinations
           1                  1
            \                  \
             2                  3
              \                /
               3              2
    ii) 1 nodes on the left and 1 nodes on the right = 1 combination
                2
               / \
              1   3
    iii) 2 nodes on the left and 0 nodes on the right = 2 combination
              3                  3
             /                  /
             1                 2
              \               /
               2             1

*/
```

83. `98. Validate Binary Search Tree.java`

```
/*
 *
 * Given a binary tree, determine if it is a valid binary search tree (BST).
 *
 * Assume a BST is defined as follows:
 *
 * The left subtree of a node contains only nodes with keys less than the node's key.
 * The right subtree of a node contains only nodes with keys greater than the node's key.
 * Both the left and right subtrees must also be binary search trees.
 * Example 1:
 *
 * Input:
 *   2
 *  / \
 * 1   3
 * Output: true
 * Example 2:
 *
 *     5
 *    / \
 *   1   4
 *  / \
 * 3   6
 * Output: false
 * Explanation: The input is: [5,1,4,null,null,3,6]. The root node's value
 * is 5 but its right child's value is 4.
 *
 *
 */

class Solution {

    public boolean isValidBST(TreeNode root) {
        return isValidBST(root, Double.NEGATIVE_INFINITY, Double.POSITIVE_INFINITY);
    }

    public boolean isValidBST(TreeNode p, double min, double max) {
        if (p == null) {
            return true;
        }

        if (p.val <= min || p.val >= max) {
            return false;
        }

        return isValidBST(p.left, min, p.val) && isValidBST(p.right, p.val, max);
    }
}
```

84. `100. Same Tree.java`

```
/*
 *
 * Given two binary trees, write a function to check if they are the same or not.
 *
 * Two binary trees are considered the same if they are structurally identical and the
nodes have the same value.
 *
 * Example 1:
 *
 * Input:     1         1
 *           / \       / \
 *          2   3     2   3
 *
 *         [1,2,3],   [1,2,3]
 *
 * Output: true
 * Example 2:
 *
 * Input:     1         1
 *           /           \
 *          2             2
 *
 *        [1,2],     [1,null,2]
 *
 * Output: false
 * Example 3:
 *
 * Input:     1         1
 *           / \       / \
 *          2   1     1   2
 *
 *         [1,2,1],   [1,1,2]
 *
 * Output: false
 *
 */

class Solution {

    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) {
            return true;
        }

        if (p == null || q == null) {
            return false;
        }

        if (p.val == q.val) {
            boolean b1 = isSameTree(p.left, q.left);
            boolean b2 = isSameTree(p.right, q.right);
```

```
            return b1 && b2;
        }
        return false; //In case all the above cases fail
    }

}
```

85. `101. Symmetric Tree.java`

```
/*
 * Given a binary tree, check whether it is a mirror of itself (ie, symmetric around its
center).
 *
 * For example, this binary tree [1,2,2,3,4,4,3] is symmetric:
 *
 *       1
 *     /  \
 *    2    2
 *   / \  / \
 *  3  4 4  3
 *
 * But the following [1,2,2,null,3,null,3] is not:
 *       1
 *      / \
 *     2   2
 *      \   \
 *      3    3
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public boolean isSymmetric(TreeNode root) {
        if (root == null) {
            return true;
        }

        return isSymmetric(root.left, root.right);

    }

    public boolean isSymmetric(TreeNode l, TreeNode r) {

        if (l == null && r == null) {
            return true;
        } else if (l == null || r == null) {
            return false;
        }
        if (l.val != r.val) {
            return false;
        }
```

```
        if (!isSymmetric(l.left, r.right)) {
            return false;
        }

        if (!isSymmetric(l.right, r.left)) {
            return false;
        }

        return true;
    }

}
```

86. `102. Binary Tree Level Order Traversal.java`

```
/*
 *
 * Given a binary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).
 *
 * For example:
 * Given binary tree [3,9,20,null,null,15,7],
 *     3
 *    / \
 *   9  20
 *  /    \
 * 15     7
 * return its level order traversal as:
 * [
 * [3],
 * [9,20],
 * [15,7]
 * ]
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> ans = new ArrayList<>();
        Queue<TreeNode> q = new LinkedList<>();
        if (root != null) {
            q.offer(root);
        }
        TreeNode cur;
        while (!q.isEmpty()) {
            int size = q.size();
            List<Integer> subAns = new LinkedList<Integer>();
```

```
            for (int i = 0; i < size; ++i) {        // traverse nodes in the same level
                cur = q.poll();
                subAns.add(cur.val);                // visit the root
                if (cur.left != null) {
                    q.offer(cur.left);              // push left child to queue if it is
not null
                }
                if (cur.right != null) {
                    q.offer(cur.right);             // push right child to queue if it is
not null
                }
            }
            ans.add(subAns);
        }
        return ans;
    }
}
```

87. `103. Binary Tree Zigzag Level Order Traversal.java`

```
/*
 * Given a binary tree, return the zigzag level order traversal of its nodes' values.
(ie, from left to right, then right to left for the next level and alternate between).
 *
 * For example:
 * Given binary tree [3,9,20,null,null,15,7],
 *      3
 *     / \
 *    9  20
 *   /     \
 *  15      7
 * return its zigzag level order traversal as:
 * [
 * [3],
 * [20,9],
 * [15,7]
 * ]
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {

        List<List<Integer>> result = new ArrayList<>();
        if (root == null) {
            return result;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        boolean zigZag = false;

        while (!queue.isEmpty()) {
            int size = queue.size();
            ArrayList<Integer> temp = new ArrayList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                temp.add(node.val);

                if (node.left != null) {
```

```
                    queue.offer(node.left);
                }
                if (node.right != null) {
                    queue.offer(node.right);
                }
            }

            if (zigZag) {
                Collections.reverse(temp);
            }

            result.add(temp);
            zigZag = zigZag ? false : true;
        }

        return result;

    }
}
```

88. `104. Maximum Depth of Binary Tree.java`

```
/*
 * Given a binary tree, find its maximum depth.
 *
 * The maximum depth is the number of nodes along the longest path from the root node
down to the farthest leaf node.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Given binary tree [3,9,20,null,null,15,7],
 *
 *      3
 *     / \
 *    9  20
 *   /    \
 *  15     7
 * return its depth = 3.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public int maxDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int left = maxDepth(root.left);// This will give me the left "subtree", not just
the left node value
        int right = maxDepth(root.right); //This will give me the right "subtree", not
just the right node value

        return Math.max(left, right) + 1;
    }
}
```

89. `107. Binary Tree Level Order Traversal II.java`

```
/*
 *
 * Given a binary tree, return the bottom-up level order traversal of its nodes' values.
(ie, from left to right, level by level from leaf to root).
 *
 * For example:
 * Given binary tree [3,9,20,null,null,15,7],
 *      3
 *     / \
 *    9  20
 *   /     \
 *  15      7
 * return its bottom-up level order traversal as:
 * [
 * [15,7],
 * [9,20],
 * [3]
 * ]
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public List<List<Integer>> levelOrderBottom(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();

        if (root == null) {
            return result;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        Stack<ArrayList<Integer>> stack = new Stack();

        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            ArrayList<Integer> list = new ArrayList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
```

```
                list.add(node.val);
                if (node.left != null) {
                    queue.offer(node.left);
                }
                if (node.right != null) {
                    queue.offer(node.right);
                }
            }

            stack.push(list);
        }

        while (!stack.isEmpty()) {
            result.add(stack.peek());
            stack.pop();
        }

        return result;

    }
}
```

90. `108. Convert Sorted Array to Binary Search`

Tree.java

```
/*
 *
 * Given an array where elements are sorted in ascending order, convert it to a height
balanced BST.
 *
 * For this problem, a height-balanced binary tree is defined as a binary tree in which
the depth of the two subtrees of every node never differ by more than 1.
 *
 * Example:
 *
 * Given the sorted array: [-10,-3,0,5,9],
 *
 * One possible answer is: [0,-3,9,-10,null,5], which represents the following height
balanced BST:
 *
 *       0
 *     /  \
 *   -3    9
 *   /     /
 * -10    5
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public TreeNode sortedArrayToBST(int[] num) {
        if (num.length == 0) {
            return null;
        }

        return sortedArrayToBST(num, 0, num.length - 1);
    }

    public TreeNode sortedArrayToBST(int[] num, int start, int end) {
        if (start > end) {
            return null;
        }

        int mid = (start + end) / 2;
        TreeNode root = new TreeNode(num[mid]);
        root.left = sortedArrayToBST(num, start, mid - 1);
        root.right = sortedArrayToBST(num, mid + 1, end);
```

```
        return root;
    }
}
```

91. `109. Convert Sorted List to Binary Search Tree.java`

```
/*
 *
 * Given a singly linked list where elements are sorted in ascending order, convert it to
a height balanced BST.
 *
 * For this problem, a height-balanced binary tree is defined as a binary tree in which
the depth of the two subtrees of every node never differ by more than 1.
 *
 * Example:
 *
 * Given the sorted linked list: [-10,-3,0,5,9],
 *
 * One possible answer is: [0,-3,9,-10,null,5], which represents the following height
balanced BST:
 *
 *      0
 *     / \
 *   -3   9
 *   /   /
 * -10  5
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public TreeNode sortedListToBST(ListNode head) {
        List<Integer> list = new ArrayList<>();
        while (head != null) {
            list.add(head.val);
            head = head.next;
        }
        return helper(list, 0, list.size() - 1);
    }
```

```
    private TreeNode helper(List<Integer> list, int low, int high) {

        if (low > high || list.isEmpty()) {
            return null;
        }
        int mid = low + (high - low) / 2;
        TreeNode root = new TreeNode(list.get(mid));
        root.left = helper(list, low, mid - 1);
        root.right = helper(list, mid + 1, high);
        return root;
    }
}
```

92. `110. Balanced Binary Tree.java`

```
/*
 *
 * Given a binary tree, determine if it is height-balanced.
 *
 * For this problem, a height-balanced binary tree is defined as:
 *
 * a binary tree in which the depth of the two subtrees of every node never differ by
more than 1.
 *
 * Example 1:
 *
 * Given the following tree [3,9,20,null,null,15,7]:
 *
 *      3
 *     / \
 *    9  20
 *   /     \
 *  15      7
 * Return true.
 *
 * Example 2:
 *
 * Given the following tree [1,2,2,3,3,null,null,4,4]:
 *
 *       1
 *      / \
 *     2   2
 *    / \
 *   3   3
 *  / \
 * 4   4
 * Return false.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public boolean isBalanced(TreeNode root) {
        if (root == null) {
            return true;
        }
        return height(root) != -1;
```

```
    }

    public int height(TreeNode node) {
        if (node == null) {
            return 0;
        }
        int lH = height(node.left);
        if (lH == -1) {
            return -1;
        }
        int rH = height(node.right);
        if (rH == -1) {
            return -1;
        }
        if (lH - rH < -1 || lH - rH > 1) {
            return -1;
        }
        return Math.max(lH, rH) + 1;
    }
}
```

93. `111. Minimum Depth of Binary Tree.java`

```
/*
 *
 * Given a binary tree, find its minimum depth.
 *
 * The minimum depth is the number of nodes along the shortest path from the root node
down to the nearest leaf node.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Given binary tree [3,9,20,null,null,15,7],
 *
 *     3
 *    / \
 *   9   20
 *  /     \
 * 15      7
 * return its minimum depth = 2.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public int minDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        return helper(root);
    }

    public int helper(TreeNode root) {
        if (root == null) {
            return Integer.MAX_VALUE;
        }
        if (root.left == null && root.right == null) {
            return 1;
        }

        int depthLeft = helper(root.left);
        int depthRight = helper(root.right);
```

```
        return Math.min(depthLeft, depthRight) + 1;
    }
}
```

94. `112. Path Sum.java`

```
/*
 *
 * Given a binary tree and a sum, determine if the tree has a root-to-leaf path such that
adding up all the values along the path equals the given sum.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Given the below binary tree and sum = 22,
 *
 *       5
 *      / \
 *     4   8
 *    /   / \
 *   11  13  4
 *  /  \      \
 * 7    2      1
 * return true, as there exist a root-to-leaf path 5->4->11->2 which sum is 22
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public boolean hasPathSum(TreeNode root, int sum) {
        if (root == null) {
            return false;
        }

        if (root.left == null && root.right == null && sum - root.val == 0) {
            return true;
        }

        return (hasPathSum(root.left, sum - root.val) || hasPathSum(root.right, sum -
root.val));

    }
}
```

95. `113. Path Sum II.java`

```
/*
 *
 * Given a binary tree and a sum, find all root-to-leaf paths where each path's sum
equals the given sum.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Given the below binary tree and sum = 22,
 *
 *        5
 *       / \
 *      4   8
 *     /   / \
 *    11  13  4
 *   /  \    / \
 *  7    2  5   1
 * Return:
 *
 * [
 * [5,4,11,2],
 * [5,8,4,5]
 * ]
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    private List<List<Integer>> result = new ArrayList<List<Integer>>();

    public List<List<Integer>> pathSum(TreeNode root, int sum) {
        helper(new ArrayList<Integer>(), root, sum);
        return result;
    }

    private void helper(List<Integer> list, TreeNode root, int sum) {
        if (root == null) {
            return;
        }
        list.add(root.val);
        sum -= root.val;
```

```
        if (root.left == null && root.right == null) {
            if (sum == 0) {
                result.add(list);
            }
            return;
        }
        helper(new ArrayList<Integer>(list), root.left, sum);
        helper(new ArrayList<Integer>(list), root.right, sum);
    }
}
```

96. `114. Flatten Binary Tree to Linked List.java`

```
/*
 * Given a binary tree, flatten it to a linked list in-place.
 *
 * For example, given the following tree:
 *
 *       1
 *      / \
 *     2   5
 *    / \   \
 *   3   4   6
 * The flattened tree should look like:
 *
 * 1
 *  \
 *   2
 *    \
 *     3
 *      \
 *       4
 *        \
 *         5
 *          \
 *           6
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public void flatten(TreeNode root) {
        if (root == null) {
            return;
        }
        Stack<TreeNode> stk = new Stack<TreeNode>();
        stk.push(root);
        while (!stk.isEmpty()) {
            TreeNode curr = stk.pop();
            if (curr.right != null) {
                stk.push(curr.right);
            }
            if (curr.left != null) {
                stk.push(curr.left);
            }
```

```
            if (!stk.isEmpty()) {
                curr.right = stk.peek();
            }
            curr.left = null;  // dont forget this!!
        }
    }
}
```

97. `116. Populating Next Right Pointers in Each`

Node.java

```
/*

You are given a perfect binary tree where all leaves are on the same level, and every
parent has two children. The binary tree has the following definition:

struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
Populate each next pointer to point to its next right node. If there is no next right
node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example:

Input: {"$id":"1","left":{"$id":"2","left":
{"$id":"3","left":null,"next":null,"right":null,"val":4},"next":null,"right":
{"$id":"4","left":null,"next":null,"right":null,"val":5},"val":2},"next":null,"right":
{"$id":"5","left":
{"$id":"6","left":null,"next":null,"right":null,"val":6},"next":null,"right":
{"$id":"7","left":null,"next":null,"right":null,"val":7},"val":3},"val":1}

Output: {"$id":"1","left":{"$id":"2","left":{"$id":"3","left":null,"next":
{"$id":"4","left":null,"next":{"$id":"5","left":null,"next":
{"$id":"6","left":null,"next":null,"right":null,"val":7},"right":null,"val":6},"right":nu
ll,"val":5},"right":null,"val":4},"next":{"$id":"7","left":
{"$ref":"5"},"next":null,"right":{"$ref":"6"},"val":3},"right":
{"$ref":"4"},"val":2},"next":null,"right":{"$ref":"7"},"val":1}

Explanation: Given the above perfect binary tree (Figure A), your function should
populate each next pointer to point to its next right node, just like in Figure B.
*/

/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val,Node _left,Node _right,Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
```

```
    }
};
*/
class Solution {
    public Node connect(Node root)
    {
         if(root == null)
            return null;

        Queue<Node> q = new LinkedList<>();
        q.add(root);
        int len = 1;

        while(!q.isEmpty()) {
            Node tmp = q.poll();
            len--;

            if(tmp.left != null)
                q.add(tmp.left);
            if(tmp.right != null)
                q.add(tmp.right);

            if(len == 0)
            {
                len = q.size();
                tmp.next = null;
            }
            else
                tmp.next = q.peek();
        }
        return root;
        /*
        Node level_start = root;
        while(level_start!=null)
        {
            Node cur = level_start;
            while(cur!=null)
            {
                if(cur.left!=null)
                    cur.left.next = cur.right;
                if(cur.right!=null && cur.next!=null)
                    cur.right.next = cur.next.left;
                cur = cur.next;
            }

            level_start = level_start.left;
        }
        return root;*/
    }
}
```

98. `117. Populating Next Right Pointers in Each Node`

II.java

```
/*

Given a binary tree

struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
Populate each next pointer to point to its next right node. If there is no next right
node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example:

Input: {"$id":"1","left":{"$id":"2","left":
{"$id":"3","left":null,"next":null,"right":null,"val":4},"next":null,"right":
{"$id":"4","left":null,"next":null,"right":null,"val":5},"val":2},"next":null,"right":
{"$id":"5","left":null,"next":null,"right":
{"$id":"6","left":null,"next":null,"right":null,"val":7},"val":3},"val":1}

Output: {"$id":"1","left":{"$id":"2","left":{"$id":"3","left":null,"next":
{"$id":"4","left":null,"next":
{"$id":"5","left":null,"next":null,"right":null,"val":7},"right":null,"val":5},"right":nu
ll,"val":4},"next":{"$id":"6","left":null,"next":null,"right":
{"$ref":"5"},"val":3},"right":{"$ref":"4"},"val":2},"next":null,"right":
{"$ref":"6"},"val":1}

Explanation: Given the above binary tree (Figure A), your function should populate each
next pointer to point to its next right node, just like in Figure B.

*/

/*
// Definition for a Node.
class Node {
    public int val;
    public Node left;
    public Node right;
    public Node next;

    public Node() {}

    public Node(int _val,Node _left,Node _right,Node _next) {
        val = _val;
        left = _left;
        right = _right;
        next = _next;
    }
```

```
};
*/
class Solution {
    public Node connect(Node root) {
        /*
            This solution can be used for the same problem Part I. Using level order traversal is going to be very useful.
            What we are doing is processing all the nodes of any particular level.
            We are processing each node without changing the left and right pointers. We are just changing each node's next pointer.
            If it the last node of that level, then make it's next pointer = null
            Otherwise, if it a general node in the level order traversal then we are going to just connect current node to the next node(by peeking in the queue and then using it as next pointer)

        */
        if(root == null)
            return null;

        Queue<Node> q = new LinkedList<>();
        q.add(root);
        int len = 1;

        while(!q.isEmpty()) {
            Node current = q.poll();
            len--;

            if(current.left != null)
                q.add(current.left);
            if(current.right != null)
                q.add(current.right);

            if(len == 0)
            {
                len = q.size();
                current.next = null;
            }
            else
                current.next = q.peek();
        }
        return root;
    }

}
```

99. `120. Triangle.java`

```
/*

Given a triangle, find the minimum path sum from top to bottom. Each step you may move to adjacent numbers on the row below.

For example, given the following triangle

[
     [2],
    [3,4],
   [6,5,7],
  [4,1,8,3]
]
The minimum path sum from top to bottom is 11 (i.e., 2 + 3 + 5 + 1 = 11).

Note:

Bonus point if you are able to do this using only O(n) extra space, where n is the total number of rows in the triangle.

*/

class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
    int size = triangle.size();
    int[] results = new int[size+1];

    for(int i=size-1; i>=0; i--) {
        List<Integer> tmp = triangle.get(i);

        for(int j=0; j<tmp.size(); j++) {
            results[j] = Math.min(results[j], results[j+1]) + tmp.get(j);
        }
    }
    return results[0];
}

}
```

100. `121. Best Time to Buy and Sell Stock.java`

```
/*
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 *
 * If you were only permitted to complete at most one transaction (i.e., buy one and sell
one share of the stock), design an algorithm to find the maximum profit.
 *
 * Note that you cannot sell a stock before you buy one.
 *
 * Example 1:
 *
 * Input: [7,1,5,3,6,4]
 * Output: 5
 * Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
 * Not 7-1 = 6, as selling price needs to be larger than buying price.
 * Example 2:
 *
 * Input: [7,6,4,3,1]
 * Output: 0
 * Explanation: In this case, no transaction is done, i.e. max profit = 0.
 *
 */

class Solution {

    public int maxProfit(int[] prices) {

        if (prices.length == 0) {
            return 0; //Base case
        }

        int min = prices[0];
        int profit = 0;

        for (int i = 1; i < prices.length; i++) {
            if (prices[i] < min) {

                min = prices[i];
            } else if (prices[i] - min > profit) {
                profit = prices[i] - min;
            }
        }

        return profit;

    }
}
```

101. 122. Best Time to Buy and Sell Stock II.java

```
/*
 *
 * Say you have an array for which the ith element is the price of a given stock on day
i.
 *
 * Design an algorithm to find the maximum profit. You may complete as many transactions
as you like (i.e., buy one and sell one share of the stock multiple times).
 *
 * Note: You may not engage in multiple transactions at the same time (i.e., you must
sell the stock before you buy again).
 *
 * Example 1:
 *
 * Input: [7,1,5,3,6,4]
 * Output: 7
 * Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
 * Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
 * Example 2:
 *
 * Input: [1,2,3,4,5]
 * Output: 4
 * Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
 * Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are
 * engaging multiple transactions at the same time. You must sell before buying again.
 * Example 3:
 *
 * Input: [7,6,4,3,1]
 * Output: 0
 * Explanation: In this case, no transaction is done, i.e. max profit = 0.
 *
 */

class Solution {

    public int maxProfit(int[] prices) {
        int profit = 0;

        for (int i = 1; i < prices.length; i++) {
            int diff = prices[i] - prices[i - 1];
            if (diff > 0) {
                profit += diff;
            }
        }
        return profit;
    }
}
```

102. `125. Valid Palindrome.java`

```
/*
 *
 * Given a string, determine if it is a palindrome, considering only alphanumeric
characters and ignoring cases.
 *
 * Note: For the purpose of this problem, we define empty string as valid palindrome.
 *
 * Example 1:
 *
 * Input: "A man, a plan, a canal: Panama"
 * Output: true
 * Example 2:
 *
 * Input: "race a car"
 * Output: false
 *
 */

class Solution {

    public boolean isPalindrome(String s) {

        /*
         * Approach 1: Simple Solution by combining strings StringBuilder
         *
         * for(int i =0;i<s.length();i++) {
         * if(Character.isLetterOrDigit(s.charAt(i))) sb.append(s.charAt(i)); }
         *
         * String input = sb.toString().toLowerCase(); int start = 0, end =
         * input.length()-1;
         *
         * while(start <= end) { if(input.charAt(start)!=input.charAt(end))
         * return false; start++; end--; }
         *
         * return true;
         */

        // Approach 2:
        if (s.isEmpty()) {
            return true;
        }

        int start = 0, end = s.length() - 1;
        char cHead, cTail;

        while (start <= end) {
            cHead = s.charAt(start);
            cTail = s.charAt(end);

            if (!Character.isLetterOrDigit(cHead)) {
                start++;
```

```
            } else if (!Character.isLetterOrDigit(cTail)) {
                end--;
            } else {
                if (Character.toLowerCase(cHead) != Character.toLowerCase(cTail)) {
                    return false;
                }

                start++;
                end--;
            }
        }

        return true;
    }
}
```

103. `127. Word Ladder.java`

```
/*
 *
 * Given two words (beginWord and endWord), and a dictionary's word list, find the length
of shortest transformation sequence from beginWord to endWord, such that:
 *
 * Only one letter can be changed at a time.
 * Each transformed word must exist in the word list. Note that beginWord is not a
transformed word.
 * Note:
 *
 * Return 0 if there is no such transformation sequence.
 * All words have the same length.
 * All words contain only lowercase alphabetic characters.
 * You may assume no duplicates in the word list.
 * You may assume beginWord and endWord are non-empty and are not the same.
 * Example 1:
 *
 * Input:
 * beginWord = "hit",
 * endWord = "cog",
 * wordList = ["hot","dot","dog","lot","log","cog"]
 *
 * Output: 5
 *
 * Explanation: As one shortest transformation is "hit" -> "hot" -> "dot" -> "dog" ->
"cog",
 * return its length 5.
 * Example 2:
 *
 * Input:
 * beginWord = "hit"
 * endWord = "cog"
 * wordList = ["hot","dot","dog","lot","log"]
 *
 * Output: 0
 *
 * Explanation: The endWord "cog" is not in wordList, therefore no possible
transformation.
 *
 */

class Solution {

    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        HashSet<String> set = new HashSet<>(wordList);
        Queue<String> queue = new LinkedList<>();
        queue.offer(beginWord);
        int steps = 1;

        while (!queue.isEmpty()) {
            int size = queue.size();
```

```
            for (int i = 0; i < size; i++) {
                String curr = queue.poll();
                if (curr.equals(endWord)) {
                    return steps;
                }

                for (int j = 0; j < curr.length(); j++) {
                    char[] chars = curr.toCharArray();
                    for (char ch = 'a'; ch <= 'z'; ch++) {
                        chars[j] = ch;
                        String sb = new String(chars);
                        if (set.contains(sb)) {
                            queue.offer(sb);
                            set.remove(sb);
                        }
                    }
                }
            }//end of for loop
            steps++;
        }
        return 0;
    }
}
```

104. `129. Sum Root to Leaf Numbers.java`

```
/*
 *
 * Given a binary tree containing digits from 0-9 only, each root-to-leaf path could
represent a number.
 *
 * An example is the root-to-leaf path 1->2->3 which represents the number 123.
 *
 * Find the total sum of all root-to-leaf numbers.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Input: [1,2,3]
 *    1
 *   / \
 *  2   3
 * Output: 25
 * Explanation:
 * The root-to-leaf path 1->2 represents the number 12.
 * The root-to-leaf path 1->3 represents the number 13.
 * Therefore, sum = 12 + 13 = 25.
 * Example 2:
 *
 * Input: [4,9,0,5,1]
 *     4
 *    / \
 *   9   0
 *  / \
 * 5   1
 * Output: 1026
 * Explanation:
 * The root-to-leaf path 4->9->5 represents the number 495.
 * The root-to-leaf path 4->9->1 represents the number 491.
 * The root-to-leaf path 4->0 represents the number 40.
 * Therefore, sum = 495 + 491 + 40 = 1026.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public int sumNumbers(TreeNode root) {
```

```
        return sum(root, 0);//Recursively call the helper function starting with the sum
= 0
    }

    public int sum(TreeNode n, int s) {
        if (n == null) {
            return 0;//If we reach the end of the tree then return 0
        }
        if (n.right == null && n.left == null) {
            return s * 10 + n.val;//reached the leaf node, so no more combinations
possible. So return after calculating the final sum
        }
        return sum(n.left, s * 10 + n.val) + sum(n.right, s * 10 + n.val); //In all other
cases recursively call the left and right subtree with the updated sum as the starting
point for future calculations
    }
}
```

105. `130. Surrounded Regions.java`

```
/*
 *
 * Given a 2D board containing 'X' and 'O' (the letter O), capture all regions surrounded
by 'X'.
 *
 * A region is captured by flipping all 'O's into 'X's in that surrounded region.
 *
 * Example:
 *
 * X X X X
 * X O O X
 * X X O X
 * X O X X
 * After running your function, the board should be:
 *
 * X X X X
 * X X X X
 * X X X X
 * X O X X
 * Explanation:
 *
 * Surrounded regions shouldn’t be on the border, which means that any 'O' on the border
of the board are not flipped to 'X'. Any 'O' that is not on the border and it is not
connected to an 'O' on the border will be flipped to 'X'. Two cells are connected if they
are adjacent cells connected horizontally or vertically.
 *
 */

class Solution {

    public void solve(char[][] board) {
        if (board == null || board.length == 0) {
            return;
        }

        int m = board.length;
        int n = board[0].length;

        //merge O's on left & right boarder
        for (int i = 0; i < m; i++) {
            if (board[i][0] == 'O') {
                merge(board, i, 0);
            }

            if (board[i][n - 1] == 'O') {
                merge(board, i, n - 1);
            }
        }

        //merge O's on top & bottom boarder
        for (int j = 0; j < n; j++) {
```

```
            if (board[0][j] == 'O') {
                merge(board, 0, j);
            }

            if (board[m - 1][j] == 'O') {
                merge(board, m - 1, j);
            }
        }

        //process the board
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == 'O') {
                    board[i][j] = 'X';
                } else if (board[i][j] == '#') {
                    board[i][j] = 'O';
                }
            }
        }
    }

    public void merge(char[][] board, int i, int j) {
        if (i < 0 || i >= board.length || j < 0 || j >= board[0].length) {
            return;
        }

        if (board[i][j] != 'O') {
            return;
        }

        board[i][j] = '#';

        merge(board, i - 1, j);
        merge(board, i + 1, j);
        merge(board, i, j - 1);
        merge(board, i, j + 1);
    }
}
```

106. `131. Palindrome Partitioning.java`

```
/*
 *
 * Given a string s, partition s such that every substring of the partition is a
palindrome.
 *
 * Return all possible palindrome partitioning of s.
 *
 * Example:
 *
 * Input: "aab"
 * Output:
 * [
 * ["aa","b"],
 * ["a","a","b"]
 * ]
 *
 */

class Solution {

    public List<List<String>> partition(String s) {
        List<List<String>> list = new ArrayList<>();
        backtrack(list, new ArrayList<>(), s, 0);
        return list;
    }

    public void backtrack(List<List<String>> list, List<String> tempList, String s, int
start) {
        if (start == s.length()) {
            list.add(new ArrayList<>(tempList));
        } else {
            for (int i = start; i < s.length(); i++) {
                if (isPalindrome(s, start, i)) {
                    tempList.add(s.substring(start, i + 1));
                    backtrack(list, tempList, s, i + 1);
                    tempList.remove(tempList.size() - 1);
                }
            }
        }
    }

    public boolean isPalindrome(String s, int low, int high) {
        while (low < high) {
            if (s.charAt(low) != s.charAt(high)) {
                return false;
            }
            low++;
            high--;

        }
        return true;
```

```
    }
}
```

107. `133. Clone Graph.java`

```
/*
 *
 * Given a reference of a node in a connected undirected graph, return a deep copy
(clone) of the graph. Each node in the graph contains a val (int) and a list (List[Node])
of its neighbors.
 *
 * Example:
 *
 * Input:
 * {"$id":"1","neighbors":[{"$id":"2","neighbors":[{"$ref":"1"},{"$id":"3","neighbors":
[{"$ref":"2"},{"$id":"4","neighbors":[{"$ref":"3"},
{"$ref":"1"}],"val":4}],"val":3}],"val":2},{"$ref":"4"}],"val":1}
 *
 * Explanation:
 * Node 1's value is 1, and it has two neighbors: Node 2 and 4.
 * Node 2's value is 2, and it has two neighbors: Node 1 and 3.
 * Node 3's value is 3, and it has two neighbors: Node 2 and 4.
 * Node 4's value is 4, and it has two neighbors: Node 1 and 3.
 *
 *
 * Note:
 *
 * The number of nodes will be between 1 and 100.
 * The undirected graph is a simple graph, which means no repeated edges and no self-
loops in the graph.
 * Since the graph is undirected, if node p has node q as neighbor, then node q must have
node p as neighbor too.
 * You must return the copy of the given node as a reference to the cloned graph.
 *
 */

 /*
 * // Definition for a Node.
 * class Node {
 * public int val;
 * public List<Node> neighbors;
 *
 * public Node() {}
 *
 * public Node(int _val,List<Node> _neighbors) {
 * val = _val;
 * neighbors = _neighbors;
 * }
 * };
 */
class Solution {

    public Node cloneGraph(Node node) {

        if (node == null) {
            return null;
```

```
        }
        Map<Node, Node> map = new HashMap<Node, Node>();

        Queue<Node> q = new LinkedList<Node>();
        map.put(node, new Node(node.val, new ArrayList<Node>()));
        q.add(node);

        while (!q.isEmpty()) {
            Node top = q.remove();
            for (Node neighbor : top.neighbors) {
                // visited
                if (!map.containsKey(neighbor)) {
                    map.put(neighbor, new Node(neighbor.val, new ArrayList<Node>()));
                    q.add(neighbor);
                }
                map.get(top).neighbors.add(map.get(neighbor));
            }
        }
        return map.get(node);

    }
}
```

108. `136. Single Number.java`

```
/*
 *
 * Given a non-empty array of integers, every element appears twice except for one. Find
that single one.
 *
 * Note:
 *
 * Your algorithm should have a linear runtime complexity. Could you implement it without
using extra memory?
 *
 * Example 1:
 *
 * Input: [2,2,1]
 * Output: 1
 * Example 2:
 *
 * Input: [4,1,2,1,2]
 * Output: 4
 *
 */

class Solution {

    static int singleNumber(int[] nums) {
        HashMap<Integer, Integer> map = new HashMap<Integer, Integer>();

        for (int i : nums) {
            if (map.containsKey(i)) {
                map.put(i, map.get(i) + 1);
            } else {
                map.put(i, 1);
            }
        }

        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (entry.getValue() == 1) {
                return entry.getKey();

            }

        }
        return 0;
    }
}
```

109. `138. Copy List with Random Pointer.java`

```
/*
 *
 * A linked list is given such that each node contains an additional random pointer which could point to any node in the list or null.
 *
 * Return a deep copy of the list.  *
 * Example 1:
 *
 * Input:
 * {"$id":"1","next":{"$id":"2","next":null,"random":{"$ref":"2"},"val":2},"random":{"$ref":"2"},"val":1}
 *
 * Explanation:
 * Node 1's value is 1, both of its next and random pointer points to Node 2.
 * Node 2's value is 2, its next pointer points to null and its random pointer points to itself.
 *
 *
 * Note:
 *
 * You must return the copy of the given head as a reference to the cloned list.
 *
 */

/**
 * Definition for singly-linked list with a random pointer.
 * class RandomListNode {
 *     int label;
 *     RandomListNode next, random;
 *     RandomListNode(int x) { this.label = x; }
 * };
 */
public class Solution {

    public RandomListNode copyRandomList(RandomListNode head) {
        if (head == null) {
            return null;
        }

        Map<RandomListNode, RandomListNode> map = new HashMap<RandomListNode, RandomListNode>();

        // loop 1. copy all the nodes
        RandomListNode node = head;
        while (node != null) {
            map.put(node, new RandomListNode(node.label));
            node = node.next;
        }

        // loop 2. assign next and random pointers
        node = head;
```

```
        while (node != null) {
            map.get(node).next = map.get(node.next);
            map.get(node).random = map.get(node.random);
            node = node.next;
        }

        return map.get(head);
    }
}
```

110. `139. Word Break.java`

```
/*
 * Given a non-empty string s and a dictionary wordDict containing a list of non-empty
words, determine if s can be segmented into a space-separated sequence of one or more
dictionary words.
 *
 * Note:
 *
 * The same word in the dictionary may be reused multiple times in the segmentation.
 * You may assume the dictionary does not contain duplicate words.
 * Example 1:
 *
 * Input: s = "leetcode", wordDict = ["leet", "code"]
 * Output: true
 * Explanation: Return true because "leetcode" can be segmented as "leet code".
 * Example 2:
 *
 * Input: s = "applepenapple", wordDict = ["apple", "pen"]
 * Output: true
 * Explanation: Return true because "applepenapple" can be segmented as "apple pen
apple".
 * Note that you are allowed to reuse a dictionary word.
 * Example 3:
 *
 * Input: s = "catsandog", wordDict = ["cats", "dog", "sand", "and", "cat"]
 * Output: false
 *
 */

class Solution {

    public boolean wordBreak(String s, List<String> wordDict1) {

        Set<String> wordDict = new HashSet(wordDict1);
        if (s == null && wordDict == null) {
            return true;
        }
        if (s == null || wordDict == null) {
            return false;
        }
        //dp[i] represents if s.substring(0, i) is wordbreakable.
        boolean[] dp = new boolean[s.length() + 1];
        dp[0] = true;
        for (int i = 1; i <= s.length(); i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && wordDict.contains(s.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }
        return dp[s.length()];
```

```
    }
}
```

111. `141. Linked List Cycle.java`

```
/*
 *
 * Given a linked list, determine if it has a cycle in it.
 *
 * To represent a cycle in the given linked list, we use an integer pos which represents
the position (0-indexed) in the linked list where tail connects to. If pos is -1, then
there is no cycle in the linked list.
 *
 *
 *
 * Example 1:
 *
 * Input: head = [3,2,0,-4], pos = 1
 * Output: true
 * Explanation: There is a cycle in the linked list, where tail connects to the second
node.
 *
 *
 * Example 2:
 *
 * Input: head = [1,2], pos = 0
 * Output: true
 * Explanation: There is a cycle in the linked list, where tail connects to the first
node.
 *
 *
 * Example 3:
 *
 * Input: head = [1], pos = -1
 * Output: false
 * Explanation: There is no cycle in the linked list.
 *
 */

/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {

    public boolean hasCycle(ListNode head) {
        ListNode fast = head;
        ListNode slow = head;
```

```
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                return true;
            }
        }

        return false;

    }
```

112. `142. Linked List Cycle II.java`

```
/*
 *
 * Given a linked list, return the node where the cycle begins. If there is no cycle,
return null.
 *
 * To represent a cycle in the given linked list, we use an integer pos which represents
the position (0-indexed) in the linked list where tail connects to. If pos is -1, then
there is no cycle in the linked list.
 *
 * Note: Do not modify the linked list.
 *
 *
 *
 * Example 1:
 *
 * Input: head = [3,2,0,-4], pos = 1
 * Output: tail connects to node index 1
 * Explanation: There is a cycle in the linked list, where tail connects to the second
node.
 *
 *
 * Example 2:
 *
 * Input: head = [1,2], pos = 0
 * Output: tail connects to node index 0
 * Explanation: There is a cycle in the linked list, where tail connects to the first
node.
 *
 *
 * Example 3:
 *
 * Input: head = [1], pos = -1
 * Output: no cycle
 * Explanation: There is no cycle in the linked list.
 *
 */

/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {

    public ListNode detectCycle(ListNode head) {
```

```
        // Find the collision point if the list has a cycle
        if (head == null) {
            return null;
        }

        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;

            if (slow == fast) {
                break;
            }
        }

        // check if the list has no cycle
        if (fast == null || fast.next == null) {
            return null;
        }

        // find the entrance of the cycle
        slow = head;
        while (slow != fast) {
            slow = slow.next;
            fast = fast.next;
        }
        return slow;
    }
}
```

113. `143. Reorder List.java`

```
/*
 *
 * Given a singly linked list L: L0→L1→…→Ln-1→Ln,
 * reorder it to: L0→Ln→L1→Ln-1→L2→Ln-2→…
 *
 * You may not modify the values in the list's nodes, only nodes itself may be changed.
 *
 * Example 1:
 *
 * Given 1->2->3->4, reorder it to 1->4->2->3.
 * Example 2:
 *
 * Given 1->2->3->4->5, reorder it to 1->5->2->4->3.
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
public class Solution {

    public void reorderList(ListNode head) {
        if (head == null) {
            return;
        }
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode mid = slow, cur = slow.next;
        if (cur != null) {
            ListNode tmp = cur.next;
            cur.next = null;
            cur = tmp;
        }

        while (cur != null) {
            ListNode tmp = cur.next;
            cur.next = mid.next;
            mid.next = cur;
            cur = tmp;
        }

        ListNode left = head, right = mid.next;
```

```
        while (right != null) {
            mid.next = right.next;
            right.next = left.next;
            left.next = right;
            left = right.next;
            right = mid.next;
        }

    }
}
```

114. 144. Binary Tree Preorder Traversal.java

```
/*
 * Given a binary tree, return the preorder traversal of its nodes' values.
 *
 * Example:
 *
 * Input: [1,null,2,3]
 *   1
 *    \
 *     2
 *    /
 *   3
 *
 * Output: [1,2,3]
 *
 */

class Solution {

    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> pre = new LinkedList<Integer>();
        if (root == null) {
            return pre;
        }
        pre.add(root.val);
        pre.addAll(preorderTraversal(root.left));
        pre.addAll(preorderTraversal(root.right));
        return pre;
    }
}
```

115. `145. Binary Tree Postorder Traversal.java`

```
/*
 *
 * Given a binary tree, return the postorder traversal of its nodes' values.
 *
 * Example:
 *
 * Input: [1,null,2,3]
 *               1
 *                \
 *         2
 *        /
 *           3
 *
 * Output: [3,2,1]
 *
 */

class Solution {

    public List<Integer> postorderTraversal(TreeNode root) {
        List<Integer> result = new ArrayList<Integer>();
        if (root != null) {
            result.addAll(postorderTraversal(root.left));

            result.addAll(postorderTraversal(root.right));
            result.add(root.val);
        }
        return result;
    }
}
```

116. `146. LRU Cache.java`

```
/*
 *
 * Design and implement a data structure for Least Recently Used (LRU) cache. It should
support the following operations: get and put.
 *
 * get(key) - Get the value (will always be positive) of the key if the key exists in the
cache, otherwise return -1.
 * put(key, value) - Set or insert the value if the key is not already present. When the
cache reached its capacity, it should invalidate the least recently used item before
inserting a new item.
 *
 * Follow up:
 * Could you do both operations in O(1) time complexity?
 *
 * Example:
 *
 * LRUCache cache = new LRUCache( 2 // capacity  );
 *
 * cache.put(1, 1);
 * cache.put(2, 2);
 * cache.get(1);       // returns 1
 * cache.put(3, 3);    // evicts key 2
 * cache.get(2);       // returns -1 (not found)
 * cache.put(4, 4);    // evicts key 1
 * cache.get(1);       // returns -1 (not found)
 * cache.get(3);       // returns 3
 * cache.get(4);       // returns 4
 *
 */

public class LRUCache {

    int capacity;
    HashMap<Integer, Node> map = new HashMap<Integer, Node>();
    Node head = null;
    Node end = null;

    public LRUCache(int capacity) {
        this.capacity = capacity;
    }

    public int get(int key) {
        if (map.containsKey(key)) {
            Node n = map.get(key);
            remove(n);
            setHead(n);
            return n.value;
        }

        return -1;
    }
```

```
    public void remove(Node n) {
        if (n.pre != null) {
            n.pre.next = n.next;
        } else {
            head = n.next;
        }

        if (n.next != null) {
            n.next.pre = n.pre;
        } else {
            end = n.pre;
        }

    }

    public void setHead(Node n) {
        n.next = head;
        n.pre = null;

        if (head != null) {
            head.pre = n;
        }

        head = n;

        if (end == null) {
            end = head;
        }
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) { //Key already exists so just update the head
            Node old = map.get(key);
            old.value = value;
            remove(old);
            setHead(old);
        } else {
            Node created = new Node(key, value);
            if (map.size() >= capacity)//Key doesn't exists but no space left in cache.
So add a new node and remove the least used node!
            {
                map.remove(end.key);
                remove(end);
                setHead(created);
            } else { //Enough space in the cache to add a new element. Add element and
assign it as the head since it is the most recently used element
                setHead(created);
            }

            map.put(key, created);
        }
    }
}
```

```
class Node {

    int key;
    int value;
    Node pre;
    Node next;

    public Node(int key, int value) {
        this.key = key;
        this.value = value;
    }
}

/**
 * Your LRUCache object will be instantiated and called as such:
 * LRUCache obj = new LRUCache(capacity);
 * int param_1 = obj.get(key);
 * obj.put(key,value);
 */
```

117. `147. Insertion Sort List.java`

```
/*

Sort a linked list using insertion sort.

Algorithm of Insertion Sort:

Insertion sort iterates, consuming one input element each repetition, and growing a
sorted output list.
At each iteration, insertion sort removes one element from the input data, finds the
location it belongs within the sorted list, and inserts it there.
It repeats until no input elements remain.

Example 1:

Input: 4->2->1->3
Output: 1->2->3->4
Example 2:

Input: -1->5->3->4->0
Output: -1->0->3->4->5

*/

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode insertionSortList(ListNode head) {
        ListNode dummy = new ListNode(0); // use dummyHead to create the resultant list
        ListNode cur = head;

        while (cur != null) {
            ListNode p = dummy; // find the correct insertion point starting from the
resultant list first node!

            //Move to the node which will be less than the current node. Skip nodes in
the current iteration which are less than the current node. Once you get a value which is
more than the current node then add that node to the resultant list!
            while (p.next != null && p.next.val < cur.val) {
                p = p.next;
            }

            //Inserting the current node to the resultant list
            ListNode next = cur.next;
            cur.next = p.next;
```

```
            p.next = cur;
            cur = next;
        }
        return dummy.next;
    }

    /*
    Eg: Input: 4 -> 2 -> 3 -> 1
    Initially: Resultant list = 0 -> null
                                |
                              dummy
               List = 4 -> 2 -> 1 -> 3
                      |    |
                    curr  next

    Iteration 1: We are going to directly insert into to our resultant list since the
while loop isn't true
                 So we get
                 Resultant list = 0 -> 4 -> null
                                  |
                                  p
               List = 2 -> 3 -> 1
                      |    |
                    curr  next

    Iteration 2: Now here we check:
                 i) Compare p.next(4) < curr(2) is false, so we are going to skip that
while loop and directly insert in between the nodes 0 and 4 in the resultant list
               Resultant list = 0 -> 2 -> 4 -> null
                                 |
                                 p
               List = 3 -> 1
                      |    |
                    curr  next
    Iteration 3: Now we check:
                 i) Compare p.next(2) < curr(3) is true so we are going to move to the
next node in the resultant list.
                 ii) Compare p.next(4) < curr(3) is false, so now we are going to skip the
while loop and insert the current node in between the node = 2 and node = 4
                 Resultant list = 0 -> 2 -> 3 -> 4 -> null
                                  |
                                  p
                 List = 1 -> null
                        |    |
                      curr  next
    Iteration 4:  Now here we check:
                 i) Compare p.next(2) < curr(1) is false, so we are going to skip that
while loop and directly insert in between the nodes 0 and 2 in the resultant list
                 Resultant list = 0 -> 1 -> 2 -> 3 -> 4 -> null
                                  |
                                  p
                 List = null
                         |
                       curr
```

```
            Now list has nothing so we are going to end the iteration and we have our
final resultant list as well

    */

}
```

118. `148. Sort List.java`

```
/*
 *
 * Sort a linked list in O(n log n) time using constant space complexity.
 *
 * Example 1:
 *
 * Input: 4->2->1->3
 * Output: 1->2->3->4
 * Example 2:
 *
 * Input: -1->5->3->4->0
 * Output: -1->0->3->4->5
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
public class Solution {

    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        // step 1. cut the list to two halves
        ListNode prev = null, slow = head, fast = head;

        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }

        prev.next = null;

        // step 2. sort each half
        ListNode l1 = sortList(head);
        ListNode l2 = sortList(slow);

        // step 3. merge l1 and l2
        return merge(l1, l2);
    }

    ListNode merge(ListNode l1, ListNode l2) {
        ListNode l = new ListNode(0), p = l;
```

```
        while (l1 != null && l2 != null) {
            if (l1.val < l2.val) {
                p.next = l1;
                l1 = l1.next;
            } else {
                p.next = l2;
                l2 = l2.next;
            }
            p = p.next;
        }

        if (l1 != null) {
            p.next = l1;
        }

        if (l2 != null) {
            p.next = l2;
        }

        return l.next;
    }

}
```

119. `151. Reverse Words in a String.java`

```
/*
 *
 * Given an input string, reverse the string word by word.
 *
 * Example 1:
 *
 * Input: "the sky is blue"
 * Output: "blue is sky the"
 * Example 2:
 *
 * Input: "  hello world!  "
 * Output: "world! hello"
 * Explanation: Your reversed string should not contain leading or trailing spaces.
 * Example 3:
 *
 * Input: "a good   example"
 * Output: "example good a"
 * Explanation: You need to reduce multiple spaces between two words to a single space in
the reversed string.
 *
 *
 * Note:
 *
 * A word is defined as a sequence of non-space characters.
 * Input string may contain leading or trailing spaces. However, your reversed string
should not contain leading or trailing spaces.
 * You need to reduce multiple spaces between two words to a single space in the reversed
string.
 *
 */

public class Solution {

    public static String reverseWords(String s) {

        StringBuilder sb = new StringBuilder();
        String[] words = s.trim().split("\\s+");

        for (int i = words.length - 1; i >= 0; i--) {
            sb.append(words[i]);
            sb.append(" ");
        }

        return sb.toString().substring(0, sb.length() - 1);
    }
}
```

120. `152. Maximum Product Subarray.java`

```
/*
 *
 * Given an integer array nums, find the contiguous subarray within an array (containing
at least one number) which has the largest product.
 *
 * Example 1:
 *
 * Input: [2,3,-2,4]
 * Output: 6
 * Explanation: [2,3] has the largest product 6.
 * Example 2:
 *
 * Input: [-2,0,-1]
 * Output: 0
 * Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
 *
 */

class Solution {

    public int maxProduct(int[] nums) {
        int[] max = new int[nums.length];
        int[] min = new int[nums.length];
        max[0] = min[0] = nums[0];
        int result = nums[0];
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] > 0) {
                max[i] = Math.max(nums[i], max[i - 1] * nums[i]);
                min[i] = Math.min(nums[i], min[i - 1] * nums[i]);
            } else {
                max[i] = Math.max(nums[i], min[i - 1] * nums[i]);
                min[i] = Math.min(nums[i], max[i - 1] * nums[i]);
            }
            result = Math.max(result, max[i]);
        }
        return result;
    }
}
```

121. `153. Find Minimum in Rotated Sorted Array.java`

```
/*
 *
 * Suppose an array sorted in ascending order is rotated at some pivot unknown to you
beforehand.
 *
 * (i.e.,  [0,1,2,4,5,6,7] might become  [4,5,6,7,0,1,2]).
 *
 * Find the minimum element.
 *
 * You may assume no duplicate exists in the array.
 *
 * Example 1:
 *
 * Input: [3,4,5,1,2]
 * Output: 1
 * Example 2:
 *
 * Input: [4,5,6,7,0,1,2]
 * Output: 0
 *
 */

class Solution {

    public int findMin(int[] num) {
        return findMin(num, 0, num.length - 1);
    }

    public int findMin(int[] num, int left, int right) {
        if (left == right) {
            return num[left];
        }
        if ((right - left) == 1) {
            return Math.min(num[left], num[right]);
        }

        int middle = left + (right - left) / 2;

        // not rotated
        if (num[left] < num[right]) {
            return num[left];

            // go right side
        } else if (num[middle] > num[left]) {
            return findMin(num, middle, right);

            // go left side
        } else {
            return findMin(num, left, middle);
        }
```

```
    }
}
```

122. `155. Min Stack.java`

```
/*
 *
 * Design a stack that supports push, pop, top, and retrieving the minimum element in
constant time.
 *
 * push(x) -- Push element x onto stack.
 * pop() -- Removes the element on top of the stack.
 * top() -- Get the top element.
 * getMin() -- Retrieve the minimum element in the stack.
 * Example:
 * MinStack minStack = new MinStack();
 * minStack.push(-2);
 * minStack.push(0);
 * minStack.push(-3);
 * minStack.getMin();   --> Returns -3.
 * minStack.pop();
 * minStack.top();      --> Returns 0.
 * minStack.getMin();   --> Returns -2.
 *
 */

class MinStack {

    int min = Integer.MAX_VALUE;
    Stack<Integer> stack = new Stack<Integer>();

    public void push(int x) {
        // only push the old minimum value when the current
        // minimum value changes after pushing the new value x
        if (x <= min) {
            stack.push(min);
            min = x;
        }
        stack.push(x);
    }

    public void pop() {
        // if pop operation could result in the changing of the current minimum value,
        // pop twice and change the current minimum value to the last minimum value.
        if (stack.pop() == min) {
            min = stack.pop();
        }
    }

    public int top() {
        return stack.peek();
    }

    public int getMin() {
        return min;
    }
```

```
}

/**
 * Your MinStack object will be instantiated and called as such: MinStack obj =
 * new MinStack(); obj.push(x); obj.pop(); int param_3 = obj.top(); int param_4
 * = obj.getMin();
 */

/*

Approach 6: Boyer-Moore Voting Algorithm
Intuition

If we had some way of counting instances of the majority element as +1+1 and instances of any other element as -1−1, summing them would make it obvious that the majority element is indeed the majority element.

Algorithm

Essentially, what Boyer-Moore does is look for a suffix sufsuf of nums where suf[0]suf[0] is the majority element in that suffix. To do this, we maintain a count, which is incremented whenever we see an instance of our current candidate for majority element and decremented whenever we see anything else. Whenever count equals 0, we effectively forget about everything in nums up to the current index and consider the current number as the candidate for majority element. It is not immediately obvious why we can get away with forgetting prefixes of nums - consider the following examples (pipes are inserted to separate runs of nonzero count).

[7, 7, 5, 7, 5, 1 | 5, 7 | 5, 5, 7, 7 | 7, 7, 7, 7]

Here, the 7 at index 0 is selected to be the first candidate for majority element. count will eventually reach 0 after index 5 is processed, so the 5 at index 6 will be the next candidate. In this case, 7 is the true majority element, so by disregarding this prefix, we are ignoring an equal number of majority and minority elements - therefore, 7 will still be the majority element in the suffix formed by throwing away the first prefix.

[7, 7, 5, 7, 5, 1 | 5, 7 | 5, 5, 7, 7 | 5, 5, 5, 5]

Now, the majority element is 5 (we changed the last run of the array from 7s to 5s), but our first candidate is still 7. In this case, our candidate is not the true majority element, but we still cannot discard more majority elements than minority elements (this would imply that count could reach -1 before we reassign candidate, which is obviously false).

Therefore, given that it is impossible (in both cases) to discard more majority elements than minority elements, we are safe in discarding the prefix and attempting to recursively solve the majority element problem for the suffix. Eventually, a suffix will be found for which count does not hit 0, and the majority element of that suffix will necessarily be the same as the majority element of the overall array.


*/

class Solution {
```

```
    public int majorityElement(int[] nums) {
        int count = 0;
        Integer candidate = null;

        for (int num : nums) {
            if (count == 0) {
                candidate = num;
            }
            count += (num == candidate) ? 1 : -1;
        }

        return candidate;
    }
}

/*

Complexity Analysis

Time complexity : O(n)

Boyer-Moore performs constant work exactly nn times, so the algorithm runs in linear
time.

Space complexity : O(1)

Boyer-Moore allocates only constant additional memory.


*/
```

123. `160. Intersection of Two Linked Lists.java`

```
/*
 *
 * Write a program to find the node at which the intersection of two singly linked lists
begins.
 *
 * For example, the following two linked lists:
 *
 *
 * begin to intersect at node c1.
 *
 *
 *
 * Example 1:
 *
 *
 * Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB
= 3
 * Output: Reference of the node with value = 8
 * Input Explanation: The intersected node's value is 8 (note that this must not be 0 if
the two lists intersect). From the head of A, it reads as [4,1,8,4,5]. From the head of
B, it reads as [5,0,1,8,4,5]. There are 2 nodes before the intersected node in A; There
are 3 nodes before the intersected node in B.
 *
 *
 * Example 2:
 *
 *
 * Input: intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
 * Output: Reference of the node with value = 2
 * Input Explanation: The intersected node's value is 2 (note that this must not be 0 if
the two lists intersect). From the head of A, it reads as [0,9,1,2,4]. From the head of
B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1
node before the intersected node in B.
 *
 *
 * Example 3:
 *
 *
 * Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
 * Output: null
 * Input Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it
reads as [1,5]. Since the two lists do not intersect, intersectVal must be 0, while skipA
and skipB can be arbitrary values.
 * Explanation: The two lists do not intersect, so return null.
 *
 *
 * Notes:
 *
 * If the two linked lists have no intersection at all, return null.
 * The linked lists must retain their original structure after the function returns.
 * You may assume there are no cycles anywhere in the entire linked structure.
```

```
 * Your code should preferably run in O(n) time and use only O(1) memory.
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {

    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        int lenA = length(headA), lenB = length(headB);
        // move headA and headB to the same start point
        while (lenA > lenB) {
            headA = headA.next;
            lenA--;
        }
        while (lenA < lenB) {
            headB = headB.next;
            lenB--;
        }
        // find the intersection until end
        while (headA != headB) {
            headA = headA.next;
            headB = headB.next;
        }
        return headA;
    }

    private int length(ListNode node) {
        int length = 0;
        while (node != null) {
            node = node.next;
            length++;
        }
        return length;
    }
}
```

124. `162. Find Peak Element.java`

```
/*
 *
 * A peak element is an element that is greater than its neighbors.
 *
 * Given an input array nums, where nums[i] ≠ nums[i+1], find a peak element and return
its index.
 *
 * The array may contain multiple peaks, in that case return the index to any one of the
peaks is fine.
 *
 * You may imagine that nums[-1] = nums[n] = -∞.
 *
 * Example 1:
 *
 * Input: nums = [1,2,3,1]
 * Output: 2
 * Explanation: 3 is a peak element and your function should return the index number 2.
 * Example 2:
 *
 * Input: nums = [1,2,1,3,5,6,4]
 * Output: 1 or 5
 * Explanation: Your function can return either index number 1 where the peak element is
2,
 * or index number 5 where the peak element is 6.
 *
 */

class Solution {

    public int findPeakElement(int[] num) {
        if (num.length <= 1) {
            return 0;
        }
        int mid = 0, l = 0, h = num.length - 1;

        while (l < h) {
            mid = (l + h) / 2;
            if (num[mid] > num[mid + 1]) {
                h = mid;
            } else if (num[mid] < num[mid + 1]) {
                l = mid + 1;
            }
        }

        return l;
    }
}
```

125. 167. Two Sum II - Input array is sorted.java

```
/*

Given an array of integers that is already sorted in ascending order, find two numbers
such that they add up to a specific target number.

The function twoSum should return indices of the two numbers such that they add up to the
target, where index1 must be less than index2.

Note:

Your returned answers (both index1 and index2) are not zero-based.
You may assume that each input would have exactly one solution and you may not use the
same element twice.
Example:

Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore index1 = 1, index2 = 2.

*/

class Solution {
    public int[] twoSum(int[] numbers, int target)
    {
        int[] answer = new int[2];
        z: for(int i=0;i<numbers.length;i++)
        {
            for(int j=i+1;j<numbers.length;j++)
            {
                if(target == (numbers[i]+ numbers[j]))
                {
                    answer[0] =i+1;
                    answer[1] = j+1;
                    break z;
                }
            }
        }
        return answer;


    }

}
```

126. `169. Majority Element.java`

```
/*
 *
 * Given an array of size n, find the majority element. The majority element is the
element that appears more than ⌊ n/2 ⌋ times.
 *
 * You may assume that the array is non-empty and the majority element always exist in
the array.
 *
 * Example 1:
 *
 * Input: [3,2,3]
 * Output: 3
 * Example 2:
 *
 * Input: [2,2,1,1,1,2,2]
 * Output: 2
 *
 */


 /*
 *
 * Approach 1: Brute Force
 * Intuition
 *
 * We can exhaust the search space in quadratic time by checking whether each element is
the majority element.
 *
 * Algorithm
 *
 * The brute force algorithm iterates over the array, and then iterates again for each
number to count its occurrences. As soon as a number is found to have appeared more than
any other can possibly have appeared, return it.
 *
 */
class Solution {

    public int majorityElement(int[] nums) {
        int majorityCount = nums.length / 2;

        for (int num : nums) {
            int count = 0;
            for (int elem : nums) {
                if (elem == num) {
                    count += 1;
                }
            }

            if (count > majorityCount) {
                return num;
            }
```

```
        }

        return -1;
    }
}

/*
 *
 * Complexity Analysis
 *
 * Time complexity : O(n^2)
 *
 * The brute force algorithm contains two nested for loops that each run for nn
 * iterations, adding up to quadratic time complexity.
 *
 * Space complexity : O(1)
 *
 * The brute force solution does not allocate additional space proportional to
 * the input size.
 *
 */

 /*
 *
 * Approach 2: HashMap Intuition
 *
 * We know that the majority element occurs more than floor(n/2) times, and a
 * HashMap allows us to count element occurrences efficiently.
 *
 * Algorithm
 *
 * We can use a HashMap that maps elements to counts in order to count
 * occurrences in linear time by looping over nums. Then, we simply return the
 * key with maximum value.
 *
 */
class Solution {

    private Map<Integer, Integer> countNums(int[] nums) {
        Map<Integer, Integer> counts = new HashMap<Integer, Integer>();
        for (int num : nums) {
            if (!counts.containsKey(num)) {
                counts.put(num, 1);
            } else {
                counts.put(num, counts.get(num) + 1);
            }
        }
        return counts;
    }

    public int majorityElement(int[] nums) {
        Map<Integer, Integer> counts = countNums(nums);
```

```
        Map.Entry<Integer, Integer> majorityEntry = null;
        for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
            if (majorityEntry == null || entry.getValue() > majorityEntry.getValue()) {
                majorityEntry = entry;
            }
        }

        return majorityEntry.getKey();
    }
}

/*
 *
 * Complexity Analysis
 *
 * Time complexity : O(n)
 *
 * We iterate over nums once and make a constant time HashMap insertion on each
 * iteration. Therefore, the algorithm runs in O(n) time.
 *
 * Space complexity : O(n)
 *
 * At most, the HashMap can contain n - floor(n/2) associations, so it occupies
 * O(n) space. This is because an arbitrary array of length nn can contain nn
 * distinct values, but nums is guaranteed to contain a majority element, which
 * will occupy (at minimum) floor(n/2) + 1 array indices. Therefore, n -
 * (floor(n/2) + 1) indices can be occupied by distinct, non-majority elements
 * (plus 1 for the majority element itself), leaving us with (at most) n -
 * floor(n/2) distinct elements.
 *
 */
/*
 * Approach 3: Sorting Intuition
 *
 * If the elements are sorted in monotonically increasing (or decreasing) order,
 * the majority element can be found at index floor(n/2) (and floor(n/2) + 1,
 * incidentally, if n is even).
 *
 * Algorithm
 *
 * For this algorithm, we simply do exactly what is described: sort nums, and
 * return the element in question. To see why this will always return the
 * majority element (given that the array has one), consider the figure below
 * (the top example is for an odd-length array and the bottom is for an
 * even-length array):
 *
 * Sorting middle index overlap
 *
 * For each example, the line below the array denotes the range of indices that
 * are covered by a majority element that happens to be the array minimum. As
 * you might expect, the line above the array is similar, but for the case where
 * the majority element is also the array maximum. In all other cases, this line
 * will lie somewhere between these two, but notice that even in these two most
 * extreme cases, they overlap at index floor(n/2) for both even- and odd-length
```

```
 * arrays. Therefore, no matter what value the majority element has in relation
 * to the rest of the array, returning the value at floor(n/2) will never be
 * wrong.
 *
 */
class Solution {

    public int majorityElement(int[] nums) {
        Arrays.sort(nums);
        return nums[nums.length / 2];
    }
}

/*
 *
 * Complexity Analysis
 *
 * Time complexity : O(nlgn)
 *
 * Sorting the array costs O(nlgn) time in Python and Java, so it dominates the
 * overall runtime.
 *
 * Space complexity : O(1) or (O(n))
 *
 * We sorted nums in place here - if that is not allowed, then we must spend
 * linear additional space on a copy of nums and sort the copy instead.
 *
 */

 /*
 *
 * Approach 4: Randomization Intuition
 *
 * Because more than floor(n/2) array indices are occupied by the majority
 * element, a random array index is likely to contain the majority element.
 *
 * Algorithm
 *
 * Because a given index is likely to have the majority element, we can just
 * select a random index, check whether its value is the majority element,
 * return if it is, and repeat if it is not. The algorithm is verifiably correct
 * because we ensure that the randomly chosen value is the majority element
 * before ever returning.
 *
 */
class Solution {

    private int randRange(Random rand, int min, int max) {
        return rand.nextInt(max - min) + min;
    }

    private int countOccurences(int[] nums, int num) {
        int count = 0;
        for (int i = 0; i < nums.length; i++) {
```

```
            if (nums[i] == num) {
                count++;
            }
        }
        return count;
    }

    public int majorityElement(int[] nums) {
        Random rand = new Random();

        int majorityCount = nums.length / 2;

        while (true) {
            int candidate = nums[randRange(rand, 0, nums.length)];
            if (countOccurences(nums, candidate) > majorityCount) {
                return candidate;
            }
        }
    }
}


/*
 *
 * Approach 5: Divide and Conquer Intuition
 *
 * If we know the majority element in the left and right halves of an array, we
 * can determine which is the global majority element in linear time.
 *
 * Algorithm
 *
 * Here, we apply a classical divide & conquer approach that recurses on the
 * left and right halves of an array until an answer can be trivially achieved
 * for a length-1 array. Note that because actually passing copies of subarrays
 * costs time and space, we instead pass lo and hi indices that describe the
 * relevant slice of the overall array. In this case, the majority element for a
 * length-1 slice is trivially its only element, so the recursion stops there.
 * If the current slice is longer than length-1, we must combine the answers for
 * the slice's left and right halves. If they agree on the majority element,
 * then the majority element for the overall slice is obviously the same[1]. If
 * they disagree, only one of them can be "right", so we need to count the
 * occurrences of the left and right majority elements to determine which
 * subslice's answer is globally correct. The overall answer for the array is
 * thus the majority element between indices 0 and nn.
 *
 */
class Solution {

    private int countInRange(int[] nums, int num, int lo, int hi) {
        int count = 0;
        for (int i = lo; i <= hi; i++) {
            if (nums[i] == num) {
                count++;
            }
```

```
        }
        return count;
    }

    private int majorityElementRec(int[] nums, int lo, int hi) {
        // base case; the only element in an array of size 1 is the majority
        // element.
        if (lo == hi) {
            return nums[lo];
        }

        // recurse on left and right halves of this slice.
        int mid = (hi - lo) / 2 + lo;
        int left = majorityElementRec(nums, lo, mid);
        int right = majorityElementRec(nums, mid + 1, hi);

        // if the two halves agree on the majority element, return it.
        if (left == right) {
            return left;
        }

        // otherwise, count each element and return the "winner".
        int leftCount = countInRange(nums, left, lo, hi);
        int rightCount = countInRange(nums, right, lo, hi);

        return leftCount > rightCount ? left : right;
    }

    public int majorityElement(int[] nums) {
        return majorityElementRec(nums, 0, nums.length - 1);
    }
}

/*

Complexity Analysis

Time complexity : O(nlgn)

Each recursive call to majority_element_rec performs two recursive calls on subslices of
size n/2 and two linear scans of length nn. Therefore, the time complexity of the divide
& conquer approach can be represented by the following recurrence relation:

T(n) = 2T(n/2) + 2n

Space complexity : O(lgn)

Although the divide & conquer does not explicitly allocate any additional memory, it uses
a non-constant amount of additional memory in stack frames due to recursion. Because the
algorithm "cuts" the array in half at each level of recursion, it follows that there can
only be O(lgn)O(lgn) "cuts" before the base case of 1 is reached. It follows from this
fact that the resulting recursion tree is balanced, and therefore all paths from the root
to a leaf are of length O(lgn)O(lgn). Because the recursion tree is traversed in a depth-
first manner, the space complexity is therefore equivalent to the length of the longest
```

```
path, which is, of course, O(lgn)

*/
```

127. `173. Binary Search Tree Iterator.java`

```
/*
 *
 * Implement an iterator over a binary search tree (BST). Your iterator will be
initialized with the root node of a BST.
 *
 * Calling next() will return the next smallest number in the BST.
 *
 *
 *
 * Example:
 *
 *
 *
 * BSTIterator iterator = new BSTIterator(root);
 * iterator.next();    // return 3
 * iterator.next();    // return 7
 * iterator.hasNext(); // return true
 * iterator.next();    // return 9
 * iterator.hasNext(); // return true
 * iterator.next();    // return 15
 * iterator.hasNext(); // return true
 * iterator.next();    // return 20
 * iterator.hasNext(); // return false
 *
 *
 * Note:
 *
 * next() and hasNext() should run in average O(1) time and uses O(h) memory, where h is
the height of the tree.
 * You may assume that next() call will always be valid, that is, there will be at least
a next smallest number in the BST when next() is called.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class BSTIterator {

    ArrayList<Integer> list = new ArrayList<>();
    int i = 0;

    public BSTIterator(TreeNode root) {
        preorder(root);
    }
```

```
    public void preorder(TreeNode root) {
        if (root == null) {
            return;
        }

        preorder(root.left);
        list.add(root.val);
        preorder(root.right);
    }

    public int next() {
        int result = -1;
        if (i < list.size()) {
            result = list.get(i);
        }
        i++;
        return result;
    }

    public boolean hasNext() {
        return (i <= list.size() - 1) ? true : false;
    }
}

/**
 * Your BSTIterator object will be instantiated and called as such:
 * BSTIterator obj = new BSTIterator(root);
 * int param_1 = obj.next();
 * boolean param_2 = obj.hasNext();
 */
```

128. `175. Combine Two Tables.java`

```
/*
Table: Person

+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| PersonId    | int     |
| FirstName   | varchar |
| LastName    | varchar |
+-------------+---------+
PersonId is the primary key column for this table.
Table: Address

+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| AddressId   | int     |
| PersonId    | int     |
| City        | varchar |
| State       | varchar |
+-------------+---------+
AddressId is the primary key column for this table.


Write a SQL query for a report that provides the following information for each person in
the Person table, regardless if there is an address for each of those people:

FirstName, LastName, City, State

*/

// SQL Query

select Person.FirstName, Person.LastName, Address.City, Address.State
from Person left Join Address on Person.PersonId = Address.PersonId
```

129. `179. Largest Number.java`

```
/*
 *
 * Given a list of non negative integers, arrange them such that they form the largest
number.
 *
 * Example 1:
 *
 * Input: [10,2]
 * Output: "210"
 * Example 2:
 *
 * Input: [3,30,34,5,9]
 * Output: "9534330"
 * Note: The result may be very large, so you need to return a string instead of an
integer.
 *
 */

class Solution {

    public String largestNumber(int[] nums) {
        if (nums == null || nums.length == 0) {
            return "";
        }
        String[] strs = new String[nums.length];
        for (int i = 0; i < nums.length; i++) {
            strs[i] = nums[i] + "";
        }

        Arrays.sort(strs, new Comparator<String>() {

            public int compare(String i, String j) {
                String s1 = i + j;
                String s2 = j + i;
                return s1.compareTo(s2);
            }
        });
        if (strs[strs.length - 1].charAt(0) == '0') {
            return "0";
        }
        String res = new String();
        for (int i = 0; i < strs.length; i++) {
            res = strs[i] + res;
        }
        return res;
    }
}
```

130. `187. Repeated DNA Sequences.java`

```
/*
 *
 * All DNA is composed of a series of nucleotides abbreviated as A, C, G, and T, for
example: "ACGAATTCCG". When studying DNA, it is sometimes useful to identify repeated
sequences within the DNA.
 *
 * Write a function to find all the 10-letter-long sequences (substrings) that occur more
than once in a DNA molecule.
 *
 * Example:
 *
 * Input: s = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"
 *
 * Output: ["AAAAACCCCC", "CCCCCAAAAA"]
 *
 */

class Solution {

    public List<String> findRepeatedDnaSequences(String s) {
        HashSet<String> seen = new HashSet<>();
        HashSet<String> repeated = new HashSet<>();

        for (int i = 0; i + 9 < s.length(); i++) {
            String temp = s.substring(i, i + 10);
            if (seen.contains(temp)) {
                repeated.add(temp);
            } else {
                seen.add(temp);
            }
        }

        return new ArrayList<String>(repeated);
    }
}
```

131. `189. Rotate Array.java`

```
/*
 *
 * Given an array, rotate the array to the right by k steps, where k is non-negative.
 *
 * Example 1:
 *
 * Input: [1,2,3,4,5,6,7] and k = 3
 * Output: [5,6,7,1,2,3,4]
 * Explanation:
 * rotate 1 steps to the right: [7,1,2,3,4,5,6]
 * rotate 2 steps to the right: [6,7,1,2,3,4,5]
 * rotate 3 steps to the right: [5,6,7,1,2,3,4]
 * Example 2:
 *
 * Input: [-1,-100,3,99] and k = 2
 * Output: [3,99,-1,-100]
 * Explanation:
 * rotate 1 steps to the right: [99,-1,-100,3]
 * rotate 2 steps to the right: [3,99,-1,-100]
 *
 */

class Solution {

    public void rotate(int[] nums, int k) {

        ArrayList<Integer> al = new ArrayList();
        if (nums.length == 0 || nums.length < k) {
            for (int i = 0; i < nums.length; i++) {
                System.out.print(nums[i]);
            }
        } else {
            int temp = k;
            while (k != 0) {

                al.add(nums[nums.length - k]);
                k--;

            }
            // for(int i=0;i<al.size();i++)
            //System.out.println(al.get(i));

            for (int i = 0; i < nums.length - temp; i++) {
                al.add(nums[i]);
                // System.out.println(nums[i]);
            }
            int a[] = new int[al.size()];
            for (int i = 0; i < al.size(); i++) {
                nums[i] = al.get(i);
            }
```

```
        }
        /*
         * if(nums == null || nums.length < 2){ return; }
         *
         * k = k % nums.length; reverse(nums, 0, nums.length - k - 1);//Full
         * array reverse(nums, nums.length - k, nums.length - 1); //Partial,
         * second part of array reverse(nums, 0, nums.length - 1);//first part
         * of the arrya
         *
         * }
         *
         * private void reverse(int[] nums, int i, int j){ int tmp = 0; while(i
         * < j){ tmp = nums[i]; nums[i] = nums[j]; nums[j] = tmp; i++; j--;
    }
         */
    }
}
```

132. `191. Number of 1 Bits.java`

```
/*
 *
 * Write a function that takes an unsigned integer and return the number of '1' bits it
has (also known as the Hamming weight).
 *
 *
 *
 * Example 1:
 *
 * Input: 00000000000000000000000000001011
 * Output: 3
 * Explanation: The input binary string 00000000000000000000000000001011 has a total of
three '1' bits.
 * Example 2:
 *
 * Input: 00000000000000000000000010000000
 * Output: 1
 * Explanation: The input binary string 00000000000000000000000010000000 has a total of
one '1' bit.
 * Example 3:
 *
 * Input: 11111111111111111111111111111101
 * Output: 31
 * Explanation: The input binary string 11111111111111111111111111111101 has a total of
thirty one '1' bits.
 *
 *
 * Note:
 *
 * Note that in some languages such as Java, there is no unsigned integer type. In this
case, the input will be given as signed integer type and should not affect your
implementation, as the internal binary representation of the integer is the same whether
it is signed or unsigned.
 * In Java, the compiler represents the signed integers using 2's complement notation.
Therefore, in Example 3 above the input represents the signed integer -3.
 *
 */

public class Solution {

    // you need to treat n as an unsigned value
    public int hammingWeight(int n) {
        int sum = 0;
        while (n != 0) {
            sum++;
            n &= (n - 1);
        }
        return sum;
    }
}
```

133. `193. Valid Phone Numbers.java`

```
awk '/^(\([0-9]{3}\) |[0-9]{3}-)[0-9]{3}-[0-9]{4}$/' file.txt
```

134. `198. House Robber.java`

```
/*
 *
 * You are a professional robber planning to rob houses along a street. Each house has a
certain amount of money stashed, the only constraint stopping you from robbing each of
them is that adjacent houses have security system connected and it will automatically
contact the police if two adjacent houses were broken into on the same night.
 *
 * Given a list of non-negative integers representing the amount of money of each house,
determine the maximum amount of money you can rob tonight without alerting the police.
 *
 * Example 1:
 *
 * Input: [1,2,3,1]
 * Output: 4
 * Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
 * Total amount you can rob = 1 + 3 = 4.
 * Example 2:
 *
 * Input: [2,7,9,3,1]
 * Output: 12
 * Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money =
1).
 * Total amount you can rob = 2 + 9 + 1 = 12.
 *
 */

class Solution {

    public int rob(int[] nums) {
        int prevMax = 0;
        int currMax = 0;
        for (int x : nums) {
            int temp = currMax;
            currMax = Math.max(prevMax + x, currMax);
            prevMax = temp;
        }
        return currMax;
    }
}
```

135. `199. Binary Tree Right Side View.java`

```
/*
 *
 * Given a binary tree, imagine yourself standing on the right side of it, return the
values of the nodes you can see ordered from top to bottom.
 *
 * Example:
 *
 * Input: [1,2,3,null,5,null,4]
 * Output: [1, 3, 4]
 * Explanation:
 *
 *     1            <---
 *   /   \
 *  2     3         <---
 *   \     \
 *    5     4       <---
 *
 */

class Solution {

    public List<Integer> rightSideView(TreeNode root) {

        ArrayList<Integer> result = new ArrayList<>();

        if (root == null) {
            return result;
        }

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                if (i == size - 1) {
                    result.add(node.val);
                }

                if (node.left != null) {
                    queue.offer(node.left);
                }
                if (node.right != null) {
                    queue.offer(node.right);
                }
            }
        }

        return result;
```

```
    }
}
```

136. `200. Number of Islands.java`

```
/*
 *
 * Given a 2d grid map of '1's (land) and '0's (water), count the number of islands. An
island is surrounded by water and is formed by connecting adjacent lands horizontally or
vertically. You may assume all four edges of the grid are all surrounded by water.
 *
 * Example 1:
 *
 * Input:
 * 11110
 * 11010
 * 11000
 * 00000
 *
 * Output: 1
 * Example 2:
 *
 * Input:
 * 11000
 * 11000
 * 00100
 * 00011
 *
 * Output: 3
 *
 */

class Solution {

    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0 || grid[0].length == 0) {
            return 0;
        }

        int m = grid.length;
        int n = grid[0].length;

        int count = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == '1') {
                    count++;
                    merge(grid, i, j);
                }
            }
        }

        return count;
    }

    public void merge(char[][] grid, int i, int j) {
```

```
        int m = grid.length;
        int n = grid[0].length;

        if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] != '1') {
            return;
        }

        grid[i][j] = 'X';

        merge(grid, i - 1, j);
        merge(grid, i + 1, j);
        merge(grid, i, j - 1);
        merge(grid, i, j + 1);
    }
}
```

137. `202. Happy Number.java`

```
/*
 *
 * Write an algorithm to determine if a number is "happy".
 *
 * A happy number is a number defined by the following process: Starting with any
positive integer, replace the number by the sum of the squares of its digits, and repeat
the process until the number equals 1 (where it will stay), or it loops endlessly in a
cycle which does not include 1. Those numbers for which this process ends in 1 are happy
numbers.
 *
 * Example:
 * Input: 19
 * Output: true
 * Explanation:
 * 12 + 92 = 82
 * 82 + 22 = 68
 * 62 + 82 = 100
 * 12 + 02 + 02 = 1
 *
 */

class Solution {

    public boolean isHappy(int n) {
        if (n <= 0) {
            return false;
        }

        while (n >= 10) {
            int sum = 0;
            while (n != 0) {
                sum += (n % 10) * (n % 10);
                n /= 10;
            }
            n = sum;
        }
        return n == 1 || n == 7;

    }
}
```

138. `203. Remove Linked List Elements.java`

```
/*
 *
 * Remove all elements from a linked list of integers that have value val.
 *
 * Example:
 *
 * Input:  1->2->6->3->4->5->6, val = 6
 * Output: 1->2->3->4->5
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode removeElements(ListNode head, int val) {
        ListNode helper = new ListNode(0);
        helper.next = head;
        ListNode p = helper;

        while (p.next != null) {
            if (p.next.val == val) {
                ListNode next = p.next;
                p.next = next.next;
            } else {
                p = p.next;
            }
        }

        return helper.next;
    }
}
```

139. `205. Isomorphic Strings.java`

```
/*
 *
 * Given two strings s and t, determine if they are isomorphic.
 *
 * Two strings are isomorphic if the characters in s can be replaced to get t.
 *
 * All occurrences of a character must be replaced with another character while
preserving the order of characters. No two characters may map to the same character but a
character may map to itself.
 *
 * Example 1:
 *
 * Input: s = "egg", t = "add"
 * Output: true
 * Example 2:
 *
 * Input: s = "foo", t = "bar"
 * Output: false
 * Example 3:
 *
 * Input: s = "paper", t = "title"
 * Output: true
 * Note:
 * You may assume both s and t have the same length.
 *
 */

class Solution {

    public boolean isIsomorphic(String s1, String s2) {
        Map<Character, Integer> m1 = new HashMap<>();
        Map<Character, Integer> m2 = new HashMap<>();

        for (Integer i = 0; i < s1.length(); i++) {

            if (m1.put(s1.charAt(i), i) != m2.put(s2.charAt(i), i)) {
                return false;
            }
        }
        return true;
    }
}
```

140. `206. Reverse Linked List.java`

```
/*
 *
 * Reverse a singly linked list.
 *
 * Example:
 *
 * Input: 1->2->3->4->5->NULL
 * Output: 5->4->3->2->1->NULL
 * Follow up:
 *
 * A linked list can be reversed either iteratively or recursively. Could you implement
both?
 *
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public ListNode reverseList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode prev = null;
        ListNode current = head;
        ListNode next = null;

        //Basically we are kind of creating a new list using the prev pointer.
        //We are starting to point the current node to the new list(null)
        //Iterate and make each node point to it's prev node
        while (current != null) {
            next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }

        head = prev;
        return head;

        /*
         * ListNode p1 = head; ListNode p2 = p1.next;
         *
```

```
         * head.next = null; while(p1!=null&& p2!=null){ ListNode t = p2.next;
         * p2.next = p1; p1 = p2; p2 = t; }
         *
         * return p1;
         */
    }
}
```

141. `207. Course Schedule.java`

```
/*
 *
 * There are a total of n courses you have to take, labeled from 0 to n-1.
 *
 * Some courses may have prerequisites, for example to take course 0 you have to first
take course 1, which is expressed as a pair: [0,1]
 *
 * Given the total number of courses and a list of prerequisite pairs, is it possible for
you to finish all courses?
 *
 * Example 1:
 *
 * Input: 2, [[1,0]]
 * Output: true
 * Explanation: There are a total of 2 courses to take.
 * To take course 1 you should have finished course 0. So it is possible.
 * Example 2:
 *
 * Input: 2, [[1,0],[0,1]]
 * Output: false
 * Explanation: There are a total of 2 courses to take.
 * To take course 1 you should have finished course 0, and to take course 0 you should
 * also have finished course 1. So it is impossible.
 * Note:
 *
 * The input prerequisites is a graph represented by a list of edges, not adjacency
matrices. Read more about how a graph is represented.
 * You may assume that there are no duplicate edges in the input prerequisites.
 *
 */

Approach 1:

    class Solution {

    public boolean canFinish(int numCourses, int[][] prerequisites) {
        ArrayList[] graph = new ArrayList[numCourses];
        for (int i = 0; i < numCourses; i++) {
            graph[i] = new ArrayList();
        }

        boolean[] visited = new boolean[numCourses];
        for (int i = 0; i < prerequisites.length; i++) {
            graph[prerequisites[i][1]].add(prerequisites[i][0]);
        }

        for (int i = 0; i < numCourses; i++) {
            if (!dfs(graph, visited, i)) {
                return false;
            }
        }
```

```
        return true;
    }

    private boolean dfs(ArrayList[] graph, boolean[] visited, int course) {
        if (visited[course]) {
            return false;
        } else {
            visited[course] = true;
        };

        for (int i = 0; i < graph[course].size(); i++) {
            if (!dfs(graph, visited, (int) graph[course].get(i))) {
                return false;
            }
        }
        visited[course] = false;
        return true;
    }

}


Approach 2:

public class Solution {

    public boolean canFinish(int numCourses, int[][] prerequisites) {

        int len = prerequisites.length;

        if (numCourses == 0 || len == 0) {
            return true;
        }

        // counter for number of prerequisites
        int[] pCounter = new int[numCourses];
        for (int i = 0; i < len; i++) {
            pCounter[prerequisites[i][0]]++;
        }

        //store courses that have no prerequisites
        LinkedList<Integer> queue = new LinkedList<Integer>();
        for (int i = 0; i < numCourses; i++) {
            if (pCounter[i] == 0) {
                queue.add(i);
            }
        }

        // number of courses that have no prerequisites
        int numNoPre = queue.size();

        while (!queue.isEmpty()) {
            int top = queue.remove();
            for (int i = 0; i < len; i++) {
```

```
                // if a course's prerequisite can be satisfied by a course in queue
                if (prerequisites[i][1] == top) {
                    pCounter[prerequisites[i][0]]--;
                    if (pCounter[prerequisites[i][0]] == 0) {
                        numNoPre++;
                        queue.add(prerequisites[i][0]);
                    }
                }
            }
        }

        return numNoPre == numCourses;
    }

}
```

142. `208. Implement Trie (Prefix Tree).java`

```
/*
 *
 * Implement a trie with insert, search, and startsWith methods.
 *
 * Example:
 *
 * Trie trie = new Trie();
 *
 * trie.insert("apple");
 * trie.search("apple");   // returns true
 * trie.search("app");     // returns false
 * trie.startsWith("app"); // returns true
 * trie.insert("app");
 * trie.search("app");     // returns true
 * Note:
 *
 * You may assume that all inputs are consist of lowercase letters a-z.
 * All inputs are guaranteed to be non-empty strings.
 *
 */

class TrieNode {

    public char val;
    public boolean isWord;
    public TrieNode[] children = new TrieNode[26];
    //  public TrieNode() {}

    TrieNode(char c) {
        TrieNode node = new TrieNode();
        node.val = c;
    }
}

public class Trie {

    TrieNode root;

    public Trie() {
        root = new TrieNode();
        root.val = ' ';
    }

    public void insert(String word) {
        TrieNode ws = root;
        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);
            if (ws.children[c - 'a'] == null) {
                ws.children[c - 'a'] = new TrieNode(c);
            }
            ws = ws.children[c - 'a'];
```

```
        }
        ws.isWord = true;
    }

    public boolean search(String word) {
        TrieNode ws = root;
        for (int i = 0; i < word.length(); i++) {
            char c = word.charAt(i);
            if (ws.children[c - 'a'] == null) {
                return false;
            }
            ws = ws.children[c - 'a'];
        }
        return ws.isWord;
    }

    public boolean startsWith(String prefix) {
        TrieNode ws = root;
        for (int i = 0; i < prefix.length(); i++) {
            char c = prefix.charAt(i);
            if (ws.children[c - 'a'] == null) {
                return false;
            }
            ws = ws.children[c - 'a'];
        }
        return true;
    }
}


/**
 * Your Trie object will be instantiated and called as such:
 * Trie obj = new Trie();
 * obj.insert(word);
 * boolean param_2 = obj.search(word);
 * boolean param_3 = obj.startsWith(prefix);
 */
```

143. `209. Minimum Size Subarray Sum.java`

```
/*
 *
 * Given an array of n positive integers and a positive integer s, find the minimal
length of a contiguous subarray of which the sum ≥ s. If there isn't one, return 0
instead.
 *
 * Example:
 * Input: s = 7, nums = [2,3,1,2,4,3]
 * Output: 2
 * Explanation: the subarray [4,3] has the minimal length under the problem constraint.
 * Follow up:
 * If you have figured out the O(n) solution, try coding another solution of which the
time complexity is O(n log n).  *
 */

class Solution {

    public int minSubArrayLen(int s, int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        int i = 0;
        int j = 0;
        int sum = 0;

        int minLen = Integer.MAX_VALUE;

        while (j < nums.length) {
            if (sum < s) {
                sum += nums[j];
                j++;
            } else {
                minLen = Math.min(minLen, j - i);
                if (i == j - 1) {
                    return 1;
                }

                sum -= nums[i];
                i++;
            }
        }

        while (sum >= s) {
            minLen = Math.min(minLen, j - i);

            sum -= nums[i++];
        }

        return minLen == Integer.MAX_VALUE ? 0 : minLen;
```

```
    }
}
```

144. `210. Course Schedule II.java`

```
/*
 *
 * There are a total of n courses you have to take, labeled from 0 to n-1.
 *
 * Some courses may have prerequisites, for example to take course 0 you have to first
take course 1, which is expressed as a pair: [0,1]
 *
 * Given the total number of courses and a list of prerequisite pairs, return the
ordering of courses you should take to finish all courses.
 *
 * There may be multiple correct orders, you just need to return one of them. If it is
impossible to finish all courses, return an empty array.
 *
 * Example 1:
 *
 * Input: 2, [[1,0]]
 * Output: [0,1]
 * Explanation: There are a total of 2 courses to take. To take course 1 you should have
finished
 * course 0. So the correct course order is [0,1] .
 * Example 2:
 *
 * Input: 4, [[1,0],[2,0],[3,1],[3,2]]
 * Output: [0,1,2,3] or [0,2,1,3]
 * Explanation: There are a total of 4 courses to take. To take course 3 you should have
finished both
 * courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.
 * So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3] .
 * Note:
 *
 * The input prerequisites is a graph represented by a list of edges, not adjacency
matrices. Read more about how a graph is represented.
 * You may assume that there are no duplicate edges in the input prerequisites.
 *
 */

class Solution {

    public int[] findOrder(int numCourses, int[][] prerequisites) {

        int len = prerequisites.length;

        //if there is no prerequisites, return a sequence of courses
        if (len == 0) {
            int[] res = new int[numCourses];
            for (int m = 0; m < numCourses; m++) {
                res[m] = m;
            }
            return res;
        }
```

```
        //records the number of prerequisites each course (0,...,numCourses-1) requires
        int[] pCounter = new int[numCourses];
        for (int i = 0; i < len; i++) {
            pCounter[prerequisites[i][0]]++;
        }

        //stores courses that have no prerequisites
        LinkedList<Integer> queue = new LinkedList<Integer>();
        for (int i = 0; i < numCourses; i++) {
            if (pCounter[i] == 0) {
                queue.add(i);
            }
        }

        int numNoPre = queue.size();

        //initialize result
        int[] result = new int[numCourses];
        int j = 0;

        while (!queue.isEmpty()) {
            int c = queue.remove();
            result[j++] = c;

            for (int i = 0; i < len; i++) {
                if (prerequisites[i][1] == c) {
                    pCounter[prerequisites[i][0]]--;
                    if (pCounter[prerequisites[i][0]] == 0) {
                        queue.add(prerequisites[i][0]);
                        numNoPre++;
                    }
                }

            }
        }

        //return result
        if (numNoPre == numCourses) {
            return result;
        } else {
            return new int[0];
        }
    }
}
```

145. `213. House Robber II.java`

```
/*
 *
 * You are a professional robber planning to rob houses along a street. Each house has a
certain amount of money stashed. All houses at this place are arranged in a circle. That
means the first house is the neighbor of the last one. Meanwhile, adjacent houses have
security system connected and it will automatically contact the police if two adjacent
houses were broken into on the same night.
 *
 * Given a list of non-negative integers representing the amount of money of each house,
determine the maximum amount of money you can rob tonight without alerting the police.
 *
 * Example 1:
 *
 * Input: [2,3,2]
 * Output: 3
 * Explanation: You cannot rob house 1 (money = 2) and then rob house 3 (money = 2),
 * because they are adjacent houses.
 * Example 2:
 *
 * Input: [1,2,3,1]
 * Output: 4
 * Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
 * Total amount you can rob = 1 + 3 = 4.
 *
 */

class Solution {

    public int rob(int[] nums) {
        if (nums == null || nums.length == 0) {
            return 0;
        }

        if (nums.length == 1) {
            return nums[0];
        }

        int max1 = robHelper(nums, 0, nums.length - 2);
        int max2 = robHelper(nums, 1, nums.length - 1);

        return Math.max(max1, max2);
    }

    public int robHelper(int[] nums, int i, int j) {
        if (i == j) {
            return nums[i];
        }

        int[] dp = new int[nums.length];
        dp[i] = nums[i];
        dp[i + 1] = Math.max(nums[i + 1], dp[i]);
```

```
        for (int k = i + 2; k <= j; k++) {
            dp[k] = Math.max(dp[k - 1], dp[k - 2] + nums[k]);
        }

        return dp[j];
    }
}
```

146. `215. Kth Largest Element in an Array.java`

```
/*
 *
 * Find the kth largest element in an unsorted array. Note that it is the kth largest
element in the sorted order, not the kth distinct element.
 *
 * Example 1:
 *
 * Input: [3,2,1,5,6,4] and k = 2
 * Output: 5
 * Example 2:
 *
 * Input: [3,2,3,1,2,4,5,5,6] and k = 4
 * Output: 4
 * Note:
 * You may assume k is always valid, 1 ≤ k ≤ array's length.
 *
 */

class Solution {

    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> queue = new PriorityQueue();

        for (int n : nums) {
            queue.offer(n);

            if (queue.size() > k) {
                queue.poll();
            }
        }
        return queue.peek();
    }
}
```

147. `217. Contains Duplicate.java`

```
/*
 *
 * Given an array of integers, find if the array contains any duplicates.
 *
 * Your function should return true if any value appears at least twice in the array, and
it should return false if every element is distinct.
 *
 * Example 1:
 *
 * Input: [1,2,3,1]
 * Output: true
 * Example 2:
 *
 * Input: [1,2,3,4]
 * Output: false
 * Example 3:
 *
 * Input: [1,1,1,3,3,4,3,2,4,2]
 * Output: true
 *
 */

class Solution {

    public boolean containsDuplicate(int[] nums) {

        Set<Integer> set = new HashSet();

        for (int i : nums) {
            if (set.contains(i)) {
                return true;
            } else {
                set.add(i);
            }
        }

        return false;

    }
}
```

148. `219. Contains Duplicate II.java`

```
/*
 *
 * Given an array of integers and an integer k, find out whether there are two distinct
indices i and j in the array such that nums[i] = nums[j] and the absolute difference
between i and j is at most k.
 *
 * Example 1:
 *
 * Input: nums = [1,2,3,1], k = 3
 * Output: true
 * Example 2:
 *
 * Input: nums = [1,0,1,1], k = 1
 * Output: true
 * Example 3:
 *
 * Input: nums = [1,2,3,1,2,3], k = 2
 * Output: false
 *
 */

class Solution {

    public boolean containsNearbyDuplicate(int[] nums, int k) {
        Set<Integer> set = new HashSet<>();
        for (int i = 0; i < nums.length; ++i) {
            if (set.contains(nums[i])) {
                return true;
            }
            set.add(nums[i]);
            if (set.size() > k) {
                set.remove(nums[i - k]);
            }
        }
        return false;
    }
}
```

149. `220. Contains Duplicate III.java`

```
/*
 *
 * Given an array of integers, find out whether there are two distinct indices i and j in
the array such that the absolute difference between nums[i] and nums[j] is at most t and
the absolute difference between i and j is at most k.
 *
 * Example 1:
 *
 * Input: nums = [1,2,3,1], k = 3, t = 0
 * Output: true
 * Example 2:
 *
 * Input: nums = [1,0,1,1], k = 1, t = 2
 * Output: true
 * Example 3:
 *
 * Input: nums = [1,5,9,1,5,9], k = 2, t = 3
 * Output: false
 *
 */

class Solution {

    public boolean containsNearbyAlmostDuplicate(int[] nums, int k, int t) {
        if (nums == null || nums.length < 2 || k < 0 || t < 0) {
            return false;
        }

        TreeSet<Long> set = new TreeSet<Long>();
        for (int i = 0; i < nums.length; i++) {
            long curr = (long) nums[i];

            long leftBoundary = (long) curr - t;
            long rightBoundary = (long) curr + t + 1; //right boundary is exclusive, so
+1
            SortedSet<Long> sub = set.subSet(leftBoundary, rightBoundary);
            if (sub.size() > 0) {
                return true;
            }

            set.add(curr);

            if (i >= k) { // or if(set.size()>=k+1)
                set.remove((long) nums[i - k]);
            }
        }

        return false;
    }
}
```

150. `221. Maximal Square.java`

```
/*
 *
 * Given a 2D binary matrix filled with 0's and 1's, find the largest square containing
only 1's and return its area.
 *
 * Example:
 *
 * Input:  *
 * 1 0 1 0 0
 * 1 0 1 1 1
 * 1 1 1 1 1
 * 1 0 0 1 0
 *
 * Output: 4
 *
 */

class Solution {

    public int maximalSquare(char[][] matrix) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return 0;
        }
        int m = matrix.length;
        int n = matrix[0].length;
        int[][] t = new int[m][n];
//top row
        for (int i = 0; i < m; i++) {
            t[i][0] = Character.getNumericValue(matrix[i][0]);
        }
//left column
        for (int j = 0; j < n; j++) {
            t[0][j] = Character.getNumericValue(matrix[0][j]);
        }
//cells inside
        for (int i = 1; i < m; i++) {
            for (int j = 1; j < n; j++) {
                if (matrix[i][j] == '1') {
                    int min = Math.min(t[i - 1][j], t[i - 1][j - 1]);
                    min = Math.min(min, t[i][j - 1]);
                    t[i][j] = min + 1;
                } else {
                    t[i][j] = 0;
                }
            }
        }
        int max = 0;
//get maximal length
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (t[i][j] > max) {
```

```
                    max = t[i][j];
                }
            }
        }
        return max * max;
    }
}
```

151. `222. Count Complete Tree Nodes.java`

```
/*
 *
 * Given a complete binary tree, count the number of nodes.
 *
 * Note:
 *
 * Definition of a complete binary tree from Wikipedia:
 * In a complete binary tree every level, except possibly the last, is completely filled,
and all nodes in the last level are as far left as possible. It can have between 1 and 2h
nodes inclusive at the last level h.
 *
 * Example:
 *
 * Input:
 * 1
 * / \
 * 2   3
 * / \  /
 * 4  5 6
 *
 * Output: 6
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {

    public int countNodes(TreeNode root) {
        Queue<TreeNode> queue = new LinkedList<>();

        if (root == null) {
            return 0;
        }

        queue.offer(root);
        int count = 0;

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            count++;

            if (node.left != null) {
                queue.offer(node.left);
```

```
            }
            if (node.right != null) {
                queue.offer(node.right);
            }
        }
        return count;
    }
}
```

152. `225. Implement Stack using Queues.java`

```
/*
 *
 *
 * Implement the following operations of a stack using queues.
 *
 * push(x) -- Push element x onto stack.
 * pop() -- Removes the element on top of the stack.
 * top() -- Get the top element.
 * empty() -- Return whether the stack is empty.
 * Example:
 *
 * MyStack stack = new MyStack();
 *
 * stack.push(1);
 * stack.push(2);
 * stack.top();   // returns 2
 * stack.pop();   // returns 2
 * stack.empty(); // returns false
 * Notes:
 *
 * You must use only standard operations of a queue -- which means only push to back,
peek/pop from front, size, and is empty operations are valid.
 * Depending on your language, queue may not be supported natively. You may simulate a
queue by using a list or deque (double-ended queue), as long as you use only standard
operations of a queue.
 * You may assume that all operations are valid (for example, no pop or top operations
will be called on an empty stack).
 *
 */

class MyStack {

    Queue<Integer> q = new LinkedList<Integer>();

    /**
     * Initialize your data structure here.
     */
    public MyStack() {

    }

    /**
     * Push element x onto stack.
     */
    public void push(int x) {
        q.add(x);

        int n = q.size();
        while (n-- > 1) {
            q.add(q.poll());
        }
```

```
    }

    /**
     * Removes the element on top of the stack and returns that element.
     */
    public int pop() {
        return q.poll();
    }

    /**
     * Get the top element.
     */
    public int top() {
        return q.peek();
    }

    /**
     * Returns whether the stack is empty.
     */
    public boolean empty() {
        return q.isEmpty();
    }
}

/**
 * Your MyStack object will be instantiated and called as such:
 * MyStack obj = new MyStack();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.top();
 * boolean param_4 = obj.empty();
 */
```

153. `226. Invert Binary Tree.java`

```
/*
 * Invert a binary tree.
 *
 * Example:
 *
 * Input:
 *
 *              4
 *        /   \
 *   2     7
 *  / \   / \
 * 1   3 6   9
 * Output:
 *
 *              4
 *    /   \
 *   7     2
 *  / \   / \
 * 9   6 3   1
 * Trivia:
 * This problem was inspired by this original tweet by Max Howell:
 *
 * Google: 90% of our engineers use the software you wrote (Homebrew), but you can’t
invert a binary tree on a whiteboard so f*** off.
 *
 */

class Solution {

    public TreeNode invertTree(TreeNode root) {
        if (root == null) {
            return null;
        }
        TreeNode right = invertTree(root.right);
        TreeNode left = invertTree(root.left);
        root.left = right;
        root.right = left;
        return root;
    }
}
```

154. `227. Basic Calculator II.java`

```
/*
 *
 * Implement a basic calculator to evaluate a simple expression string.
 *
 * The expression string contains only non-negative integers, +, -, *, / operators and
empty spaces . The integer division should truncate toward zero.
 *
 * Example 1:
 *
 * Input: "3+2*2"
 * Output: 7
 * Example 2:
 *
 * Input: " 3/2 "
 * Output: 1
 * Example 3:
 *
 * Input: " 3+5 / 2 "
 * Output: 5
 * Note:
 *
 * You may assume that the given expression is always valid.
 * Do not use the eval built-in library function.
 *
 */

class Solution {

    public int calculate(String s) {
        if (s == null || s.length() == 0) {
            return 0;
        }

        int num = 0, res = 0;
        char op = '+';
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (Character.isDigit(c)) {
                num = num * 10 + c - '0'; //Shift the already present result by 1 place
            }
            if (i == s.length() - 1 || (!Character.isDigit(c) && c != ' ')) {
                if (op == '+') {
                    stack.push(num);
                } else if (op == '-') {
                    stack.push(-num);
                } else if (op == '*') {
                    stack.push(stack.pop() * num);
                } else if (op == '/') {
                    stack.push(stack.pop() / num);
```

```
                }

                op = c;
                num = 0;
            }
        }

        while (!stack.isEmpty()) {
            res += stack.pop();
        }

        return res;

    }
}
```

155. `228. Summary Ranges.java`

```
/*
 *
 * Given a sorted integer array without duplicates, return the summary of its ranges.
 *
 * Example 1:
 *
 * Input:  [0,1,2,4,5,7]
 * Output: ["0->2","4->5","7"]
 * Explanation: 0,1,2 form a continuous range; 4,5 form a continuous range.
 * Example 2:
 *
 * Input:  [0,2,3,4,6,8,9]
 * Output: ["0","2->4","6","8->9"]
 * Explanation: 2,3,4 form a continuous range; 8,9 form a continuous range.
 *
 */

class Solution {

    public List<String> summaryRanges(int[] nums) {
        List<String> result = new ArrayList<String>();
        int start = 0, end = 0;

        for (int i = 0; i < nums.length; i++) {
            while (i < nums.length - 1 && (1 + nums[i] == nums[i + 1])) {
                end++;
                i++;
            }

            String s = null;
            if (start == end) {
                s = "" + nums[start];
            } else {
                s = "" + nums[start] + "->" + nums[end];
            }

            if (end + 1 < nums.length) {
                start = end + 1;
                end = end + 1;
            }

            result.add(s);

        }
        return result;

    }
}
```

156. `230. Kth Smallest Element in a BST.java`

```
/*
 *
 * Given a binary search tree, write a function kthSmallest to find the kth smallest
element in it.
 *
 * Note:
 * You may assume k is always valid, 1 ≤ k ≤ BST's total elements.
 *
 * Example 1:
 *
 * Input: root = [3,1,4,null,2], k = 1
 *    3
 *   / \
 *  1   4
 *   \
 *    2
 * Output: 1
 * Example 2:
 *
 * Input: root = [5,3,6,2,4,null,null,1], k = 3
 *      5
 *     / \
 *    3   6
 *   / \
 *  2   4
 * /
 * 1
 * Output: 3
 *
 */

class Solution {

    ArrayList<Integer> list = new ArrayList<>();

    public int kthSmallest(TreeNode root, int k) {

        inorder(root);
        int result = -1;

        for (int i = 0; i < k; i++) {
            if (i == k - 1) {
                result = list.get(i);
            }
        }

        return result;
    }

    void inorder(TreeNode root) {
```

```
        if (root == null) {
            return;
        }

        inorder(root.left);
        list.add(root.val);
        inorder(root.right);
    }
}

/*
 * We can optimize the above solution. When we are doing the inorder traversal
 * and trying to add nodes to the list, we can check whether we reach K nodes
 * while traversing. So this will make sure that we don't do unnecessary
 * calculations. Like for example, if there are 100 nodes in a tree and most of
 * the nodes are on the right side of the tree, and K = 1, then we are doing
 * unnecessary computations of getting the inorder traversal, while our result
 * lies on the left side by only doing a few computations. The code is:
 *
 * int count = 0; int result = Integer.MIN_VALUE;
 *
 * public int kthSmallest(TreeNode root, int k){
        inorder(root, k); return
 *      result;
    }
 *
 * public void inorder(TreeNode root, int k) {
        if(root == null) return;
 *
 *      inorder(root.left, k);
        count ++;//Increment counter after we are going to theleft subtree, to skip the
parent root.val of the subtree
 *
 *      if(count == k)//If we are at K, then that is the answer
            result = root.val;
 *
 *      traverse(root.right, k);
    }
 */
```

157. `231. Power of Two.java`

```
/*
 *
 * Given an integer, write a function to determine if it is a power of two.
 *
 * Example 1:
 *
 * Input: 1
 * Output: true
 * Explanation: 20 = 1
 * Example 2:
 *
 * Input: 16
 * Output: true
 * Explanation: 24 = 16
 * Example 3:
 *
 * Input: 218
 * Output: false
 *
 */

class Solution {

    public boolean isPowerOfTwo(int n) {
        /*
         * If n is the power of two:
         *
         * n = 2 ^ 0 = 1 = 0b0000...00000001, and (n - 1) = 0 = 0b0000...0000. n
         * = 2 ^ 1 = 2 = 0b0000...00000010, and (n - 1) = 1 = 0b0000...0001. n =
         * 2 ^ 2 = 4 = 0b0000...00000100, and (n - 1) = 3 = 0b0000...0011. n = 2
         * ^ 3 = 8 = 0b0000...00001000, and (n - 1) = 7 = 0b0000...0111. we have
         * n & (n-1) == 0b0000...0000 == 0
         *
         * Otherwise, n & (n-1) != 0.
         *
         * For example, n =14 = 0b0000...1110, and (n - 1) = 13 = 0b0000...1101.
         */
        return n > 0 && ((n & (n - 1)) == 0);
    }
}
```

158. `232. Implement Queue using Stacks.java`

```
/*
 *
 * Implement the following operations of a queue using stacks.
 *
 * push(x) -- Push element x to the back of queue.
 * pop() -- Removes the element from in front of queue.
 * peek() -- Get the front element.
 * empty() -- Return whether the queue is empty.
 * Example:
 *
 * MyQueue queue = new MyQueue();
 *
 * queue.push(1);
 * queue.push(2);
 * queue.peek();  // returns 1
 * queue.pop();   // returns 1
 * queue.empty(); // returns false
 * Notes:
 *
 * You must use only standard operations of a stack -- which means only push to top,
peek/pop from top, size, and is empty operations are valid.
 * Depending on your language, stack may not be supported natively. You may simulate a
stack by using a list or deque (double-ended queue), as long as you use only standard
operations of a stack.
 * You may assume that all operations are valid (for example, no pop or peek operations
will be called on an empty queue).
 *
 */

class MyQueue {

    Stack<Integer> temp = new Stack<Integer>();
    Stack<Integer> value = new Stack<Integer>();

    /**
     * Initialize your data structure here.
     */
    public MyQueue() {

    }

    /**
     * Push element x to the back of queue.
     */
    public void push(int x) {
        if (value.isEmpty()) {
            value.push(x);
        } else {
            while (!value.isEmpty()) {
                temp.push(value.pop());
            }
```

```
            value.push(x);

            while (!temp.isEmpty()) {
                value.push(temp.pop());
            }
        }
    }

    /**
     * Removes the element from in front of queue and returns that element.
     */
    public int pop() {
        return value.pop();
    }

    /**
     * Get the front element.
     */
    public int peek() {
        return value.peek();
    }

    /**
     * Returns whether the queue is empty.
     */
    public boolean empty() {
        return value.isEmpty();
    }
}

/**
 * Your MyQueue object will be instantiated and called as such:
 * MyQueue obj = new MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * boolean param_4 = obj.empty();
 */
```

159. `234. Palindrome Linked List.java`

```
/*
 *
 * Given a singly linked list, determine if it is a palindrome.
 *
 * Example 1:
 *
 * Input: 1->2
 * Output: false
 * Example 2:
 *
 * Input: 1->2->2->1
 * Output: true
 *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {

    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) {
            return true;
        }
        ListNode tmp = new ListNode(0);
        tmp.next = head;
        Stack<ListNode> stack = new Stack<>();
        ListNode slow = tmp, fast = tmp;
        // 1 1   1 0 1
        while (fast != null && fast.next != null) {
            fast = fast.next.next;
            slow = slow.next;
            stack.push(slow);

        }
        if (fast != null && fast.next == null) {
            slow = slow.next;
        }
        while (!stack.isEmpty()) {
            if (stack.pop().val != slow.val) {
                return false;
            }
            slow = slow.next;
        }
        return true;
```

```
    }
}
```

160. `235. Lowest Common Ancestor of a Binary Search Tree.java`

```
/*
 *
 * Given a binary search tree (BST), find the lowest common ancestor (LCA) of two given
nodes in the BST.
 *
 * According to the definition of LCA on Wikipedia: “The lowest common ancestor is
defined between two nodes p and q as the lowest node in T that has both p and q as
descendants (where we allow a node to be a descendant of itself).”
 *
 * Given binary search tree:  root = [6,2,8,0,4,7,9,null,null,3,5]
 *
 * Example 1:
 *
 * Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
 * Output: 6
 * Explanation: The LCA of nodes 2 and 8 is 6.
 * Example 2:
 *
 * Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
 * Output: 2
 * Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself
according to the LCA definition.
 *
 *
 * Note:
 *
 * All of the nodes' values will be unique.
 * p and q are different and both values will exist in the BST.
 *
 */

class Solution {

    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode m = root;

        if (m.val > p.val && m.val < q.val) {
            return m;
        } else if (m.val > p.val && m.val > q.val) {
            return lowestCommonAncestor(root.left, p, q);
        } else if (m.val < p.val && m.val < q.val) {
            return lowestCommonAncestor(root.right, p, q);
        }

        return root;

    }
}
```

161. `236. Lowest Common Ancestor of a Binary Tree.java`

```
/*
 *
 * Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the
tree.
 *
 * According to the definition of LCA on Wikipedia: “The lowest common ancestor is
defined between two nodes p and q as the lowest node in T that has both p and q as
descendants (where we allow a node to be a descendant of itself).”
 *
 * Given the following binary tree:  root = [3,5,1,6,2,0,8,null,null,7,4]
 *
 *
 *
 *
 * Example 1:
 *
 * Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
 * Output: 3
 * Explanation: The LCA of nodes 5 and 1 is 3.
 * Example 2:
 *
 * Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
 * Output: 5
 * Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself
according to the LCA definition.
 *
 */

class Solution {

    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) {
            return null;
        }

        if (root == p || root == q) {
            return root;
        }

        TreeNode l = lowestCommonAncestor(root.left, p, q);
        TreeNode r = lowestCommonAncestor(root.right, p, q);

        if (l != null && r != null) {
            return root;
        } else if (l == null && r == null) {
            return null;
        } else {
            return l == null ? r : l;
        }
    }
}
```

162. `237. Delete Node in a Linked List.java`

```
/*
 *
 * Write a function to delete a node (except the tail) in a singly linked list, given
only access to that node.
 *
 * Given linked list -- head = [4,5,1,9], which looks like following:
 *
 *
 * Example 1:
 *
 * Input: head = [4,5,1,9], node = 5
 * Output: [4,1,9]
 * Explanation: You are given the second node with value 5, the linked list should become
4 -> 1 -> 9 after calling your function.
 * Example 2:
 *
 * Input: head = [4,5,1,9], node = 1
 * Output: [4,5,9]
 * Explanation: You are given the third node with value 1, the linked list should become
4 -> 5 -> 9 after calling your function.
 *
 *
 * Note:
 *
 * The linked list will have at least two elements.
 * All of the nodes' values will be unique.
 * The given node will not be the tail and it will always be a valid node of the linked
list.
 * Do not return anything from your function.
 *
 */

class Solution {

    public void deleteNode(ListNode node) {
        node.val = node.next.val;
        node.next = node.next.next;
    }
}
```

163. `238. Product of Array Except Self.java`

```
/*
 *
 * Given an array nums of n integers where n > 1,  return an array output such that
output[i] is equal to the product of all the elements of nums except nums[i].
 *
 * Example:
 *
 * Input:  [1,2,3,4]
 * Output: [24,12,8,6]
 * Note: Please solve it without division and in O(n).
 *
 * Follow up:
 * Could you solve it with constant space complexity? (The output array does not count as
extra space for the purpose of space complexity analysis.)
 *
 */

class Solution {

    public int[] productExceptSelf(int[] nums) {
        int[] result = new int[nums.length];

        int[] t1 = new int[nums.length];
        int[] t2 = new int[nums.length];

        t1[0] = 1;
        t2[nums.length - 1] = 1;

        //scan from left to right
        for (int i = 0; i < nums.length - 1; i++) {
            t1[i + 1] = nums[i] * t1[i];
        }

        //scan from right to left
        for (int i = nums.length - 1; i > 0; i--) {
            t2[i - 1] = t2[i] * nums[i];
        }

        //multiply
        for (int i = 0; i < nums.length; i++) {
            result[i] = t1[i] * t2[i];
        }

        return result;
    }
}
```

164. `240. Search a 2D Matrix II.java`

```
/*
 *
 * Write an efficient algorithm that searches for a value in an m x n matrix. This matrix
has the following properties:
 *
 * Integers in each row are sorted in ascending from left to right.
 * Integers in each column are sorted in ascending from top to bottom.
 * Example:
 *
 * Consider the following matrix:
 *
 * [
 * [1,   4,  7, 11, 15],
 * [2,   5,  8, 12, 19],
 * [3,   6,  9, 16, 22],
 * [10, 13, 14, 17, 24],
 * [18, 21, 23, 26, 30]
 * ]
 * Given target = 5, return true.
 *
 * Given target = 20, return false
 *
 */

public class Solution {

    public boolean searchMatrix(int[][] matrix, int target) {
        if (matrix == null || matrix.length < 1 || matrix[0].length < 1) {
            return false;
        }

        int col = matrix[0].length - 1;
        int row = 0;

        while (col >= 0 && row <= matrix.length - 1) {
            if (target == matrix[row][col]) {
                return true;
            } else if (target < matrix[row][col]) {
                col--;
            } else if (target > matrix[row][col]) {
                row++;
            }

        }
        return false;
    }
}
```

165. `242. Valid Anagram.java`

```
/*
 *
 * Given two strings s and t , write a function to determine if t is an anagram of s.
 *
 * Example 1:
 *
 * Input: s = "anagram", t = "nagaram"
 * Output: true
 * Example 2:
 *
 * Input: s = "rat", t = "car"
 * Output: false
 * Note:
 * You may assume the string contains only lowercase alphabets.
 *
 * Follow up:
 * What if the inputs contain unicode characters? How would you adapt your solution to
such case?
 *
 */

class Solution {

    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) {
            return false;
        }

        HashMap<Character, Integer> map = new HashMap<Character, Integer>();

        for (int i = 0; i < s.length(); i++) {
            char c1 = s.charAt(i);
            if (map.containsKey(c1)) {
                map.put(c1, map.get(c1) + 1);
            } else {
                map.put(c1, 1);
            }
        }

        for (int i = 0; i < s.length(); i++) {
            char c2 = t.charAt(i);
            if (map.containsKey(c2)) {
                if (map.get(c2) == 1) {
                    map.remove(c2);
                } else {
                    map.put(c2, map.get(c2) - 1);
                }
            } else {
                return false;
            }
        }
```

```
        if (map.size() > 0) {
            return false;
        }

        return true;
    }
}
```

166. `257. Binary Tree Paths.java`

```
/*
 *
 * Given a binary tree, return all root-to-leaf paths.
 *
 * Note: A leaf is a node with no children.
 *
 * Example:
 *
 * Input:
 *
 * 1
 * /   \
 * 2     3
 * \
 * 5
 *
 * Output: ["1->2->5", "1->3"]
 *
 * Explanation: All root-to-leaf paths are: 1->2->5, 1->3
 *
 */

class Solution {

    public List<String> binaryTreePaths(TreeNode root) {
        ArrayList<String> res = new ArrayList<String>();
        helper(root, "", res);
        return res;
    }

    void helper(TreeNode root, String path, ArrayList<String> res) {
        if (root == null) {
            return;
        }
        if (root.left == null && root.right == null) {
            res.add(path + root.val);
        }
        helper(root.left, path + root.val + "->", res);
        helper(root.right, path + root.val + "->", res);
    }
}
```

167. `258. Add Digits.java`

```
/*
 *
 * Given a non-negative integer num, repeatedly add all its digits until the result has
only one digit.
 *
 * Example:
 *
 * Input: 38
 * Output: 2
 * Explanation: The process is like: 3 + 8 = 11, 1 + 1 = 2.
 * Since 2 has only one digit, return it.
 * Follow up:
 * Could you do it without any loop/recursion in O(1) runtime?
 *
 */

class Solution {

    public int addDigits(int num) {
        int res = num % 9;
        return (res != 0 || num == 0) ? res : 9;

    }
}
```

168. `264. Ugly Number II.java`

```
/*
 *
 * Write a program to find the n-th ugly number.
 *
 * Ugly numbers are positive numbers whose prime factors only include 2, 3, 5.  *
 * Example:
 *
 * Input: n = 10
 * Output: 12
 * Explanation: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12 is the sequence of the first 10 ugly
numbers.
 * Note:  *
 * 1 is typically treated as an ugly number.
 * n does not exceed 1690.
 *
 */

class Solution {

    public int nthUglyNumber(int n) {

        if (n == 1) {
            return n;
        }

        Queue<Long> q = new PriorityQueue<Long>();
        int[] nums = {2, 3, 5};
        Long result = Long.valueOf(1);
        q.offer(result);

        for (int i = 0; i < n; i++) {
            // Each time we poll the peak value of q, is the ith number
            result = q.poll();
            for (int num : nums) {
                Long uglyNum = result * num;
                if (!q.contains(uglyNum)) {
                    q.offer(uglyNum);
                }
            }
        }
        return result.intValue();
    }
}

/*
   public int nthUglyNumber(int n) {
    if(n<=0)
        return 0;

    ArrayList<Integer> list = new ArrayList<Integer>();
    list.add(1);
```

```
    int i=0;
    int j=0;
    int k=0;

    while(list.size()<n){
        int m2 = list.get(i)*2;
        int m3 = list.get(j)*3;
        int m5 = list.get(k)*5;

        int min = Math.min(m2, Math.min(m3, m5));
        list.add(min);

        if(min==m2)
            i++;

        if(min==m3)
            j++;

        if(min==m5)
            k++;
    }

    return list.get(list.size()-1);
}*/
```

169. `268. Missing Number.java`

```
/*
 *
 * Given an array containing n distinct numbers taken from 0, 1, 2, ..., n, find the one
that is missing from the array.
 *
 * Example 1:
 *
 * Input: [3,0,1]
 * Output: 2
 * Example 2:
 *
 * Input: [9,6,4,2,3,5,7,0,1]
 * Output: 8
 * Note:
 * Your algorithm should run in linear runtime complexity. Could you implement it using
only constant extra space complexity?
 *
 */

class Solution {

    public int missingNumber(int[] nums) {
        int res = nums.length;
        for (int i = 0; i < nums.length; i++) {
            res ^= i;
            res ^= nums[i];
        }
        return res;
    }
}
```

170. `278. First Bad Version.java`

```
/*
 *
 * You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.
 *
 * Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one, which causes all the following ones to be bad.
 *
 * You are given an API bool isBadVersion(version) which will return whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.
 *
 * Example:
 *
 * Given n = 5, and version = 4 is the first bad version.
 *
 * call isBadVersion(3) -> false
 * call isBadVersion(5) -> true
 * call isBadVersion(4) -> true
 *
 * Then 4 is the first bad version.  *
 */

 /*
 * The isBadVersion API is defined in the parent class VersionControl.
 * boolean isBadVersion(int version);
 */
public class Solution extends VersionControl {

    public int firstBadVersion(int n) {
        int mid, left = 1, right = n;
        int ans = -1;
        while (left < right) {
            mid = left + (right - left) / 2;
            boolean res = isBadVersion(mid);

            if (res) {
                right = mid;
            } else {
                left = mid + 1;
            }

        }
        return left;
    }
}
```

171. `279. Perfect Squares.java`

```
/*
 *
 * Given a positive integer n, find the least number of perfect square numbers (for
example, 1, 4, 9, 16, ...) which sum to n.
 *
 * Example 1:
 *
 * Input: n = 12
 * Output: 3
 * Explanation: 12 = 4 + 4 + 4.
 * Example 2:
 *
 * Input: n = 13
 * Output: 2
 * Explanation: 13 = 4 + 9.
 *
 */

class Solution {

    public int numSquares(int n) {
        int[] DP = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            int min = Integer.MAX_VALUE;
            for (int j = 1; j * j <= i; j++) {
                min = Math.min(min, DP[i - j * j] + 1);
            }
            DP[i] = min;
        }
        return DP[n];
    }
}
```

172. `283. Move Zeroes.java`

```
/*
 *
 * Given an array nums, write a function to move all 0's to the end of it while
maintaining the relative order of the non-zero elements.
 *
 * Example:
 *
 * Input: [0,1,0,3,12]
 * Output: [1,3,12,0,0]
 * Note:
 *
 * You must do this in-place without making a copy of the array.
 * Minimize the total number of operations.
 *
 */

class Solution {

    public void moveZeroes(int[] nums) {
        int i = 0;
        int j = 0;

        while (j < nums.length) {
            if (nums[j] == 0) {
                j++;
            } else {
                nums[i] = nums[j];
                i++;
                j++;
            }
        }

        while (i < nums.length) {
            nums[i] = 0;
            i++;
        }
    }
}
```

173. `287. Find the Duplicate Number.java`

```
/*
 *
 * Given an array nums containing n + 1 integers where each integer is between 1 and n
(inclusive), prove that at least one duplicate number must exist. Assume that there is
only one duplicate number, find the duplicate one.
 *
 * Example 1:
 *
 * Input: [1,3,4,2,2]
 * Output: 2
 * Example 2:
 *
 * Input: [3,1,3,4,2]
 * Output: 3
 * Note:
 *
 * You must not modify the array (assume the array is read only).
 * You must use only constant, O(1) extra space.
 * Your runtime complexity should be less than O(n2).
 * There is only one duplicate number in the array, but it could be repeated more than
once.
 *
 */

class Solution {

    public int findDuplicate(int[] values) {
        HashMap<Integer, Integer> map = new HashMap<>();
        int result = -1;

        for (int value : values) {
            if (map.containsKey(value)) {
                result = value;
                break;
            } else {
                map.put(value, 1);
            }

        }

        return result;

    }
}
```

174. `289. Game of Life.java`

```
/*
 *
 * According to the Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."
 *
 * Given a board with m by n cells, each cell has an initial state live (1) or dead (0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):
 *
 * Any live cell with fewer than two live neighbors dies, as if caused by under-population.
 * Any live cell with two or three live neighbors lives on to the next generation.
 * Any live cell with more than three live neighbors dies, as if by over-population..
 * Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.
 * Write a function to compute the next state (after one update) of the board given its current state. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously.
 *
 * Example:
 *
 * Input:
 * [
 * [0,1,0],
 * [0,0,1],
 * [1,1,1],
 * [0,0,0]
 * ]
 * Output:
 * [
 * [0,0,0],
 * [1,0,1],
 * [0,1,1],
 * [0,1,0]
 * ]
 * Follow up:
 *
 * Could you solve it in-place? Remember that the board needs to be updated at the same time: You cannot update some cells first and then use their updated values to update other cells.
 * In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area encroaches the border of the array. How would you address these problems?
 *
 */

class Solution {

    int[][] dir = {{1, -1}, {1, 0}, {1, 1}, {0, -1}, {0, 1}, {-1, -1}, {-1, 0}, {-1, 1}};

    public void gameOfLife(int[][] board) {
```

```
        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                int live = 0;
                for (int[] d : dir) {
                    int nx = d[0] + i;
                    int ny = d[1] + j;

                    if (nx < 0 || nx >= board.length || ny < 0 || ny >= board[0].length)
{
                        continue;
                    }
                    if (board[nx][ny] == 1 || board[nx][ny] == 2) {
                        live++;
                    }
                }

                if (board[i][j] == 0 && live == 3) {
                    board[i][j] = 3;//It is a dead cell. Make it a live cell.
                }
                if (board[i][j] == 1 && (live < 2 || live > 3)) {
                    board[i][j] = 2;//It is a live cell. But it needs to be dead cell
since there is over and under population scenario
                }
            }
        }

        for (int i = 0; i < board.length; i++) {
            for (int j = 0; j < board[0].length; j++) {
                board[i][j] %= 2; //Because at 0 and 1 we have dead and live cell. At 2
and 3 we have the "new" live and dead cell.
            }
        }
    }
}
```

175. `295. Find Median from Data Stream.java`

```
/*
 *
 * Median is the middle value in an ordered integer list. If the size of the list is
even, there is no middle value. So the median is the mean of the two middle value.
 *
 * For example,
 * [2,3,4], the median is 3
 *
 * [2,3], the median is (2 + 3) / 2 = 2.5
 *
 * Design a data structure that supports the following two operations:
 *
 * void addNum(int num) - Add a integer number from the data stream to the data
structure.
 * double findMedian() - Return the median of all elements so far.
 *
 *
 * Example:
 *
 * addNum(1)
 * addNum(2)
 * findMedian() -> 1.5
 * addNum(3)
 * findMedian() -> 2
 *
 */

class MedianFinder {

    PriorityQueue<Integer> maxHeap;//lower half
    PriorityQueue<Integer> minHeap;//higher half

    public MedianFinder() {
        maxHeap = new PriorityQueue<Integer>(Collections.reverseOrder());
        minHeap = new PriorityQueue<Integer>();
    }

    // Adds a number into the data structure.
    public void addNum(int num) {
        maxHeap.offer(num);
        minHeap.offer(maxHeap.poll());
        //First add the number to the max heap then add it to the minheap, and then check
if the heaps are balanced or not!
        //We are adding all the elements to the maxHeap first bcoz then in FindMedian()
we can see in the else part that we are returning the maxHeap.peek(). So inorder to get
the result correctly we need to add all the elements to the maxHeap first, and then one
by one see which element would fit in maxHeap and which one would fit in minHeap

        if (maxHeap.size() < minHeap.size()) {
            maxHeap.offer(minHeap.poll());
        }
```

```
    }

    // Returns the median of current data stream
    public double findMedian() {
        if (maxHeap.size() == minHeap.size()) {
            return (double) (maxHeap.peek() + (minHeap.peek())) / 2;
        } else {
            return maxHeap.peek();
        }
    }
}

/**
 * Your MedianFinder object will be instantiated and called as such:
 * MedianFinder obj = new MedianFinder();
 * obj.addNum(num);
 * double param_2 = obj.findMedian();
 */
```

176. `297. Serialize and Deserialize Binary Tree.java`

```
/*
 *
 * Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.
 *
 * Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.
 *
 * Example:  *
 * You may serialize the following tree:
 *
 * 1
 * / \
 * 2   3
 * / \
 * 4   5
 *
 * as "[1,2,3,null,null,4,5]"
 * Clarification: The above format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.
 *
 * Note: Do not use class member/global/static variables to store states. Your serialize and deserialize algorithms should be stateless.
 *
 */

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
public class Codec {

    private final static String NULL = "null";
    private final static String SEP = ",";

    // Encodes a tree to a single string.
    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        serializeHelper(root, sb);

        sb.deleteCharAt(sb.length() - 1);
```

```
        return sb.toString();
    }

    private void serializeHelper(TreeNode node, StringBuilder sb) {
        if (node == null) {
            sb.append(NULL);
            sb.append(SEP);
            return;
        }
        sb.append(String.valueOf(node.val));
        sb.append(SEP);

        serializeHelper(node.left, sb);
        serializeHelper(node.right, sb);
    }

    private int index;

    // Decodes your encoded data to tree.
    public TreeNode deserialize(String data) {
        String[] ss = data.split(",");//split on comma
        index = 0;
        return deserializeHelper(ss);
    }

    private TreeNode deserializeHelper(String[] ss) {
        if (ss[index].equals(NULL)) {
            index++;
            return null;
        } else {
            TreeNode node = new TreeNode(Integer.parseInt(ss[index]));
            index++;
            node.left = deserializeHelper(ss);
            node.right = deserializeHelper(ss);
            return node;
        }
    }
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.deserialize(codec.serialize(root));
```

177. `299. Bulls and Cows.java`

```
/*
 *
 * You are playing the following Bulls and Cows game with your friend: You write down a
number and ask your friend to guess what the number is. Each time your friend makes a
guess, you provide a hint that indicates how many digits in said guess match your secret
number exactly in both digit and position (called "bulls") and how many digits match the
secret number but locate in the wrong position (called "cows"). Your friend will use
successive guesses and hints to eventually derive the secret number.
 *
 * Write a function to return a hint according to the secret number and friend's guess,
use A to indicate the bulls and B to indicate the cows.  *
 * Please note that both secret number and friend's guess may contain duplicate digits.
 *
 * Example 1:
 *
 * Input: secret = "1807", guess = "7810"
 *
 * Output: "1A3B"
 *
 * Explanation: 1 bull and 3 cows. The bull is 8, the cows are 0, 1 and 7.
 * Example 2:
 *
 * Input: secret = "1123", guess = "0111"
 *
 * Output: "1A1B"
 *
 * Explanation: The 1st 1 in friend's guess is a bull, the 2nd or 3rd 1 is a cow.
 * Note: You may assume that the secret number and your friend's guess only contain
digits, and their lengths are always equal.
 *
 */

class Solution {

    public String getHint(String secret, String guess) {
        int bulls = 0;
        int cows = 0;
        int[] numbers = new int[10];
        for (int i = 0; i < secret.length(); i++) {
            if (secret.charAt(i) == guess.charAt(i)) {
                bulls++;
            } else {
                if (numbers[secret.charAt(i) - '0']++ < 0) {
                    cows++;
                }
                if (numbers[guess.charAt(i) - '0']-- > 0) {
                    cows++;
                }
            }
        }
        return bulls + "A" + cows + "B";
```

```
    }
}
```

178. `300. Longest Increasing Subsequence.java`

```
/*
 *
 * Given an unsorted array of integers, find the length of longest increasing
subsequence.
 *
 * Example:
 *
 * Input: [10,9,2,5,3,7,101,18]
 * Output: 4
 * Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length
is 4.
 * Note:
 *
 * There may be more than one LIS combination, it is only necessary for you to return the
length.
 * Your algorithm should run in O(n2) complexity.
 * Follow up: Could you improve it to O(n log n) time complexity?
 *
 */

class Solution {

    public int lengthOfLIS(int[] nums) {
        //Explanation: Tushar Roy -> https://www.youtube.com/watch?v=CE2b_-XfVDk

        // Base case
        if (nums.length <= 1) {
            return nums.length;
        }

        // This will be our array to track longest sequence length
        int T[] = new int[nums.length];

        // Fill each position with value 1 in the array
        for (int i = 0; i < nums.length; i++) {
            T[i] = 1;
        }

        // Mark one pointer at i. For each i, start from j=0.
        for (int i = 1; i < nums.length; i++) {
            for (int j = 0; j < i; j++) {
                // It means next number contributes to increasing sequence.
                if (nums[j] < nums[i]) {
                    // But increase the value only if it results in a larger value of the
sequence than T[i]
                    // It is possible that T[i] already has larger value from some
previous j'th iteration
                    if (T[j] + 1 > T[i]) {
                        T[i] = T[j] + 1;
                    }
                }
```

```
            }
        }

        // Find the maximum length from the array that we just generated
        int maxLength = 0;
        for (int i = 0; i < T.length; i++) {
            maxLength = Math.max(maxLength, T[i]);
        }

        return maxLength;
    }
}
```

179. `301. Remove Invalid Parentheses.java`

```
/*
 *
 * Remove the minimum number of invalid parentheses in order to make the input string
valid. Return all possible results.
 *
 * Note: The input string may contain letters other than the parentheses ( and ).
 *
 * Example 1:
 *
 * Input: "()())()"
 * Output: ["()()()", "(())()"]
 * Example 2:
 *
 * Input: "(a())()"
 * Output: ["(a)()()", "(a())()"]
 * Example 3:
 *
 * Input: ")("
 * Output: [""]
 *
 */

public class Solution {

    public List<String> removeInvalidParentheses(String s) {
        List<String> res = new ArrayList<>();

        // sanity check
        if (s == null) {
            return res;
        }

        Set<String> visited = new HashSet<>();
        Queue<String> queue = new LinkedList<>();

        // initialize
        queue.add(s);
        visited.add(s);

        boolean found = false;

        while (!queue.isEmpty()) {
            s = queue.poll();

            if (isValid(s)) {
                // found an answer, add to the result
                res.add(s);
                found = true;
            }

            if (found) {
```

```
                continue;
            }

            // generate all possible states
            for (int i = 0; i < s.length(); i++) {
                // we only try to remove left or right paren
                if (s.charAt(i) != '(' && s.charAt(i) != ')') {
                    continue; //for any other random character, ignore this iteration
                }
                // Remove the current char i in the string
                // So for that we are going to create 2 substrings
                // First string is from (0 to i-1) and second is (i+1 to end), and then
concatenate the two substrings and check if the new string has already been processed or
not!
                String t = s.substring(0, i) + s.substring(i + 1);

                if (!visited.contains(t)) {
                    // for each state, if it's not visited, add it to the queue
                    queue.add(t);
                    visited.add(t);
                }
            }
        }

        return res;
    }

    // helper function checks if string s contains valid parantheses
    boolean isValid(String s) {
        int count = 0;

        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c == '(') {
                count++;
            }
            if (c == ')' && count-- == 0) {
                return false;
            }
        }

        return count == 0;
    }
}
```

180. `303. Range Sum Query - Immutable.java`

```
/*
 *
 * Given an integer array nums, find the sum of the elements between indices i and j (i ≤ j), inclusive.
 *
 * Example:
 * Given nums = [-2, 0, 3, -5, 2, -1]
 *
 * sumRange(0, 2) -> 1
 * sumRange(2, 5) -> -1
 * sumRange(0, 5) -> -3
 * Note:
 * You may assume that the array does not change.
 * There are many calls to sumRange function.
 *
 */

class NumArray {

    private int[] data;

    public NumArray(int[] nums) {
        data = nums;
    }

    public int sumRange(int i, int j) {
        int sum = 0;
        while (i <= j) {
            sum += data[i];
            i++;
        }
        return sum;
    }
}

/**
 * Your NumArray object will be instantiated and called as such:
 * NumArray obj = new NumArray(nums);
 * int param_1 = obj.sumRange(i,j);
 */
```

181. `304. Range Sum Query 2D - Immutable.java`

```
/*
 *
 * Given a 2D matrix matrix, find the sum of the elements inside the rectangle defined by
its upper left corner (row1, col1) and lower right corner (row2, col2).
 *
 * Range Sum Query 2D
 * The above rectangle (with the red border) is defined by (row1, col1) = (2, 1) and
(row2, col2) = (4, 3), which contains sum = 8.
 *
 * Example:
 * Given matrix = [
 * [3, 0, 1, 4, 2],
 * [5, 6, 3, 2, 1],
 * [1, 2, 0, 1, 5],
 * [4, 1, 0, 1, 7],
 * [1, 0, 3, 0, 5]
 * ]
 *
 * sumRegion(2, 1, 4, 3) -> 8
 * sumRegion(1, 1, 2, 2) -> 11
 * sumRegion(1, 2, 2, 4) -> 12
 * Note:
 * You may assume that the matrix does not change.
 * There are many calls to sumRegion function.
 * You may assume that row1 ≤ row2 and col1 ≤ col2.
 *
 */

class NumMatrix {

    private int[][] dp;

    public NumMatrix(int[][] matrix) {
        if (matrix.length == 0 || matrix[0].length == 0) {
            return;
        }
        dp = new int[matrix.length][matrix[0].length + 1];
        for (int r = 0; r < matrix.length; r++) {
            for (int c = 0; c < matrix[0].length; c++) {
                dp[r][c + 1] = dp[r][c] + matrix[r][c];
            }
        }
    }

    public int sumRegion(int row1, int col1, int row2, int col2) {
        int sum = 0;
        for (int row = row1; row <= row2; row++) {
            sum += dp[row][col2 + 1] - dp[row][col1];
        }
        return sum;
    }
```

```
}

/**
 * Your NumMatrix object will be instantiated and called as such:
 * NumMatrix obj = new NumMatrix(matrix);
 * int param_1 = obj.sumRegion(row1,col1,row2,col2);
 */
```

182. `310. Minimum Height Trees.java`

```
/*

For an undirected graph with tree characteristics, we can choose any node as the root.
The result graph is then a rooted tree. Among all possible rooted trees, those with
minimum height are called minimum height trees (MHTs). Given such a graph, write a
function to find all the MHTs and return a list of their root labels.

Format
The graph contains n nodes which are labeled from 0 to n - 1. You will be given the
number n and a list of undirected edges (each edge is a pair of labels).

You can assume that no duplicate edges will appear in edges. Since all edges are
undirected, [0, 1] is the same as [1, 0] and thus will not appear together in edges.

Example 1 :

Input: n = 4, edges = [[1, 0], [1, 2], [1, 3]]

        0
        |
        1
       / \
      2   3

Output: [1]
Example 2 :

Input: n = 6, edges = [[0, 3], [1, 3], [2, 3], [4, 3], [5, 4]]

     0  1  2
      \ | /
        3
        |
        4
        |
        5

Output: [3, 4]
Note:

According to the definition of tree on Wikipedia: “a tree is an undirected graph in which
any two vertices are connected by exactly one path. In other words, any connected graph
without simple cycles is a tree.”
The height of a rooted tree is the number of edges on the longest downward path between
the root and a leaf.

*/

class Solution {

    public List<Integer> findMinHeightTrees(int n, int[][] edges) {
```

```
        List<List<Integer>> myGraph = new ArrayList<List<Integer>>();
        List<Integer> res = new ArrayList<Integer>();
        if (n == 1) {
            res.add(0);
            return res;
        }
        int[] degree = new int[n];
        for (int i = 0; i < n; i++) {
            myGraph.add(new ArrayList<Integer>());
        }
        for (int i = 0; i < edges.length; i++) {
            myGraph.get(edges[i][0]).add(edges[i][1]);
            myGraph.get(edges[i][1]).add(edges[i][0]);
            degree[edges[i][0]]++;
            degree[edges[i][1]]++;
        }
        Queue<Integer> myQueue = new ArrayDeque<Integer>();

        for (int i = 0; i < n; i++) {
            if (degree[i] == 0) {
                return res;
            } else if (degree[i] == 1) {
                myQueue.offer(i);
            }
        }

        while (!myQueue.isEmpty()) {
            res = new ArrayList<Integer>();
            int count = myQueue.size();

            for (int i = 0; i < count; i++) {
                int curr = myQueue.poll();
                res.add(curr);
                degree[curr]--;
                for (int k = 0; k < myGraph.get(curr).size(); k++) {
                    int next = myGraph.get(curr).get(k);
                    if (degree[next] == 0) {
                        continue;
                    }
                    if (degree[next] == 2) {
                        myQueue.offer(next);
                    }
                    degree[next]--;
                }
            }
        }
        return res;
    }
}
```

183. `322. Coin Change.java`

```
/*
 *
 * You are given coins of different denominations and a total amount of money amount.
Write a function to compute the fewest number of coins that you need to make up that
amount. If that amount of money cannot be made up by any combination of the coins, return
-1.
 *
 * Example 1:
 *
 * Input: coins = [1, 2, 5], amount = 11
 * Output: 3
 * Explanation: 11 = 5 + 5 + 1
 * Example 2:
 *
 * Input: coins = [2], amount = 3
 * Output: -1
 * Note:
 * You may assume that you have an infinite number of each kind of coin.
 *
 */

class Solution {

    public int coinChange(int[] coins, int amount) {

        if (coins == null || coins.length == 0 || amount < 1) {
            return 0;
        }

        Deque<Integer> queue = new ArrayDeque<Integer>();
        Set<Integer> visited = new HashSet<Integer>();
        queue.addFirst(amount);
        visited.add(amount);
        int level = 0;

        //We are going to explore every amount possible in  terms of level
        while (!queue.isEmpty()) {
            int size = queue.size();

            while (size-- > 0) {
                int curr = queue.removeLast();
                if (curr == 0) {
                    return level;
                }

                if (curr < 0) {
                    continue;
                }

                for (int coin : coins) {
                    int next = curr - coin;
```

```
                    if (next >= 0 && !visited.contains(next)) {
                        queue.addFirst(next);
                        visited.add(next);
                    }
                }
            }

            level++;
        }

        return -1;
    }
}
    //For example, first we have 11 as amount.
    /*            11
                /   |    \
              10     9      6
             /|\    /|\    /|\
            9 8 5  8 9 4  5 4 1 ..... so on
            We explore the amount in this way using a BFS

    */


    /*
    public static int coinChange(int[] coins, int amount) {
    if (coins == null || coins.length == 0 || amount <= 0)
        return 0;
    int[] minNumber = new int[amount + 1];
    Arrays.fill(minNumber, Integer.MAX_VALUE );
    minNumber[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < coins.length; j++) {
            if (coins[j] <= i && minNumber[i - coins[j]] != Integer.MAX_VALUE)
                minNumber[i] = Integer.min(minNumber[i], 1 + minNumber[i - coins[j]]);
        }
    }
    if (minNumber[amount] == Integer.MAX_VALUE)
        return -1;
    else
        return minNumber[amount];
}*/
```

184. `328. Odd Even Linked List.java`

```
/*
 *
 * Given a singly linked list, group all odd nodes together followed by the even nodes.
Please note here we are talking about the node number and not the value in the nodes.
 *
 * You should try to do it in place. The program should run in O(1) space complexity and
O(nodes) time complexity.
 *
 * Example 1:
 *
 * Input: 1->2->3->4->5->NULL
 * Output: 1->3->5->2->4->NULL
 * Example 2:
 *
 * Input: 2->1->3->5->6->4->7->NULL
 * Output: 2->3->6->7->1->5->4->NULL
 * Note:
 *
 * The relative order inside both the even and odd groups should remain as it was in the
input.
 * The first node is considered odd, the second node even and so on  *
 */

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
public class Solution {

    public ListNode oddEvenList(ListNode head) {
        if (head == null) {
            return null;
        }
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next;
            odd = odd.next;
            even.next = odd.next;
            even = even.next;
        }
        odd.next = evenHead;
        return head;
    }
}
```

185. `329. Longest Increasing Path in a Matrix.java`

```
/*
 *
 * Given an integer matrix, find the length of the longest increasing path.
 *
 * From each cell, you can either move to four directions: left, right, up or down. You
may NOT move diagonally or move outside of the boundary (i.e. wrap-around is not
allowed).
 *
 * Example 1:
 *
 * Input: nums =
 * [
 * [9,9,4],
 * [6,6,8],
 * [2,1,1]
 * ]
 * Output: 4
 * Explanation: The longest increasing path is [1, 2, 6, 9].
 * Example 2:
 *
 * Input: nums =
 * [
 * [3,4,5],
 * [3,2,6],
 * [2,2,1]
 * ]
 * Output: 4
 * Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not
allowed.
 *
 */

class Solution {

    public int longestIncreasingPath(int[][] matrix) {
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return 0;
        }

        int[] max = new int[1];
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[0].length; j++) {
                dfs(matrix, i, j, max, 1);
            }
        }

        return max[0];
    }

    public void dfs(int[][] matrix, int i, int j, int[] max, int len) {
```

```
        max[0] = Math.max(max[0], len);

        int m = matrix.length;
        int n = matrix[0].length;

        int[] dx = {-1, 0, 1, 0};
        int[] dy = {0, 1, 0, -1};

        for (int k = 0; k < 4; k++) {
            int x = i + dx[k];
            int y = j + dy[k];

            if (x >= 0 && x < m && y >= 0 && y < n && matrix[x][y] > matrix[i][j]) {
                dfs(matrix, x, y, max, len + 1);
            }
        }
    }
}
```

186. `336. Palindrome Pairs.java`

```
/*
 *
 * Given a list of unique words, find all pairs of distinct indices (i, j) in the given
list, so that the concatenation of the two words, i.e. words[i] + words[j] is a
palindrome.
 *
 * Example 1:
 *
 * Input: ["abcd","dcba","lls","s","sssll"]
 * Output: [[0,1],[1,0],[3,2],[2,4]]
 * Explanation: The palindromes are ["dcbaabcd","abcddcba","slls","llssssll"]
 * Example 2:
 *
 * Input: ["bat","tab","cat"]
 * Output: [[0,1],[1,0]]
 * Explanation: The palindromes are ["battab","tabbat"]
 *
 */

class Solution {

    public List<List<Integer>> palindromePairs(String[] words) {

        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < words.length; i++) {
            for (int j = 0; j != i && j < words.length; j++) {
                StringBuilder sb1 = new StringBuilder(words[i]);
                sb1.append(words[j]);
                if (isPalindrome(sb1.toString())) {
                    List<Integer> list = new ArrayList<>();
                    list.add(i);
                    list.add(j);
                    result.add(list);
                }

                StringBuilder sb2 = new StringBuilder(words[j]);
                sb2.append(words[i]);
                if (isPalindrome(sb2.toString())) {
                    List<Integer> list = new ArrayList<>();
                    list.add(j);
                    list.add(i);
                    result.add(list);
                }

            }
        }

        return result;
    }
```

```
    public boolean isPalindrome(String text) {
        StringBuilder plain = new StringBuilder(text);
        StringBuilder reverse = plain.reverse();
        return (reverse.toString()).equals(text);
    }
}
```

187. `337. House Robber III.java`

```
/*

The thief has found himself a new place for his thievery again. There is only one
entrance to this area, called the "root." Besides the root, each house has one and only
one parent house. After a tour, the smart thief realized that "all houses in this place
forms a binary tree". It will automatically contact the police if two directly-linked
houses were broken into on the same night.

Determine the maximum amount of money the thief can rob tonight without alerting the
police.

Example 1:

Input: [3,2,3,null,3,null,1]

     3
    / \
   2   3
    \   \
     3   1

Output: 7
Explanation: Maximum amount of money the thief can rob = 3 + 3 + 1 = 7.
Example 2:

Input: [3,4,5,1,3,null,1]

     3
    / \
   4   5
  / \   \
 1   3   1

Output: 9
Explanation: Maximum amount of money the thief can rob = 4 + 5 = 9.

*/


class Solution {

    public int rob(TreeNode root) {
        if (root == null) {
            return 0;
        }

        int[] result = helper(root);
        return Math.max(result[0], result[1]);
    }

    public int[] helper(TreeNode root) {
```

```
        if (root == null) {
            int[] result = {0, 0};
            return result;
        }

        int[] result = new int[2];
        int[] left = helper(root.left);
        int[] right = helper(root.right);

        // result[0] is when root is selected, result[1] is when not.
        result[0] = root.val + left[1] + right[1];
        result[1] = Math.max(left[0], left[1]) + Math.max(right[0], right[1]);

        return result;
    }
}
```

188. `338. Counting Bits.java`

```
/*
 *
 * Given a non negative integer number num. For every numbers i in the range 0 ≤ i ≤ num
calculate the number of 1's in their binary representation and return them as an array.
 *
 * Example 1:
 *
 * Input: 2
 * Output: [0,1,1]
 * Example 2:
 *
 * Input: 5
 * Output: [0,1,1,2,1,2]
 * Follow up:
 *
 * It is very easy to come up with a solution with run time O(n*sizeof(integer)). But can
you do it in linear time O(n) /possibly in a single pass?
 * Space complexity should be O(n).
 * Can you do it like a boss? Do it without using any builtin function like
__builtin_popcount in c++ or in any other language.
 *
 */

class Solution {

    public int[] countBits(int num) {
        int[] result = new int[num + 1];

        for (int i = 0; i <= num; i++) {
            result[i] = countEach(i);
        }

        return result;
    }

    public int countEach(int num) {
        int result = 0;

        while (num != 0) {
            if (num % 2 == 1) {
                result++;
            }
            num = num / 2;
        }

        return result;
    }
}
```

189. `341. Flatten Nested List Iterator.java`

```
/*
 *
 * Given a nested list of integers, implement an iterator to flatten it.
 *
 * Each element is either an integer, or a list -- whose elements may also be integers or
other lists.
 *
 * Example 1:
 *
 * Input: [[1,1],2,[1,1]]
 * Output: [1,1,2,1,1]
 * Explanation: By calling next repeatedly until hasNext returns false,
 * the order of elements returned by next should be: [1,1,2,1,1].
 * Example 2:
 *
 * Input: [1,[4,[6]]]
 * Output: [1,4,6]
 * Explanation: By calling next repeatedly until hasNext returns false,
 * the order of elements returned by next should be: [1,4,6].
 *
 */

/**
 * // This is the interface that allows for creating nested lists.
 * // You should not implement it, or speculate about its implementation
 * public interface NestedInteger {
 *
 *     // @return true if this NestedInteger holds a single integer, rather than a nested
list.
 *     public boolean isInteger();
 *
 *     // @return the single integer that this NestedInteger holds, if it holds a single
integer
 *     // Return null if this NestedInteger holds a nested list
 *     public Integer getInteger();
 *
 *     // @return the nested list that this NestedInteger holds, if it holds a nested
list
 *     // Return null if this NestedInteger holds a single integer
 *     public List<NestedInteger> getList(); }
 */
public class NestedIterator implements Iterator<Integer> {

    Stack<NestedInteger> stack = new Stack<NestedInteger>();

    public NestedIterator(List<NestedInteger> nestedList) {
        if (nestedList == null) {
            return;
        }

        for (int i = nestedList.size() - 1; i >= 0; i--) {
```

```
            stack.push(nestedList.get(i));
        }
    }

    @Override
    public Integer next() {
        return stack.pop().getInteger();
    }

    @Override
    public boolean hasNext() {
        while (!stack.isEmpty()) {
            NestedInteger top = stack.peek();
            if (top.isInteger()) {
                return true;
            } else {
                stack.pop();
                for (int i = top.getList().size() - 1; i >= 0; i--) {
                    stack.push(top.getList().get(i));
                }
            }
        }

        return false;
    }
}

/**
 * Your NestedIterator object will be instantiated and called as such:
 * NestedIterator i = new NestedIterator(nestedList);
 * while (i.hasNext()) v[f()] = i.next();
 */
```

190. `344. Reverse String.java`

```
/*
 *
 * Write a function that reverses a string. The input string is given as an array of
characters char[].
 *
 * Do not allocate extra space for another array, you must do this by modifying the input
array in-place with O(1) extra memory.
 *
 * You may assume all the characters consist of printable ascii characters.
 *
 *
 *
 * Example 1:
 *
 * Input: ["h","e","l","l","o"]
 * Output: ["o","l","l","e","h"]
 * Example 2:
 *
 * Input: ["H","a","n","n","a","h"]
 * Output: ["h","a","n","n","a","H"]
 *
 */

class Solution {

    public String reverseString(String s) {
        int len = s.length();
        len = len * 2;
        char[] temp = new char[len];

        for (int i = 0; i < s.length(); i++) {
            temp[i] = s.charAt(i);
        }
        // System.out.println(temp.length);
        int k = s.length() - 1;
        char[] c = new char[s.length()];
        for (int i = 0; i < s.length() && k >= 0; i++) {
            temp[i + s.length()] = temp[k];
            k--;
            c[i] = temp[i + s.length()];
            // System.out.println(c[i]);
        }
        //System.out.println();

        return String.valueOf(c);

    }
}
```

191. `345. Reverse Vowels of a String.java`

```
/*
 *
 * Write a function that takes a string as input and reverse only the vowels of a string.
 *
 * Example 1:
 *
 * Input: "hello"
 * Output: "holle"
 * Example 2:
 *
 * Input: "leetcode"
 * Output: "leotcede"
 * Note:
 * The vowels does not include the letter "y".
 *
 *
 */

class Solution {

    public String reverseVowels(String s) {
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == 'a' || s.charAt(i) == 'e' || s.charAt(i) == 'i' ||
s.charAt(i) == 'o' || s.charAt(i) == 'u'
                    || s.charAt(i) == 'A' || s.charAt(i) == 'E' || s.charAt(i) == 'I' ||
s.charAt(i) == 'O' || s.charAt(i) == 'U') {
                sb.append(s.charAt(i));

            }
        }
        sb = sb.reverse();//oe
        int k = 0;
        StringBuilder ans = new StringBuilder();

        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == 'a' || s.charAt(i) == 'e' || s.charAt(i) == 'i' ||
s.charAt(i) == 'o' || s.charAt(i) == 'u'
                    || s.charAt(i) == 'A' || s.charAt(i) == 'E' || s.charAt(i) == 'I' ||
s.charAt(i) == 'O' || s.charAt(i) == 'U') {
                ans.append(sb.charAt(k));
                k++;
            } else {
                ans.append(s.charAt(i));
            }
        }

        return ans.toString();
    }
}
```

192. `347. Top K Frequent Elements.java`

```
/*
 *
 * Given a non-empty array of integers, return the k most frequent elements.
 *
 * Example 1:
 *
 * Input: nums = [1,1,1,2,2,3], k = 2
 * Output: [1,2]
 * Example 2:
 *
 * Input: nums = [1], k = 1
 * Output: [1]
 * Note:
 *
 * You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
 * Your algorithm's time complexity must be better than O(n log n), where n is the
array's size.
 *
 */

public class Solution {

    public List<Integer> topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int n : nums) {
            map.put(n, map.getOrDefault(n, 0) + 1);
        }

        PriorityQueue<Map.Entry<Integer, Integer>> maxHeap
                = new PriorityQueue<>((a, b) -> (b.getValue() - a.getValue()));
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            maxHeap.add(entry);
        }

        List<Integer> res = new ArrayList<>();
        while (res.size() < k) {
            Map.Entry<Integer, Integer> entry = maxHeap.poll();
            res.add(entry.getKey());
        }
        return res;
    }
}
```

193. `349. Intersection of Two Arrays.java`

```
/*
 *
 * Given two arrays, write a function to compute their intersection.
 *
 * Example 1:
 *
 * Input: nums1 = [1,2,2,1], nums2 = [2,2]
 * Output: [2]
 * Example 2:
 *
 * Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
 * Output: [9,4]
 * Note:
 *
 * Each element in the result must be unique.
 * The result can be in any order.
 *
 */

class Solution {

    public int[] intersection(int[] nums1, int[] nums2) {
        HashSet<Integer> set1 = new HashSet<Integer>();
        for (int i : nums1) {
            set1.add(i);
        }

        HashSet<Integer> set2 = new HashSet<Integer>();
        for (int i : nums2) {
            if (set1.contains(i)) {
                set2.add(i);
            }
        }

        int[] result = new int[set2.size()];
        int i = 0;
        for (int n : set2) {
            result[i++] = n;
        }

        return result;
    }
}
```

194. `350. Intersection of Two Arrays II.java`

```
/*
 *
 * Given two arrays, write a function to compute their intersection.
 *
 * Example 1:
 *
 * Input: nums1 = [1,2,2,1], nums2 = [2,2]
 * Output: [2,2]
 * Example 2:
 *
 * Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
 * Output: [4,9]
 * Note:
 *
 * Each element in the result should appear as many times as it shows in both arrays.
 * The result can be in any order.
 * Follow up:
 *
 * What if the given array is already sorted? How would you optimize your algorithm?
 * What if nums1's size is small compared to nums2's size? Which algorithm is better?
 * What if elements of nums2 are stored on disk, and the memory is limited such that you
cannot load all elements into the memory at once?
 *
 */

class Solution {

    public int[] intersect(int[] nums1, int[] nums2) {
        HashMap<Integer, Integer> map = new HashMap<Integer, Integer>();
        for (int i : nums1) {
            if (map.containsKey(i)) {
                map.put(i, map.get(i) + 1);
            } else {
                map.put(i, 1);
            }
        }

        ArrayList<Integer> list = new ArrayList<Integer>();
        for (int i : nums2) {
            if (map.containsKey(i)) {
                if (map.get(i) > 1) {
                    map.put(i, map.get(i) - 1);
                } else {
                    map.remove(i);
                }
                list.add(i);
            }
        }

        int[] result = new int[list.size()];
        int i = 0;
```

```
        while (i < list.size()) {
            result[i] = list.get(i);
            i++;
        }

        return result;
    }
}
```

195. `355. Design Twitter.java`

```
/*
 *
 * Design a simplified version of Twitter where users can post tweets, follow/unfollow
another user and is able to see the 10 most recent tweets in the user's news feed. Your
design should support the following methods:
 *
 * postTweet(userId, tweetId): Compose a new tweet.
 * getNewsFeed(userId): Retrieve the 10 most recent tweet ids in the user's news feed.
Each item in the news feed must be posted by users who the user followed or by the user
herself. Tweets must be ordered from most recent to least recent.
 * follow(followerId, followeeId): Follower follows a followee.
 * unfollow(followerId, followeeId): Follower unfollows a followee.
 * Example:
 *
 * Twitter twitter = new Twitter();
 *
 * // User 1 posts a new tweet (id = 5).
 * twitter.postTweet(1, 5);
 *
 * // User 1's news feed should return a list with 1 tweet id -> [5].
 * twitter.getNewsFeed(1);
 *
 * // User 1 follows user 2.
 * twitter.follow(1, 2);
 *
 * // User 2 posts a new tweet (id = 6).
 * twitter.postTweet(2, 6);
 *
 * // User 1's news feed should return a list with 2 tweet ids -> [6, 5].
 * // Tweet id 6 should precede tweet id 5 because it is posted after tweet id 5.
 * twitter.getNewsFeed(1);
 *
 * // User 1 unfollows user 2.
 * twitter.unfollow(1, 2);
 *
 * // User 1's news feed should return a list with 1 tweet id -> [5],
 * // since user 1 is no longer following user 2.
 * twitter.getNewsFeed(1);
 *
 */

public class Twitter {

    private static int timeStamp = 0;

    // easy to find if user exist
    private Map<Integer, User> userMap;

    // Tweet link to next Tweet so that we can save a lot of time
    // when we execute getNewsFeed(userId)
    private class Tweet {
```

```
    public int id;
    public int time;
    public Tweet next;

    public Tweet(int id) {
        this.id = id;
        time = timeStamp++;
        next = null;
    }
}

// OO design so User can follow, unfollow and post itself
public class User {

    public int id;
    public Set<Integer> followed;
    public Tweet tweet_head;

    public User(int id) {
        this.id = id;
        followed = new HashSet<>();
        follow(id); // first follow itself
        tweet_head = null;
    }

    public void follow(int id) {
        followed.add(id);
    }

    public void unfollow(int id) {
        followed.remove(id);
    }

    // everytime user post a new tweet, add it to the head of tweet list.
    public void post(int id) {
        Tweet t = new Tweet(id);
        t.next = tweet_head;
        tweet_head = t;
    }
}

/**
 * Initialize your data structure here.
 */
public Twitter() {
    userMap = new HashMap<Integer, User>();
}

/**
 * Compose a new tweet.
 */
public void postTweet(int userId, int tweetId) {
    if (!userMap.containsKey(userId)) {
```

```
            User u = new User(userId);
            userMap.put(userId, u);
        }
        userMap.get(userId).post(tweetId);

    }

    // Best part of this.
    // first get all tweets lists from one user including itself and all people it
followed.
    // Second add all heads into a max heap. Every time we poll a tweet with
    // largest time stamp from the heap, then we add its next tweet into the heap.
    // So after adding all heads we only need to add 9 tweets at most into this
    // heap before we get the 10 most recent tweet.
    public List<Integer> getNewsFeed(int userId) {
        List<Integer> res = new LinkedList<>();

        if (!userMap.containsKey(userId)) {
            return res;
        }

        Set<Integer> users = userMap.get(userId).followed;
        PriorityQueue<Tweet> q = new PriorityQueue<Tweet>(users.size(), (a, b) -> (b.time
- a.time));
        for (int user : users) {
            Tweet t = userMap.get(user).tweet_head;
            // very imporant! If we add null to the head we are screwed.
            if (t != null) {
                q.add(t);
            }
        }
        int n = 0;
        while (!q.isEmpty() && n < 10) {
            Tweet t = q.poll();
            res.add(t.id);
            n++;
            if (t.next != null) {
                q.add(t.next);
            }
        }

        return res;

    }

    /**
     * Follower follows a followee. If the operation is invalid, it should be a
     * no-op.
     */
    public void follow(int followerId, int followeeId) {
        if (!userMap.containsKey(followerId)) {
            User u = new User(followerId);
            userMap.put(followerId, u);
        }
```

```
        if (!userMap.containsKey(followeeId)) {
            User u = new User(followeeId);
            userMap.put(followeeId, u);
        }
        userMap.get(followerId).follow(followeeId);
    }

    /**
     * Follower unfollows a followee. If the operation is invalid, it should be
     * a no-op.
     */
    public void unfollow(int followerId, int followeeId) {
        if (!userMap.containsKey(followerId) || followerId == followeeId) {
            return;
        }
        userMap.get(followerId).unfollow(followeeId);
    }
}
```

196. `367. Valid Perfect Square.java`

```
/*
 *
 * Given a positive integer num, write a function which returns True if num is a perfect
square else False.
 *
 * Note: Do not use any built-in library function such as sqrt.
 *
 * Example 1:
 *
 * Input: 16
 * Output: true
 * Example 2:
 *
 * Input: 14
 * Output: false
 *
 */

class Solution {

    public boolean isPerfectSquare(int n) {
        long sum = 0;
        for (int i = 1; sum < n; i = i + 2) {
            sum = sum + i;
            if (sum == n) {
                return true;
            }
        }
        return false;
    }
}
```

197. `371. Sum of Two Integers.java`

```
/*
 *
 * Calculate the sum of two integers a and b, but you are not allowed to use the operator
+ and -.
 *
 * Example 1:
 *
 * Input: a = 1, b = 2
 * Output: 3
 * Example 2:
 *
 * Input: a = -2, b = 3
 * Output: 1
 *
 */

class Solution {

    public int getSum(int a, int b) {
        if (a == 0) {
            return b;
        }
        if (b == 0) {
            return a;
        }

        while (b != 0) {
            int carry = a & b;
            a = a ^ b;
            b = carry << 1;
        }

        return a;
    }
}
```

198. `373. Find K Pairs with Smallest Sums.java`

```
/*
 *
 * You are given two integer arrays nums1 and nums2 sorted in ascending order and an
integer k.
 *
 * Define a pair (u,v) which consists of one element from the first array and one element
from the second array.
 *
 * Find the k pairs (u1,v1),(u2,v2) ...(uk,vk) with the smallest sums.
 *
 * Example 1:
 *
 * Input: nums1 = [1,7,11], nums2 = [2,4,6], k = 3
 * Output: [[1,2],[1,4],[1,6]]
 * Explanation: The first 3 pairs are returned from the sequence:
 * [1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
 * Example 2:
 *
 * Input: nums1 = [1,1,2], nums2 = [1,2,3], k = 2
 * Output: [1,1],[1,1]
 * Explanation: The first 2 pairs are returned from the sequence:
 * [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]
 * Example 3:
 *
 * Input: nums1 = [1,2], nums2 = [3], k = 3
 * Output: [1,3],[2,3]
 * Explanation: All possible pairs are returned from the sequence: [1,3],[2,3]
 *
 */

public class Solution {

    public List<int[]> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        List<int[]> result = new LinkedList<int[]>();
        if (nums1 == null || nums2 == null || nums1.length == 0 || nums2.length == 0 || k
== 0) {
            return result;
        }

        PriorityQueue<Triple> queue = new PriorityQueue<Triple>(nums1.length, new
Comparator<Triple>() {
            public int compare(Triple a, Triple b) {
                return Integer.compare(a.sum, b.sum);
            }
        });

// add the first column
        for (int i = 0; i < nums1.length; i++) {
            queue.add(new Triple(nums1[i] + nums2[0], i, 0));
        }
```

```
        while (k-- > 0 && !queue.isEmpty()) {
            Triple current = queue.poll();
            result.add(new int[]{nums1[current.index1], nums2[current.index2]});
            // if the current one has a right candidate, add it to the queue.
            if (current.index2 + 1 < nums2.length) {
                queue.add(new Triple(nums1[current.index1] + nums2[current.index2 + 1],
current.index1, current.index2 + 1));
            }
        }

        return result;
    }

// Triple is used to store the sum, the index in nums1 and the index in nums2.
    class Triple {

        int sum;
        int index1;
        int index2;

        Triple(int sum, int index1, int index2) {
            this.sum = sum;
            this.index1 = index1;
            this.index2 = index2;
        }
    }
}


/*
    It is actually the same as how we merge k sorted list, in this question, the
following are the k sorted list(each number in nums1[] full-mesh with the numbers in
nums2[].

(1,2) -> (1,9) -> (1,10) -> (1,15)
(7,2) -> (7,9) -> (7,10) -> (7,15)
(11,2) -> (11,9) -> (11,10) -> (11,15)
(16,2) -> (16,9) -> (16,10) -> (16,15)

Remember how we do in "merge k sorted list"? We simply add the head of the list into the
heap and when a node is poll(), we just add the node.next
    */

    /*public List<int[]> kSmallestPairs(int[] nums1, int[] nums2, int k) {
        PriorityQueue<int[]> que = new PriorityQueue<>((a,b)->a[0]+a[1]-b[0]-b[1]);
        List<int[]> res = new ArrayList<>();

        if(nums1.length==0 || nums2.length==0 || k==0)
            return res;

        //In the Priority Queue we maintain only k elements!
        //Those k elements we are getting by traversing the nums1 array, and only
considering j=0
        for(int i=0; i<nums1.length && i<k; i++)
```

```
            que.offer(new int[]{nums1[i], nums2[0], 0});
        //In the integer array we are storing the element from array1, array2, and index
where we are in array2

        //We will add elements to the queue only if we don't have k elements in our final
list
        while(k-- > 0 && !que.isEmpty()){
            int[] cur = que.poll();
            res.add(new int[]{cur[0], cur[1]});

            if(cur[2] == nums2.length-1)
                continue;//If we have reached the end of the nums2 array then skip the
current iteration

            que.offer(new int[]{cur[0],nums2[cur[2]+1], cur[2]+1});
        }
        return res;
    }*/
```

199. `374. Guess Number Higher or Lower.java`

```
/*
 *
 * We are playing the Guess Game. The game is as follows:
 *
 * I pick a number from 1 to n. You have to guess which number I picked.
 *
 * Every time you guess wrong, I'll tell you whether the number is higher or lower.
 *
 * You call a pre-defined API guess(int num) which returns 3 possible results (-1, 1, or
0):
 *
 * -1 : My number is lower
 * 1 : My number is higher
 * 0 : Congrats! You got it!
 * Example :
 *
 * Input: n = 10, pick = 6
 * Output: 6
 *
 */

 /*
 * The guess API is defined in the parent class GuessGame.
 * @param num, your guess
 * @return -1 if my number is lower, 1 if my number is higher, otherwise return 0
 * int guess(int num);
 */
public class Solution extends GuessGame {

    public int guessNumber(int n) {
        int mid, left = 1, right = n;

        while (left <= right) {
            mid = left + (right - left) / 2;
            int res = guess(mid);

            if (res == 0) {
                return mid;
            } else if (res == -1) {
                right = mid - 1;
            } else if (res == 1) {
                left = mid + 1;
            }
        }
        return -1;
    }
}
```

200. `378. Kth Smallest Element in a Sorted Matrix.java`

```
/*
 *
 * Given a n x n matrix where each of the rows and columns are sorted in ascending order,
find the kth smallest element in the matrix.
 *
 * Note that it is the kth smallest element in the sorted order, not the kth distinct
element.
 *
 * Example:
 *
 * matrix = [
 * [ 1,  5,  9],
 * [10, 11, 13],
 * [12, 13, 15]
 * ],
 * k = 8,
 *
 * return 13.
 *
 */

class Solution {

    public int kthSmallest(int[][] matrix, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<Integer>(k, (a, b) -> (a -
b));

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[0].length; j++) {
                minHeap.offer(matrix[i][j]);
            }
        }

        int result = 0;
        while (k > 0) {
            result = minHeap.poll();
            k--;
        }
        return result;
    }
}
```

201. `380. Insert Delete GetRandom O(1).java`

```
/*
 *
 * Design a data structure that supports all following operations in average O(1) time.
 *
 * insert(val): Inserts an item val to the set if not already present.
 * remove(val): Removes an item val from the set if present.
 * getRandom: Returns a random element from current set of elements. Each element must
have the same probability of being returned.
 * Example:
 *
 * // Init an empty set.
 * RandomizedSet randomSet = new RandomizedSet();
 *
 * // Inserts 1 to the set. Returns true as 1 was inserted successfully.
 * randomSet.insert(1);
 *
 * // Returns false as 2 does not exist in the set.
 * randomSet.remove(2);
 *
 * // Inserts 2 to the set, returns true. Set now contains [1,2].
 * randomSet.insert(2);
 *
 * // getRandom should return either 1 or 2 randomly.
 * randomSet.getRandom();
 *
 * // Removes 1 from the set, returns true. Set now contains [2].
 * randomSet.remove(1);
 *
 * // 2 was already in the set, so return false.
 * randomSet.insert(2);
 *
 * // Since 2 is the only number in the set, getRandom always return 2.
 * randomSet.getRandom();
 *
 */

public class RandomizedSet {

    ArrayList<Integer> nums;
    HashMap<Integer, Integer> locs;
    java.util.Random rand = new java.util.Random();

    /**
     * Initialize your data structure here.
     */
    public RandomizedSet() {
        nums = new ArrayList<Integer>();
        locs = new HashMap<Integer, Integer>();
    }

    /**
```

```
     * Inserts a value to the set. Returns true if the set did not already
     * contain the specified element.
     */
    public boolean insert(int val) {
        boolean contain = locs.containsKey(val);

        if (contain) {
            return false;
        }

        locs.put(val, nums.size());
        nums.add(val);
        return true;
    }

    /**
     * Removes a value from the set. Returns true if the set contained the
     * specified element.
     */
    public boolean remove(int val) {
        boolean contain = locs.containsKey(val);

        if (!contain) {
            return false;
        }

        int loc = locs.get(val);
        if (loc < nums.size() - 1) { // not the last one than swap the last one with this
val
            int lastone = nums.get(nums.size() - 1);
            nums.set(loc, lastone);
            locs.put(lastone, loc);
        }

        locs.remove(val);
        nums.remove(nums.size() - 1);
        return true;
    }

    /**
     * Get a random element from the set.
     */
    public int getRandom() {
        return nums.get(rand.nextInt(nums.size()));
    }
}
```

202. `383. Ransom Note.java`

```
/*
 *
 * Given an arbitrary ransom note string and another string containing letters from all the magazines, write a function that will return true if the ransom note can be constructed from the magazines ; otherwise, it will return false.
 *
 * Each letter in the magazine string can only be used once in your ransom note.
 *
 * Note:
 * You may assume that both strings contain only lowercase letters.
 *
 * canConstruct("a", "b") -> false
 * canConstruct("aa", "ab") -> false
 * canConstruct("aa", "aab") -> true
 *
 */

class Solution {

    public boolean canConstruct(String ransomNote, String magazine) {
        int[] cnt = new int[26];
        for (char c : magazine.toCharArray()) {
            cnt[c - 'a']++;
        }

        for (char c : ransomNote.toCharArray()) {
            if (--cnt[c - 'a'] < 0) {
                return false;
            }
        }
        return true;
    }
}
```

203. `384. Shuffle an Array.java`

```
/*
 *
 * Shuffle a set of numbers without duplicates.
 *
 * Example:
 *
 * // Init an array with set 1, 2, and 3.
 * int[] nums = {1,2,3};
 * Solution solution = new Solution(nums);
 *
 * // Shuffle the array [1,2,3] and return its result. Any permutation of [1,2,3] must
equally likely to be returned.
 * solution.shuffle();
 *
 * // Resets the array back to its original configuration [1,2,3].
 * solution.reset();
 *
 * // Returns the random shuffling of array [1,2,3].
 * solution.shuffle();
 *
 */

class Solution {

    private int[] nums;

    public Solution(int[] nums) {
        this.nums = nums;
    }

    /**
     * Resets the array to its original configuration and return it.
     */
    public int[] reset() {
        return nums;
    }

    /**
     * Returns a random shuffling of the array.
     */
    public int[] shuffle() {
        int[] rand = new int[nums.length];
        for (int i = 0; i < nums.length; i++) {
            int r = (int) (Math.random() * (i + 1));
            rand[i] = rand[r];
            rand[r] = nums[i];
        }
        return rand;
    }
}
```

```
/**
 * Your Solution object will be instantiated and called as such:
 * Solution obj = new Solution(nums);
 * int[] param_1 = obj.reset();
 * int[] param_2 = obj.shuffle();
 */
```

204. `385. Mini Parser.java`

```
/*
 *
 * Given a nested list of integers represented as a string, implement a parser to
deserialize it.
 *
 * Each element is either an integer, or a list -- whose elements may also be integers or
other lists.
 *
 * Note: You may assume that the string is well-formed:
 *
 * String is non-empty.
 * String does not contain white spaces.
 * String contains only digits 0-9, [, - ,, ].
 * Example 1:
 *
 * Given s = "324",
 *
 * You should return a NestedInteger object which contains a single integer 324.
 * Example 2:
 *
 * Given s = "[123,[456,[789]]]",
 *
 * Return a NestedInteger object containing a nested list with 2 elements:
 *
 * 1. An integer containing value 123.
 * 2. A nested list containing two elements:
 * i.  An integer containing value 456.
 * ii. A nested list with one element:
 * a. An integer containing value 789.
 *
 */

/**
 * // This is the interface that allows for creating nested lists.
 * // You should not implement it, or speculate about its implementation
 * public interface NestedInteger {
 *     // Constructor initializes an empty nested list.
 *     public NestedInteger();
 *
 *     // Constructor initializes a single integer.
 *     public NestedInteger(int value);
 *
 *     // @return true if this NestedInteger holds a single integer, rather than a nested
list.
 *     public boolean isInteger();
 *
 *     // @return the single integer that this NestedInteger holds, if it holds a single
integer
 *     // Return null if this NestedInteger holds a nested list
 *     public Integer getInteger();
 *
```

```
 *     // Set this NestedInteger to hold a single integer.
 *     public void setInteger(int value);
 *
 *     // Set this NestedInteger to hold a nested list and adds a nested integer to it.
 *     public void add(NestedInteger ni);
 *
 *     // @return the nested list that this NestedInteger holds, if it holds a nested
list
 *     // Return null if this NestedInteger holds a single integer
 *     public List<NestedInteger> getList(); }
 */
class Solution {

    public NestedInteger deserialize(String s) {
        Stack<NestedInteger> stack = new Stack<>();
        Integer num = null;
        int sig = 1;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (c == '[') {
                stack.push(new NestedInteger());
            } else if (c == '-') {
                sig = -1;
            } else if (c == ',' || c == ']') {
                if (num != null) { //found a number
                    stack.peek().add(new NestedInteger(num.intValue() * sig));
                    num = null;
                    sig = 1;
                }
                if (c == ']') {
                    NestedInteger n = stack.pop();
                    if (stack.empty()) {
                        return n;
                    }
                    stack.peek().add(n);
                }
            } else { //number
                if (num == null) {
                    num = 0;
                }
                num = num * 10 + c - '0';
            }
        }
        return new NestedInteger(num.intValue() * sig); //When s only contains numbers
(String is non-empty according to the problem description)
    }
}
```

205. `387. First Unique Character in a String.java`

```
/*
 *
 * Given a string, find the first non-repeating character in it and return it's index. If
it doesn't exist, return -1.
 *
 * Examples:
 *
 * s = "leetcode"
 * return 0.
 *
 * s = "loveleetcode",
 * return 2.
 * Note: You may assume the string contain only lowercase letters.
 *
 *
 */

class Solution {

    public int firstUniqChar(String s) {
        HashMap<Character, Integer> map = new HashMap<>();

        for (char c : s.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        for (int i = 0; i < s.length(); i++) {
            if (map.get(s.charAt(i)) == 1) {
                return i;
            }
        }

        return -1;
    }
}
```

206. `389. Find the Difference.java`

```
/*
 *
 * Given two strings s and t which consist of only lowercase letters.
 *
 * String t is generated by random shuffling string s and then add one more letter at a
random position.
 *
 * Find the letter that was added in t.
 *
 * Example:
 *
 * Input:
 * s = "abcd"
 * t = "abcde"
 *
 * Output:
 * e
 *
 * Explanation:
 * 'e' is the letter that was added.
 *
 */

public class Solution {

    public char findTheDifference(String s, String t) {
        char c = 0;
        for (int i = 0; i < s.length(); i++) {
            c ^= s.charAt(i);
        }
        for (int i = 0; i < t.length(); i++) {
            c ^= t.charAt(i);
        }
        return c;
    }
}
```

207. `392. Is Subsequence.java`

```
/*
 *
 * Given a string s and a string t, check if s is subsequence of t.
 *
 * You may assume that there is only lower case English letters in both s and t. t is
potentially a very long (length ~= 500,000) string, and s is a short string (<=100).
 *
 * A subsequence of a string is a new string which is formed from the original string by
deleting some (can be none) of the characters without disturbing the relative positions
of the remaining characters. (ie, "ace" is a subsequence of "abcde" while "aec" is not).
 *
 * Example 1:
 * s = "abc", t = "ahbgdc"
 *
 * Return true.
 *
 * Example 2:
 * s = "axc", t = "ahbgdc"
 *
 * Return false.
 *
 * Follow up:
 * If there are lots of incoming S, say S1, S2, ... , Sk where k >= 1B, and you want to
check one by one to see if T has its subsequence. In this scenario, how would you change
your code?
 *
 *
 */

class Solution {

    public boolean isSubsequence(String s, String t) {
        if (s.length() == 0) {
            return true;
        }
        int indexS = 0, indexT = 0;

        while (indexT < t.length()) {
            if (t.charAt(indexT) == s.charAt(indexS)) {
                indexS++;
                if (indexS == s.length()) {
                    return true;
                }
            }
            indexT++;
        }
        return false;
    }
}
```

208. `394. Decode String.java`

```
/*
 *
 * Given an encoded string, return it's decoded string.
 *
 * The encoding rule is: k[encoded_string], where the encoded_string inside the square
brackets is being repeated exactly k times. Note that k is guaranteed to be a positive
integer.
 *
 * You may assume that the input string is always valid; No extra white spaces, square
brackets are well-formed, etc.
 *
 * Furthermore, you may assume that the original data does not contain any digits and
that digits are only for those repeat numbers, k. For example, there won't be input like
3a or 2[4].
 *
 * Examples:
 *
 * s = "3[a]2[bc]", return "aaabcbc".
 * s = "3[a2[c]]", return "accaccacc".
 * s = "2[abc]3[cd]ef", return "abcabccdcdcdef".
 *
 */

class Solution {

    private int i = 0;

    public String decodeString(String s) {
        StringBuilder sb = new StringBuilder();
        int num = 0;

        while (i < s.length()) {
            char c = s.charAt(i);

            if (Character.isDigit(c)) {
                num = num * 10 + c - '0';
            } else if (c == '[') {
                i++;
                String str = decodeString(s);
                for (int k = 0; k < num; k++) {
                    sb.append(str);
                }
                num = 0;
            } else if (c == ']') {
                return sb.toString();
            } else {
                sb.append(c);
            }
            i++;
        }
```

```
        return sb.toString();
    }
}
```

209. `395. Longest Substring with At Least K Repeating`

Characters.java

```
/*
 *
 * Find the length of the longest substring T of a given string (consists of lowercase
letters only) such that every character in T appears no less than k times.
 *
 * Example 1:
 *
 * Input:
 * s = "aaabb", k = 3
 *
 * Output:
 * 3
 *
 * The longest substring is "aaa", as 'a' is repeated 3 times.
 * Example 2:
 *
 * Input:
 * s = "ababbc", k = 2
 *
 * Output:
 * 5
 *
 * The longest substring is "ababb", as 'a' is repeated 2 times and 'b' is repeated 3
times.
 *
 */

class Solution {

    public int longestSubstring(String s, int k) {
        HashMap<Character, Integer> counter = new HashMap<Character, Integer>();

        for (int i = 0; i < s.length(); i++) {

            char c = s.charAt(i);
            if (counter.containsKey(c)) {
                counter.put(c, counter.get(c) + 1);
            } else {
                counter.put(c, 1);
            }

        }

        HashSet<Character> splitSet = new HashSet<Character>();
        for (char c : counter.keySet()) {
            if (counter.get(c) < k) {
                splitSet.add(c);
            }
        }

        if (splitSet.isEmpty()) {
```

```
            return s.length();
        }

        int max = 0;
        int i = 0, j = 0;
        while (j < s.length()) {
            char c = s.charAt(j);
            if (splitSet.contains(c)) {
                if (j != i) {
                    max = Math.max(max, longestSubstring(s.substring(i, j), k));
                }
                i = j + 1;
            }
            j++;
        }

        if (i != j) {
            max = Math.max(max, longestSubstring(s.substring(i, j), k));
        }

        return max;
    }
}
```

210. `402. Remove K Digits.java`

```
/*
 *
 * Given a non-negative integer num represented as a string, remove k digits from the
number so that the new number is the smallest possible.
 *
 * Note:
 * The length of num is less than 10002 and will be ≥ k.
 * The given num does not contain any leading zero.
 * Example 1:
 *
 * Input: num = "1432219", k = 3
 * Output: "1219"
 * Explanation: Remove the three digits 4, 3, and 2 to form the new number 1219 which is
the smallest.
 * Example 2:
 *
 * Input: num = "10200", k = 1
 * Output: "200"
 * Explanation: Remove the leading 1 and the number is 200. Note that the output must not
contain leading zeroes.
 * Example 3:
 *
 * Input: num = "10", k = 2
 * Output: "0"
 * Explanation: Remove all the digits from the number and it is left with nothing which
is 0.
 *
 */

class Solution {

    public String removeKdigits(String num, int k) {
        Deque<Character> stack = new ArrayDeque<>();
        int n = num.length();
        if (k >= n) {
            return "0";
        }

        for (int i = 0; i < n; i++) {
            while (k > 0 && !stack.isEmpty() && stack.peek() > num.charAt(i)) {
                stack.pop();//Basically we want to check if the value that we have in the
stack, is it more than the current value, then just pop the value from the stack till the
time you get a value which is less than the current value.
                k--;
            }
            stack.push(num.charAt(i));
        }

        while (k > 0) {
            stack.pop(); //We want only k characters in our stack
            k--;
```

```
        }

        StringBuilder sb = new StringBuilder();

        while (!stack.isEmpty()) {
            sb.append(stack.pop());//Remove all the k characters from the stack
        }
        sb.reverse();//Since Stack is LIFO, we need to reverse the result

        while (sb.length() > 1 && sb.charAt(0) == '0') {
            sb.deleteCharAt(0);//In case we have a leading zero in our result
        }
        return sb.toString();//Return the result string
    }
}
```

211. `412. Fizz Buzz.java`

```
/*
 *
 * Write a program that outputs the string representation of numbers from 1 to n.
 *
 * But for multiples of three it should output “Fizz” instead of the number and for the
multiples of five output “Buzz”. For numbers which are multiples of both three and five
output “FizzBuzz”.
 *
 * Example:
 *
 * n = 15,
 *
 * Return:
 * [
 * "1",
 * "2",
 * "Fizz",
 * "4",
 * "Buzz",
 * "Fizz",
 * "7",
 * "8",
 * "Fizz",
 * "Buzz",
 * "11",
 * "Fizz",
 * "13",
 * "14",
 * "FizzBuzz"
 * ]
 *
 */

class Solution {

    public List<String> fizzBuzz(int n) {
        List<String> list = new ArrayList<>();
        for (int i = 1; i <= n; i++) {
            if (i % 3 == 0 && i % 5 == 0) {
                list.add("FizzBuzz");
            } else if (i % 3 == 0) {
                list.add("Fizz");
            } else if (i % 5 == 0) {
                list.add("Buzz");
            } else {
                list.add(String.valueOf(i));
            }
        }
        return list;
    }
}
```

212. `414. Third Maximum Number.java`

```
/*
 *
 * Given a non-empty array of integers, return the third maximum number in this array. If
it does not exist, return the maximum number. The time complexity must be in O(n).
 *
 * Example 1:
 * Input: [3, 2, 1]
 *
 * Output: 1
 *
 * Explanation: The third maximum is 1.
 * Example 2:
 * Input: [1, 2]
 *
 * Output: 2
 *
 * Explanation: The third maximum does not exist, so the maximum (2) is returned instead.
 * Example 3:
 * Input: [2, 2, 3, 1]
 *
 * Output: 1
 *
 * Explanation: Note that the third maximum here means the third maximum distinct number.
 * Both numbers with value 2 are both considered as second maximum.
 *
 */

class Solution {

    public int thirdMax(int[] nums) {

        //Solution using Priority Queue similar to Q215
        TreeSet<Integer> s = new TreeSet<Integer>();

        for (int i : nums) {
            s.add(i);
        }

        List<Integer> al = new ArrayList<Integer>(s);

        int k = 3;
        PriorityQueue<Integer> queue = new PriorityQueue();
        if (s.size() == 2) {
            return Math.max(al.get(0), al.get(1));
        }
        if (s.size() == 1) {
            return al.get(0);
        }
        if (s.size() == 0) {
            return 0;
        } else {
```

```
            for (int i : s) {
                queue.offer(i);
                if (queue.size() > 3) {
                    queue.poll();
                }
            }
            return queue.peek();

        }

    }

}
```

213. `415. Add Strings.java`

```
/*
 * Given two non-negative integers num1 and num2 represented as string, return the sum of
num1 and num2.
 *
 * Note:
 *
 * The length of both num1 and num2 is < 5100.
 * Both num1 and num2 contains only digits 0-9.
 * Both num1 and num2 does not contain any leading zero.
 * You must not use any built-in BigInteger library or convert the inputs to integer
directly.
 *
 */

class Solution {

    public String addStrings(String num1, String num2) {
        if (num1 == null || num2 == null) {
            return null;
        }

        StringBuilder sb = new StringBuilder();
        int carry = 0;
        int l1 = num1.length() - 1;
        int l2 = num2.length() - 1;

        while (l1 >= 0 || l2 >= 0) {
            int n1 = l1 >= 0 ? num1.charAt(l1) - '0' : 0;
            int n2 = l2 >= 0 ? num2.charAt(l2) - '0' : 0;
            int sum = n1 + n2 + carry;
            carry = sum > 9 ? 1 : 0;
            sb.insert(0, sum % 10);
            l1--;
            l2--;
        }

        if (carry == 1) {
            sb.insert(0, 1);
        }

        return sb.toString();
    }
}
```

214. `417. Pacific Atlantic Water Flow.java`

```
/*
 *
 * Given an m x n matrix of non-negative integers representing the height of each unit
cell in a continent, the "Pacific ocean" touches the left and top edges of the matrix and
the "Atlantic ocean" touches the right and bottom edges.
 *
 * Water can only flow in four directions (up, down, left, or right) from a cell to
another one with height equal or lower.
 *
 * Find the list of grid coordinates where water can flow to both the Pacific and
Atlantic ocean.
 *
 * Note:
 * The order of returned grid coordinates does not matter.
 * Both m and n are less than 150.
 * Example:
 *
 * Given the following 5x5 matrix:
 *
 * Pacific ~   ~   ~   ~   ~
 * ~  1   2   2   3  (5) *
 * ~  3   2   3  (4) (4) *
 * ~  2   4  (5)  3   1  *
 * ~ (6) (7)  1   4   5  *
 * ~ (5)  1   1   2   4  *
 *   *   *   *   * Atlantic
 *
 * Return:
 *
 * [[0, 4], [1, 3], [1, 4], [2, 2], [3, 0], [3, 1], [4, 0]] (positions with parentheses
in above matrix).
 *
 */

public class Solution {

    int[][] dir = new int[][]{{1, 0}, {-1, 0}, {0, 1}, {0, -1}};

    public List<int[]> pacificAtlantic(int[][] matrix) {
        List<int[]> res = new LinkedList<>();
        if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
            return res;
        }

        int m = matrix.length, n = matrix[0].length;

        //One visited map for each ocean
        boolean[][] pacific = new boolean[m][n];
        boolean[][] atlantic = new boolean[m][n];

        Queue<int[]> pQueue = new LinkedList<>();
```

```
        Queue<int[]> aQueue = new LinkedList<>();

        //We are given tht vertical border on left and right are Pacific and Atlantic
oceans respectively
        for (int i = 0; i < m; i++) {
            pQueue.offer(new int[]{i, 0});
            aQueue.offer(new int[]{i, n - 1});
            pacific[i][0] = true;
            atlantic[i][n - 1] = true;
        }

        //Similarly we are given tht horizontal border on top and bottom are Pacific and
Atlantic oceans respectively
        for (int i = 0; i < n; i++) {
            pQueue.offer(new int[]{0, i});
            aQueue.offer(new int[]{m - 1, i});
            pacific[0][i] = true;
            atlantic[m - 1][i] = true;
        }

        bfs(matrix, pQueue, pacific);
        bfs(matrix, aQueue, atlantic);

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (pacific[i][j] && atlantic[i][j]) {
                    res.add(new int[]{i, j});
                }
            }
        }

        return res;
    }

    public void bfs(int[][] matrix, Queue<int[]> queue, boolean[][] visited) {
        int m = matrix.length, n = matrix[0].length;

        while (!queue.isEmpty()) {
            int[] cur = queue.poll();
            for (int[] d : dir) {
                int x = cur[0] + d[0];
                int y = cur[1] + d[1];
                if (x >= 0 && x < m && y >= 0 && y < n && !visited[x][y] && matrix[x][y]
>= matrix[cur[0]][cur[1]]) {
                    visited[x][y] = true;
                    queue.offer(new int[]{x, y});
                }
            }
        }
    }
}
```

215. `429. N-ary Tree Level Order Traversal.java`

```
/*
 *
 * Given an n-ary tree, return the level order traversal of its nodes' values. (ie, from
left to right, level by level).
 *
 * For example, given a 3-ary tree:
 *
 *
 * We should return its level order traversal:
 *
 * [
 * [1],
 * [3,2,4],
 * [5,6]
 * ]
 *
 *
 * Note:
 *
 * The depth of the tree is at most 1000.
 * The total number of nodes is at most 5000.
 *
 */

 /*
 * // Definition for a Node.
 * class Node {
 * public int val;
 * public List<Node> children;
 *
 * public Node() {}
 *
 * public Node(int _val,List<Node> _children) {
 * val = _val;
 * children = _children;
 * }
 * };
 */
class Solution {

    public List<List<Integer>> levelOrder(Node root) {

        List<List<Integer>> result = new ArrayList<List<Integer>>();

        if (root == null) {
            return result;
        }
        Queue<Node> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
```

```
            List<Integer> curLevel = new LinkedList<>();
            int len = queue.size();

            for (int i = 0; i < len; i++) {
                Node curr = queue.poll();
                curLevel.add(curr.val);
                for (Node c : curr.children) {
                    queue.offer(c);
                }
            }
            result.add(curLevel);
        }
        return result;
    }
}
```

216. `430. Flatten a Multilevel Doubly Linked List.java`

```
/*

You are given a doubly linked list which in addition to the next and previous pointers, it could have a child pointer, which may or may not point to a separate doubly linked list. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure, as shown in the example below.

Flatten the list so that all the nodes appear in a single-level, doubly linked list. You are given the head of the first level of the list.



Example:

Input:
 1---2---3---4---5---6--NULL
         |
         7---8---9---10--NULL
             |
             11--12--NULL

Output:
1-2-3-7-8-11-12-9-10-4-5-6-NULL

*/


class Solution {

    public Node flatten(Node head) {
        if (head == null) {
            return head;
        }

        // Pointer
        Node p = head;
        while (p != null) {
            /*
             * CASE 1: if no child, proceed
             */
            if (p.child == null) {
                p = p.next;
                continue; //Skip forward since there is no child
            }

            /*
             * CASE 2: got child, find the tail of the child and link it to
             * p.next
             */
            Node temp = p.child;
```

```
            // Find the tail of the child
            while (temp.next != null) {
                temp = temp.next;
            }
            // Connect tail with p.next, if it is not null
            temp.next = p.next;

            if (p.next != null) {
                p.next.prev = temp;
            }

            // Connect p with p.child, and remove p.child
            p.next = p.child;
            p.child.prev = p;
            p.child = null;
        }
        return head;
    }
}



/*

Iteration 1,2 will be skipped since they don't have any children
Iteration 3: We have p = 3, which has a child, so we are going to connect our child list
to the current node = p
    So we are going to have a list like this:
            1---2---3(p)         4---5---6--NULL
                    |            |
           (temp)-> 7---8---9---10
                        |
                        11--12--NULL

Iteration 4 will be skipped since we have p = 7, and it doesn't have child so skip it
Iteration 5: When we have p = 8, we will be traversing the whole child list and just go
on and change the tail node pointer of the child list!
             1---2----3               4---5---6--NULL
                      |               |
                      7---8(p)    9---10
                           |      |
                 (temp)->  11----12

Rest of the iterations don't have any children so need to do anything else. They will
just go on skipping the iterations till the end of the list!

*/
```

217. `433. Minimum Genetic Mutation.java`

```
/*
 * A gene string can be represented by an 8-character long string, with choices from "A",
"C", "G", "T".
 *
 * Suppose we need to investigate about a mutation (mutation from "start" to "end"),
where ONE mutation is defined as ONE single character changed in the gene string.
 *
 * For example, "AACCGGTT" -> "AACCGGTA" is 1 mutation.
 *
 * Also, there is a given gene "bank", which records all the valid gene mutations. A gene
must be in the bank to make it a valid gene string.
 *
 * Now, given 3 things - start, end, bank, your task is to determine what is the minimum
number of mutations needed to mutate from "start" to "end". If there is no such a
mutation, return -1.
 *
 * Note:
 *
 * Starting point is assumed to be valid, so it might not be included in the bank.
 * If multiple mutations are needed, all mutations during in the sequence must be valid.
 * You may assume start and end string is not the same.
 *
 *
 * Example 1:
 *
 * start: "AACCGGTT"
 * end:   "AACCGGTA"
 * bank: ["AACCGGTA"]
 *
 * return: 1
 *
 *
 * Example 2:
 *
 * start: "AACCGGTT"
 * end:   "AAACGGTA"
 * bank: ["AACCGGTA", "AACCGCTA", "AAACGGTA"]
 *
 * return: 2
 *
 *
 * Example 3:
 *
 * start: "AAAAACCC"
 * end:   "AACCCCCC"
 * bank: ["AAAACCCC", "AAACCCCC", "AACCCCCC"]
 *
 * return: 3
 *
 */
class Solution {
```

```
    public int minMutation(String start, String end, String[] bank) {

        HashSet<String> set = new HashSet<String>();
        for (String s : bank) {
            set.add(s);
        }

        Queue<String> queue = new LinkedList<String>();
        queue.offer(start);
        int steps = 0;
        char[] charSet = {'A', 'C', 'G', 'T'};

        while (!queue.isEmpty()) {
            int size = queue.size(); //We need a special variable to store size because
later we are adding elements in the queue, which is changing the size of the queue, which
we don't want to happen. So use an extra size variable instead of for(int
i=0;i<queue.size();i++)
            for (int i = 0; i < size; i++) {
                String current = queue.poll();
                if (current.equals(end)) {
                    return steps;
                }

                for (int j = 0; j < current.length(); j++) {
                    for (char ch : charSet) {
                        char[] chars = current.toCharArray();
                        chars[j] = ch;
                        String temp = new String(chars);
                        if (set.contains(temp)) {
                            queue.offer(temp);
                            set.remove(temp);
                        }
                    }
                }
            }
            steps++;
        }

        return -1;
    }
}
```

218. `434. Number of Segments in a String.java`

```
/*
 *
 * Count the number of segments in a string, where a segment is defined to be a
contiguous sequence of non-space characters.
 *
 * Please note that the string does not contain any non-printable characters.
 *
 * Example:
 *
 * Input: "Hello, my name is John"
 * Output: 5
 *
 */

class Solution {

    public int countSegments(String s) {
        s = s.trim();

        if (s.length() == 0) //everything was just space then return 0
        {
            return 0;
        }

        return s.split("\\s+").length;
    }
}
```

219. `438. Find All Anagrams in a String.java`

```
/*
 *
 * Given a string s and a non-empty string p, find all the start indices of p's anagrams
in s.
 *
 * Strings consists of lowercase English letters only and the length of both strings s
and p will not be larger than 20,100.
 *
 * The order of output does not matter.
 *
 * Example 1:
 *
 * Input:
 * s: "cbaebabacd" p: "abc"
 *
 * Output:
 * [0, 6]
 *
 * Explanation:
 * The substring with start index = 0 is "cba", which is an anagram of "abc".
 * The substring with start index = 6 is "bac", which is an anagram of "abc".
 * Example 2:
 *
 * Input:
 * s: "abab" p: "ab"
 *
 * Output:
 * [0, 1, 2]
 *
 * Explanation:
 * The substring with start index = 0 is "ab", which is an anagram of "ab".
 * The substring with start index = 1 is "ba", which is an anagram of "ab".
 * The substring with start index = 2 is "ab", which is an anagram of "ab".
 *
 */

public class Solution {

    public List<Integer> findAnagrams(String s, String t) {
        List<Integer> result = new LinkedList<>();
        if (t.length() > s.length()) {
            return result;
        }
        Map<Character, Integer> map = new HashMap<>();
        for (char c : t.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }
        int counter = map.size();

        int begin = 0, end = 0;
        int head = 0;
```

```
        int len = Integer.MAX_VALUE;

        while (end < s.length()) {
            char c = s.charAt(end);
            if (map.containsKey(c)) {
                map.put(c, map.get(c) - 1);
                if (map.get(c) == 0) {
                    counter--;
                }
            }
            end++;

            while (counter == 0) {
                char tempc = s.charAt(begin);

                if (map.containsKey(tempc)) {
                    if (map.get(tempc) == 0) {
                        counter++;  //need to increase counter when it first adds back to
the map.
                    }
                    map.put(tempc, map.get(tempc) + 1);//plus or minus one
                }
                // if(map.containsKey(tempc)){
                //     map.put(tempc, map.get(tempc) + 1);
                //     if(map.get(tempc) > 0){
                //         counter++;
                //     }
                // }
                if (end - begin == t.length()) {
                    result.add(begin);
                }
                begin++;
            }

        }
        return result;
    }
}
```

220. `443. String Compression.java`

```
/*
 *
 * Given an array of characters, compress it in-place.
 *
 * The length after compression must always be smaller than or equal to the original
array.
 *
 * Every element of the array should be a character (not int) of length 1.
 *
 * After you are done modifying the input array in-place, return the new length of the
array.
 *
 *
 * Follow up:
 * Could you solve it using only O(1) extra space?
 *
 *
 * Example 1:
 *
 * Input:
 * ["a","a","b","b","c","c","c"]
 *
 * Output:
 * Return 6, and the first 6 characters of the input array should be:
["a","2","b","2","c","3"]
 *
 * Explanation:
 * "aa" is replaced by "a2". "bb" is replaced by "b2". "ccc" is replaced by "c3".
 *
 *
 * Example 2:
 *
 * Input:
 * ["a"]
 *
 * Output:
 * Return 1, and the first 1 characters of the input array should be: ["a"]
 *
 * Explanation:
 * Nothing is replaced.
 *
 *
 * Example 3:
 *
 * Input:
 * ["a","b","b","b","b","b","b","b","b","b","b","b","b"]
 *
 * Output:
 * Return 4, and the first 4 characters of the input array should be: ["a","b","1","2"].
 *
 * Explanation:
```

```
 * Since the character "a" does not repeat, it is not compressed. "bbbbbbbbbbbb" is
replaced by "b12".
 * Notice each digit has it's own entry in the array.
 *
 *
 * Note:
 *
 * All characters have an ASCII value in [35, 126].
 * 1 <= len(chars) <= 1000.
 *
 */

class Solution {

    public int compress(char[] chars) {
        int indexAns = 0, index = 0;
        while (index < chars.length) {
            char currentChar = chars[index];
            int count = 0;
            while (index < chars.length && chars[index] == currentChar) {
                index++;
                count++;
            }
            chars[indexAns++] = currentChar;
            if (count != 1) {
                for (char c : Integer.toString(count).toCharArray()) {
                    chars[indexAns++] = c;
                }
            }
        }
        return indexAns;
    }
}
```

221. `445. Add Two Numbers II.java`

```
/*
 *
 * You are given two non-empty linked lists representing two non-negative integers. The
most significant digit comes first and each of their nodes contain a single digit. Add
the two numbers and return it as a linked list.
 *
 * You may assume the two numbers do not contain any leading zero, except the number 0
itself.
 *
 * Follow up:
 * What if you cannot modify the input lists? In other words, reversing the lists is not
allowed.
 *
 * Example:
 *
 * Input: (7 -> 2 -> 4 -> 3) + (5 -> 6 -> 4)
 * Output: 7 -> 8 -> 0 -> 7
 *
 */

//Approach 1: Using Stack

import java.math.BigDecimal;



class Solution {

    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode head = null;
        Stack<Integer> s1 = new Stack<>();
        Stack<Integer> s2 = new Stack<>();

        while (l1 != null) {
            s1.push(l1.val);
            l1 = l1.next;
        }

        while (l2 != null) {
            s2.push(l2.val);
            l2 = l2.next;
        }

        int carry = 0;
        while (!s1.isEmpty() || !s2.isEmpty() || carry != 0) {
            int a = s1.isEmpty() ? 0 : s1.pop();
            int b = s2.isEmpty() ? 0 : s2.pop();
            int sum = a + b + carry;
            carry = sum / 10;

            ListNode node = new ListNode(sum % 10);
```

```
            node.next = head;
            head = node;

        }

        return head;
    }
}

//Approach 2: Using BigDecimal Class

public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        if (l1==null) return l2;
        if (l2==null) return l1;
        StringBuilder s1 = new StringBuilder();
        StringBuilder s2 = new StringBuilder();
        while (l1 != null) {
            s1.append(l1.val);
            l1 = l1.next;
        }
        while (l2 != null) {
            s2.append(l2.val);
            l2 = l2.next;
        }

        BigDecimal b1 = new BigDecimal(s1.toString());
        BigDecimal b2 = new BigDecimal(s2.toString());
        BigDecimal add = b1.add(b2);
        String str = String.valueOf(add.toString());
        ListNode dummy = new ListNode(-1);
        ListNode cur = dummy;

        for (int i = 0; i < str.length(); i++) {
            cur.next = new ListNode(Integer.valueOf(Character.toString(str.charAt(i))));
            cur = cur.next;
        }
        return dummy.next;
    }
```

222. `448. Find All Numbers Disappeared in an Array.java`

```
/*
 *
 * Given an array of integers where 1 ≤ a[i] ≤ n (n = size of array), some elements
appear twice and others appear once.
 *
 * Find all the elements of [1, n] inclusive that do not appear in this array.
 *
 * Could you do it without extra space and in O(n) runtime? You may assume the returned
list does not count as extra space.
 *
 * Example:
 *
 * Input:
 * [4,3,2,7,8,2,3,1]
 *
 * Output:
 * [5,6]
 *
 */

class Solution {

    public List<Integer> findDisappearedNumbers(int[] nums) {
        int[] lookup = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            lookup[nums[i]]++;
        }

        ArrayList ret = new ArrayList();
        for (int i = 0; i < lookup.length; i++) {
            if (lookup[i] == 0 && i != 0) {
                ret.add(i);
            }
        }
        return ret;
    }
}
```

223. `451. Sort Characters By Frequency.java`

```
/*
 *
 * Given a string, sort it in decreasing order based on the frequency of characters.
 *
 * Example 1:
 *
 * Input:
 * "tree"
 *
 * Output:
 * "eert"
 *
 * Explanation:
 * 'e' appears twice while 'r' and 't' both appear once.
 * So 'e' must appear before both 'r' and 't'. Therefore "eetr" is also a valid answer.
 * Example 2:
 *
 * Input:
 * "cccaaa"
 *
 * Output:
 * "cccaaa"
 *
 * Explanation:
 * Both 'c' and 'a' appear three times, so "aaaccc" is also a valid answer.
 * Note that "cacaca" is incorrect, as the same characters must be together.
 * Example 3:
 *
 * Input:
 * "Aabb"
 *
 * Output:
 * "bbAa"
 *
 * Explanation:
 * "bbaA" is also a valid answer, but "Aabb" is incorrect.
 * Note that 'A' and 'a' are treated as two different characters.
 *
 */

class Solution {

    public String frequencySort(String s) {
        HashMap<Character, Integer> map = new HashMap<Character, Integer>();
        for (int i = 0; i < s.length(); i++) {
            map.put(s.charAt(i), map.getOrDefault(s.charAt(i), 0) + 1);
        }

        PriorityQueue<Map.Entry<Character, Integer>> maxHeap = new PriorityQueue<>((a, b)
-> (b.getValue() - a.getValue()));
```

```
        maxHeap.addAll(map.entrySet());

        StringBuilder sb = new StringBuilder();
        while (!maxHeap.isEmpty()) {
            Map.Entry<Character, Integer> entry = maxHeap.poll();
            for (int i = 0; i < entry.getValue(); i++) {
                sb.append(entry.getKey());
            }
        }

        return sb.toString();

    }
}
```

224. `456. 132 Pattern.java`

```
/*
 *
 * Given a sequence of n integers a1, a2, ..., an, a 132 pattern is a subsequence ai, aj,
ak such that i < j < k and ai < ak < aj. Design an algorithm that takes a list of n
numbers as input and checks whether there is a 132 pattern in the list.
 *
 * Note: n will be less than 15,000.
 *
 * Example 1:
 * Input: [1, 2, 3, 4]
 *
 * Output: False
 *
 * Explanation: There is no 132 pattern in the sequence.
 * Example 2:
 * Input: [3, 1, 4, 2]
 *
 * Output: True
 *
 * Explanation: There is a 132 pattern in the sequence: [1, 4, 2].
 * Example 3:
 * Input: [-1, 3, 2, 0]
 *
 * Output: True
 *
 * Explanation: There are three 132 patterns in the sequence: [-1, 3, 2], [-1, 3, 0] and
[-1, 2, 0].
 *
 */

class Solution {

    public boolean find132pattern(int[] nums) {
        for (int j = 0, min = Integer.MAX_VALUE; j < nums.length; j++) {
            min = Math.min(nums[j], min);
            if (min == nums[j]) {
                continue;
            }

            for (int k = nums.length - 1; k > j; k--) {
                if (min < nums[k] && nums[k] < nums[j]) {
                    return true;
                }
            }
        }
        return false;

    }
}
```

225. `459. Repeated Substring Pattern.java`

```
/*
 *
 * Given a non-empty string check if it can be constructed by taking a substring of it and appending multiple copies of the substring together. You may assume the given string consists of lowercase English letters only and its length will not exceed 10000.
 *
 * Example 1:
 *
 * Input: "abab"
 * Output: True
 * Explanation: It's the substring "ab" twice.
 * Example 2:
 *
 * Input: "aba"
 * Output: False
 * Example 3:
 *
 * Input: "abcabcabcabc"
 * Output: True
 * Explanation: It's the substring "abc" four times. (And the substring "abcabc" twice.)
 *
 */

class Solution {

    public boolean repeatedSubstringPattern(String str) {
        String s = str + str;
        return s.substring(1, s.length() - 1).contains(str);
    }
}
```

226. `460. LFU Cache.java`

```
/*
 *
 * Design and implement a data structure for Least Frequently Used (LFU) cache. It should support the following operations: get and put.
 *
 * get(key) - Get the value (will always be positive) of the key if the key exists in the cache, otherwise return -1.
 * put(key, value) - Set or insert the value if the key is not already present. When the cache reaches its capacity, it should invalidate the least frequently used item before inserting a new item. For the purpose of this problem, when there is a tie (i.e., two or more keys that have the same frequency), the least recently used key would be evicted.
 *
 * Follow up:
 * Could you do both operations in O(1) time complexity?
 *
 * Example:
 *
 * LFUCache cache = new LFUCache( 2 );
 *
 * cache.put(1, 1);
 * cache.put(2, 2);
 * cache.get(1);       // returns 1
 * cache.put(3, 3);    // evicts key 2
 * cache.get(2);       // returns -1 (not found)
 * cache.get(3);       // returns 3.
 * cache.put(4, 4);    // evicts key 1.
 * cache.get(1);       // returns -1 (not found)
 * cache.get(3);       // returns 3
 * cache.get(4);       // returns 4
 *
 */

public class LFUCache {

    HashMap<Integer, Integer> vals;
    HashMap<Integer, Integer> counts;
    HashMap<Integer, LinkedHashSet<Integer>> lists;
    int cap;
    int min = -1;

    public LFUCache(int capacity) {
        cap = capacity;
        vals = new HashMap<>();
        counts = new HashMap<>();
        lists = new HashMap<>();
        lists.put(1, new LinkedHashSet<>());
    }

    public int get(int key) {
        if (!vals.containsKey(key)) {
            return -1;
```

```
        }
        int count = counts.get(key);
        counts.put(key, count + 1);
        lists.get(count).remove(key);
        if (count == min && lists.get(count).size() == 0) {
            min++;
        }
        if (!lists.containsKey(count + 1)) {
            lists.put(count + 1, new LinkedHashSet<>());
        }
        lists.get(count + 1).add(key);
        return vals.get(key);
    }

    public void set(int key, int value) {
        if (cap <= 0) {
            return;
        }
        if (vals.containsKey(key)) {
            vals.put(key, value);
            get(key);
            return;
        }
        if (vals.size() >= cap) {
            int evit = lists.get(min).iterator().next();
            lists.get(min).remove(evit);
            vals.remove(evit);
        }
        vals.put(key, value);
        counts.put(key, 1);
        min = 1;
        lists.get(1).add(key);
    }
}
```

227. `461. Hamming Distance.java`

```
/*

The Hamming distance between two integers is the number of positions at which the
corresponding bits are different.

Given two integers x and y, calculate the Hamming distance.

Note:
0 ≤ x, y < 231.

Example:

Input: x = 1, y = 4

Output: 2

Explanation:
1   (0 0 0 1)
4   (0 1 0 0)
       ↑   ↑

The above arrows point to positions where the corresponding bits are different.

*/

public class Solution {
    public int hammingDistance(int x, int y)
    {

         int n = x ^ y;

        int count = 0;
        while (n != 0) {
            count++;
            n = n & (n - 1);
        }
        return count;
    }
}
```

228. `463. Island Perimeter.java`

```
/*
 *
 * You are given a map in form of a two-dimensional integer grid where 1 represents land
and 0 represents water.
 *
 * Grid cells are connected horizontally/vertically (not diagonally). The grid is
completely surrounded by water, and there is exactly one island (i.e., one or more
connected land cells).
 *
 * The island doesn't have "lakes" (water inside that isn't connected to the water around
the island). One cell is a square with side length 1. The grid is rectangular, width and
height don't exceed 100. Determine the perimeter of the island.
 *
 *
 *
 * Example:
 *
 * Input:
 * [[0,1,0,0],
 * [1,1,1,0],
 * [0,1,0,0],
 * [1,1,0,0]]
 *
 * Output: 16
 *
 */

class Solution {

    public int islandPerimeter(int[][] grid) {
        if (grid.length == 0) {
            return 0;
        }
        boolean[][] visited = new boolean[grid.length][grid[0].length];
        for (int i = 0; i < grid.length; i++) {
            for (int j = 0; j < grid[0].length; j++) {
                if (grid[i][j] == 1) { //If it is an island then do a DFS
                    return dfs(grid, visited, i, j);
                }
            }
        }
        return 0;
    }

    public int dfs(int[][] grid, boolean[][] visited, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length) {
            return 1;//If it is out of boundary then add one edge
        }

        if (grid[r][c] == 0) {
            return 1;//If is a water then add one
```

```
        }

        if (visited[r][c]) {
            return 0;//If it already visited island, no need to add it one more time
        }
        visited[r][c] = true;

        return dfs(grid, visited, r + 1, c) + dfs(grid, visited, r - 1, c) + dfs(grid,
visited, r, c + 1) + dfs(grid, visited, r, c - 1); //Recursively go in the 4 directions

    }
}
```

229. `468. Validate IP Address.java`

```
/*
 *
 * Write a function to check whether an input string is a valid IPv4 address or IPv6
address or neither.
 *
 * IPv4 addresses are canonically represented in dot-decimal notation, which consists of
four decimal numbers, each ranging from 0 to 255, separated by dots ("."),
e.g.,172.16.254.1;
 *
 * Besides, leading zeros in the IPv4 is invalid. For example, the address 172.16.254.01
is invalid.
 *
 * IPv6 addresses are represented as eight groups of four hexadecimal digits, each group
representing 16 bits. The groups are separated by colons (":"). For example, the address
2001:0db8:85a3:0000:0000:8a2e:0370:7334 is a valid one. Also, we could omit some leading
zeros among four hexadecimal digits and some low-case characters in the address to upper-
case ones, so 2001:db8:85a3:0:0:8A2E:0370:7334 is also a valid IPv6 address(Omit leading
zeros and using upper cases).
 *
 * However, we don't replace a consecutive group of zero value with a single empty group
using two consecutive colons (::) to pursue simplicity. For example,
2001:0db8:85a3::8A2E:0370:7334 is an invalid IPv6 address.
 *
 * Besides, extra leading zeros in the IPv6 is also invalid. For example, the address
02001:0db8:85a3:0000:0000:8a2e:0370:7334 is invalid.
 *
 * Note: You may assume there is no extra space or special characters in the input
string.
 *
 * Example 1:
 * Input: "172.16.254.1"
 *
 * Output: "IPv4"
 *
 * Explanation: This is a valid IPv4 address, return "IPv4".
 * Example 2:
 * Input: "2001:0db8:85a3:0:0:8A2E:0370:7334"
 *
 * Output: "IPv6"
 *
 * Explanation: This is a valid IPv6 address, return "IPv6".
 * Example 3:
 * Input: "256.256.256.256"
 *
 * Output: "Neither"
 *
 * Explanation: This is neither a IPv4 address nor a IPv6 address.
 *
 */

class Solution {
```

```
public String validIPAddress(String IP) {
    if (isValidIPv4(IP)) {
        return "IPv4";
    } else if (isValidIPv6(IP)) {
        return "IPv6";
    } else {
        return "Neither";
    }
}

public boolean isValidIPv4(String ip) {
    if (ip.length() < 7) {
        return false;
    }
    if (ip.charAt(0) == '.') {
        return false;
    }
    if (ip.charAt(ip.length() - 1) == '.') {
        return false;
    }
    String[] tokens = ip.split("\\.");
    if (tokens.length != 4) {
        return false;
    }
    for (String token : tokens) {
        if (!isValidIPv4Token(token)) {
            return false;
        }
    }
    return true;
}

public boolean isValidIPv4Token(String token) {
    if (token.startsWith("0") && token.length() > 1) {
        return false;
    }
    try {
        int parsedInt = Integer.parseInt(token);
        if (parsedInt < 0 || parsedInt > 255) {
            return false;
        }
        if (parsedInt == 0 && token.charAt(0) != '0') {
            return false;
        }
    } catch (NumberFormatException nfe) {
        return false;
    }
    return true;
}

public boolean isValidIPv6(String ip) {
    if (ip.length() < 15) {
        return false;
```

```
        }
        if (ip.charAt(0) == ':') {
            return false;
        }
        if (ip.charAt(ip.length() - 1) == ':') {
            return false;
        }
        String[] tokens = ip.split(":");
        if (tokens.length != 8) {
            return false;
        }
        for (String token : tokens) {
            if (!isValidIPv6Token(token)) {
                return false;
            }
        }
        return true;
    }

    public boolean isValidIPv6Token(String token) {
        if (token.length() > 4 || token.length() == 0) {
            return false;
        }
        char[] chars = token.toCharArray();
        for (char c : chars) {
            boolean isDigit = c >= 48 && c <= 57;
            boolean isUppercaseAF = c >= 65 && c <= 70;
            boolean isLowerCaseAF = c >= 97 && c <= 102;
            if (!(isDigit || isUppercaseAF || isLowerCaseAF)) {
                return false;
            }
        }
        return true;
    }
}
```

230. `473. Matchsticks to Square.java`

```
/*
 *
 * Remember the story of Little Match Girl? By now, you know exactly what matchsticks the little match girl has, please find out a way you can make one square by using up all those matchsticks. You should not break any stick, but you can link them up, and each matchstick must be used exactly one time.
 *
 * Your input will be several matchsticks the girl has, represented with their stick length. Your output will either be true or false, to represent whether you could make one square using all the matchsticks the little match girl has.
 *
 * Example 1:
 * Input: [1,1,2,2,2]
 * Output: true
 *
 * Explanation: You can form a square with length 2, one side of the square came two sticks with length 1.
 * Example 2:
 * Input: [3,3,3,3,4]
 * Output: false
 *
 * Explanation: You cannot find a way to form a square with all the matchsticks.
 * Note:
 * The length sum of the given matchsticks is in the range of 0 to 10^9.
 * The length of the given matchstick array will not exceed 15.
 *
 */

class Solution {

    public boolean makesquare(int[] nums) {
        if (nums == null || nums.length < 4) {
            return false;
        }
        int sum = 0;
        for (int num : nums) {
            sum += num;
        }
        if (sum % 4 != 0) {
            return false;
        }

        Arrays.sort(nums);
        // reverse(nums);

        return dfs(nums, new int[4], 0, sum / 4);
    }

    private boolean dfs(int[] nums, int[] sums, int index, int target) {
        if (index == nums.length) {
            if (sums[0] == target && sums[1] == target && sums[2] == target) {
```

```
                return true;
            }
            return false;
        }

        for (int i = 0; i < 4; i++) {
            if (sums[i] + nums[index] > target) {
                continue;
            }
            sums[i] += nums[index];
            if (dfs(nums, sums, index + 1, target)) {
                return true;
            }
            sums[i] -= nums[index];
        }

        return false;
    }
}
```

231. `476. Number Complement.java`

```
/*
 *
 * Given a positive integer, output its complement number. The complement strategy is to
flip the bits of its binary representation.
 *
 * Note:
 * The given integer is guaranteed to fit within the range of a 32-bit signed integer.
 * You could assume no leading zero bit in the integer’s binary representation.
 * Example 1:
 * Input: 5
 * Output: 2
 * Explanation: The binary representation of 5 is 101 (no leading zero bits), and its
complement is 010. So you need to output 2.
 * Example 2:
 * Input: 1
 * Output: 0
 * Explanation: The binary representation of 1 is 1 (no leading zero bits), and its
complement is 0. So you need to output 0.
 *
 */

public class Solution {

    public int findComplement(int num) {
        int i = 0;
        int j = 0;

        while (i < num) {
            i += Math.pow(2, j);
            j++;
        }

        return i - num;
    }
}
```

232. `482. License Key Formatting.java`

```
/*
 *
 * You are given a license key represented as a string S which consists only alphanumeric character and dashes. The string is separated into N+1 groups by N dashes.
 *
 * Given a number K, we would want to reformat the strings such that each group contains exactly K characters, except for the first group which could be shorter than K, but still must contain at least one character. Furthermore, there must be a dash inserted between two groups and all lowercase letters should be converted to uppercase.
 *
 * Given a non-empty string S and a number K, format the string according to the rules described above.
 *
 * Example 1:
 * Input: S = "5F3Z-2e-9-w", K = 4
 *
 * Output: "5F3Z-2E9W"
 *
 * Explanation: The string S has been split into two parts, each part has 4 characters.
 * Note that the two extra dashes are not needed and can be removed.
 * Example 2:
 * Input: S = "2-5g-3-J", K = 2
 *
 * Output: "2-5G-3J"
 *
 * Explanation: The string S has been split into three parts, each part has 2 characters except the first part as it could be shorter as mentioned above.
 * Note:
 * The length of string S will not exceed 12,000, and K is a positive integer.
 * String S consists only of alphanumerical characters (a-z and/or A-Z and/or 0-9) and dashes(-).
 * String S is non-empty.
 *
 */

public class Solution {

    public String licenseKeyFormatting(String s, int K) {
        StringBuilder sb = new StringBuilder();
        s = s.replace("-", "");
        String s1 = s.toUpperCase();

        Stack stk = new Stack();
        for (int i = 0; i < s1.length(); i++) {
            stk.push(s1.charAt(i));
        }

        int count = 0;
        char[] s2 = s1.toCharArray();

        while (!stk.empty()) {
```

```
            count++;
            if (count != K + 1) {
                sb.append(stk.peek());
                stk.pop();
            }
            if (count == K + 1) {
                count = 0;
                sb.append("-");
                continue;
            }
        }
        //System.out.println(sb.toString());
        return sb.reverse().toString();
    }
}
```

233. `485. Max Consecutive Ones.java`

```
/*
 *
 * Given a binary array, find the maximum number of consecutive 1s in this array.
 *
 * Example 1:
 * Input: [1,1,0,1,1,1]
 * Output: 3
 * Explanation: The first two digits or the last three digits are consecutive 1s.
 * The maximum number of consecutive 1s is 3.
 * Note:
 *
 * The input array will only contain 0 and 1.
 * The length of input array is a positive integer and will not exceed 10,000
 *
 */

class Solution {

    public int findMaxConsecutiveOnes(int[] num) {
        int result = 0;
        int count = 0;
        int max = -1;
        int temp = -1;
        for (int i = 0; i < num.length; i++) {
            if (num[i] == 1) {
                count++;
                temp = count;
            } else {
                temp = count;
                if (num[i] == 0) {
                    count = 0;
                }
            }
            if (max < temp) {
                max = temp;
            }
        }

        return max;
    }

}
```

234. `494. Target Sum.java`

```
/*
 *
 * You are given a list of non-negative integers, a1, a2, ..., an, and a target, S. Now
you have 2 symbols + and -. For each integer, you should choose one from + and - as its
new symbol.
 *
 * Find out how many ways to assign symbols to make sum of integers equal to target S.
 *
 * Example 1:
 * Input: nums is [1, 1, 1, 1, 1], S is 3.
 * Output: 5
 * Explanation:  *
 * -1+1+1+1+1 = 3
 * +1-1+1+1+1 = 3
 * +1+1-1+1+1 = 3
 * +1+1+1-1+1 = 3
 * +1+1+1+1-1 = 3
 *
 * There are 5 ways to assign symbols to make the sum of nums be target 3.
 * Note:
 * The length of the given array is positive and will not exceed 20.
 * The sum of elements in the given array will not exceed 1000.
 * Your output answer is guaranteed to be fitted in a 32-bit integer.
 *
 */

class Solution {

    int result = 0;

    public int findTargetSumWays(int[] nums, int S) {
        if (nums == null || nums.length == 0) {
            return result;
        }
        helper(nums, S, 0, 0);
        return result;
    }

    public void helper(int[] nums, int target, int pos, long eval) {
        if (pos == nums.length) {
            if (target == eval) {
                result++;
            }
            return;
        }
        helper(nums, target, pos + 1, eval + nums[pos]);
        helper(nums, target, pos + 1, eval - nums[pos]);
    }
}
```

235. `500. Keyboard Row.java`

```
/*
 *
 * Given a List of words, return the words that can be typed using letters of alphabet on only one row's of American keyboard like the image below.
 *
 * Example:
 *
 * Input: ["Hello", "Alaska", "Dad", "Peace"]
 * Output: ["Alaska", "Dad"]
 *
 *
 * Note:
 *
 * You may use one character in the keyboard more than once.
 * You may assume the input string will only contain letters of alphabet.
 *
 */

public class Solution {

    public static String[] findWords(String[] words) {
        String s1 = "qwertyuiop", s2 = "asdfghjkl", s3 = "zxcvbnm";
        List<String> list = new ArrayList<String>();
        int count1 = 0, count2 = 0, count3 = 0;
        for (int i = 0; i != words.length; i++) {
            for (int j = 0; j != words[i].length(); j++) {
                if (s1.indexOf(words[i].toLowerCase().charAt(j)) != -1) {
                    count1++;
                }
                if (s2.indexOf(words[i].toLowerCase().charAt(j)) != -1) {
                    count2++;
                }
                if (s3.indexOf(words[i].toLowerCase().charAt(j)) != -1) {
                    count3++;
                }
            }
            if (count1 == words[i].length() || count2 == words[i].length() || count3 == words[i].length()) {
                list.add(words[i]);
            }
            count1 = 0;
            count2 = 0;
            count3 = 0;
        }
        return list.toArray(new String[0]);
    }

}
```

236. `509. Fibonacci Number.java`

```
/*
 *
 * The Fibonacci numbers, commonly denoted F(n) form a sequence, called the Fibonacci
sequence, such that each number is the sum of the two preceding ones, starting from 0 and
1. That is,
 *
 * F(0) = 0,   F(1) = 1
 * F(N) = F(N - 1) + F(N - 2), for N > 1.
 * Given N, calculate F(N).
 *
 *
 *
 * Example 1:
 *
 * Input: 2
 * Output: 1
 * Explanation: F(2) = F(1) + F(0) = 1 + 0 = 1.
 * Example 2:
 *
 * Input: 3
 * Output: 2
 * Explanation: F(3) = F(2) + F(1) = 1 + 1 = 2.
 * Example 3:
 *
 * Input: 4
 * Output: 3
 * Explanation: F(4) = F(3) + F(2) = 2 + 1 = 3.
 *
 */

class Solution {

    public int fib(int N) {
        if (N <= 1) {
            return N;
        } else {
            return fib(N - 1) + fib(N - 2);
        }
    }
}
```

237. `513. Find Bottom Left Tree Value.java`

```
/*
Given a binary tree, find the leftmost value in the last row of the tree.

Example 1:
Input:

    2
   / \
  1   3

Output:
1
Example 2:
Input:

        1
       / \
      2   3
     /   / \
    4   5   6
       /
      7

Output:
7
Note: You may assume the tree (i.e., the given root node) is not NULL.


*/

class Solution {

    public int findBottomLeftValue(TreeNode root) {
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);
        TreeNode n = null;

        while (!queue.isEmpty()) {
            n = queue.poll();
            if (n.right != null) {
                queue.offer(n.right);
            }
            if (n.left != null) {
                queue.offer(n.left);
            }
        }
        return n.val;

    }
}
```

238. `515. Find Largest Value in Each Tree Row.java`

```
/*

You need to find the largest value in each row of a binary tree.

Example:
Input:

          1
         / \
        3   2
       / \   \
      5   3   9

Output: [1, 3, 9]

*/

class Solution {

    public List<Integer> largestValues(TreeNode root) {
        Queue<TreeNode> queue = new LinkedList<TreeNode>();
        List<Integer> res = new ArrayList<Integer>();
        queue.add(root);

        int queueSize = root == null ? 0 : 1;

        while (queueSize > 0) {
            int largestElement = Integer.MIN_VALUE;

            for (int i = 0; i < queueSize; i++) {
                TreeNode cur = queue.poll();
                largestElement = Math.max(cur.val, largestElement);

                if (cur.left != null) {
                    queue.add(cur.left);
                }
                if (cur.right != null) {
                    queue.add(cur.right);
                }
            }

            res.add(largestElement);
            queueSize = queue.size();
        }

        return res;

    }
}
```

239. `518. Coin Change 2.java`

```
/*
 *
 * You are given coins of different denominations and a total amount of money. Write a
function to compute the number of combinations that make up that amount. You may assume
that you have infinite number of each kind of coin.
 *
 *
 *
 * Example 1:
 *
 * Input: amount = 5, coins = [1, 2, 5]
 * Output: 4
 * Explanation: there are four ways to make up the amount:
 * 5=5
 * 5=2+2+1
 * 5=2+1+1+1
 * 5=1+1+1+1+1
 * Example 2:
 *
 * Input: amount = 3, coins = [2]
 * Output: 0
 * Explanation: the amount of 3 cannot be made up just with coins of 2.
 * Example 3:
 *
 * Input: amount = 10, coins = [10]
 * Output: 1
 *
 *
 * Note:
 *
 * You can assume that
 *
 * 0 <= amount <= 5000
 * 1 <= coin <= 5000
 * the number of coins is less than 500
 * the answer is guaranteed to fit into signed 32-bit integer
 *
 */

class Solution {

    public int change(int amount, int[] coins) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int coin : coins) {
            for (int i = 1; i <= amount; i++) {
                if (i - coin >= 0) {
                    dp[i] += dp[i - coin];
                }
            }
        }
```

```
        return dp[amount];
    }
}
```

240. `520. Detect Capital.java`

```
/*
 * Given a word, you need to judge whether the usage of capitals in it is right or not.
 *
 * We define the usage of capitals in a word to be right when one of the following cases
holds:
 *
 * All letters in this word are capitals, like "USA".
 * All letters in this word are not capitals, like "leetcode".
 * Only the first letter in this word is capital if it has more than one letter, like
"Google".
 * Otherwise, we define that this word doesn't use capitals in a right way.
 * Example 1:
 * Input: "USA"
 * Output: True
 * Example 2:
 * Input: "FlaG"
 * Output: False
 * Note: The input will be a non-empty word consisting of uppercase and lowercase latin
letters.
 *
 *
 */

class Solution {

    public boolean detectCapitalUse(String word) {
        return word.equals(word.toUpperCase())
                || word.equals(word.toLowerCase())
                || Character.isUpperCase(word.charAt(0))
                && word.substring(1).equals(word.substring(1).toLowerCase());
    }
}
```

241. `524. Longest Word in Dictionary through Deleting.java`

```
/*
 * Given a string and a string dictionary, find the longest string in the dictionary that can be formed by deleting some characters of the given string. If there are more than one possible results, return the longest word with the smallest lexicographical order. If there is no possible result, return the empty string.
 *
 * Example 1:
 * Input:
 * s = "abpcplea", d = ["ale","apple","monkey","plea"]
 *
 * Output:
 * "apple"
 * Example 2:
 * Input:
 * s = "abpcplea", d = ["a","b","c"]
 *
 * Output:
 * "a"
 * Note:
 * All the strings in the input will only contain lower-case letters.
 * The size of the dictionary won't exceed 1,000.
 * The length of all the strings in the input won't exceed 1,000.
 *
 */

class Solution {

    public String findLongestWord(String s, List<String> d) {
        Collections.sort(d, (a, b) -> {
            if (a.length() != b.length()) {
                return b.length() - a.length();
            }
            return a.compareTo(b);
        });

        for (String dictWord : d) {
            int i = 0;
            for (char c : s.toCharArray()) {
                if (i < dictWord.length() && c == dictWord.charAt(i)) {
                    i++;
                }
            }

            if (i == dictWord.length()) {
                return dictWord;
            }
        }
        return "";
    }
}
```

242. `529. Minesweeper.java`

```
/*
 *
 * Let's play the minesweeper game (Wikipedia, online game)!
 *
 * You are given a 2D char matrix representing the game board. 'M' represents an
unrevealed mine, 'E' represents an unrevealed empty square, 'B' represents a revealed
blank square that has no adjacent (above, below, left, right, and all 4 diagonals) mines,
digit ('1' to '8') represents how many mines are adjacent to this revealed square, and
finally 'X' represents a revealed mine.
 *
 * Now given the next click position (row and column indices) among all the unrevealed
squares ('M' or 'E'), return the board after revealing this position according to the
following rules:
 *
 * If a mine ('M') is revealed, then the game is over - change it to 'X'.
 * If an empty square ('E') with no adjacent mines is revealed, then change it to
revealed blank ('B') and all of its adjacent unrevealed squares should be revealed
recursively.
 * If an empty square ('E') with at least one adjacent mine is revealed, then change it
to a digit ('1' to '8') representing the number of adjacent mines.
 * Return the board when no more squares will be revealed.
 *
 *
 * Example 1:
 *
 * Input:  *
 * [['E', 'E', 'E', 'E', 'E'],
 * ['E', 'E', 'M', 'E', 'E'],
 * ['E', 'E', 'E', 'E', 'E'],
 * ['E', 'E', 'E', 'E', 'E']]
 *
 * Click : [3,0]
 *
 * Output:  *
 * [['B', '1', 'E', '1', 'B'],
 * ['B', '1', 'M', '1', 'B'],
 * ['B', '1', '1', '1', 'B'],
 * ['B', 'B', 'B', 'B', 'B']]
 *
 * Explanation:
 *
 * Example 2:
 *
 * Input:  *
 * [['B', '1', 'E', '1', 'B'],
 * ['B', '1', 'M', '1', 'B'],
 * ['B', '1', '1', '1', 'B'],
 * ['B', 'B', 'B', 'B', 'B']]
 *
 * Click : [1,2]
 *
```

```
 * Output:  *
 * [['B', '1', 'E', '1', 'B'],
 * ['B', '1', 'X', '1', 'B'],
 * ['B', '1', '1', '1', 'B'],
 * ['B', 'B', 'B', 'B', 'B']]
 *
 * Explanation:
 *
 *
 *
 * Note:
 *
 * The range of the input matrix's height and width is [1,50].
 * The click position will only be an unrevealed square ('M' or 'E'), which also means the input board contains at least one clickable square.
 * The input board won't be a stage when game is over (some mines have been revealed).
 * For simplicity, not mentioned rules should be ignored in this problem. For example, you don't need to reveal all the unrevealed mines when the game is over, consider any cases that you will win the game or flag any squares.
 *
 */

public class Solution {

    public char[][] updateBoard(char[][] board, int[] click) {
        int m = board.length, n = board[0].length;
        Queue<int[]> q = new LinkedList<>();

        if (board[click[0]][click[1]] == 'M') {
            board[click[0]][click[1]] = 'X';
            return board;
        } else {
            board[click[0]][click[1]] = 'B';
        }

        int[] dx = {1, 1, 0, -1, -1, -1, 0, 1};
        int[] dy = {0, 1, 1, 1, 0, -1, -1, -1};

        q.offer(click);

        while (!q.isEmpty()) {
            int[] cur = q.poll();

            int count = 0;

            for (int k = 0; k < 8; k++) {
                int i = cur[0] + dx[k], j = cur[1] + dy[k];
                if (i < 0 || i >= m || j < 0 || j >= n) {
                    continue;
                }
                if (board[i][j] == 'M') {
                    count++;
                }
            }
```

```
            if (count != 0) {
                board[cur[0]][cur[1]] = (char) ('0' + count);
            } else {
                for (int k = 0; k < 8; k++) {
                    int i = cur[0] + dx[k], j = cur[1] + dy[k];

                    if (i < 0 || i >= m || j < 0 || j >= n) {
                        continue;
                    }
                    if (board[i][j] == 'E') {
                        board[i][j] = 'B';
                        q.offer(new int[]{i, j});
                    }
                }
            }
        }

        return board;
    }
}
```

243. `530. Minimum Absolute Difference in BST.java`

```
/*

Given a binary search tree with non-negative values, find the minimum absolute difference
between values of any two nodes.

Example:

Input:

   1
    \
     3
    /
   2

Output:
1

Explanation:
The minimum absolute difference is 1, which is the difference between 2 and 1 (or between
2 and 3).


Note: There are at least two nodes in this BST.


*/

class Solution {

    private int min = Integer.MAX_VALUE;
    private TreeNode prev = null;

    public int getMinimumDifference(TreeNode root) {
        if (root == null) {
            return min; //Reached the leaf node so we have got a minimum value from that
subtree
        }
        getMinimumDifference(root.left);

        if (prev != null) {
            min = Math.min(min, root.val - prev.val);
        }

        prev = root;
        getMinimumDifference(root.right);

        return min;
    }
}
```

244. `532. K-diff Pairs in an Array.java`

```
/*
 *
 * Given an array of integers and an integer k, you need to find the number of unique k-
diff pairs in the array. Here a k-diff pair is defined as an integer pair (i, j), where i
and j are both numbers in the array and their absolute difference is k.
 *
 * Example 1:
 * Input: [3, 1, 4, 1, 5], k = 2
 * Output: 2
 * Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).
 * Although we have two 1s in the input, we should only return the number of unique
pairs.
 * Example 2:
 * Input:[1, 2, 3, 4, 5], k = 1
 * Output: 4
 * Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4,
5).
 * Example 3:
 * Input: [1, 3, 1, 5, 4], k = 0
 * Output: 1
 * Explanation: There is one 0-diff pair in the array, (1, 1).
 * Note:
 * The pairs (i, j) and (j, i) count as the same pair.
 * The length of the array won't exceed 10,000.
 * All the integers in the given input belong to the range: [-1e7, 1e7].
 *
 */

class Solution {

    public int findPairs(int[] nums, int k) {
        if (nums == null || nums.length == 0 || k < 0) {
            return 0;
        }

        Map<Integer, Integer> map = new HashMap<>();
        int count = 0;
        for (int i : nums) {
            map.put(i, map.getOrDefault(i, 0) + 1);
        }

        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            if (k == 0) {
                //count how many elements in the array that appear more than twice,
because they will have a difference = 0
                if (entry.getValue() >= 2) {
                    count++;
                }
            } else //We can even do entry.getKey() - k, since our difference is absolute.
So both + and - will work completely fine!
            if (map.containsKey(entry.getKey() + k)) {
```

```
                count++;
            }
        }

        return count;
    }
}
```

245. `535. Encode and Decode TinyURL.java`

```
/*
 *
 * Note: This is a companion problem to the System Design problem: Design TinyURL.
 * TinyURL is a URL shortening service where you enter a URL such as
https://leetcode.com/problems/design-tinyurl and it returns a short URL such as
http://tinyurl.com/4e9iAk.
 *
 * Design the encode and decode methods for the TinyURL service. There is no restriction
on how your encode/decode algorithm should work. You just need to ensure that a URL can
be encoded to a tiny URL and the tiny URL can be decoded to the original URL.
 *
 */

public class Codec {

    Map<Integer, String> map = new HashMap<>();
    int i = 0;

    public String encode(String longUrl) {
        map.put(i, longUrl);
        return "http://tinyurl.com/" + i++;
    }

    public String decode(String shortUrl) {
        return map.get(Integer.parseInt(shortUrl.replace("http://tinyurl.com/", "")));
    }
}

// Your Codec object will be instantiated and called as such:
// Codec codec = new Codec();
// codec.decode(codec.encode(url));
```

246. `542. 01 Matrix.java`

```
/*
 *
 * Given a matrix consists of 0 and 1, find the distance of the nearest 0 for each cell.
 *
 * The distance between two adjacent cells is 1.
 *
 *
 *
 * Example 1:
 *
 * Input:
 * [[0,0,0],
 * [0,1,0],
 * [0,0,0]]
 *
 * Output:
 * [[0,0,0],
 * [0,1,0],
 * [0,0,0]]
 * Example 2:
 *
 * Input:
 * [[0,0,0],
 * [0,1,0],
 * [1,1,1]]
 *
 * Output:
 * [[0,0,0],
 * [0,1,0],
 * [1,2,1]]
 *
 *
 * Note:
 *
 * The number of elements of the given matrix will not exceed 10,000.
 * There are at least one 0 in the given matrix.
 * The cells are adjacent in only four directions: up, down, left and right.
 *
 */

public class Solution {

    public int[][] updateMatrix(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;

        Queue<int[]> queue = new LinkedList<>();
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (matrix[i][j] == 0) {
                    queue.offer(new int[]{i, j});
```

```
                } else {
                    matrix[i][j] = Integer.MAX_VALUE;
                }
            }
        }

        int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};

        while (!queue.isEmpty()) {
            int[] cell = queue.poll();

            for (int[] d : dirs) {
                int nx = cell[0] + d[0];
                int ny = cell[1] + d[1];

                if (nx >= 0 && nx < m && ny >= 0 && ny < n
                        && matrix[nx][ny] > matrix[cell[0]][cell[1]] + 1) {
                    queue.add(new int[]{nx, ny});
                    matrix[nx][ny] = matrix[cell[0]][cell[1]] + 1;
                }
            }
        }

        return matrix;
    }
}
```

247. `543. Diameter of Binary Tree.java`

```
/*
 *
 * Given a binary tree, you need to compute the length of the diameter of the tree. The
diameter of a binary tree is the length of the longest path between any two nodes in a
tree. This path may or may not pass through the root.
 *
 * Example:
 * Given a binary tree
 *     1
 *    / \
 *   2   3
 *  / \
 * 4   5
 * Return 3, which is the length of the path [4,2,1,3] or [5,2,1,3].
 *
 * Note: The length of path between two nodes is represented by the number of edges
between them.
 *
 *
 */

class Solution {

    public int diameterOfBinaryTree(TreeNode root) {
        if (root == null) {
            return 0;
        }

        /*
         * Get the left and right height of the tree.
         */
        int leftHeight = getHeight(root.left);
        int rightHeight = getHeight(root.right);

        /*
         * Get the left and right diameter of the tree.
         */
        int leftDiameter = diameterOfBinaryTree(root.left);
        int rightDiameter = diameterOfBinaryTree(root.right);

        return Math.max(leftHeight + rightHeight, Math.max(leftDiameter, rightDiameter));
    }

    public int getHeight(TreeNode root) {
        if (root == null) {
            return 0;
        }

        if (root.left == null && root.right == null) {
            return 1;
        }
```

```
        return Math.max(getHeight(root.left), getHeight(root.right)) + 1;
    }
}
```

248. `559. Maximum Depth of N-ary Tree.java`

```
/*

Given a n-ary tree, find its maximum depth.

The maximum depth is the number of nodes along the longest path from the root node down
to the farthest leaf node.

For example, given a 3-ary tree:

We should return its max depth, which is 3.

Note:

The depth of the tree is at most 1000.
The total number of nodes is at most 5000.

*/

/*
// Definition for a Node.
class Node {
    public int val;
    public List<Node> children;

    public Node() {}

    public Node(int _val,List<Node> _children) {
        val = _val;
        children = _children;
    }
};
*/
class Solution {
      public int maxDepth(Node root) {
        return helper(root);
    }

    public int helper(Node root) {
        if(root == null)
            return 0;
        if(root.children == null)
            return 1;

                int max=0;
        for(Node child: root.children)
            max = Math.max(max, helper(child));

        return 1+max;
    }
}
```

249. `560. Subarray Sum Equals K.java`

```
/*

Given an array of integers and an integer k, you need to find the total number of
continuous subarrays whose sum equals to k.

Example 1:
Input:nums = [1,1,1], k = 2
Output: 2
Note:
The length of the array is in range [1, 20,000].
The range of numbers in the array is [-1000, 1000] and the range of the integer k is
[-1e7, 1e7].

*/

class Solution {
    public int subarraySum(int[] nums, int k) {
        if(nums.length == 1 && k==1)
            return k;

        int result = 0;
        for(int i=0;i<nums.length;i++)
        {
           int sum=nums[i];
            if(sum==k)
                result++; //we have reached the destination in a single element

            for(int j=i+1;j<nums.length;j++){
                sum= sum + nums[j];
                if(sum==k)
                    result++;
            }
        }

        return result;
    }
}
```

250. `561. Array Partition I.java`

```
/*

Given an array of 2n integers, your task is to group these integers into n pairs of integer, say (a1, b1), (a2, b2), ..., (an, bn) which makes sum of min(ai, bi) for all i from 1 to n as large as possible.

Example 1:
Input: [1,4,3,2]

Output: 4
Explanation: n is 2, and the maximum sum of pairs is 4 = min(1, 2) + min(3, 4).
Note:
n is a positive integer, which is in the range of [1, 10000].
All the integers in the array will be in the range of [-10000, 10000].

*/

public class Solution {
    public int arrayPairSum(int[] nums) {
        Arrays.sort(nums);
        int sum=0;
        for(int i=0;i<nums.length;i+=2){
            sum+=nums[i];
        }
        return sum;
    }
}
```

251. `563. Binary Tree Tilt.java`

```
/*

Given a binary tree, return the tilt of the whole tree.

The tilt of a tree node is defined as the absolute difference between the sum of all left subtree node values and the sum of all right subtree node values. Null node has tilt 0.

The tilt of the whole tree is defined as the sum of all nodes' tilt.

Example:
Input:
         1
       /   \
      2     3
Output: 1
Explanation:
Tilt of node 2 : 0
Tilt of node 3 : 0
Tilt of node 1 : |2-3| = 1
Tilt of binary tree : 0 + 0 + 1 = 1
Note:

The sum of node values in any subtree won't exceed the range of 32-bit integer.
All the tilt values won't exceed the range of 32-bit integer.

*/

class Solution {
    int tilt=0;
    public int findTilt(TreeNode root) {
        traverse(root);
        return tilt;
    }
    public int traverse(TreeNode root)
    {
        if(root==null )
            return 0;
        int left=traverse(root.left);
        int right=traverse(root.right);
        tilt+=Math.abs(left-right);
        return left+right+root.val;
    }
}
```

252. `566. Reshape the Matrix.java`

```
/*

In MATLAB, there is a very useful function called 'reshape', which can reshape a matrix
into a new one with different size but keep its original data.

You're given a matrix represented by a two-dimensional array, and two positive integers r
and c representing the row number and column number of the wanted reshaped matrix,
respectively.

The reshaped matrix need to be filled with all the elements of the original matrix in the
same row-traversing order as they were.

If the 'reshape' operation with given parameters is possible and legal, output the new
reshaped matrix; Otherwise, output the original matrix.

Example 1:
Input:
nums =
[[1,2],
 [3,4]]
r = 1, c = 4
Output:
[[1,2,3,4]]
Explanation:
The row-traversing of nums is [1,2,3,4]. The new reshaped matrix is a 1 * 4 matrix, fill
it row by row by using the previous list.
Example 2:
Input:
nums =
[[1,2],
 [3,4]]
r = 2, c = 4
Output:
[[1,2],
 [3,4]]
Explanation:
There is no way to reshape a 2 * 2 matrix to a 2 * 4 matrix. So output the original
matrix.
Note:
The height and width of the given matrix is in range [1, 100].
The given r and c are all positive.

*/


import java.util.*;
class Solution
{
    public int[][] matrixReshape(int[][] nums, int r, int c)
    {
        List<Integer> al = new ArrayList<Integer>();
```

```
        int rows = nums.length;
        int columns = nums[0].length;
        int[][] answer = new int[r][c];

        if((rows*columns) == (r*c))
        {
            //System.out.println("Possible");
            for(int i=0;i<rows;i++)
                for(int j=0;j<columns;j++)
                {
                    al.add(nums[i][j]);
                }
           // System.out.println(al.get(0)+""+al.get(1)+""+al.get(2)+""+al.get(3));
           int count = 0;
            for(int i=0;i<r;i++)
            {  for(int j=0;j<c;j++)
                {
                    //System.out.println(count);
                    answer[i][j] = al.get(count);
                    count++;
                }
            }

            return answer;
        }
        else
        {
            //System.out.println("Not possible");
            /*for(int i=0;i<rows;i++)
            {
                for(int j=0;j<columns;j++)
                {
                   answer[rows][columns] = nums[rows][colums];
                }
            }*/
                return nums;
        }


    }



}
```

253. `572. Subtree of Another Tree.java`

```
/*

Given two non-empty binary trees s and t, check whether tree t has exactly the same
structure and node values with a subtree of s. A subtree of s is a tree consists of a
node in s and all of this node's descendants. The tree s could also be considered as a
subtree of itself.

Example 1:
Given tree s:

     3
    / \
   4   5
  / \
 1   2
Given tree t:
   4
  / \
 1   2
Return true, because t has the same structure and node values with a subtree of s.
Example 2:
Given tree s:

     3
    / \
   4   5
  / \
 1   2
    /
   0
Given tree t:
   4
  / \
 1   2
Return false.

*/

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public boolean isSubtree(TreeNode s, TreeNode t) {
        if(s==null)
            return t == null; //If source tree is null then see target is also null
```

```
        return isSame(s, t) || isSubtree(s.left,t) || isSubtree(s.right,t);
        //Check if current subtree starting with s is equal to target tree t
        //It may or may be the equal
        //So we are also calling the left subtree as well as right subtree to see if
there is a match with the target tree!
        //We are having OR condition because we can get same subtree at current node or
at the left subtree or at the right subtree
    }

    private boolean isSame(TreeNode s, TreeNode t){
        if(s==null || t == null)
            return t == s; //This is there to avoid a scenario like in above Example 2,
wheere node 2 last a left child in source tree

        //Check if the current source tree's root value is equal to target tree's root
value
        //Simultaneously we also need to check the left node of S and T, as well as right
node of S and T, are they equal
        //If all three are equal then only we return that we have found the subtree
        return s.val == t.val && isSame(s.left, t.left) && isSame(s.right, t.right);
    }
}
```

254. `581. Shortest Unsorted Continuous Subarray.java`

```
/*
Given an integer array, you need to find one continuous subarray that if you only sort
this subarray in ascending order, then the whole array will be sorted in ascending order,
too.

You need to find the shortest such subarray and output its length.

Example 1:
Input: [2, 6, 4, 8, 10, 9, 15]
Output: 5
Explanation: You need to sort [6, 4, 8, 10, 9] in ascending order to make the whole array
sorted in ascending order.
Note:
Then length of the input array is in range [1, 10,000].
The input array may contain duplicates, so ascending order here means <=.

*/

class Solution {
    public int findUnsortedSubarray(int[] nums) {
        int length = nums.length;
        int start =-1 ;
        int end = -2;
        int min = nums[length-1];
        int max = nums[0];

        for(int i=1;i<length;i++){
            max = Math.max(max, nums[i]);
            min = Math.min(min, nums[length-i-1]);

            if(nums[i] < max){
                end = i;
            }

            if(nums[length-1-i] > min){
                start = length-1-i;
            }
        }
        return end-start+1;
    }
}
```

255. `583. Delete Operation for Two Strings.java`

```
/*

Given two words word1 and word2, find the minimum number of steps required to make word1
and word2 the same, where in each step you can delete one character in either string.

Example 1:
Input: "sea", "eat"
Output: 2
Explanation: You need one step to make "sea" to "ea" and another step to make "eat" to
"ea".
Note:
The length of given words won't exceed 500.
Characters in given words can only be lower-case letters.

*/

class Solution {
    //This solution is similar to 72. Edit Distance question
    //The only difference from there is here we are converting word1 and word2 into two
equal matching words. That is word1 is becoming from sea -> ea and word2 is becoming from
eat -> ea. So basically we can't take value from the diagonal, just take value from the
top and the left. Since we are just supporting deleting a character, and no insertion, we
are taking the values from the top and the left!

    public int minDistance(String word1, String word2) {
        int m = word2.length();
        int n = word1.length();

        int[][] dp = new int[m+1][n+1];
        for(int i=0;i<m+1;i++)
            dp[i][0] = i;
        for(int i=0;i<n+1;i++)
            dp[0][i] = i;

        char[] w1 = word1.toCharArray();
        char[] w2 = word2.toCharArray();

        for(int i=1;i<m+1;i++)
            for(int j=1;j<n+1;j++)
            {
                if(w2[i-1] == w1[j-1])
                    dp[i][j] = dp[i-1][j-1];
                else
                    dp[i][j] = 1+  Math.min(dp[i-1][j], dp[i][j-1]);
            }

        return dp[m][n];
    }
}
```

256. `599. Minimum Index Sum of Two Lists.java`

```
/*

Suppose Andy and Doris want to choose a restaurant for dinner, and they both have a list
of favorite restaurants represented by strings.

You need to help them find out their common interest with the least list index sum. If
there is a choice tie between answers, output all of them with no order requirement. You
could assume there always exists an answer.

Example 1:
Input:
["Shogun", "Tapioca Express", "Burger King", "KFC"]
["Piatti", "The Grill at Torrey Pines", "Hungry Hunter Steakhouse", "Shogun"]
Output: ["Shogun"]
Explanation: The only restaurant they both like is "Shogun".
Example 2:
Input:
["Shogun", "Tapioca Express", "Burger King", "KFC"]
["KFC", "Shogun", "Burger King"]
Output: ["Shogun"]
Explanation: The restaurant they both like and have the least index sum is "Shogun" with
index sum 1 (0+1).
Note:
The length of both lists will be in the range of [1, 1000].
The length of strings in both lists will be in the range of [1, 30].
The index is starting from 0 to the list length minus 1.
No duplicates in both lists.

*/

class Solution {
    public String[] findRestaurant(String[] list1, String[] list2) {
        Map<String, Integer> map = new HashMap<>();
    List<String> res = new LinkedList<>();
    int minSum = Integer.MAX_VALUE;
    for (int i=0;i<list1.length;i++) map.put(list1[i], i);
    for (int i=0;i<list2.length;i++) {
        Integer j = map.get(list2[i]);
        if (j != null && i + j <= minSum) {
            if (i + j < minSum) { res.clear(); minSum = i+j; }//Can also use:
res.removeAll(res);

            res.add(list2[i]);
        }
    }
    return res.toArray(new String[res.size()]);
    }
}
```

257. `605. Can Place Flowers.java`

```
/*

Suppose you have a long flowerbed in which some of the plots are planted and some are not. However, flowers cannot be planted in adjacent plots - they would compete for water and both would die.

Given a flowerbed (represented as an array containing 0 and 1, where 0 means empty and 1 means not empty), and a number n, return if n new flowers can be planted in it without violating the no-adjacent-flowers rule.

Example 1:
Input: flowerbed = [1,0,0,0,1], n = 1
Output: True
Example 2:
Input: flowerbed = [1,0,0,0,1], n = 2
Output: False
Note:
The input array won't violate no-adjacent-flowers rule.
The input array size is in the range of [1, 20000].
n is a non-negative integer which won't exceed the input array size.

*/

public class Solution {
    public boolean canPlaceFlowers(int[] flowerbed, int n) {
        int count = 0;
        for(int i = 0; i < flowerbed.length && count < n; i++) {
            if(flowerbed[i] == 0) {
             //get next and prev flower bed slot values. If i lies at the ends the next and prev are considered as 0.
                int next = (i == flowerbed.length - 1) ? 0 : flowerbed[i + 1];
                int prev = (i == 0) ? 0 : flowerbed[i - 1];
                if(next == 0 && prev == 0) {
                    flowerbed[i] = 1;
                    count++;
                }
            }
        }

        return count == n;
    }
}
```

258. 617. Merge Two Binary Trees.java

```
/*

Given two binary trees and imagine that when you put one of them to cover the other, some
nodes of the two trees are overlapped while the others are not.

You need to merge them into a new binary tree. The merge rule is that if two nodes
overlap, then sum node values up as the new value of the merged node. Otherwise, the NOT
null node will be used as the node of new tree.

Example 1:

Input:
        Tree 1                     Tree 2
          1                         2
         / \                       / \
        3   2                     1   3
       /                           \   \
      5                             4   7
Output:
Merged tree:
             3
            / \
           4   5
          / \   \
         5   4   7


Note: The merging process must start from the root nodes of both trees.


*/

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */

    public class Solution {
    public TreeNode mergeTrees(TreeNode t1, TreeNode t2) {
        if (t1 == null) return t2;
        if (t2 == null) return t1;
        TreeNode result = new TreeNode(t1.val + t2.val);
        result.left = mergeTrees(t1.left, t2.left);
        result.right = mergeTrees(t1.right, t2.right);
        return result;
```

```
    }
}
```

259. `621. Task Scheduler.java`

```
/*

Given a char array representing tasks CPU need to do. It contains capital letters A to Z where different letters represent different tasks. Tasks could be done without original order. Each task could be done in one interval. For each interval, CPU could finish one task or just be idle.

However, there is a non-negative cooling interval n that means between two same tasks, there must be at least n intervals that CPU are doing different tasks or just be idle.

You need to return the least number of intervals the CPU will take to finish all the given tasks.



Example:

Input: tasks = ["A","A","A","B","B","B"], n = 2
Output: 8
Explanation: A -> B -> idle -> A -> B -> idle -> A -> B.


Note:

The number of tasks is in the range [1, 10000].
The integer n is in the range [0, 100].

*/

class Solution {
    public int leastInterval(char[] tasks, int n) {
        int[] count = new int[26];
        int max = 0, taskCountOfMax = 0;
        for (char task : tasks) {
            int c = ++count[task - 'A'];
            if (c == max) {
                taskCountOfMax++;
            } else if (c > max) {
                max = c;
                taskCountOfMax = 1;
            }
        }
        return Math.max(tasks.length, (max - 1) * (n + 1) + taskCountOfMax);
    }
}

/*

Imagine task A appeared 4 times, others less than 4. And n=2. You only need to arrange A in the way that doesn't violate the rule first, then insert other tasks in any order:
A - - A - - A - - A
```

```
It's obvious that we need 6 other tasks to fill it. If other tasks are less 6, we need (4 - 1) * (n + 1) + 1 = 10 tasks in total, if other tasks are equal to or more than 6, tasks.length will be our result.
Now if we have more than one tasks have the same max occurrence, the scheduling will look like this:
A B - A B - A B - A B
So we only need to modify the formula by replacing 1 with the different amount of tasks that has the max occurrence: (4 - 1) * (n + 1) + taskCountOfMax = 11
*/
```

260. `628. Maximum Product of Three Numbers.java`

```
/*

Given an integer array, find three numbers whose product is maximum and output the
maximum product.

Example 1:

Input: [1,2,3]
Output: 6


Example 2:

Input: [1,2,3,4]
Output: 24


Note:

The length of the given array will be in range [3,104] and all elements are in the range
[-1000, 1000].
Multiplication of any three numbers in the input won't exceed the range of 32-bit signed
integer.

*/

class Solution {
    public int maximumProduct(int[] nums) {

       int min1 = Integer.MAX_VALUE, min2 = Integer.MAX_VALUE;
        int max1 = Integer.MIN_VALUE, max2 = Integer.MIN_VALUE, max3 = Integer.MIN_VALUE;

        for (int n: nums) {
            if (n >= max1) {            // n is greater than max1, max2 and max3
                max3 = max2;
                max2 = max1;
                max1 = n;
            } else if (n >= max2) {     // n lies betweeen max1 and max2
                max3 = max2;
                max2 = n;
            } else if (n >= max3) {     // n lies betwen max2 and max3
                max3 = n;
            }

            if (n <= min1) {
                min2 = min1;
                min1 = n;
            } else if (n <= min2) {     // n lies between min1 and min2
                min2 = n;
            }
```

```
        }
        return Math.max(min1 * min2 * max1, max1 * max2 * max3);
    }
}
```

261. `637. Average of Levels in Binary Tree.java`

```
/*

Given a non-empty binary tree, return the average value of the nodes on each level in the
form of an array.
Example 1:
Input:
    3
   / \
  9  20
    /  \
   15   7
Output: [3, 14.5, 11]
Explanation:
The average value of nodes on level 0 is 3,  on level 1 is 14.5, and on level 2 is 11.
Hence return [3, 14.5, 11].
Note:
The range of node's value is in the range of 32-bit signed integer

*/

class Solution {
    public List<Double> averageOfLevels(TreeNode root) {

        List<Double> result = new ArrayList<>();
        if(root == null)
            return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while(!queue.isEmpty())
        {
            int size = queue.size();
            double sum = 0;

            for(int i = 0;i<size;i++)
            {
                TreeNode node = queue.poll();
                sum = sum + node.val;

                if(node.left!=null)
                    queue.offer(node.left);
                if(node.right!=null)
                    queue.offer(node.right);
            }

            result.add((sum/size));
        }

        return result;
```

```
    }
}
```

262. `647. Palindromic Substrings.java`

```
/*

Given a string, your task is to count how many palindromic substrings in this string.

The substrings with different start indexes or end indexes are counted as different
substrings even they consist of same characters.

Example 1:

Input: "abc"
Output: 3
Explanation: Three palindromic strings: "a", "b", "c".


Example 2:

Input: "aaa"
Output: 6
Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".


Note:

The input string length won't exceed 1000.

*/

public class Solution {
    int count = 0;

    public int countSubstrings(String s) {
        if (s == null || s.length() == 0) return 0;

        for (int i = 0; i < s.length(); i++) { // i is the mid point
            extendPalindrome(s, i, i); // odd length;
            extendPalindrome(s, i, i + 1); // even length
        }

        return count;
    }

    private void extendPalindrome(String s, int left, int right) {
        while (left >=0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            count++; left--; right++;
        }
    }
}
```

263. `648. Replace Words.java`

```
/*
In English, we have a concept called root, which can be followed by some other words to
form another longer word - let's call this word successor. For example, the root an,
followed by other, which can form another word another.

Now, given a dictionary consisting of many roots and a sentence. You need to replace all
the successor in the sentence with the root forming it. If a successor has many roots can
form it, replace it with the root with the shortest length.

You need to output the sentence after the replacement.

Example 1:

Input: dict = ["cat", "bat", "rat"]
sentence = "the cattle was rattled by the battery"
Output: "the cat was rat by the bat"


Note:

The input will only have lower-case letters.
1 <= dict words number <= 1000
1 <= sentence words number <= 1000
1 <= root length <= 100
1 <= sentence words length <= 1000

*/

public class Solution {
     public String replaceWords(List<String> dict, String sentence) {
        dict.sort(Comparator.comparingInt(String::length));
        String[] words = sentence.split(" ");
        StringBuilder sb = new StringBuilder();
        for (String word : words) {
            for (String d : dict) {
                if (word.startsWith(d)) {
                    word = d;
                    break;
                }
            }
            sb.append(word + " ");
        }
        return sb.toString().trim();
    }
}
```

264. `654. Maximum Binary Tree.java`

```
/*

Given an integer array with no duplicates. A maximum tree building on this array is
defined as follow:

The root is the maximum number in the array.
The left subtree is the maximum tree constructed from left part subarray divided by the
maximum number.
The right subtree is the maximum tree constructed from right part subarray divided by the
maximum number.
Construct the maximum tree by the given array and output the root node of this tree.

Example 1:
Input: [3,2,1,6,0,5]
Output: return the tree root node representing the following tree:

      6
    /   \
   3     5
    \    /
     2  0
       \
        1
Note:
The size of the given array will be in the range [1,1000].

*/

/**
 * Definition for a binary tree node.
 * public class TreeNode {
 *     int val;
 *     TreeNode left;
 *     TreeNode right;
 *     TreeNode(int x) { val = x; }
 * }
 */
class Solution {
    public TreeNode constructMaximumBinaryTree(int[] nums) {
        if (nums == null || nums.length == 0) {
            return null;
        }

        return helper(nums, 0, nums.length);
    }

    TreeNode helper(int[] nums, int left, int right)
    {
        //Incase we have only single element in the subarray
        if (left == right)
            return null;
```

```
        int maxIndex = left;
        //Get the maxIndex from our subarray of [left, right] index limit
        for (int i = left; i < right; i++) {
            maxIndex = (nums[i] > nums[maxIndex]) ? i : maxIndex;
        }

        //Create a root node using the maximum node from the given subarray of [left,
right] limit
        TreeNode root = new TreeNode(nums[maxIndex]);

        root.left = helper(nums, left, maxIndex);
        root.right = helper(nums, maxIndex + 1, right);

        return root;
    }
}
```

265. `657. Robot Return to Origin.java`

```
/*

There is a robot starting at position (0, 0), the origin, on a 2D plane. Given a sequence
of its moves, judge if this robot ends up at (0, 0) after it completes its moves.

The move sequence is represented by a string, and the character moves[i] represents its
ith move. Valid moves are R (right), L (left), U (up), and D (down). If the robot returns
to the origin after it finishes all of its moves, return true. Otherwise, return false.

Note: The way that the robot is "facing" is irrelevant. "R" will always make the robot
move to the right once, "L" will always make it move left, etc. Also, assume that the
magnitude of the robot's movement is the same for each move.

Example 1:

Input: "UD"
Output: true
Explanation: The robot moves up once, and then down once. All moves have the same
magnitude, so it ended up at the origin where it started. Therefore, we return true.


Example 2:

Input: "LL"
Output: false
Explanation: The robot moves left twice. It ends up two "moves" to the left of the
origin. We return false because it is not at the origin at the end of its moves.

*/

class Solution {
    public boolean judgeCircle(String moves) {
        int x = 0, y = 0;
        for (char move: moves.toCharArray()) {
            if (move == 'U') y--;
            else if (move == 'D') y++;
            else if (move == 'L') x--;
            else if (move == 'R') x++;
        }
        return x == 0 && y == 0;
    }
}
```

266. `658. Find K Closest Elements.java`

```
/*

Given a sorted array, two integers k and x, find the k closest elements to x in the
array. The result should also be sorted in ascending order. If there is a tie, the
smaller elements are always preferred.

Example 1:
Input: [1,2,3,4,5], k=4, x=3
Output: [1,2,3,4]
Example 2:
Input: [1,2,3,4,5], k=4, x=-1
Output: [1,2,3,4]
Note:
The value k is positive and will always be smaller than the length of the sorted array.
Length of the given array is positive and will not exceed 104
Absolute value of elements in the array and x will not exceed 104
UPDATE (2017/9/19):
The arr parameter had been changed to an array of integers (instead of a list of
integers). Please reload the code definition to get the latest changes.


*/

class Solution {
     public List<Integer> findClosestElements(int[] arr, int k, int x) {
        int start = 0, end = arr.length-k;

        while(start<end) {
            int mid = (start + end)/2;
            if (x - arr[mid] > arr[mid+k]-x)
                start = mid + 1;
            else
                end = mid;
        }

        List<Integer> results = new ArrayList<Integer>();
        for(int i=start;i<start+k;i++){
            results.add(arr[i]);
        }
        return results;

    }
}
```

267. `671. Second Minimum Node In a Binary Tree.java`

```
/*

Given a non-empty special binary tree consisting of nodes with the non-negative value, where each node in this tree has exactly two or zero sub-node. If the node has two sub-nodes, then this node's value is the smaller value among its two sub-nodes.

Given such a binary tree, you need to output the second minimum value in the set made of all the nodes' value in the whole tree.

If no such second minimum value exists, output -1 instead.

Example 1:

Input:
    2
   / \
  2   5
     / \
    5   7

Output: 5
Explanation: The smallest value is 2, the second smallest value is 5.


Example 2:

Input:
    2
   / \
  2   2

Output: -1
Explanation: The smallest value is 2, but there isn't any second smallest value.

*/

class Solution {
    int min1;
    long ans = Long.MAX_VALUE;

    public void dfs(TreeNode root) {
        if (root != null) {
            if (min1 < root.val && root.val < ans) {
                ans = root.val;
            } else if (min1 == root.val) {
                dfs(root.left);
                dfs(root.right);
            }
        }
    }
    public int findSecondMinimumValue(TreeNode root) {
```

```
        min1 = root.val;
        dfs(root);
        return ans < Long.MAX_VALUE ? (int) ans : -1;
    }
}
```

268. `675. Cut Off Trees for Golf Event.java`

```
/*

You are asked to cut off trees in a forest for a golf event. The forest is represented as
a non-negative 2D map, in this map:

0 represents the obstacle can't be reached.
1 represents the ground can be walked through.
The place with number bigger than 1 represents a tree can be walked through, and this
positive number represents the tree's height.


You are asked to cut off all the trees in this forest in the order of tree's height -
always cut off the tree with lowest height first. And after cutting, the original place
has the tree will become a grass (value 1).

You will start from the point (0, 0) and you should output the minimum steps you need to
walk to cut off all the trees. If you can't cut off all the trees, output -1 in that
situation.

You are guaranteed that no two trees have the same height and there is at least one tree
needs to be cut off.

Example 1:

Input:
[
 [1,2,3],
 [0,0,4],
 [7,6,5]
]
Output: 6


Example 2:

Input:
[
 [1,2,3],
 [0,0,0],
 [7,6,5]
]
Output: -1


Example 3:

Input:
[
 [2,3,4],
 [0,0,5],
 [8,7,6]
```

```
]
Output: 6
Explanation: You started from the point (0,0) and you can cut off the tree in (0,0)
directly without walking.

*/

class Solution {
    int[][] dirs = new int[][]{{-1,0},{1,0},{0,-1},{0,1}};
    public int cutOffTree(List<List<Integer>> forest) {

        //create a list of int[] {x, y, Height}
        List<int[]> trees = new ArrayList<>();
        for (int i = 0; i < forest.size(); i++) {
            for (int j = 0; j < forest.get(0).size(); j++) {
                int height = forest.get(i).get(j);
                if (height > 1)
                    trees.add(new int[]{i, j, height});
            }
        }

        //Sort all the trees based on their height
        Collections.sort(trees, (a, b)->(a[2]-b[2]));

        //Run thru each test tree given to us
        int res = 0, x = 0, y = 0;
        for (int[] tree: trees) {
        //Do a BFS traversal from current location(x,y) to that tree location(tree[0],
tree[1])
            int dist = bfs(forest, x, y, tree[0], tree[1]);
            if (dist < 0)
                return -1;
            else
            {
                res = res + dist;
                x = tree[0];
                y = tree[1];
            }
        }
        return res;
    }

    private int bfs(List<List<Integer>> forest, int x, int y, int tx, int ty)
    {
        //A typical BFS approach that we use
        int m = forest.size(), n = forest.get(0).size();
        Queue<int[]> queue = new LinkedList<>();
        boolean[][] visited = new boolean[m][n];

        //Add the current source coordinates to the queue and mark it as visited
        queue.add(new int[]{x, y});
        visited[x][y] = true;

        int dist = 0;
```

```
        while (!queue.isEmpty()) {
            int size = queue.size();

            for (int j = 0; j < size; j++) {
                int[] cur = queue.poll();

//If we have reached the tree at (tx,ty) through traversal then that means we can return
the distance covered from (x,y) to (tx,ty)
                if (cur[0] == tx && cur[1] == ty)
                    return dist;

                //Traverse in all 4 directions and then process it if it meets the
conditions
                for (int i = 0; i < 4; i++)
                {
                    int nx = cur[0]+dirs[i][0];
                    int ny = cur[1]+dirs[i][1];

                    if (nx >= 0 && nx < m && ny >= 0 && ny < n &&
                        !visited[nx][ny] && forest.get(nx).get(ny) >= 1)
                    {
                      visited[nx][ny] = true;
                      queue.add(new int[]{nx, ny});
                   }
                }
            }
            dist++;
        }
        return -1;
    }
}
```

269. `690. Employee Importance.java`

```
/*

You are given a data structure of employee information, which includes the employee's unique id, his importance value and his direct subordinates' id.

For example, employee 1 is the leader of employee 2, and employee 2 is the leader of employee 3. They have importance value 15, 10 and 5, respectively. Then employee 1 has a data structure like [1, 15, [2]], and employee 2 has [2, 10, [3]], and employee 3 has [3, 5, []]. Note that although employee 3 is also a subordinate of employee 1, the relationship is not direct.

Now given the employee information of a company, and an employee id, you need to return the total importance value of this employee and all his subordinates.

Example 1:

Input: [[1, 5, [2, 3]], [2, 3, []], [3, 3, []]], 1
Output: 11
Explanation:
Employee 1 has importance value 5, and he has two direct subordinates: employee 2 and employee 3. They both have importance value 3. So the total importance value of employee 1 is 5 + 3 + 3 = 11.


Note:

One employee has at most one direct leader and may have several subordinates.
The maximum number of employees won't exceed 2000.

*/

/*
// Employee info
class Employee {
    // It's the unique id of each node;
    // unique id of this employee
    public int id;
    // the importance value of this employee
    public int importance;
    // the id of direct subordinates
    public List<Integer> subordinates;
};
*/
class Solution {
    public int getImportance(List<Employee> employees, int id) {
        int sum=0;
        Map<Integer, Employee> map = new HashMap<>();
        for(Employee e: employees){
            map.put(e.id, e);
        }
        Queue<Employee> queue = new LinkedList();
```

```
        queue.offer(map.get(id));

        while(!queue.isEmpty()){
            Employee current = queue.poll();
            sum += current.importance;

            for(Integer subId: current.subordinates){
                queue.offer(map.get(subId));
            }
        }
        return sum;
    }
}
```

270. `692. Top K Frequent Words.java`

```
/*

Given a non-empty list of words, return the k most frequent elements.

Your answer should be sorted by frequency from highest to lowest. If two words have the
same frequency, then the word with the lower alphabetical order comes first.

Example 1:
Input: ["i", "love", "leetcode", "i", "love", "coding"], k = 2
Output: ["i", "love"]
Explanation: "i" and "love" are the two most frequent words.
    Note that "i" comes before "love" due to a lower alphabetical order.
Example 2:
Input: ["the", "day", "is", "sunny", "the", "the", "the", "sunny", "is", "is"], k = 4
Output: ["the", "is", "sunny", "day"]
Explanation: "the", "is", "sunny" and "day" are the four most frequent words,
    with the number of occurrence being 4, 3, 2 and 1 respectively.
Note:
You may assume k is always valid, 1 ≤ k ≤ number of unique elements.
Input words contain only lowercase letters.
Follow up:
Try to solve it in O(n log k) time and O(n) extra space.

*/

class Solution {
    public List<String> topKFrequent(String[] words, int k) {
        HashMap<String,Integer> map = new HashMap<>();
        for(String word: words)
        {
            map.put(word, map.getOrDefault(word,0)+1);
        }

         // if same frequency, then sort alphabetical .
        PriorityQueue<Map.Entry<String,Integer>> maxHeap = new PriorityQueue<>((a,b) ->
            a.getValue()==b.getValue() ? a.getKey().compareTo(b.getKey()) : b.getValue()-
a.getValue());

        for(Map.Entry<String, Integer> entry: map.entrySet())
            maxHeap.add(entry);

        ArrayList<String> list = new ArrayList<>();

        while(list.size() < k)
        {
            Map.Entry<String, Integer> entry = maxHeap.poll();
            list.add(entry.getKey());
        }
        return list;
    }
}
```

271. `695. Max Area of Island.java`

```
/*

Given a non-empty 2D array grid of 0's and 1's, an island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

Find the maximum area of an island in the given 2D array. (If there is no island, the maximum area is 0.)

Example 1:

[[0,0,1,0,0,0,0,1,0,0,0,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
 [0,1,1,0,1,0,0,0,0,0,0,0,0],
 [0,1,0,0,1,1,0,0,1,0,1,0,0],
 [0,1,0,0,1,1,0,0,1,1,1,0,0],
 [0,0,0,0,0,0,0,0,0,0,1,0,0],
 [0,0,0,0,0,0,0,1,1,1,0,0,0],
 [0,0,0,0,0,0,0,1,1,0,0,0,0]]
Given the above grid, return 6. Note the answer is not 11, because the island must be connected 4-directionally.
Example 2:

[[0,0,0,0,0,0,0,0]]
Given the above grid, return 0.
Note: The length of each dimension in the given grid does not exceed 50.


*/

class Solution {
   public int maxAreaOfIsland(int[][] grid) {
        int max_area = 0;
        for(int i = 0; i < grid.length; i++)
            for(int j = 0; j < grid[0].length; j++)
                if(grid[i][j] == 1)
                    max_area = Math.max(max_area, AreaOfIsland(grid, i, j));
        return max_area;
    }

    public int AreaOfIsland(int[][] grid, int i, int j){
//         if( i < 0 || i > grid.length || j < 0 || j > grid[0].length || grid[i][j] != 1)
//             return

         if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] == 0) {
            return 0;
        }
    grid[i][j] = 0;
    return 1 + AreaOfIsland(grid, i+1, j) + AreaOfIsland(grid, i-1, j) +
```

```
AreaOfIsland(grid, i, j-1) + AreaOfIsland(grid, i, j+1);
        //       return 0;
    }
}
```

272. `698. Partition to K Equal Sum Subsets.java`

```
/*

Given an array of integers nums and a positive integer k, find whether it's possible to
divide this array into k non-empty subsets whose sums are all equal.



Example 1:

Input: nums = [4, 3, 2, 3, 5, 2, 1], k = 4
Output: True
Explanation: It's possible to divide it into 4 subsets (5), (1, 4), (2,3), (2,3) with
equal sums.


Note:

1 <= k <= len(nums) <= 16.
0 < nums[i] < 10000.

*/

class Solution {
    public boolean canPartitionKSubsets(int[] A, int k) {
        if (k > A.length)
            return false;

        int sum = 0;
        for (int num : A)
            sum += num;

        if (sum % k != 0)
            return false;

        boolean[] visited = new boolean[A.length];
        //Arrays.sort(A);
        return dfs(A, 0, A.length - 1, visited, sum / k, k);
    }

    public boolean dfs(int[] A, int sum, int lastIndex, boolean[] visited, int target,
int round) {
        if (round == 0)
            return true;
        if (sum == target && dfs(A, 0, A.length - 1, visited, target, round - 1))
            return true;

        for (int i = lastIndex; i >= 0; --i) {
            if (!visited[i] && sum + A[i] <= target) {
                visited[i] = true;
                if (dfs(A, sum + A[i], i - 1, visited, target, round))
                    return true;
```

```
                visited[i] = false;
            }
        }
        return false;
    }

    /*
    public boolean canPartitionKSubsets(int[] nums, int k) {
        int sum = 0;
        //First get the sum of all nums. If sum is not divisible by k then return false.
        for(int num:nums)
            sum += num;
        if(k <= 0 || sum%k != 0)
            return false;
        int[] visited = new int[nums.length];
        return canPartition(nums, visited, 0, k, 0, 0, sum/k);
    }

    public boolean canPartition(int[] nums, int[] visited, int start_index, int k, int
cur_sum, int cur_num, int target)
    {
        if(k==1)
            return true;

        //If one of the subsets equals the target sum of subset, then we return true
        if(cur_sum == target && cur_num>0)
            return canPartition(nums, visited, 0, k-1, 0, 0, target);

        for(int i = start_index; i<nums.length; i++){
            if(visited[i] == 0){
                visited[i] = 1;
                if(canPartition(nums, visited, i+1, k, cur_sum + nums[i], cur_num++,
target))
                    return true;
                visited[i] = 0;
            }
        }
        return false;
    }
*/
}
```

273. `703. Kth Largest Element in a Stream.java`

```
/*

Design a class to find the kth largest element in a stream. Note that it is the kth
largest element in the sorted order, not the kth distinct element.

Your KthLargest class will have a constructor which accepts an integer k and an integer
array nums, which contains initial elements from the stream. For each call to the method
KthLargest.add, return the element representing the kth largest element in the stream.

Example:

int k = 3;
int[] arr = [4,5,8,2];
KthLargest kthLargest = new KthLargest(3, arr);
kthLargest.add(3);   // returns 4
kthLargest.add(5);   // returns 5
kthLargest.add(10);  // returns 5
kthLargest.add(9);   // returns 8
kthLargest.add(4);   // returns 8

*/

class KthLargest {

    Queue<Integer> pq;
    int capacity;
    public KthLargest(int k, int[] nums) {
        capacity = k;
        pq = new PriorityQueue<Integer>();
        for(int i : nums) {
            add(i);
        }
    }

    public int add(int val) {
        // cannot directly do following
        // pq.offer(val)
        // return pq.peek()
        if(pq.size() < capacity)
            pq.offer(val);
        else {
            pq.offer(val);
            pq.poll();
        }
        return pq.peek();
    }
}

/**
 * Your KthLargest object will be instantiated and called as such:
 * KthLargest obj = new KthLargest(k, nums);
```

```
 * int param_1 = obj.add(val);
 */
```

274. `704. Binary Search.java`

```
/*

Given a sorted (in ascending order) integer array nums of n elements and a target value,
write a function to search target in nums. If target exists, then return its index,
otherwise return -1.


Example 1:

Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4

Example 2:

Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1


Note:

You may assume that all elements in nums are unique.
n will be in the range [1, 10000].
The value of each element in nums will be in the range [-9999, 9999].

*/

class Solution {
    public int search(int[] nums, int target) {
         int left = 0, right = nums.length - 1;
       while(left <= right)
       {
           int mid = left + (right-left)/2;
           if(nums[mid] == target)
               return mid;
           else
               if(nums[mid] > target )
                   right = mid - 1;
           else
                   left = mid + 1;
       }

       return -1;
    }
}
```

275. `705. Design HashSet.java`

```
/*

Design a HashSet without using any built-in hash table libraries.

To be specific, your design should include these functions:

add(value): Insert a value into the HashSet.
contains(value) : Return whether the value exists in the HashSet or not.
remove(value): Remove a value in the HashSet. If the value does not exist in the HashSet,
do nothing.

Example:

MyHashSet hashSet = new MyHashSet();
hashSet.add(1);
hashSet.add(2);
hashSet.contains(1);    // returns true
hashSet.contains(3);    // returns false (not found)
hashSet.add(2);
hashSet.contains(2);    // returns true
hashSet.remove(2);
hashSet.contains(2);    // returns false (already removed)

Note:

All values will be in the range of [0, 1000000].
The number of operations will be in the range of [1, 10000].
Please do not use the built-in HashSet library.

*/

class MyHashSet {

    int[] arr;

    /** Initialize your data structure here. */
    public MyHashSet() {
        arr = new int[1000000];
    }

    public void add(int key) {
        arr[hash(key)] = key+1;  // handle the case key=0
    }

    public void remove(int key) {
        arr[hash(key)] = 0;
    }

    /** Returns true if this set did not already contain the specified element */
    public boolean contains(int key) {
        return (arr[hash(key)]!=0);
```

```
    }

    private int hash(int i){
        return i;
    }
}

/**
 * Your MyHashSet object will be instantiated and called as such:
 * MyHashSet obj = new MyHashSet();
 * obj.add(key);
 * obj.remove(key);
 * boolean param_3 = obj.contains(key);
 */
```

276. `707. Design Linked List.java`

```
/*

Design your implementation of the linked list. You can choose to use the singly linked
list or the doubly linked list. A node in a singly linked list should have two
attributes: val and next. val is the value of the current node, and next is a
pointer/reference to the next node. If you want to use the doubly linked list, you will
need one more attribute prev to indicate the previous node in the linked list. Assume all
nodes in the linked list are 0-indexed.

Implement these functions in your linked list class:

get(index) : Get the value of the index-th node in the linked list. If the index is
invalid, return -1.
addAtHead(val) : Add a node of value val before the first element of the linked list.
After the insertion, the new node will be the first node of the linked list.
addAtTail(val) : Append a node of value val to the last element of the linked list.
addAtIndex(index, val) : Add a node of value val before the index-th node in the linked
list. If index equals to the length of linked list, the node will be appended to the end
of linked list. If index is greater than the length, the node will not be inserted.
deleteAtIndex(index) : Delete the index-th node in the linked list, if the index is
valid.
Example:

MyLinkedList linkedList = new MyLinkedList();
linkedList.addAtHead(1);
linkedList.addAtTail(3);
linkedList.addAtIndex(1, 2);  // linked list becomes 1->2->3
linkedList.get(1);            // returns 2
linkedList.deleteAtIndex(1);  // now the linked list is 1->3
linkedList.get(1);            // returns 3
Note:

All values will be in the range of [1, 1000].
The number of operations will be in the range of [1, 1000].
Please do not use the built-in LinkedList library.

*/

class MyLinkedList {
    class Node {
        int val;
        Node next;
        public Node(int val) {
            this.val = val;
        }
    }
    private Node head;
    private int size;

    /** Initialize your data structure here. */
```

```
    public MyLinkedList() {

    }

    /** Get the value of the index-th node in the linked list. If the index is invalid,
return -1. */
    public int get(int index) {
        if (index >= size) return -1;
        Node current = head;
        for (int i = 0; i < index; i++) {
            current = current.next;
        }
        return current.val;
    }

    /** Add a node of value val before the first element of the linked list. After the
insertion, the new node will be the first node of the linked list. */
    public void addAtHead(int val) {
        Node prev = head;
        head = new Node(val);
        head.next = prev;
        size++;
    }

    /** Append a node of value val to the last element of the linked list. */
    public void addAtTail(int val) {
        Node node = new Node(val);
        size++;
        if (head == null) {
            head = node;
        } else {
            Node current = head;
            while (current.next != null) {
                current = current.next;
            }
            current.next = node;
        }
    }

    /** Add a node of value val before the index-th node in the linked list. If index
equals to the length of linked list, the node will be appended to the end of linked list.
If index is greater than the length, the node will not be inserted. */
    public void addAtIndex(int index, int val) {
        if (index > size) return;
        if (index == 0) {
            addAtHead(val);
        } else {
            size++;
            Node current = head;
            for (int i = 0; i < index - 1; i++) {
                current = current.next;
            }
            Node node = new Node(val);
            node.next = current.next;
```

```
            current.next = node;
        }
    }

    /** Delete the index-th node in the linked list, if the index is valid. */
    public void deleteAtIndex(int index) {
        if (index >= size) return;
        size--;
        Node current = head;
        for (int i = 0; i < index - 1; i++) {
            current = current.next;
        }
        current.next = current.next.next;
    }
}
```

277. `709. To Lower Case.java`

```
/*

Implement function ToLowerCase() that has a string parameter str, and returns the same
string in lowercase.



Example 1:

Input: "Hello"
Output: "hello"
Example 2:

Input: "here"
Output: "here"
Example 3:

Input: "LOVELY"
Output: "lovely"

*/

class Solution {
    public String toLowerCase(String str) {
        return str.toLowerCase();
    }
}
```

278. 712. Minimum ASCII Delete Sum for Two Strings.java

```
/*

Given two strings s1, s2, find the lowest ASCII sum of deleted characters to make two
strings equal.

Example 1:
Input: s1 = "sea", s2 = "eat"
Output: 231
Explanation: Deleting "s" from "sea" adds the ASCII value of "s" (115) to the sum.
Deleting "t" from "eat" adds 116 to the sum.
At the end, both strings are equal, and 115 + 116 = 231 is the minimum sum possible to
achieve this.
Example 2:
Input: s1 = "delete", s2 = "leet"
Output: 403
Explanation: Deleting "dee" from "delete" to turn the string into "let",
adds 100[d]+101[e]+101[e] to the sum.  Deleting "e" from "leet" adds 101[e] to the sum.
At the end, both strings are equal to "let", and the answer is 100+101+101+101 = 403.
If instead we turned both strings into "lee" or "eet", we would get answers of 433 or
417, which are higher.
Note:

0 < s1.length, s2.length <= 1000.
All elements of each string will have an ASCII value in [97, 122].

*/

class Solution {
    public int minimumDeleteSum(String s1, String s2) {
       int m = s1.length();
        int n = s2.length();

        int[][] dp = new int[m+1][n+1];
        dp[0][0] = 0;

       for (int i = 1; i < m + 1; ++i)
                    dp[i][0] = dp[i - 1][0] + s1.charAt(i - 1);

        for (int j = 1; j < n + 1; ++j)
                    dp[0][j] = dp[0][j - 1] + s2.charAt(j - 1);

    char[] w1 = s1.toCharArray();
    char[] w2 = s2.toCharArray();

        for (int i = 1; i < m + 1; ++i) {
                char c1 = s1.charAt(i - 1);
                for (int j = 1; j < n + 1; ++j) {
                        char c2 = s2.charAt(j - 1);
                        int a = c1 == c2 ? dp[i - 1][j - 1] : Integer.MAX_VALUE;
                        int b = dp[i - 1][j] + c1;
                        int c = dp[i][j - 1] + c2;
```

```
                        dp[i][j] = Math.min(a, Math.min(b, c));
                }
        }
        return dp[m][n];
    }
}
```

279. `719. Find K-th Smallest Pair Distance.java`

```
/*

Given an integer array, return the k-th smallest distance among all the pairs. The
distance of a pair (A, B) is defined as the absolute difference between A and B.

Example 1:
Input:
nums = [1,3,1]
k = 1
Output: 0
Explanation:
Here are all the pairs:
(1,3) -> 2
(1,1) -> 0
(3,1) -> 2
Then the 1st smallest distance pair is (1,1), and its distance is 0.
Note:
2 <= len(nums) <= 10000.
0 <= nums[i] < 1000000.
1 <= k <= len(nums) * (len(nums) - 1) / 2.

*/

class Solution {
    public int smallestDistancePair(int[] nums, int k) {
        // nlog(n)
        Arrays.sort(nums);
        PriorityQueue<Node> heap = new PriorityQueue<>();
        // nlog(n)
        for(int i = 0; i < nums.length - 1; i++) {
            heap.offer(new Node(i, i+1, nums[i+1] - nums[i]));
        }
        int cnt = 0;
        Node node = null;
        // klog(n)
        while(cnt < k && !heap.isEmpty()) {
            node = heap.poll();
            if (node.end < nums.length - 1) {
                heap.offer(new Node(node.start, node.end + 1, nums[node.end + 1] -
nums[node.start]));
            }
            cnt++;
        }
        if (node != null) {
            return node.dist;
        }
        return -1;
    }

    private static class Node implements Comparable<Node> {
        int start;
```

```
        int end;
        int dist;
        public Node(int start, int end, int dist) {
            this.start = start;
            this.end = end;
            this.dist = dist;
        }
        public int compareTo(Node n) {
            return this.dist - n.dist;
        }
    }

}
```

280. `725. Split Linked List in Parts.java`

```
/*

Given a (singly) linked list with head node root, write a function to split the linked
list into k consecutive linked list "parts".

The length of each part should be as equal as possible: no two parts should have a size
differing by more than 1. This may lead to some parts being null.

The parts should be in order of occurrence in the input list, and parts occurring earlier
should always have a size greater than or equal parts occurring later.

Return a List of ListNode's representing the linked list parts that are formed.

Examples 1->2->3->4, k = 5 // 5 equal parts [ [1], [2], [3], [4], null ]
Example 1:
Input:
root = [1, 2, 3], k = 5
Output: [[1],[2],[3],[],[]]
Explanation:
The input and each element of the output are ListNodes, not arrays.
For example, the input root has root.val = 1, root.next.val = 2, \root.next.next.val = 3,
and root.next.next.next = null.
The first element output[0] has output[0].val = 1, output[0].next = null.
The last element output[4] is null, but it's string representation as a ListNode is [].
Example 2:
Input:
root = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], k = 3
Output: [[1, 2, 3, 4], [5, 6, 7], [8, 9, 10]]
Explanation:
The input has been split into consecutive parts with size difference at most 1, and
earlier parts are a larger size than the later parts.
Note:

The length of root will be in the range [0, 1000].
Each value of a node in the input will be an integer in the range [0, 999].
k will be an integer in the range [1, 50].

*/

class Solution {
     public ListNode[] splitListToParts(ListNode root, int k) {
        ListNode[] res = new ListNode[k];
        int n = 0;

        for (ListNode curr = root; curr != null; curr = curr.next) {
            n++;
        }

        int perpart = n / k;
        int remain = n % k;
```

```
        ListNode head = root;
        ListNode prev = null;

        for (int i = 0; i < k; i++) {
            res[i] = head;
            int partSize = perpart + (remain > 0 ? 1 : 0);

            for (int j = 0; j < partSize; j++) {
                prev = head;
                head = head.next;
            }

            remain--;

            if (prev != null) {
                prev.next = null;
            }

        }
        return res;
    }
}
```

281. `729. My Calendar I.java`

```
/*

Implement a MyCalendar class to store your events. A new event can be added if adding the event will not cause a double booking.

Your class will have the method, book(int start, int end). Formally, this represents a booking on the half open interval [start, end), the range of real numbers x such that start <= x < end.

A double booking happens when two events have some non-empty intersection (ie., there is some time that is common to both events.)

For each call to the method MyCalendar.book, return true if the event can be added to the calendar successfully without causing a double booking. Otherwise, return false and do not add the event to the calendar.

Your class will be called like this: MyCalendar cal = new MyCalendar();
MyCalendar.book(start, end)
Example 1:

MyCalendar();
MyCalendar.book(10, 20); // returns true
MyCalendar.book(15, 25); // returns false
MyCalendar.book(20, 30); // returns true
Explanation:
The first event can be booked.  The second can't because time 15 is already booked by another event.
The third event can be booked, as the first event takes every time less than 20, but not including 20.


Note:

The number of calls to MyCalendar.book per test case will be at most 1000.
In calls to MyCalendar.book(start, end), start and end are integers in the range [0, 10^9].

*/

class MyCalendar {

    class Interval {
        int start;
        int end;

        public Interval(int start, int end) {
            this.start = start;
            this.end = end;
        }

        public int getStart() { return start; }
```

```
        public int getEnd() { return end; }
    }

    List<Interval> list;

    public MyCalendar() {
        list = new ArrayList<>();
    }

    public boolean book(int start, int end) {
        for (Interval interval: list) {
            if (start < interval.getEnd() && end > interval.getStart()) {
                return false;
            }
        }

        list.add(new Interval(start, end));
        return true;
    }
}

/*
class MyCalendar {

    TreeSet<int[]> books = new TreeSet<int[]>((int[] a, int[] b) -> a[0] - b[0]);// Min
sorted based on the start values

    public boolean book(int s, int e) {
        int[] book = new int[] { s, e }, floor = books.floor(book), ceiling =
books.ceiling(book);


        if (floor != null && s < floor[1])
            return false; // (s, e) start within floor
        if (ceiling != null && ceiling[0] < e)
            return false; // ceiling start within (s, e)

        books.add(book);
        return true;
    }
}
*/

/**
 * Your MyCalendar object will be instantiated and called as such:
 * MyCalendar obj = new MyCalendar();
 * boolean param_1 = obj.book(start,end);
 */
```

282. `731. My Calendar II.java`

```
/*

Implement a MyCalendarTwo class to store your events. A new event can be added if adding the event will not cause a triple booking.

Your class will have one method, book(int start, int end). Formally, this represents a booking on the half open interval [start, end), the range of real numbers x such that start <= x < end.

A triple booking happens when three events have some non-empty intersection (ie., there is some time that is common to all 3 events.)

For each call to the method MyCalendar.book, return true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Your class will be called like this: MyCalendar cal = new MyCalendar();
MyCalendar.book(start, end)
Example 1:

MyCalendar();
MyCalendar.book(10, 20); // returns true
MyCalendar.book(50, 60); // returns true
MyCalendar.book(10, 40); // returns true
MyCalendar.book(5, 15); // returns false
MyCalendar.book(5, 10); // returns true
MyCalendar.book(25, 55); // returns true
Explanation:
The first two events can be booked.  The third event can be double booked.
The fourth event (5, 15) can't be booked, because it would result in a triple booking.
The fifth event (5, 10) can be booked, as it does not use time 10 which is already double booked.
The sixth event (25, 55) can be booked, as the time in [25, 40) will be double booked with the third event;
the time [40, 50) will be single booked, and the time [50, 55) will be double booked with the second event.


Note:

The number of calls to MyCalendar.book per test case will be at most 1000.
In calls to MyCalendar.book(start, end), start and end are integers in the range [0, 10^9].

*/

class MyCalendarTwo {

    private List<int[]> books = new ArrayList<>();

    public boolean book(int s, int e) {
```

```
        MyCalendar overlaps = new MyCalendar();

        for (int[] b : books)
            if (Math.max(b[0], s) < Math.min(b[1], e)) // overlap exist
                if (!overlaps.book(Math.max(b[0], s), Math.min(b[1], e)))
                    return false; // overlaps overlapped

        books.add(new int[]{ s, e });
        return true;
    }

    private static class MyCalendar {
        List<int[]> books = new ArrayList<>();

        public boolean book(int start, int end) {
            for (int[] b : books)
                if (Math.max(b[0], start) < Math.min(b[1], end))
                    return false;

            books.add(new int[]{ start, end });
            return true;
        }
    }
}
```

283. `733. Flood Fill.java`

```
/*

An image is represented by a 2-D array of integers, each integer representing the pixel
value of the image (from 0 to 65535).

Given a coordinate (sr, sc) representing the starting pixel (row and column) of the flood
fill, and a pixel value newColor, "flood fill" the image.

To perform a "flood fill", consider the starting pixel, plus any pixels connected 4-
directionally to the starting pixel of the same color as the starting pixel, plus any
pixels connected 4-directionally to those pixels (also with the same color as the
starting pixel), and so on. Replace the color of all of the aforementioned pixels with
the newColor.

At the end, return the modified image.

Example 1:
Input:
image = [[1,1,1],[1,1,0],[1,0,1]]
sr = 1, sc = 1, newColor = 2
Output: [[2,2,2],[2,2,0],[2,0,1]]
Explanation:
From the center of the image (with position (sr, sc) = (1, 1)), all pixels connected
by a path of the same color as the starting pixel are colored with the new color.
Note the bottom corner is not colored 2, because it is not 4-directionally connected
to the starting pixel.
Note:

The length of image and image[0] will be in the range [1, 50].
The given starting pixel will satisfy 0 <= sr < image.length and 0 <= sc <
image[0].length.
The value of each color in image[i][j] and newColor will be an integer in [0, 65535].

*/

class Solution {
    public int[][] floodFill(int[][] image, int sr, int sc, int newColor) {
        if (image[sr][sc] == newColor)
            return image;
        dfs(image, sr, sc, image[sr][sc], newColor);
        return image;
    }

    private void dfs(int[][] image, int sr, int sc, int color, int newColor) {
        if (sr < 0 || sr >= image.length || sc < 0 || sc >= image[0].length || image[sr]
[sc] != color)
            return;

        image[sr][sc] = newColor;
        dfs(image, sr + 1, sc, color, newColor);
        dfs(image, sr - 1, sc, color, newColor);
```

```
        dfs(image, sr, sc + 1, color, newColor);
        dfs(image, sr, sc - 1, color, newColor);
    }
}
```

284. `739. Daily Temperatures.java`

```
/*

Given a list of daily temperatures T, return a list such that, for each day in the input, tells you how many days you would have to wait until a warmer temperature. If there is no future day for which this is possible, put 0 instead.

For example, given the list of temperatures T = [73, 74, 75, 71, 69, 72, 76, 73], your output should be [1, 1, 4, 2, 1, 1, 0, 0].

Note: The length of temperatures will be in the range [1, 30000]. Each temperature will be an integer in the range [30, 100].


*/

class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int m = temperatures.length;
        int[] ans = new int[m];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < m; i++)
        {
            while (!stack.empty() && temperatures[stack.peek()] < temperatures[i])
                ans[stack.peek()] = i - stack.pop();

            stack.push(i);
        }
        return ans;
    }
}
```

285. `743. Network Delay Time.java`

```
/*

There are N network nodes, labelled 1 to N.

Given times, a list of travel times as directed edges times[i] = (u, v, w), where u is the source node, v is the target node, and w is the time it takes for a signal to travel from source to target.

Now, we send a signal from a certain node K. How long will it take for all nodes to receive the signal? If it is impossible, return -1.

Note:

N will be in the range [1, 100].
K will be in the range [1, N].
The length of times will be in the range [1, 6000].
All edges times[i] = (u, v, w) will have 1 <= u, v <= N and 0 <= w <= 100.

*/

class Solution {
    public int networkDelayTime(int[][] times, int N, int K) {
        Map<Integer, Map<Integer,Integer>> map = new HashMap<>();
        for(int[] time : times){
            map.putIfAbsent(time[0], new HashMap<>());
            map.get(time[0]).put(time[1], time[2]);
        }

        //distance, node into pq
        Queue<int[]> pq = new PriorityQueue<>((a,b) -> (a[0] - b[0]));

        pq.add(new int[]{0, K});

        boolean[] visited = new boolean[N+1];
        int res = 0;

        while(!pq.isEmpty()){
            int[] cur = pq.remove();
            int curNode = cur[1];
            int curDist = cur[0];
            if(visited[curNode])
                continue;

            visited[curNode] = true;
            res = curDist;
            N--;

            if(map.containsKey(curNode))
            {
                for(int next : map.get(curNode).keySet())
                {
```

```
                    pq.add(new int[]{curDist + map.get(curNode).get(next), next});
                }
            }
        }
        return N == 0 ? res : -1;

    }
}
```

286. `746. Min Cost Climbing Stairs.java`

```
/*

On a staircase, the i-th step has some non-negative cost cost[i] assigned (0 indexed).

Once you pay the cost, you can either climb one or two steps. You need to find minimum
cost to reach the top of the floor, and you can either start from the step with index 0,
or the step with index 1.

Example 1:
Input: cost = [10, 15, 20]
Output: 15
Explanation: Cheapest is start on cost[1], pay that cost and go to the top.
Example 2:
Input: cost = [1, 100, 1, 1, 1, 100, 1, 1, 100, 1]
Output: 6
Explanation: Cheapest is start on cost[0], and only step on 1s, skipping cost[3].
Note:
cost will have a length in the range [2, 1000].
Every cost[i] will be an integer in the range [0, 999].

*/

class Solution {
    public int minCostClimbingStairs(int[] cost) {
        for (int i = 2; i < cost.length; i++) {
            cost[i] += Math.min(cost[i-1], cost[i-2]);
        }
        return Math.min(cost[cost.length-1], cost[cost.length-2]);
    }
}
```

287. `752. Open the Lock.java`

```
/*

You have a lock in front of you with 4 circular wheels. Each wheel has 10 slots: '0',
'1', '2', '3', '4', '5', '6', '7', '8', '9'. The wheels can rotate freely and wrap
around: for example we can turn '9' to be '0', or '0' to be '9'. Each move consists of
turning one wheel one slot.

The lock initially starts at '0000', a string representing the state of the 4 wheels.

You are given a list of deadends dead ends, meaning if the lock displays any of these
codes, the wheels of the lock will stop turning and you will be unable to open it.

Given a target representing the value of the wheels that will unlock the lock, return the
minimum total number of turns required to open the lock, or -1 if it is impossible.

Example 1:
Input: deadends = ["0201","0101","0102","1212","2002"], target = "0202"
Output: 6
Explanation:
A sequence of valid moves would be "0000" -> "1000" -> "1100" -> "1200" -> "1201" ->
"1202" -> "0202".
Note that a sequence like "0000" -> "0001" -> "0002" -> "0102" -> "0202" would be
invalid,
because the wheels of the lock become stuck after the display becomes the dead end
"0102".
Example 2:
Input: deadends = ["8888"], target = "0009"
Output: 1
Explanation:
We can turn the last wheel in reverse to move from "0000" -> "0009".
Example 3:
Input: deadends = ["8887","8889","8878","8898","8788","8988","7888","9888"], target =
"8888"
Output: -1
Explanation:
We can't reach the target without getting stuck.
Example 4:
Input: deadends = ["0000"], target = "8888"
Output: -1
Note:
The length of deadends will be in the range [1, 500].
target will not be in the list deadends.
Every string in deadends and the string target will be a string of 4 digits from the
10,000 possibilities '0000' to '9999'.

*/

class Solution {
    public static int openLock(String[] deadends, String target) {
        Queue<String> q = new LinkedList<>();
        Set<String> deads = new HashSet<>(Arrays.asList(deadends));
```

```
        Set<String> visited = new HashSet<>();

        int depth = 0;
        String marker = "*";
        q.addAll(Arrays.asList("0000", "*"));
        while(!q.isEmpty()) {
            String node = q.poll();
            if(node.equals(target))
                return depth;
            if(visited.contains(node) || deads.contains(node))
                continue;
            if(node.equals(marker) && q.isEmpty())
                return -1;
            if(node.equals(marker)) {
                q.add(marker);
                depth += 1;
            } else {
                visited.add(node);
                q.addAll(getSuccessors(node));
            }
        }
        return depth;
    }

    private static List<String> getSuccessors(String str) {
        List<String> res = new LinkedList<>();
        for (int i = 0; i < str.length(); i++) {
            res.add(str.substring(0, i) + (str.charAt(i) == '0' ? 9 :  str.charAt(i) -
'0' - 1) + str.substring(i+1));
            res.add(str.substring(0, i) + (str.charAt(i) == '9' ? 0 :  str.charAt(i) -
'0' + 1) + str.substring(i+1));
        }
        return res;
    }
}
```

288. `767. Reorganize String.java`

```
/*

Given a string S, check if the letters can be rearranged so that two characters that are
adjacent to each other are not the same.

If possible, output any possible result.  If not possible, return the empty string.

Example 1:

Input: S = "aab"
Output: "aba"
Example 2:

Input: S = "aaab"
Output: ""
Note:

S will consist of lowercase letters and have length in range [1, 500].

*/

class Solution {
    public String reorganizeString(String S) {

        Map<Character, Integer> map = new HashMap<Character, Integer>();
        for(char c: S.toCharArray())
            map.put(c, map.getOrDefault(c, 0)+1); //Character Frequency count

        PriorityQueue<Map.Entry<Character, Integer>> maxHeap = new PriorityQueue<>((a,b)
-> (b.getValue() - a.getValue()));
        maxHeap.addAll(map.entrySet());//Adding all elements to our heap

        StringBuilder result = new StringBuilder();

        //The reason we need "prev" Entry is because we don't want 2 consecutive same
characters!
        Map.Entry<Character, Integer> prev = null; //Since this is the start of the
resultant string we have to keep prev = null. We don't want to add anything before the
start of the resultant string.

        while(!maxHeap.isEmpty())
        {
            Map.Entry<Character, Integer> curr = maxHeap.poll(); //Get the element with
the highest freq from the PQ
            result.append(curr.getKey());//Append the current character

            curr.setValue(curr.getValue()-1); //Since we have used one character from the
current Entry, we update the frequency count
            if(prev != null)
                maxHeap.offer(prev); //Means that our previous entry still has characters
available to use, so add it to the Priority Queue for further processing.
```

```
            //After updating the current node's frequency count, if we are still having
some characters available, so update the prev with the current value.
            if(curr.getValue() > 0)
                prev = curr;
            else
                prev = null;
        }
        System.out.println(result);

        return result.length() == S.length() ? result.toString() : "";
    }
}
```

289. `771. Jewels and Stones.java`

```
/*

You're given strings J representing the types of stones that are jewels, and S
representing the stones you have.  Each character in S is a type of stone you have.  You
want to know how many of the stones you have are also jewels.

The letters in J are guaranteed distinct, and all characters in J and S are letters.
Letters are case sensitive, so "a" is considered a different type of stone from "A".

Example 1:

Input: J = "aA", S = "aAAbbbb"
Output: 3
Example 2:

Input: J = "z", S = "ZZ"
Output: 0
Note:

S and J will consist of letters and have length at most 50.
The characters in J are distinct.

*/

class Solution {
    public int numJewelsInStones(String J, String S) {
        HashSet<Character> set = new HashSet<>();
        char[] j = J.toCharArray();
        char[] s = S.toCharArray();
        int count =0;
        for(char c: j)
            if(!set.contains(c))
                set.add(c);

        for(char c: s)
        {
            if(set.contains(c))
                count++;

        }
        return count;

    }
}
```

290. `787. Cheapest Flights Within K Stops.java`

```
/*

There are n cities connected by m flights. Each fight starts from city u and arrives at v
with a price w.

Now given all the cities and flights, together with starting city src and the destination
dst, your task is to find the cheapest price from src to dst with up to k stops. If there
is no such route, output -1.

Example 1:
Input:
n = 3, edges = [[0,1,100],[1,2,100],[0,2,500]]
src = 0, dst = 2, k = 1
Output: 200
Explanation:
The graph looks like this:


The cheapest price from city 0 to city 2 with at most 1 stop costs 200, as marked red in
the picture.
Example 2:
Input:
n = 3, edges = [[0,1,100],[1,2,100],[0,2,500]]
src = 0, dst = 2, k = 0
Output: 500
Explanation:
The graph looks like this:


The cheapest price from city 0 to city 2 with at most 0 stop costs 500, as marked blue in
the picture.
Note:

The number of nodes n will be in range [1, 100], with nodes labeled from 0 to n - 1.
The size of flights will be in range [0, n * (n - 1) / 2].
The format of each flight will be (src, dst, price).
The price of each flight will be in the range [1, 10000].
k is in the range of [0, n - 1].
There will not be any duplicated flights or self cycles

*/

class Solution {
    public int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        //Create a map with <Source, <Destination, Price>> entry
        Map<Integer, Map<Integer, Integer>> prices = new HashMap<>();

        for (int[] f : flights)
        {
           //Add all flights from Source to various destinations inside our map with the
prices
```

```
            if (!prices.containsKey(f[0]))
                prices.put(f[0], new HashMap<>());
            //Or you can use:  prices.putIfAbsent(f[0], new HashMap<>());

            prices.get(f[0]).put(f[1], f[2]);
        }


        //Our PQ consists of {price, source, number_of_steps}, and we are having a
minHeap based on the price
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> (Integer.compare(a[0],
b[0])));
        pq.add(new int[] {0, src, k + 1});

        while (!pq.isEmpty())
        {
            int[] top = pq.poll();
            int price = top[0];
            int city = top[1];
            int stops = top[2];

            if (city == dst)
                return price;

            if (stops > 0)
            {
                Map<Integer, Integer> adj = prices.getOrDefault(city, new HashMap<>());
                for (int a : adj.keySet())
                {
                    pq.add(new int[] {price + adj.get(a), a, stops - 1});
                }
            }
        }
        return -1;
    }
}
```

291. `791. Custom Sort String.java`

```
/*

S and T are strings composed of lowercase letters. In S, no letter occurs more than once.

S was sorted in some custom order previously. We want to permute the characters of T so
that they match the order that S was sorted. More specifically, if x occurs before y in
S, then x should occur before y in the returned string.

Return any permutation of T (as a string) that satisfies this property.

Example :
Input:
S = "cba"
T = "abcd"
Output: "cbad"
Explanation:
"a", "b", "c" appear in S, so the order of "a", "b", "c" should be "c", "b", and "a".
Since "d" does not appear in S, it can be at any position in T. "dcba", "cdba", "cbda"
are also valid outputs.


Note:

S has length at most 26, and no character is repeated in S.
T has length at most 200.
S and T consist of lowercase letters only.

*/

class Solution {
     public String customSortString(String S, String T) {
        int[] bucket = new int[26];
        for (char c : T.toCharArray()) {
            bucket[c - 'a']++;
        }

        StringBuilder sb = new StringBuilder();
        for (char c : S.toCharArray()) {
            for (int i = 0; i < bucket[c - 'a']; i++) {
                sb.append(c);
            }
            bucket[c - 'a'] = 0;
        }

        for (int i = 0; i < 26; i++) {
            for (int j = 0; j < bucket[i]; j++) {
                sb.append((char) (i + 'a'));
            }
        }

        return sb.toString();
```

```
    }
}
```

292. `792. Number of Matching Subsequences.java`

```
/*

Given string S and a dictionary of words words, find the number of words[i] that is a
subsequence of S.

Example :
Input:
S = "abcde"
words = ["a", "bb", "acd", "ace"]
Output: 3
Explanation: There are three words in words that are a subsequence of S: "a", "acd",
"ace".
Note:

All words in words and S will only consists of lowercase letters.
The length of S will be in the range of [1, 50000].
The length of words will be in the range of [1, 5000].
The length of words[i] will be in the range of [1, 50].

*/

class Solution {
    public int numMatchingSubseq(String S, String[] words) {
        int count = 0;
        for (String w : words) {
            int i = 0;   // word index
            int j = 0;   // S index
            while (j < S.length() && i < w.length()) {
                if (S.charAt(j) == w.charAt(i)) {
                    i ++;
                }
               j ++;

            }
            if (i == w.length())
                count ++;
        }
        return count;
    }
}
```

293. `794. Valid Tic-Tac-Toe State.java`

```
/*

A Tic-Tac-Toe board is given as a string array board. Return True if and only if it is possible to reach this board position during the course of a valid tic-tac-toe game.

The board is a 3 x 3 array, and consists of characters " ", "X", and "O".  The " " character represents an empty square.

Here are the rules of Tic-Tac-Toe:

Players take turns placing characters into empty squares (" ").
The first player always places "X" characters, while the second player always places "O" characters.
"X" and "O" characters are always placed into empty squares, never filled ones.
The game ends when there are 3 of the same (non-empty) character filling any row, column, or diagonal.
The game also ends if all squares are non-empty.
No more moves can be played if the game is over.
Example 1:
Input: board = ["O  ", "   ", "   "]
Output: false
Explanation: The first player always plays "X".

Example 2:
Input: board = ["XOX", " X ", "   "]
Output: false
Explanation: Players take turns making moves.

Example 3:
Input: board = ["XXX", "   ", "OOO"]
Output: false

Example 4:
Input: board = ["XOX", "O O", "XOX"]
Output: true
Note:

board is a length-3 array of strings, where each string board[i] has length 3.
Each board[i][j] is a character in the set {" ", "X", "O"}

*/

class Solution {
    public boolean validTicTacToe(String[] board) {
        int turns = 0;
        boolean xwin = false;
        boolean owin = false;
        int[] rows = new int[3];
        int[] cols = new int[3];
        int diag = 0;
        int antidiag = 0;
```

```
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[i].charAt(j) == 'X') {
                    turns++;
                    rows[i]++;
                    cols[j]++;
                    if (i == j)
                        diag++;
                    if (i + j == 2)
                        antidiag++;
                }
                else if (board[i].charAt(j) == 'O') {
                    turns--;
                    rows[i]--;
                    cols[j]--;
                    if (i == j)
                        diag--;
                    if (i + j == 2)
                        antidiag--;
                }
            }
        }

        xwin = rows[0] == 3 || rows[1] == 3 || rows[2] == 3 ||
               cols[0] == 3 || cols[1] == 3 || cols[2] == 3 ||
               diag == 3 || antidiag == 3;

        owin = rows[0] == -3 || rows[1] == -3 || rows[2] == -3 ||
               cols[0] == -3 || cols[1] == -3 || cols[2] == -3 ||
               diag == -3 || antidiag == -3;

        if (xwin && turns == 0 || owin && turns == 1) {
            return false;
        }
        return (turns == 0 || turns == 1) && (!xwin || !owin);
    }
}
```

294. `804. Unique Morse Code Words.java`

```
/*

International Morse Code defines a standard encoding where each letter is mapped to a
series of dots and dashes, as follows: "a" maps to ".-", "b" maps to "-...", "c" maps to
"-.-.", and so on.

For convenience, the full table for the 26 letters of the English alphabet is given
below:

[".-","-...","-.-.","-..",".","..-.","--.","....","..",".---","-.-",".-..","--","-.","---
",".--.","--.-",".-.","...","-","..-","...-",".--","-..-","-.--","--.."]
Now, given a list of words, each word can be written as a concatenation of the Morse code
of each letter. For example, "cba" can be written as "-.-..--...", (which is the
concatenation "-.-." + "-..." + ".-"). We'll call such a concatenation, the
transformation of a word.

Return the number of different transformations among all words we have.

Example:
Input: words = ["gin", "zen", "gig", "msg"]
Output: 2
Explanation:
The transformation of each word is:
"gin" -> "--...-."
"zen" -> "--...-."
"gig" -> "--...--."
"msg" -> "--...--."

There are 2 different transformations, "--...-." and "--...--.".
Note:

The length of words will be at most 100.
Each words[i] will have length in range [1, 12].
words[i] will only consist of lowercase letters.

*/

class Solution {
    public int uniqueMorseRepresentations(String[] words) {

            String[] morse = {".-","-...","-.-.","-..",".","..-.","--.","....","..",".---
",".-.-",".-..","--","-.","---",".--.","--.-",".-.","...","-","..-","...-",".--","-..-
","-.--","--.."};
            HashSet<String> result_set = new HashSet<String>();

            for(String word: words)
            {
                StringBuilder sb = new StringBuilder();
                for(char c: word.toCharArray())
                {
                    sb.append(morse[c - 'a']);
```

```
                }
              //  System.out.println(sb.toString());
                result_set.add(sb.toString());
            }

            return result_set.size();
    }
}
```

295. `814. Binary Tree Pruning.java`

```
/*

We are given the head node root of a binary tree, where additionally every node's value
is either a 0 or a 1.

Return the same tree where every subtree (of the given tree) not containing a 1 has been
removed.

(Recall that the subtree of a node X is X, plus every node that is a descendant of X.)

Example 1:
Input: [1,null,0,0,1]
Output: [1,null,0,null,1]

Explanation:
Only the red nodes satisfy the property "every subtree not containing a 1".
The diagram on the right represents the answer.


Example 2:
Input: [1,0,1,0,0,0,1]
Output: [1,null,1,null,1]



Example 3:
Input: [1,1,0,1,1,0,1,0]
Output: [1,1,0,1,1,null,1]



Note:

The binary tree will have at most 100 nodes.
The value of each node will only be 0 or 1.

*/

class Solution {
     public TreeNode pruneTree(TreeNode root) {
        if (root == null)
            return null;

        root.left = pruneTree(root.left);
        root.right = pruneTree(root.right);

        if (root.left == null && root.right == null && root.val == 0)
            return null;
        return root;
    }
}
```

296. `815. Bus Routes.java`

```
/*

We have a list of bus routes. Each routes[i] is a bus route that the i-th bus repeats
forever. For example if routes[0] = [1, 5, 7], this means that the first bus (0-th
indexed) travels in the sequence 1->5->7->1->5->7->1->... forever.

We start at bus stop S (initially not on a bus), and we want to go to bus stop T.
Travelling by buses only, what is the least number of buses we must take to reach our
destination? Return -1 if it is not possible.

Example:
Input:
routes = [[1, 2, 7], [3, 6, 7]]
S = 1
T = 6
Output: 2
Explanation:
The best strategy is take the first bus to the bus stop 7, then take the second bus to
the bus stop 6.
Note:

1 <= routes.length <= 500.
1 <= routes[i].length <= 500.
0 <= routes[i][j] < 10 ^ 6.

*/

class Solution {
    public int numBusesToDestination(int[][] routes, int S, int T) {

    //Map stores the routes(key) that a particular bus(value) follows
    //That is Bus0 goes on route0 with stops = [1,2,7]
    //Map = {1=[0], 2=[0], 3=[1], 6=[1], 7=[0, 1]}
    //Just to avoid repetition of a particular bus's number we can take it as BusA
instead of Bus0, so that it doesn't interfere with our bus stop numbers to create
confusion
    //Map can be seen as :
    // Map = {  1=[A], 2=[A],
    //          3=[B], 6=[B],
    //          7=[A,B]  }

    HashMap<Integer, Set<Integer>> map = new HashMap();
    for(int i = 0; i < routes.length; i++){
        for(int j = 0; j < routes[i].length; j++){
            Set set = map.getOrDefault(routes[i][j], new HashSet());
            set.add(i);
            map.put(routes[i][j], set);
        }
    }

    Queue<Integer> queue = new LinkedList();
```

```
    Set<Integer> seen = new HashSet();

    //Add the starting point in the queue for processing
    //Add starting point to the visited set, because we are having the routes[i] in a
repeated order, so that we have don't keep on visiting the same route's bus stop
    queue.add(S);
    seen.add(S);
    int level = 0;

    while(!queue.isEmpty()){
        int count = queue.size();

        while(count-- > 0){
            int busStop = queue.poll();//First we get busStop = 1
            if (busStop == T)
                return level;

            //When we do map.get(1), we get get the busNumber=0, and subsequently we
traverse the remaining stops by that particular busNumber(here bus0), so add 2 and 7 to
queue(since they are not visited)
            for(int bus : map.get(busStop)){
                //Explore all the busStops on the particular bus route
                for(int j = 0 ; j < routes[bus].length; j++){
                    if(!seen.contains(routes[bus][j])){
                        queue.add(routes[bus][j]);
                        seen.add(routes[bus][j]);
                    }
                }
            }
        }
        level++;
    }
    return -1;
}
}

/*
    The way we explore the BFS is:
    routes = [[1, 2, 7], [3, 6, 7]], S = 1, T = 6
                  |          |
              Bus0/BusA   Bus1/BusB


            1
          /   \
         2     7     <--- Level 1
             /  \
            3    6(Target)   <--- Level 2

    One more case:
    [[1,2,7],[3,6,7], [4,5,6]], S=2, T = 4

            1
          /   \
```

```
         2     7     <--- Level 1
             /  \
            3    6   <--- Level 2
                /  \
          4(Target)  5  <--- Level 3

*/
```

297. `817. Linked List Components.java`

```
/*

We are given head, the head node of a linked list containing unique integer values.

We are also given the list G, a subset of the values in the linked list.

Return the number of connected components in G, where two values are connected if they
appear consecutively in the linked list.

Example 1:

Input:
head: 0->1->2->3
G = [0, 1, 3]
Output: 2
Explanation:
0 and 1 are connected, so [0, 1] and [3] are the two connected components.
Example 2:

Input:
head: 0->1->2->3->4
G = [0, 3, 1, 4]
Output: 2
Explanation:
0 and 1 are connected, 3 and 4 are connected, so [0, 1] and [3, 4] are the two connected
components.
Note:

If N is the length of the linked list given by head, 1 <= N <= 10000.
The value of each node in the linked list will be in the range [0, N - 1].
1 <= G.length <= 10000.
G is a subset of all values in the linked list.

*/

/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) { val = x; }
 * }
 */
class Solution {
    public int numComponents(ListNode head, int[] G) {
        Set<Integer> set = new HashSet<Integer>();
        for(int i: G)
            set.add(i);
        int result = 0;

        //We increment the result when current element
```

```
        while(head != null){
            if(set.contains(head.val) && (head.next == null ||
!set.contains(head.next.val) ) )
                result++;
            head = head.next;
        }
        return result;
    }
}
```

298. `819. Most Common Word.java`

```
/*

Given a paragraph and a list of banned words, return the most frequent word that is not
in the list of banned words.  It is guaranteed there is at least one word that isn't
banned, and that the answer is unique.

Words in the list of banned words are given in lowercase, and free of punctuation.  Words
in the paragraph are not case sensitive.  The answer is in lowercase.



Example:

Input:
paragraph = "Bob hit a ball, the hit BALL flew far after it was hit."
banned = ["hit"]
Output: "ball"
Explanation:
"hit" occurs 3 times, but it is a banned word.
"ball" occurs twice (and no other word does), so it is the most frequent non-banned word
in the paragraph.
Note that words in the paragraph are not case sensitive,
that punctuation is ignored (even if adjacent to words, such as "ball,"),
and that "hit" isn't the answer even though it occurs more because it is banned.


Note:

1 <= paragraph.length <= 1000.
0 <= banned.length <= 100.
1 <= banned[i].length <= 10.
The answer is unique, and written in lowercase (even if its occurrences in paragraph may
have uppercase symbols, and even if it is a proper noun.)
paragraph only consists of letters, spaces, or the punctuation symbols !?',;.
There are no hyphens or hyphenated words.
Words only consist of letters, never apostrophes or other punctuation symbols.

*/

class Solution {
    public String mostCommonWord(String paragraph, String[] banned) {
        String[] words = paragraph.toLowerCase().split("[ !?',;.]+");
        HashMap<String, Integer> map = new HashMap<>();
        for(String word : words) map.put(word, map.getOrDefault(word, 0) + 1);
        for(String word : banned) if(map.containsKey(word)) map.remove(word);
        String res = null;
        for(String word : map.keySet())
            if(res == null || map.get(word) > map.get(res))
                res = word;
        return res;
```

```
    }
}
```

299. `822. Card Flipping Game.java`

```
/*

On a table are N cards, with a positive integer printed on the front and back of each
card (possibly different).

We flip any number of cards, and after we choose one card.

If the number X on the back of the chosen card is not on the front of any card, then this
number X is good.

What is the smallest number that is good?  If no number is good, output 0.

Here, fronts[i] and backs[i] represent the number on the front and back of card i.

A flip swaps the front and back numbers, so the value on the front is now on the back and
vice versa.

Example:

Input: fronts = [1,2,4,4,7], backs = [1,3,4,1,3]
Output: 2
Explanation: If we flip the second card, the fronts are [1,3,4,4,7] and the backs are
[1,2,4,1,3].
We choose the second card, which has number 2 on the back, and it isn't on the front of
any card, so 2 is good.


Note:

1 <= fronts.length == backs.length <= 1000.
1 <= fronts[i] <= 2000.
1 <= backs[i] <= 2000.

*/

class Solution {
    public int flipgame(int[] fronts, int[] backs) {
        Set<Integer> same = new HashSet();
        for (int i = 0; i < fronts.length; ++i)
            if (fronts[i] == backs[i])
                same.add(fronts[i]);

        int ans = 9999;
        for (int x: fronts)
            if (!same.contains(x))
                ans = Math.min(ans, x);

        for (int x: backs)
            if (!same.contains(x))
                ans = Math.min(ans, x);
```

```
        return ans % 9999;
    }
}
```

300. `825. Friends Of Appropriate Ages.java`

```
/*
Some people will make friend requests. The list of their ages is given and ages[i] is the age of the ith person.

Person A will NOT friend request person B (B != A) if any of the following conditions are true:

age[B] <= 0.5 * age[A] + 7
age[B] > age[A]
age[B] > 100 && age[A] < 100
Otherwise, A will friend request B.

Note that if A requests B, B does not necessarily request A.  Also, people will not friend request themselves.

How many total friend requests are made?

Example 1:

Input: [16,16]
Output: 2
Explanation: 2 people friend request each other.
Example 2:

Input: [16,17,18]
Output: 2
Explanation: Friend requests are made 17 -> 16, 18 -> 17.
Example 3:

Input: [20,30,100,110,120]
Output:
Explanation: Friend requests are made 110 -> 100, 120 -> 110, 120 -> 100.


Notes:

1 <= ages.length <= 20000.
1 <= ages[i] <= 120.

*/

class Solution {
    public int numFriendRequests(int[] ages) {
        int freq [] = new int [121];
        int ret = 0;

        for(int i = 0; i < ages.length; i++){
            freq[ages[i]]++;
        }

        for(int i = 1; i <= 120; i++){
```

```
            for(int j = 1; j <= 120; j++){
                if(!((j <= 0.5 * i + 7) || (j > i) || (j > 100 && i < 100))){
                    if(i == j) {
                        ret = ret + ((freq[i] * (freq[i] - 1)));
                    } else {
                        ret = ret + (freq[i] * freq[j]);
                    }
                }
            }
        }

        return ret;
    }
}
```

301. `832. Flipping an Image.java`

```
/*

Given a binary matrix A, we want to flip the image horizontally, then invert it, and
return the resulting image.

To flip an image horizontally means that each row of the image is reversed.  For example,
flipping [1, 1, 0] horizontally results in [0, 1, 1].

To invert an image means that each 0 is replaced by 1, and each 1 is replaced by 0. For
example, inverting [0, 1, 1] results in [1, 0, 0].

Example 1:

Input: [[1,1,0],[1,0,1],[0,0,0]]
Output: [[1,0,0],[0,1,0],[1,1,1]]
Explanation: First reverse each row: [[0,1,1],[1,0,1],[0,0,0]].
Then, invert the image: [[1,0,0],[0,1,0],[1,1,1]]
Example 2:

Input: [[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]]
Output: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
Explanation: First reverse each row: [[0,0,1,1],[1,0,0,1],[1,1,1,0],[0,1,0,1]].
Then invert the image: [[1,1,0,0],[0,1,1,0],[0,0,0,1],[1,0,1,0]]
Notes:

1 <= A.length = A[0].length <= 20
0 <= A[i][j] <= 1

*/

class Solution {
    public int[][] flipAndInvertImage(int[][] A) {
        int n = A.length;
        for (int[] row : A)
            for (int i = 0; i * 2 < n; i++)
                if (row[i] == row[n - i - 1])
                    row[i] = row[n - i - 1] ^= 1;
        return A;
    }
}
```

302. `833. Find And Replace in String.java`

```
/*

To some string S, we will perform some replacement operations that replace groups of
letters with new ones (not necessarily the same size).

Each replacement operation has 3 parameters: a starting index i, a source word x and a
target word y.  The rule is that if x starts at position i in the original string S, then
we will replace that occurrence of x with y.  If not, we do nothing.

For example, if we have S = "abcd" and we have some replacement operation i = 2, x =
"cd", y = "ffff", then because "cd" starts at position 2 in the original string S, we
will replace it with "ffff".

Using another example on S = "abcd", if we have both the replacement operation i = 0, x =
"ab", y = "eee", as well as another replacement operation i = 2, x = "ec", y = "ffff",
this second operation does nothing because in the original string S[2] = 'c', which
doesn't match x[0] = 'e'.

All these operations occur simultaneously.  It's guaranteed that there won't be any
overlap in replacement: for example, S = "abc", indexes = [0, 1], sources = ["ab","bc"]
is not a valid test case.

Example 1:

Input: S = "abcd", indexes = [0,2], sources = ["a","cd"], targets = ["eee","ffff"]
Output: "eeebffff"
Explanation: "a" starts at index 0 in S, so it's replaced by "eee".
"cd" starts at index 2 in S, so it's replaced by "ffff".
Example 2:

Input: S = "abcd", indexes = [0,2], sources = ["ab","ec"], targets = ["eee","ffff"]
Output: "eeecd"
Explanation: "ab" starts at index 0 in S, so it's replaced by "eee".
"ec" doesn't starts at index 2 in the original S, so we do nothing.
Notes:

0 <= indexes.length = sources.length = targets.length <= 100
0 < indexes[i] < S.length <= 1000
All characters in given inputs are lowercase letters

*/
class Solution {
      public String findReplaceString(String S, int[] indexes, String[] sources, String[]
targets) {
        List<int[]> sorted = new ArrayList<>();
        for (int i = 0 ; i < indexes.length; i++) sorted.add(new int[]{indexes[i], i});
        Collections.sort(sorted, Comparator.comparing(i -> -i[0]));
        for (int[] ind: sorted) {
            int i = ind[0], j = ind[1];
            String s = sources[j], t = targets[j];
            if (S.substring(i, i + s.length()).equals(s)) S = S.substring(0, i) + t +
```

```
S.substring(i + s.length());
        }
        return S;
    }
}
```

303. `842. Split Array into Fibonacci Sequence.java`

```
/*

Given a string S of digits, such as S = "123456579", we can split it into a Fibonacci-
like sequence [123, 456, 579].

Formally, a Fibonacci-like sequence is a list F of non-negative integers such that:

0 <= F[i] <= 2^31 - 1, (that is, each integer fits a 32-bit signed integer type);
F.length >= 3;
and F[i] + F[i+1] = F[i+2] for all 0 <= i < F.length - 2.
Also, note that when splitting the string into pieces, each piece must not have extra
leading zeroes, except if the piece is the number 0 itself.

Return any Fibonacci-like sequence split from S, or return [] if it cannot be done.

Example 1:

Input: "123456579"
Output: [123,456,579]
Example 2:

Input: "11235813"
Output: [1,1,2,3,5,8,13]
Example 3:

Input: "112358130"
Output: []
Explanation: The task is impossible.
Example 4:

Input: "0123"
Output: []
Explanation: Leading zeroes are not allowed, so "01", "2", "3" is not valid.
Example 5:

Input: "1101111"
Output: [110, 1, 111]
Explanation: The output [11, 0, 11, 11] would also be accepted.
Note:

1 <= S.length <= 200
S contains only digits.

*/

class Solution {
    public List<Integer> splitIntoFibonacci(String S) {
    List<Integer> ans = new ArrayList<>();
    helper(S, ans, 0);
    return ans;
}
```

```
public boolean helper(String s, List<Integer> ans, int idx) {
    if (idx == s.length() && ans.size() >= 3) {
        return true;
    }
    for (int i=idx; i<s.length(); i++) {
        if (s.charAt(idx) == '0' && i > idx) {
            break;
        }
        long num = Long.parseLong(s.substring(idx, i+1));
        if (num > Integer.MAX_VALUE) {
            break;
        }
        int size = ans.size();
        // early termination
        if (size >= 2 && num > ans.get(size-1)+ans.get(size-2)) {
            break;
        }
        if (size <= 1 || num == ans.get(size-1)+ans.get(size-2)) {
            ans.add((int)num);
            // branch pruning. if one branch has found fib seq, return true to upper call
            if (helper(s, ans, i+1)) {
                return true;
            }
            ans.remove(ans.size()-1);
        }
    }
    return false;
}
}
```

304. `844. Backspace String Compare.java`

```
/*

Given two strings S and T, return if they are equal when both are typed into empty text
editors. # means a backspace character.

Example 1:

Input: S = "ab#c", T = "ad#c"
Output: true
Explanation: Both S and T become "ac".
Example 2:

Input: S = "ab##", T = "c#d#"
Output: true
Explanation: Both S and T become "".
Example 3:

Input: S = "a##c", T = "#a#c"
Output: true
Explanation: Both S and T become "c".
Example 4:

Input: S = "a#c", T = "b"
Output: false
Explanation: S becomes "c" while T becomes "b".
Note:

1 <= S.length <= 200
1 <= T.length <= 200
S and T only contain lowercase letters and '#' characters.
Follow up:

Can you solve it in O(N) time and O(1) space?

*/

class Solution {
    public boolean backspaceCompare(String S, String T) {
        return build(S).equals(build(T));
    }

    public String build(String S) {
        Stack<Character> ans = new Stack();
        for (char c: S.toCharArray()) {
            if (c != '#')
                ans.push(c);
            else if (!ans.empty())
                ans.pop();
        }
        return String.valueOf(ans);
```

```
    }
}
```

305. `852. Peak Index in a Mountain Array.java`

```
/*

Let's call an array A a mountain if the following properties hold:

A.length >= 3
There exists some 0 < i < A.length - 1 such that A[0] < A[1] < ... A[i-1] < A[i] > A[i+1]
> ... > A[A.length - 1]
Given an array that is definitely a mountain, return any i such that A[0] < A[1] < ...
A[i-1] < A[i] > A[i+1] > ... > A[A.length - 1].

Example 1:

Input: [0,1,0]
Output: 1
Example 2:

Input: [0,2,1,0]
Output: 1
Note:

3 <= A.length <= 10000
0 <= A[i] <= 10^6
A is a mountain, as defined above.

*/

class Solution {
    public int peakIndexInMountainArray(int[] num) {
        int max = num[0];
        int index = 0;
        for(int i=1; i<=num.length-2; i++){
            int prev = num[i-1];
            int curr = num[i];
            int next = num[i+1];

            System.out.print(next);
            if(curr > prev && curr > next && curr > max){
                index = i;
                max = curr;
            }
        }

        if(num[num.length-1] > max){
            return num.length-1;
        }

        return index;
    }
}
```

306. 863. All Nodes Distance K in Binary Tree.java

```
/*

We are given a binary tree (with root node root), a target node, and an integer value K.

Return a list of the values of all nodes that have a distance K from the target node.
The answer can be returned in any order.



Example 1:

Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, K = 2

Output: [7,4,1]

Explanation:
The nodes that are a distance 2 from the target node (with value 5)
have values 7, 4, and 1.



Note that the inputs "root" and "target" are actually TreeNodes.
The descriptions of the inputs above are just serializations of these objects.


Note:

The given tree is non-empty.
Each node in the tree has unique values 0 <= node.val <= 500.
The target node is a node in the tree.
0 <= K <= 1000.

*/

class Solution {
    public List<Integer> distanceK(TreeNode root, TreeNode target, int K)
    {
         Map<TreeNode, TreeNode> nodeToParentMap = new HashMap();
         populateNodeToParentMap(nodeToParentMap, root, null);

         Queue<TreeNode> queue = new LinkedList();
         queue.add(target);

        Set<TreeNode> seen = new HashSet();
        seen.add(target);

        List<Integer> result = new ArrayList<Integer>();
         int currentLayer = 0;

        while (!queue.isEmpty())
        {
```

```
            if (currentLayer == K)
            {
                for (TreeNode node: queue) {
                    result.add(node.val);
                }
                return result;
            }

            int layerSize = queue.size();

            for (int i = 0; i < layerSize; i++)
            {

                 TreeNode currentNode = queue.poll();

                  if (currentNode.left != null && !seen.contains(currentNode.left))
                  {
                      seen.add(currentNode.left);
                      queue.offer(currentNode.left);
                  }

                  if (currentNode.right != null && !seen.contains(currentNode.right))
                  {
                     seen.add(currentNode.right);
                     queue.offer(currentNode.right);
                  }

                  TreeNode parentOfCurrentNode = nodeToParentMap.get(currentNode);
                  if (parentOfCurrentNode != null && !seen.contains(parentOfCurrentNode))
                  {
                     seen.add(parentOfCurrentNode);
                     queue.offer(parentOfCurrentNode);




                  }

            }

            currentLayer++;

        }

  return new ArrayList<Integer>();
    }



    void populateNodeToParentMap(Map<TreeNode, TreeNode> nodeToParentMap,TreeNode root,
TreeNode parent)
    {
        if (root == null)
```

```
          return;

         nodeToParentMap.put(root, parent);
         populateNodeToParentMap(nodeToParentMap, root.left, root);
         populateNodeToParentMap(nodeToParentMap, root.right, root);

    }
}
```

307. `872. Leaf-Similar Trees.java`

```
/*

Consider all the leaves of a binary tree.  From left to right order, the values of those leaves form a leaf value sequence.

For example, in the given tree above, the leaf value sequence is (6, 7, 4, 9, 8).

Two binary trees are considered leaf-similar if their leaf value sequence is the same.

Return true if and only if the two given trees with head nodes root1 and root2 are leaf-similar.

Note:

Both of the given trees will have between 1 and 100 nodes

*/

class Solution {
    public boolean leafSimilar(TreeNode root1, TreeNode root2) {
        List<Integer> leaves1 = new ArrayList();
        List<Integer> leaves2 = new ArrayList();
        dfs(root1, leaves1);
        dfs(root2, leaves2);
        return leaves1.equals(leaves2);
    }

    public void dfs(TreeNode node, List<Integer> leafValues) {
        if (node != null) {
            if (node.left == null && node.right == null)
                leafValues.add(node.val);
            dfs(node.left, leafValues);
            dfs(node.right, leafValues);
        }
    }
}
```

308. `876. Middle of the Linked List.java`

```
/*

Given a non-empty, singly linked list with head node head, return a middle node of linked
list.

If there are two middle nodes, return the second middle node.



Example 1:

Input: [1,2,3,4,5]
Output: Node 3 from this list (Serialization: [3,4,5])
The returned node has value 3.  (The judge's serialization of this node is [3,4,5]).
Note that we returned a ListNode object ans, such that:
ans.val = 3, ans.next.val = 4, ans.next.next.val = 5, and ans.next.next.next = NULL.
Example 2:

Input: [1,2,3,4,5,6]
Output: Node 4 from this list (Serialization: [4,5,6])
Since the list has two middle nodes with values 3 and 4, we return the second one.


Note:

The number of nodes in the given list will be between 1 and 100.

*/

class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

309. `880. Decoded String at Index.java`

```
/*

An encoded string S is given.  To find and write the decoded string to a tape, the
encoded string is read one character at a time and the following steps are taken:

If the character read is a letter, that letter is written onto the tape.
If the character read is a digit (say d), the entire current tape is repeatedly written
d-1 more times in total.
Now for some encoded string S, and an index K, find and return the K-th letter (1
indexed) in the decoded string.



Example 1:

Input: S = "leet2code3", K = 10
Output: "o"
Explanation:
The decoded string is "leetleetcodeleetleetcodeleetleetcode".
The 10th letter in the string is "o".
Example 2:

Input: S = "ha22", K = 5
Output: "h"
Explanation:
The decoded string is "hahahaha".  The 5th letter is "h".
Example 3:

Input: S = "a2345678999999999999999", K = 1
Output: "a"
Explanation:
The decoded string is "a" repeated 8301530446056247680 times.  The 1st letter is "a".


Note:

2 <= S.length <= 100
S will only contain lowercase letters and digits 2 through 9.
S starts with a letter.
1 <= K <= 10^9
The decoded string is guaranteed to have less than 2^63 letters.

*/

class Solution {
    public String decodeAtIndex(String S, int K) {
        long size = 0;
        int N = S.length();

        // Find size = length of decoded string
        for (int i = 0; i < N; ++i) {
```

```
            char c = S.charAt(i);
            if (Character.isDigit(c))
                size *= c - '0';
            else
                size++;
        }

        for (int i = N-1; i >= 0; --i) {
            char c = S.charAt(i);
            K %= size;
            if (K == 0 && Character.isLetter(c))
                return Character.toString(c);

            if (Character.isDigit(c))
                size /= c - '0';
            else
                size--;
        }

        throw null;
    }
}
```

310. `897. Increasing Order Search Tree.java`

```
/*

Given a tree, rearrange the tree in in-order so that the leftmost node in the tree is now
the root of the tree, and every node has no left child and only 1 right child.

Example 1:
Input: [5,3,6,2,4,null,8,1,null,null,null,7,9]

       5
      / \
    3    6
   / \    \
  2   4    8
 /        / \
1        7   9

Output: [1,null,2,null,3,null,4,null,5,null,6,null,7,null,8,null,9]

 1
  \
   2
    \
     3
      \
       4
        \
         5
          \
           6
            \
             7
              \
               8
                \
                 9
Note:

The number of nodes in the given tree will be between 1 and 100.
Each node will have a unique integer value from 0 to 1000.

*/

class Solution {
    public TreeNode increasingBST(TreeNode root) {
        List<Integer> vals = new ArrayList();
        inorder(root, vals);
        TreeNode ans = new TreeNode(0), cur = ans;
        for (int v: vals) {
            cur.right = new TreeNode(v);
            cur = cur.right;
        }
```

```
        return ans.right;
    }

    public void inorder(TreeNode node, List<Integer> vals) {
        if (node == null) return;
        inorder(node.left, vals);
        vals.add(node.val);
        inorder(node.right, vals);
    }
}
```

311. `904. Fruit Into Baskets.java`

```
/*

In a row of trees, the i-th tree produces fruit with type tree[i].

You start at any tree of your choice, then repeatedly perform the following steps:

Add one piece of fruit from this tree to your baskets.  If you cannot, stop.
Move to the next tree to the right of the current tree.  If there is no tree to the
right, stop.
Note that you do not have any choice after the initial choice of starting tree: you must
perform step 1, then step 2, then back to step 1, then step 2, and so on until you stop.

You have two baskets, and each basket can carry any quantity of fruit, but you want each
basket to only carry one type of fruit each.

What is the total amount of fruit you can collect with this procedure?



Example 1:

Input: [1,2,1]
Output: 3
Explanation: We can collect [1,2,1].
Example 2:

Input: [0,1,2,2]
Output: 3
Explanation: We can collect [1,2,2].
If we started at the first tree, we would only collect [0, 1].
Example 3:

Input: [1,2,3,2,2]
Output: 4
Explanation: We can collect [2,3,2,2].
If we started at the first tree, we would only collect [1, 2].
Example 4:

Input: [3,3,3,1,2,1,1,2,3,3,4]
Output: 5
Explanation: We can collect [1,2,1,1,2].
If we started at the first tree or the eighth tree, we would only collect 4 fruits.


Note:

1 <= tree.length <= 40000
0 <= tree[i] < tree.length

*/
```

```
class Solution {
       public int totalFruit(int[] tree) {
        Map<Integer, Integer> count = new HashMap<Integer, Integer>();
        int res = 0, i = 0;
        for (int j = 0; j < tree.length; ++j) {
            count.put(tree[j], count.getOrDefault(tree[j], 0) + 1);
            while (count.size() > 2) {
                count.put(tree[i], count.get(tree[i]) - 1);
                if (count.get(tree[i]) == 0) count.remove(tree[i]);
                i++;
            }
            res = Math.max(res, j - i + 1);
        }
        return res;
    }
}
```

312. `905. Sort Array By Parity.java`

```
/*

Given an array A of non-negative integers, return an array consisting of all the even
elements of A, followed by all the odd elements of A.

You may return any answer array that satisfies this condition.



Example 1:

Input: [3,1,2,4]
Output: [2,4,3,1]
The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.


Note:

1 <= A.length <= 5000
0 <= A[i] <= 5000

*/

class Solution {
    public int[] sortArrayByParity(int[] A) {
        int[] ans = new int[A.length];
        int t = 0;

        for (int i = 0; i < A.length; ++i)
            if (A[i] % 2 == 0)
                ans[t++] = A[i];

        for (int i = 0; i < A.length; ++i)
            if (A[i] % 2 == 1)
                ans[t++] = A[i];

        return ans;
    }
}
```

313. `909. Snakes and Ladders.java`

```
/*

On an N x N board, the numbers from 1 to N*N are written boustrophedonically starting
from the bottom left of the board, and alternating direction each row.  For example, for
a 6 x 6 board, the numbers are written as follows:


You start on square 1 of the board (which is always in the last row and first column).
Each move, starting from square x, consists of the following:

You choose a destination square S with number x+1, x+2, x+3, x+4, x+5, or x+6, provided
this number is <= N*N.
(This choice simulates the result of a standard 6-sided die roll: ie., there are always
at most 6 destinations, regardless of the size of the board.)
If S has a snake or ladder, you move to the destination of that snake or ladder.
Otherwise, you move to S.
A board square on row r and column c has a "snake or ladder" if board[r][c] != -1.  The
destination of that snake or ladder is board[r][c].

Note that you only take a snake or ladder at most once per move: if the destination to a
snake or ladder is the start of another snake or ladder, you do not continue moving.
(For example, if the board is `[[4,-1],[-1,3]]`, and on the first move your destination
square is `2`, then you finish your first move at `3`, because you do not continue moving
to `4`.)

Return the least number of moves required to reach square N*N.  If it is not possible,
return -1.

Example 1:

Input: [
[-1,-1,-1,-1,-1,-1],
[-1,-1,-1,-1,-1,-1],
[-1,-1,-1,-1,-1,-1],
[-1,35,-1,-1,13,-1],
[-1,-1,-1,-1,-1,-1],
[-1,15,-1,-1,-1,-1]]
Output: 4
Explanation:
At the beginning, you start at square 1 [at row 5, column 0].
You decide to move to square 2, and must take the ladder to square 15.
You then decide to move to square 17 (row 3, column 5), and must take the snake to square
13.
You then decide to move to square 14, and must take the ladder to square 35.
You then decide to move to square 36, ending the game.
It can be shown that you need at least 4 moves to reach the N*N-th square, so the answer
is 4.
Note:

2 <= board.length = board[0].length <= 20
board[i][j] is between 1 and N*N or is equal to -1.
```

```
The board square with number 1 has no snake or ladder.
The board square with number N*N has no snake or ladder.

*/

class Solution {
    public int snakesAndLadders(int[][] board) {
        if (board == null || board.length == 0 || board[0].length == 0) {
            return -1;
        }
        int rows = board.length;
        int cols = board[0].length;
        int dest = rows * cols;
        Queue<Integer> queue = new LinkedList<>();
        queue.offer(1);
        Set<Integer> set = new HashSet<>();
        set.add(1);
        int steps = 0;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int curr = queue.poll();
                if (curr == dest) {
                    return steps;
                }
                for (int diff = 1; diff <= 6 && curr + diff <= dest; diff++) {
                    int[] pos = getCoordinate(curr + diff, rows, cols);
                    int next = board[pos[0]][pos[1]] == -1 ? curr + diff : board[pos[0]]
[pos[1]];
                    if (!set.contains(next)) {
                        queue.offer(next);
                        set.add(next);
                    }
                }
            }
            steps++;
        }
        return -1;
    }

    public int[] getCoordinate(int n, int rows, int cols) {
        int r = rows - 1 - (n - 1) / cols;
        int c = (n - 1) % cols;
        if (r % 2 == rows % 2) {
            return new int[]{r, cols - 1 - c};
        } else {
            return new int[]{r, c};
        }
    }
}
```

314. `911. Online Election.java`

```
/*

In an election, the i-th vote was cast for persons[i] at time times[i].

Now, we would like to implement the following query function: TopVotedCandidate.q(int t)
will return the number of the person that was leading the election at time t.

Votes cast at time t will count towards our query.  In the case of a tie, the most recent
vote (among tied candidates) wins.



Example 1:

Input: ["TopVotedCandidate","q","q","q","q","q","q"], [[[0,1,1,0,0,1,0],
[0,5,10,15,20,25,30]],[3],[12],[25],[15],[24],[8]]
Output: [null,0,1,1,0,0,1]
Explanation:
At time 3, the votes are [0], and 0 is leading.
At time 12, the votes are [0,1,1], and 1 is leading.
At time 25, the votes are [0,1,1,0,0,1], and 1 is leading (as ties go to the most recent
vote.)
This continues for 3 more queries at time 15, 24, and 8.


Note:

1 <= persons.length = times.length <= 5000
0 <= persons[i] <= persons.length
times is a strictly increasing array with all elements in [0, 10^9].
TopVotedCandidate.q is called at most 10000 times per test case.
TopVotedCandidate.q(int t) is always called with t >= times[0]

*/

class TopVotedCandidate {

   private Map<Integer, Integer> map = new HashMap<>(); // time and leading candidate
    private int[] times;

public TopVotedCandidate(int[] persons, int[] times)
{
    this.times = times;
    int[] count = new int[persons.length + 1]; // count[i]: count of votes for persons[i]

    for (int i = 0, winner = -1; i < times.length; ++i)
    {
        ++count[persons[i]]; // at times[i], persons[i] got a vote.
        // is persons[i] leading? update winner.
        if (map.isEmpty() || count[winner] <= count[persons[i]])
        {
```

```
            winner = persons[i];
        }

        map.put(times[i], winner); // update time and winner.
    }
}
public int q(int t)
{
    int idx = Arrays.binarySearch(times, t); // search for the time slot.

    //We have a problem when we do Arrays.binarySearch()
    //If the element is present in the array then we return that index
    //Otherwise we return the index = { -(insertion point) - 1}
    //Source: https://stackoverflow.com/questions/13734694/array-binarysearch-returning-
negative-numbers
    //So inorder to counter the negative index that we are getting, we need to make it
positive by using (-idx - 2)
    return map.get(times[idx < 0 ? -idx - 2 : idx]); // fetch the corresponding
information.
}
}

/**
 * Your TopVotedCandidate object will be instantiated and called as such:
 * TopVotedCandidate obj = new TopVotedCandidate(persons, times);
 * int param_1 = obj.q(t);
 */
```

315. `929. Unique Email Addresses.java`

```
/*

Every email consists of a local name and a domain name, separated by the @ sign.

For example, in alice@leetcode.com, alice is the local name, and leetcode.com is the
domain name.

Besides lowercase letters, these emails may contain '.'s or '+'s.

If you add periods ('.') between some characters in the local name part of an email
address, mail sent there will be forwarded to the same address without dots in the local
name.  For example, "alice.z@leetcode.com" and "alicez@leetcode.com" forward to the same
email address.  (Note that this rule does not apply for domain names.)

If you add a plus ('+') in the local name, everything after the first plus sign will be
ignored. This allows certain emails to be filtered, for example m.y+name@email.com will
be forwarded to my@email.com.  (Again, this rule does not apply for domain names.)

It is possible to use both of these rules at the same time.

Given a list of emails, we send one email to each address in the list.  How many
different addresses actually receive mails?



Example 1:

Input:
["test.email+alex@leetcode.com","test.e.mail+bob.cathy@leetcode.com","testemail+david@lee
.tcode.com"]
Output: 2
Explanation: "testemail@leetcode.com" and "testemail@lee.tcode.com" actually receive
mails


Note:

1 <= emails[i].length <= 100
1 <= emails.length <= 100
Each emails[i] contains exactly one '@' character.
All local and domain names are non-empty.
Local names do not start with a '+' character.

*/

class Solution {
    public int numUniqueEmails(String[] emails) {
        Set<String> seen = new HashSet();
        for (String email: emails) {
            int i = email.indexOf('@');
            String local = email.substring(0, i);
```

```
            String rest = email.substring(i);
            if (local.contains("+")) {
                local = local.substring(0, local.indexOf('+'));
            }
            local = local.replaceAll(".", "");
            seen.add(local + rest);
        }

        return seen.size();
    }
}
```

316. `934. Shortest Bridge.java`

```
/*

In a given 2D binary array A, there are two islands.  (An island is a 4-directionally
connected group of 1s not connected to any other 1s.)

Now, we may change 0s to 1s so as to connect the two islands together to form 1 island.

Return the smallest number of 0s that must be flipped.  (It is guaranteed that the answer
is at least 1.)



Example 1:

Input: [[0,1],[1,0]]
Output: 1
Example 2:

Input: [[0,1,0],[0,0,0],[0,0,1]]
Output: 2
Example 3:

Input: [[1,1,1,1,1],[1,0,0,0,1],[1,0,1,0,1],[1,0,0,0,1],[1,1,1,1,1]]
Output: 1


Note:

1 <= A.length = A[0].length <= 100
A[i][j] == 0 or A[i][j] == 1


*/

class Solution {

    class Pos
    {
      int x;
      int y;
      public Pos(int x, int y)
      {
        this.x = x;
        this.y = y;
      }
    }

public int shortestBridge(int[][] A)
{
    // find one island
    boolean found = false;
```

```
    Queue<Pos> q = new LinkedList<>();

    for(int i = 0; i < A.length; i++)
    {
        if(found)
            break;
        for(int j = 0; j < A[0].length; j++)
        {
            if(A[i][j] == 1)
            {
                dfs(A, i, j, q);  // find one island, use dfs to find connected conponent
                found = true;
                break;
            }
        }
    }


    // use bfs to find the shortest path
    int steps = 0;
    int[][] dirs = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    while(!q.isEmpty())
    {
        int size = q.size();

        //extend boundary until we find the other island
        while(size-- != 0)
        {
            Pos p = q.poll();
            int x = p.x;
            int y = p.y;

            for(int i = 0; i < 4; i++)
            {
                int tx = x+dirs[i][0];
                int ty = y+dirs[i][1];

                if(tx < 0 || tx >= A.length || ty < 0 || ty >= A[0].length || A[tx][ty]
== 2)
                    continue;

                if(A[tx][ty] == 1)
                    return steps;

                A[tx][ty] = 2;
                q.offer(new Pos(tx, ty));
            }
        }

        steps++;
    }

    return -1;
```

```
}

private void dfs(int[][] A, int x, int y, Queue<Pos> q){
    if(x < 0 || y < 0 || x >= A.length || y >= A[0].length || A[x][y] != 1)
        return;

    A[x][y] = 2;

    q.offer(new Pos(x, y));
    dfs(A, x+1, y, q);
    dfs(A, x-1, y, q);
    dfs(A, x, y+1, q);
    dfs(A, x, y-1, q);
}

}
```

317. `946. Validate Stack Sequences.java`

```
/*

Given two sequences pushed and popped with distinct values, return true if and only if
this could have been the result of a sequence of push and pop operations on an initially
empty stack.



Example 1:

Input: pushed = [1,2,3,4,5], popped = [4,5,3,2,1]
Output: true
Explanation: We might do the following sequence:
push(1), push(2), push(3), push(4), pop() -> 4,
push(5), pop() -> 5, pop() -> 3, pop() -> 2, pop() -> 1
Example 2:

Input: pushed = [1,2,3,4,5], popped = [4,3,5,1,2]
Output: false
Explanation: 1 cannot be popped before 2.


Note:

0 <= pushed.length == popped.length <= 1000
0 <= pushed[i], popped[i] < 1000
pushed is a permutation of popped.
pushed and popped have distinct values.

*/

class Solution {
     public boolean validateStackSequences(int[] pushed, int[] popped) {
        Stack<Integer> s = new Stack<>();
        int i = 0;
        for (int x : pushed) {
            s.push(x);
            while (!s.empty() && s.peek() == popped[i]) {
                s.pop();
                i++;
            }
        }
        return i == popped.length;
    }
}
```

318. `973. K Closest Points to Origin.java`

```
/*

We have a list of points on the plane.  Find the K closest points to the origin (0, 0).

(Here, the distance between two points on a plane is the Euclidean distance.)

You may return the answer in any order.  The answer is guaranteed to be unique (except
for the order that it is in.)



Example 1:

Input: points = [[1,3],[-2,2]], K = 1
Output: [[-2,2]]
Explanation:
The distance between (1, 3) and the origin is sqrt(10).
The distance between (-2, 2) and the origin is sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
We only want the closest K = 1 points from the origin, so the answer is just [[-2,2]].
Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], K = 2
Output: [[3,3],[-2,4]]
(The answer [[-2,4],[3,3]] would also be accepted.)


Note:

1 <= K <= points.length <= 10000
-10000 < points[i][0] < 10000
-10000 < points[i][1] < 10000

*/

class Solution {
   public int[][] kClosest(int[][] points, int K) {
    PriorityQueue<int[]> pq = new PriorityQueue<int[]>((p1, p2) -> p2[0] * p2[0] + p2[1]
* p2[1] - p1[0] * p1[0] - p1[1] * p1[1]);
    for (int[] p : points) {
        pq.offer(p);
        if (pq.size() > K) {
            pq.poll();
        }
    }
    int[][] res = new int[K][2];
    while (K > 0) {
        res[--K] = pq.poll();
    }
    return res;
```

```
}
}
```

319. `976. Largest Perimeter Triangle.java`

```
/*

Given an array A of positive lengths, return the largest perimeter of a triangle with
non-zero area, formed from 3 of these lengths.

If it is impossible to form any triangle of non-zero area, return 0.



Example 1:

Input: [2,1,2]
Output: 5
Example 2:

Input: [1,2,1]
Output: 0
Example 3:

Input: [3,2,3,4]
Output: 10
Example 4:

Input: [3,6,2,3]
Output: 8


Note:

3 <= A.length <= 10000
1 <= A[i] <= 10^6

*/

class Solution {
   public int largestPerimeter(int[] A) {
       //For a >= b >= c, a,b,c can form a triangle if a < b + c

        Arrays.sort(A);
        for (int i = A.length - 1; i > 1; --i)
            if (A[i] < A[i - 1] + A[i - 2])
                return A[i] + A[i - 1] + A[i - 2];
        return 0;
    }
}
```

320. `994. Rotting Oranges.java`

```
/*

In a given grid, each cell can have one of three values:

the value 0 representing an empty cell;
the value 1 representing a fresh orange;
the value 2 representing a rotten orange.
Every minute, any fresh orange that is adjacent (4-directionally) to a rotten orange
becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange.
If this is impossible, return -1 instead.



Example 1:



Input: [[2,1,1],[1,1,0],[0,1,1]]
Output: 4
Example 2:

Input: [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation:  The orange in the bottom left corner (row 2, column 0) is never rotten,
because rotting only happens 4-directionally.
Example 3:

Input: [[0,2]]
Output: 0
Explanation:  Since there are already no fresh oranges at minute 0, the answer is just 0.


Note:

1 <= grid.length <= 10
1 <= grid[0].length <= 10
grid[i][j] is only 0, 1, or 2.

*/

class Solution {
    public int orangesRotting(int[][] grid) {

        if(grid == null || grid.length == 0)
            return 0;
        int rows = grid.length;
        int cols = grid[0].length;

        Queue<int[]> queue = new LinkedList<>();
```

```
        int count_fresh = 0;

        for(int i = 0 ; i < rows ; i++) {
            for(int j = 0 ; j < cols ; j++) {
                if(grid[i][j] == 2) {
                    queue.offer(new int[]{i , j});
                }
                else if(grid[i][j] == 1) {
                    count_fresh++;
                }
            }
        }

        if(count_fresh == 0)
            return 0;

        int count = 0;
        int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};

        while(!queue.isEmpty()) {
            count++;

            int size = queue.size();

            for(int i = 0 ; i < size ; i++) {
                int[] point = queue.poll();

                for(int dir[] : dirs) {
                    int x = point[0] + dir[0];
                    int y = point[1] + dir[1];

                    if(x < 0 || y < 0 || x >= rows || y >= cols || grid[x][y] == 0 ||
grid[x][y] == 2)
                        continue;

                    grid[x][y] = 2;
                    queue.offer(new int[]{x , y});
                    //decrease the count of fresh oranges by 1
                    count_fresh--;
                }
            }
        }

        return count_fresh == 0 ? count-1 : -1;
    }
}
```

321. `1013. Partition Array Into Three Parts With Equal`

Sum.java

```
/*

Given an array A of integers, return true if and only if we can partition the array into three non-empty parts with equal sums.

Formally, we can partition the array if we can find indexes i+1 < j with (A[0] + A[1] + ... + A[i] == A[i+1] + A[i+2] + ... + A[j-1] == A[j] + A[j-1] + ... + A[A.length - 1])



Example 1:

Input: [0,2,1,-6,6,-7,9,1,2,0,1]
Output: true
Explanation: 0 + 2 + 1 = -6 + 6 - 7 + 9 + 1 = 2 + 0 + 1
Example 2:

Input: [0,2,1,-6,6,7,9,-1,2,0,1]
Output: false
Example 3:

Input: [3,3,6,5,-2,2,5,1,-9,4]
Output: true
Explanation: 3 + 3 = 6 = 5 - 2 + 2 + 5 + 1 - 9 + 4


Note:

3 <= A.length <= 50000
-10000 <= A[i] <= 10000

*/

class Solution {
    public boolean canThreePartsEqualSum(int[] A) {
        int sum = 0;

        for (int a : A)
            sum += a;

        if (sum % 3 != 0)
            return false;
        int target = sum / 3, count = 0, curr = 0;

        for (int a : A) {
            curr = curr + a;
            if (curr == target) {
                curr = 0;
                count++;
            }
        }
```

```
        return count >= 3;
    }
}
```